

Untitled Fighter Game

Jeremy Matina



Contents

Untitled Fighter Game	2
Analysis	9
Problem Identification	9
Stakeholders	9
Research.....	9
Computational methods	11
Data storage.....	11
Decomposition	11
Decision-making.....	11
Object-oriented programming.....	11
Limitations	11
Requirements.....	12
Hardware	12
Software.....	12
Interview/Questionnaire.....	12
Interview Conclusion.....	13
Key features (using research)	14
Key binding.....	14
CPU.....	14
Music/Sound Effects	14
Animations	14
Success Criteria	16
Must have	16
Should have.....	17
Could have	18
Won't have.....	18
Design.....	19
Design Objectives.....	19
Design Interview	19
Design Interview Results.....	23
System Flowchart.....	25
Success Criteria 1: Background Image	26
Justification	26
Flowchart	26
Success Criteria 2: Start Menu	27
Justification	27

Flowchart	27
Success Criteria 3: Game title	28
Justification	28
Flowchart	28
Success Criteria 4: Quit button on Main Menu	28
Justification	29
Flowchart 1	29
Flowchart 2	29
Success Criteria 5: Play button on Main Menu	29
Justification	30
Flowchart 1	30
Flowchart 2	31
Success Criteria 6: Settings button on Main Menu.....	32
Justification	32
Flowchart 1	32
Flowchart 2	33
Success Criteria 7: Character Selection screen	34
Justification	34
Flowchart	35
Success Criteria 8: Character 1 Selection Button.....	36
Justification	36
Flowchart	36
Success Criteria 9: Character 2 Selection Button.....	37
Justification	37
Flowchart	37
Success Criteria 10: CPU Selection Button.....	38
Justification	38
Flowchart	38
Success Criteria 11: Start Battle Button	39
Justification	39
Flowchart 1	40
Flowchart 2	41
Success Criteria 12: Battle Screen.....	42
Justification	42
Flowchart 1	43
Success Criteria 13: Timer/Timeout Screen.....	44
Justification	44

Flowchart 1	45
Flowchart 2	46
Success Criteria 14 & 15: Health Bars for Characters	47
Justification	47
Flowchart 1	47
Flowchart 2	49
Success Criteria 16 & 17: Character Image on Battle Screen.....	50
Justification	50
Flowchart 1	50
Success Criteria 18 & 23: Character Idle Animation	50
Justification	51
Flowchart 1	52
Flowchart 2	53
Success Criteria 19 & 24: Character Run Animation	54
Justification	54
Flowchart 1	54
Success Criteria 20 & 25: Character Jump Animation.....	55
Justification	55
Flowchart 1	55
Success Criteria 21 & 26: Character Attack 1 Animation	56
Justification	56
Flowchart 1	56
Success Criteria 22 & 27: Character Attack 2 Animation	57
Justification	57
Flowchart 1	57
Success Criteria 28: Knock Out Death.....	58
Justification	58
Flowchart 1	58
Success Criteria 29: Win Screen.....	59
Justification	59
Flowchart 1	59
Success Criteria 30: Restart Button on Win Screen	60
Justification	60
Flowchart 1	60
Success Criteria 31: Main Menu Button on Win Screen	61
Justification	61
Flowchart 1	61

Screen Designs	62
Main Menu.....	62
Character Selection	63
Battle Screen	64
Win Screen	65
Data Structures	66
Menu Variables	66
Fighter Class	66
Post-development Data	68
Test Plan.....	68
Implementation	81
Overview	81
Stage One: Main Menu Screen	81
Base template	81
Success Criteria 1: Background Image	81
Success Criteria 2: Main Menu.....	83
Success Criteria 4,5,6	87
Success Criteria 3: Game Title.....	87
Success Criteria 4: Quit Button	92
Success Criteria 5: Play Button.....	93
(Should have) Success Criteria 13: Highlighted buttons	94
Main Menu Screen: Review	98
Stage Two – Battle Screen	99
Success Criteria 17: Character Image.....	99
Success Criteria 18 & 23: Character Idle Animation	107
Success Criteria 19 & 24: Character Movement Function & Animation.....	108
Extra fix 1: Flipping the characters.....	113
Extra fix 2: Keeping Characters on Screen	115
Success Criteria 20 & 25: Character Jump Function & Animation	118
Success Criteria 14 & 15: Character Health Bars	124
Success Criteria 21 & 26: Character Attack 1 Function & Animation	127
Extra feature 1: Knockback on attacks.....	135
Success Criteria 13: Timer	139
(Should have) Success Criteria 1 & 3: Ability 1 Animation.....	143
(Should have) Success Criteria 7: Battle Screen Music	160
Battle Screen: Review	164
Stage Three – Character Selection Screen	166

Success Criteria 8: Character 1 Button.....	166
Success Criteria 10: CPU Selection Button.....	172
Success Criteria 11: Start Battle Button.....	177
Character Selection Screen: Review	179
Stage Four – Win/Timeout Screen.....	180
Success Criteria 28: Knock Out Death.....	180
Success Criteria 29: Win Screen.....	183
Success Criteria 13: TimeOut Screen	187
Win/Timeout Screen: Review	189
Evaluation	190
Testing for Evaluation	190
Success Criteria 1: Background Image	190
Success Criteria 2 & 3: Main Menu and Game Title.....	192
Success Criteria 4: Quit	192
Success Criteria 5: Play Button.....	194
Success Criteria 6: Options Button.....	195
Success Criteria 7: Character Selection screen	196
Success Criteria 8: Character 1 Button.....	197
Success Criteria 9: Character 2 Button.....	199
Success Criteria 10: CPU Button.....	202
Success Criteria 11: Start Battle Button	205
Success Criteria 12: Battle Screen.....	206
Success Criteria 13: Timer/Timeout Screen.....	207
Success Criteria 14 & 15: Health Bars	209
Success Criteria 16 & 17: Character Image	211
Success Criteria 18 & 23: Character Idle Animation	211
Success Criteria 19 & 24 : Character Run Animation	214
Success Criteria 20 & 25 : Character Jump Animation.....	217
Success Criteria 21, 22, 26 & 27 : Character 1 & 2, Attack 1 & 2 Animation	220
Success Criteria 28 : Knock Out Death	222
Success Criteria 29 : Win Screen	223
Success Criteria 30 : Restart Button.....	224
Success Criteria 31 : Main Menu Button.....	225
User Interview: Usability & Maintenance.....	226
User Interview: General Feedback and Limitations.....	227
User Interview: Conclusion	228
Bibliography	229

Appendix	230
Background image	230
Shinobi	230
Samurai	232
Red Fire Ball	234
UFG1.py.....	234
UFGfighter.py.....	256
UFGability.py.....	267

Analysis

Problem Identification

In many versions of old, small-scale 2D fighter games, they had no CPU enemy (an enemy controlled by the game's algorithms) which a user could face. This meant that one user couldn't play the game as they may not have anyone to battle with. So, to solve this problem, I will program a 2D fighting game using Python, with the Pygame library, which will include a CPU enemy to deliver a difficult challenge in battle. The objective of the game is to defeat the other opponent in the game using attack skills to deplete the opponent's health to 0. The game will have a menu which will have two options: CPU mode and 2-player mode. The CPU challenge mode will lead into the main game where the user can control a character and will use various attacks to defeat the CPU opponent. It will get more difficult to kill when users increase the difficulty of the CPU, which gives it more buffs to its attributes and makes it more satisfying to defeat. The 2-player mode allows two users on the same device to fight each other using different keys to control a character on the screen.

Stakeholders

The main demographic my project is aimed at are teenagers and children (approximately aged 10-19), but my game can be played by anyone who has an interest in gaming and entertainment whilst also wanting a challenging experience. My project will allow two of these users to play my game at the same time on the same machine or a single player can play alone by battling a CPU opponent. In my interview, I will ask students of a sixth form about their requirements and expectations of my fighting game, and I will collate the results of their needs to form my success criteria for my project.

Research

In my project, I will be using inspiration from different aspects of successful fighting games which I will incorporate into my project. I will be looking at Mortal Kombat (1993), Super Smash Flash 2 (2006), Street Fighter II (1991).



Mortal Kombat (1993) is a very distinguished fighting game that is popular among a range of ages. The game consists of a main menu, character selection screen and a battleground for users to control their characters to fight each other. The point of the game is for two users to select a character and fight each other using the attack moves set for each character chosen until one player's health goes to zero, meaning that the other player has won the fight.

I am inspired the way different characters have unique attacks and skills, and I think this will be a good addition to my project because players will be able to explore, learn and master the different attacks and skills that my sprites will have, making it more fun to play repeatedly for the user. The game also has a background selection screen where you can choose the scenery in which your players will fight. I may not have time to

include many backgrounds, but I will make sure that there will be a background selection screen as it provides more choice and entertainment for the player which is my aim of this project.

I like how realistic the game is, and the theme of bloody battles is evident throughout the game. I think having a solid theme that runs throughout a game will help it to keep its player base whilst also gathering unaccustomed users towards the game, so I believe it would be helpful for my project to have an overarching theme. This also coincides with the colour selection of the game and how it should match with the theme.

The font on the health bars and the “FIGHT!!” text before the battle is really intriguing as they add to the brutal theme of the game. Naming of the characters also open the scope of the characters in the game to have some identification and a backstory. This element of games is what many argue how a game keeps and attracts users, but for my project, it may be time-consuming given my circumstances, so a story is an unrealistic goal for this project. Moreover, since my game is targeting children and teenagers, having a similar brutal theme to Mortal Kombat will be unsuitable for children.



Super Smash Flash 2 (2006) was an outstanding fighting game during its time and operated like the game Super Smash Bros (1999) where you would select from a range of different characters and use their attacks and skills to knock off other players from the fighting stage. In my game, it would be unrealistic to have a multitude of

different characters because of the time constraint that I have on this project, but I will have at least two different characters that have different moves to use in my game to allow for versatility in the way each character can be learned and used.

I like how the game allows users to adjust how quickly a battle can go on for. This helps users challenge themselves by learning how to play under low time conditions or users can play a laxer battle with no time pressure until a player wins the match. I will make each round timed in my game, but I may also incorporate the ability to adjust this time so that the users of my game can play to their ability and challenge themselves according to their skill level.

What I like most about this game is the “knock off” mechanic whereby each hit you receive increases how much knockback you gain from subsequent attacks. This unique style of play was a remarkably successful element of the game, and I may consider it for my project because it adds a humorous mechanic into my game which will make it more enjoyable for the user.



Street Fighter II (1991) was and is loved by many for its detailed sprites and gameplay design. Uniquely, the game features a win count on the top of each characters' health bar and I think this is quite useful because this will bring out a competitive side towards users and will make them want to play more to get a higher win count than the other player.

I like how each character has different abilities that can be short-range or long-range, making

it possible to master different characters on the game.

I like how the game has different music for the different backgrounds that you can select to play on. This adds distinct themes to each background and provides immersive battle music for the user to play with, enhancing the gaming experience the user is having.

Computational methods

Data storage

My game will need to record and store user inputs such as keyboard inputs for the movement of the selected character and storing data on player health so that the computer can check whether a player has died. Using a computer will make this issue much easier to tackle as data can be easily accessed and stored.

Decomposition

The creation of my game can easily be decomposed into smaller, manageable tasks which can then be completed and reassembled back to make the whole game. This can be done in many ways for my fighting game (function and aesthetic, screen by screen, etc) but I have chosen to split each stage by each screen because the stages will be easier to focus on and test.

Decision-making

My project will require decision-making logic to be used to control CPU movements and attacks, which makes it suitable for a computational approach. Data on the player and enemy entity (Health, damage, dead, etc..) will need to be stored and referenced on a computer so that the CPU can actively be make decisions effectively according to its situation.

Object-oriented programming

I will use a class for the fighters so that I can assign attributes (health, speed, type of attacks) and methods (move, updating animations, attack) to eventually create two instances of these fighters that will interact with each other using these methods and attributes.

Limitations

Unfortunately, there are limitations to my project that must be addressed to emphasise the scope of this project.

My game will not feature online playability because it would be difficult to maintain costly servers and prevent issues such as latency and data security under the duration of time I have for this project.

My game will be 2D instead of 3D because 3D games are more time-consuming and I will require more research of advanced concepts such as rendering, lighting and shaders. This, paired with the fact that 3D games require much more processing power (may require a GPU), makes adding 3D elements to my game unrealistic for this project.

The animations in my game will be borrowed and credited from a website that offers free sprite characters ([2D Game Assets Store & Free - CraftPix.net](https://craftpix.net)), as such, I will not be creating any sprites. I would want to prioritise coding the game instead of designing every frame of each animation of each character.

The game I will create will, of course, not be the best 2D fighter game ever because this is a solo project being performed by a sixth form student programmer. I have many commitments alongside this project so not all my time will be spent doing this project, thus, I will not be able to make any fancy animations or menu screens and I will prioritise functionality over aesthetics.

Requirements

My game will be written using Python3 and Pygame, so based on this, I will use the minimum requirements to use Python3 on a computer as well as examining the disk space my program may take up, having a mouse, a keyboard, and a monitor.

Hardware

Hardware	Minimum Requirement	Justification
Processor	1.5GHz	Minimum processing power to use Python3
Disk Space	100MB	Minimum space needed to store my program and its assets
RAM	32MB	Minimum for running Python
Keyboard	Simple, working keyboard	Allows for inputs of character movement and attacks
Mouse	Simple, working mouse	Allows for the selection of menu icons
Monitor	Video Graphics Array screen or later versions	To output the images of the game

Software

Software	Requirement	Justification
Operating system	Windows 95 or later	Minimum requirement to use Python3
Pygame	Pygame version 2 or later	Minimum requirement to ensure good compatibility

Interview/Questionnaire

I plan to interview three sixth form students about suitable features that I will feedback into the success criteria of my project. I will ask:

- Have you played any 2D fighting games before?
- What did you enjoy most about this game?
- What did you dislike most about this game?

- Was there an CPU enemy that you could play against? If so, how difficult were they?

I have compiled their responses and will show each response separately for questions that I have permitted multiple responses for. For these questions, I have asked them to provide a reason for their answer because it will help me justify why I should factor their thoughts into my game. I have also decided to keep the game names they mention undisclosed as I would like to focus on the key features that make a game enjoyable for the user, as such, I have edited the responses so that the key information is extracted from their answers:

Have you played any 2D fighting games before?

All three students said 'Yes' to this question.

I asked them to refer to their favourite game when answering the next few questions.

What did you enjoy most about this game?

Student 1: "I would say that I enjoyed the control layout of the game because it made the game much easier to play and to learn the controls of it."

Student 2: "I enjoyed how the game had clear and simple moves to learn since it did not have many key-binds (*)."

Student 3: "I enjoyed the wide variety of attacks that I could use from different characters, and this helped the game feel less boring to play frequently because there were more moves to learn!"

What did you dislike about this game?

Student 1: "The game I played had an issue with balancing the powers of the characters, meaning that some characters or moves were overpowering others. This made the game quite frustrating because my opponent had an advantage if they used this character or move."

Student 2: "This game had high system requirements and operated poorly my last low-end machine which put me at a disadvantage in the game and led to me saving for a new computer."

Student 3: "My game frequently had choppy graphics, and it distracted me from the actual gameplay of the game."

Was there an CPU enemy that you could play against? If so, how difficult were they?

Student 1 answered 'No' to this question, however Students 2 and 3 said yes and provided answers:

Student 2: "The CPU enemies I've faced in my game have been very difficult to beat because it would counter everything you did and could predict your moves."

Student 3: "The CPU enemies were very basic, and they spammed the same moves which made them predictable and wasn't fun to play after a while."

Interview Conclusion

From the interview and my research, I gathered a basic idea of what the game I will create will look like which meets the user requirements from the interview:

- There will be a main menu screen.
- Users will be able to select what character they want.

- There will be two players for which characters can be selected.
- Characters can be selected to be controlled by the computer.
- Users will control their character's movements and fight actions when battling.
- Characters will have animated movements and actions.
- One user will win when the other user's health goes to 0.

These are general ideas for my game that I will develop and make more specific in my success criteria and further on in the latter stages of development so that user requirements are fully met.

Key features (using research)

From the information collected and interpreted from the interview and research of existing games, I have created a list of key features that my project should have because it will help me measure how successful my project will be during the evaluation stage.

Key binding

I will need to assign keys to these different animations that my game will have, such as moving forwards and backwards, jumping, and attacking. A computational approach would be necessary here because I can create an algorithm that takes in the user's keyboard input and the algorithm will output the necessary animation that is bound to the key the user pressed.

CPU

There needs to be a CPU enemy for the user to fight. The difficulty of this CPU enemy should be adjustable to fit the skill level of the player. I will use different levels (Like easy, medium and hard) to show the difficulty level of the CPU. As the CPU gets more difficult, I will increase its health, speed, and attack damage at differing increments (depending on the level) so that it becomes harder to defeat.

Music/Sound Effects

In my fighter game, there will need to be background music and sound effects for each screen because it will add to the fighting atmosphere of my game. Here is a list of links of music/sound effects that I may potentially use:

- Battle music: <https://youtu.be/635tej6SFj4?si=i6SF430QOe1AxRUH>
- AI-generated music, Prompt: Epic Main Menu Music, [Generate AI Music Free Online with AI Music Generator](#)
- Sound Effects: [Free Sword Sound Effects Download - Pixabay](#)

Animations

I will need to create and use an algorithm that can take in animation frames of a sprite as an input and output a cycle of each frame of an animation one after another. This I will also need to include how fast each frame is switched and outputted onto the screen as well to differentiate one cycle of animations (walking) from another (jumping). This process is much suited to a computational approach as I will be using algorithms and a way of outputting the results of this algorithm.

After searching for websites where I could use royalty-free sprite images and animations, I found out about CraftPix, [2D Game Assets Store & Free - CraftPix.net](#), and it offers sprites free sprites that can be used for non-commercial projects. After reading the terms and conditions of using the sprites offered and concluding that this is a project for school, I decided that this website is suitable for this project as the sprites are allowed to be used non-commercially.

For the abilities, I will be using sprites from [Top free game assets tagged 2D and Animation - itch.io](#).

After scouring CraftPix, I found two fitting pixelated sprites that would suit my game. One is described to be a “Shinobi” whilst the other is a “Samurai”. They are shown below:



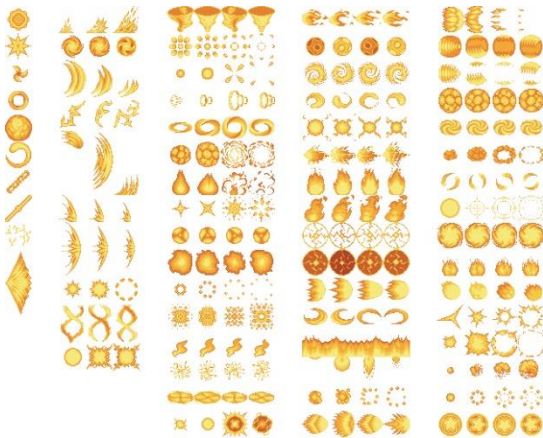
(The Samurai)



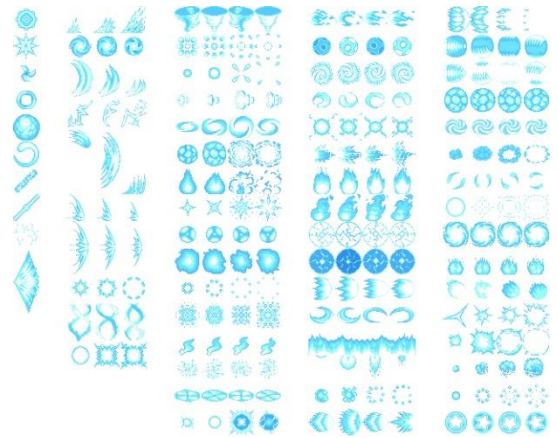
(The Shinobi)

The Shinobi uses its fists to attack the player whereas the Samurai wields a sword. Each animation provided came separately when downloading this file from the publisher.

From itch.io, I used [Free Effect Bullet Impact Explosion 32x32 by BDragon1727 \(itch.io\)](https://itch.io/item/1111111) for the abilities. I chose this because they were pixelated, which matched with the character art design that I was using, and this animation effect bundle had abilities suitable for a sword-wielding character and a fist-fighting one too. The bundle also has different colours for the effects which will help users differentiate what character used the ability.



(red fire effects)



(blue fire effects)

To be able to animate these sprites in my project, I will need to convert the frames of the different animations (run, idle, jump, etc..) into sprite sheets that I can then cut out and iterate through using the pygame library. In the design stage, I will create an algorithm for this purpose.

Success Criteria

Must have

Number	Requirement	Justification
1	Background Image	Colour makes game attractive to younger audience
2	Start Menu	Acts as the introduction to the game
3	Game title in Main Menu	Displays the name of the game to the user
4	Quit button in Main Menu	Allows user to quit the program
5	Play button in Main Menu	Allows user to move onto next screen in the game
6	Settings button in Main Menu	Allows user to move onto settings screen to configure settings
7	Character Selection Screen	Allow the user to see which characters they want to use
8	Character 1 Selection Button	User can pick character 1
9	Character 2 Selection Button	User can pick character 2
10	CPU Selection Button	Allow user to choose if they want a player to be CPU controlled
11	Start Battle Button	Allow user to move onto the next screen (Battle screen)
12	Battle screen	User can see their selected character and opponent that they are going to fight
13	Timer/Timeout Screen	To stop endless battles from happening and to make battles fast paced. Timeout screen shows battle was a draw.
14	Health Bar for Character 1	To show player 1 how much health they have left
15	Health Bar for Character 2	To show player 2 how much health they have left
16	Character 1 Image on Battle screen	To display the user's character
17	Character 2 Image on Battle screen	To display the user's character
18	Character 1 Idle Animation	To show when the character is idle
19	Character 1 Run Animation	To show when the character is moving forward or backwards

20	Character 1 Jump Animation	To show when the character is jumping
21	Character 1, Attack 1 Animation	To show that the character is using its first attack option
22	Character 1, Attack 2 Animation	To show that the character is using its second attack option
23	Character 2 Idle Animation	To show when the character is idle
24	Character 2 Run Animation	To show when the character is moving forward or backwards
25	Character 2 Jump Animation	To show when the character is jumping
26	Character 2, Attack 1 Animation	To show that the character is using its first attack option
27	Character 2, Attack 2 Animation	To show that the character is using its second attack option
28	Knock-out death	To show that the character has been defeated/knocked out of arena
29	Win Screen	To display which character has won
30	Restart Button on Win screen	To allow another battle to happen
31	Main Menu Button on Win screen	To allow user(s) to return to the main menu

Should have

Number	Requirement	Justification
1	Character 1, Ability 1 Animation	To show when the character uses its first power-up
2	Character 1, Ability 2 Animation	To show when the character uses its second power-up
3	Character 2, Ability 1 Animation	To show when the character uses its first power-up
4	Character 2, Ability 2 Animation	To show when the character uses its second power-up
5	Start menu music	To set the initial mood of the game
6	Mute button in options	To allows the user to remove all music if they prefer
7	Battle screen music	To set the battle atmosphere
8	Punch sound effect	To signify if the character's punch has landed
9	Jump sound effect	To signify that the character has jumped
10	Damage sound effect	To signify that the character has taken damage
11	Ability Bar	To show when the character is able to use its ability

12	Adjustable CPU difficulty	To allow the user to face a more difficult CPU enemy
13	Highlight buttons when mouse hovers over them	Makes it clear to the user what button they are about to press and signifies if they can press the button
14	Pause Button on Battle screen	To allow user(s) to pause the game
15	Pause Screen	To show the options available to the user when the battle screen is paused
16	Resume Button on Pause Screen	To allow the user to continue the paused battle
17	Main Menu Button on Pause Screen	To allow the user to move back to the Main Menu when the battle is paused

Could have

Number	Requirement	Justification
1	High Score Table	To show the quickest time that a character has been defeated
2	Hit Counter	To show how many hits have landed on a character
3	Adjustable timer in settings	To allow for shorter/longer battles
4	Blood when hit	To show that a character has been hurt
5	Background Selection Screen	Changes the atmosphere of the Battle screen
6	Obstacles in Battle screen	Could act as a way to hide from user attacks and/or abilities

Won't have

Number	Requirement	Justification
1	Online Play	It would be too time consuming for the duration that I am given for this project
2	3D models	Difficult to work with on Python3 and time-consuming
3	Story/Plot	Not a key feature of the game
4	Realistic Physics	Time-consuming as I would need to learn how to do this
5	Detailed Art	I am borrowing sprites so that art style is dependent on that
6	Microtransactions	This project is non-commercial
7	Many characters	This game will primarily focus on two different characters

Design

Design Objectives

To design this project, I will refer to the success criteria that has been outputted by the analysis stage. Using these criteria, I will create solutions using flowcharts to address these criteria, justifying each solution as I make them. As I am using the pygame library, I will incorporate their functions into the designs.

Design Interview

I will ask a group of 5 students a series of questions in relation to how the game should be designed. I came to them with my prior ideas for the game and asked them to vote on what they would prefer the most. The number in the bracket is the number of votes for the design.

Main Menu:

Is the background screen selected by me a good choice to use?

- Yes (4)
- No (1)

Should the Title be present on the main menu screen?

- Yes (5)
- No (0)

What colour should the Title be?

- Blue (0)
- Black (1)
- Red (1)
- Yellow (0)
- White (3)

What colour should the menu buttons be?

- Blue (1)
- Black (2)
- Red (1)
- Yellow (1)
- White (1)

What colour should the highlighted buttons be?

- Blue (3)
- Black (0)
- Red (1)
- Yellow (1)
- White (0)

How big should the menu buttons be?

- Small, 80 x 30 (1)
- Medium, 150 x 50 (2)
- Large, 250 x 100 (2)

Since this result was a draw, I will aim to make the menu between both values.

How big should the Title be?

- Small, 300 x 150 (1)
- Medium, 450 x 200 (3)
- Large, 600 x 250 (1)

Character Selection:

How big should the Character buttons be?

- Small, 50 x 80 (1)
- Medium, 125 x 175 (4)
- Large, 250 x 300 (0)

Where should the start button be?

- Top of the screen (0)
- Middle of the screen (2)
- Bottom of the screen (3)

Should the character names be inside the button or their image?

- Image (5)
- Name (0)

What colour should the character buttons be?

- Blue (0)
- Black (4)
- Red (0)
- Yellow (0)
- White (1)

What colour should the CPU button be?

- Blue (0)
- Black (4)
- Red (0)
- Yellow (0)
- White (1)

Where should the CPU button be?

- Above Character Buttons (0)
- Below Character Buttons (5)

Battle Screen:

How big should the characters be?

- 50 x 100 (0)
- 80 x 150 (4)

- 110 x 200 (1)

How fast should the characters be?

- Walking pace (2)
- Quite fast (3)
- Very fast (0)

How high should the characters be able to jump?

- To the middle of the screen (1)
- To the top of the screen (4)

How big should the Shinobi hitbox be?

- Same size as character
- Long but thin
- Short but thick

How big should the Samurai hitbox be?

- Same size as character
- Long but thin
- Short but thick

How much damage should attacks deal? (out of 100)

- 5 (2)

- 10 (2)
- 15 (1)

Since this result was a tie, I will aim to make the damage in between these values.

How big should the health bar be?

- 150 x 10 (1)
- 300 x 20 (1)
- 400 x 30 (2)

What colour should the health bar be?

- Blue (0)
- Black (0)
- Green (5)
- Yellow (0)
- White (0)

How long should battles last?

- 30 secs (1)
- 45 secs (3)
- 1 min (1)

Design Interview Results

From the answers received, I am able to map out a more detailed design of my game and how it will look. These are the requirements I have gathered:

General:

- Screen will be 1000 x 600

Main Menu:

- My chosen background will be visible
- The title will be visible
- The title will be white and medium-sized
- The menu buttons will be black (play, options and quit)
- The menu buttons will be highlighted blue when the mouse hovers over it
- The menu buttons will be medium-large

Character selection screen:

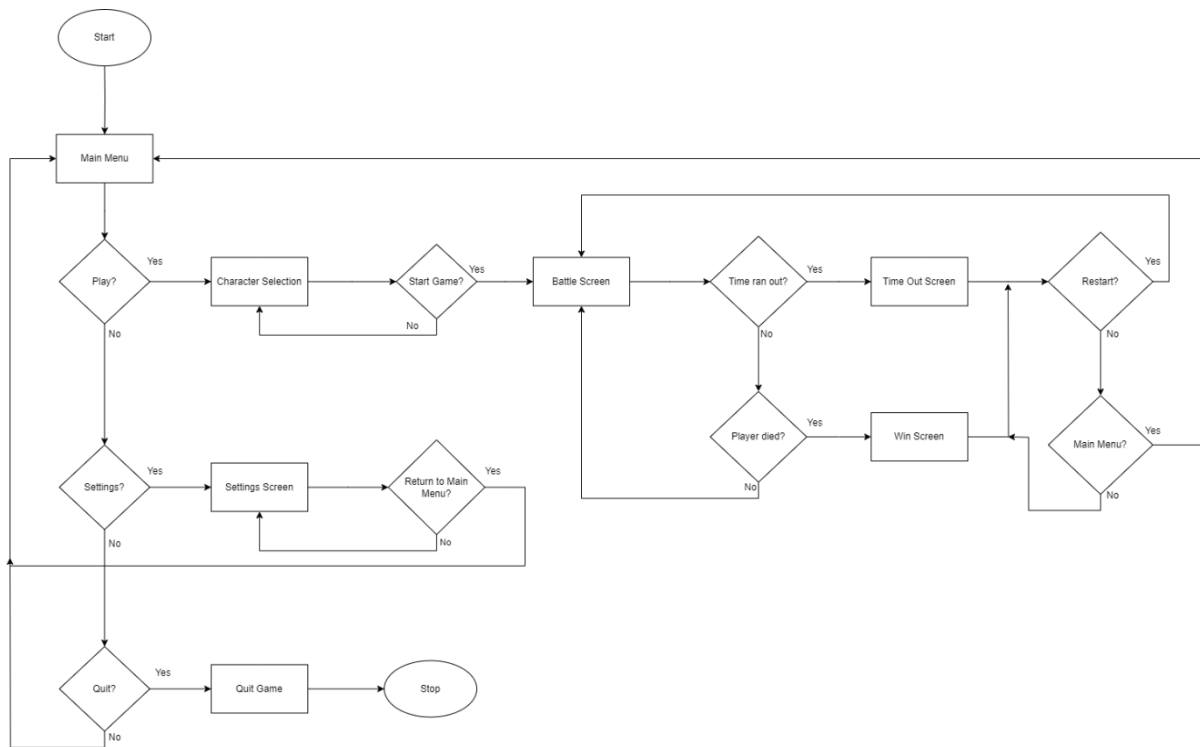
- Four Character buttons (two for each player) will be visible
- The character buttons will have the character image inside of them
- The character buttons will be black buttons
- The character buttons will be medium-sized
- The start button will be at the bottom of the screen
- The CPU buttons will be below the sets of two character buttons for each player
- The CPU buttons will be black

Battle screen:

- Two characters will be visible
- They will be medium-sized
- They will be able to move quite fast
- They will be able to jump high to the top of the screen
- The health bar for each player will be at the top of the screen
- The health bar will be green
- The health bar will be 400x30
- The timer will be at the top of the screen
- Battles will last 45 seconds

System Flowchart

Here, I designed the overall flow of the game I will be making using a flowchart. This includes all the major user interfaces that I will be designing and using throughout my game. To simplify this flowchart, I have focused only on the major stages if the system it will not include any minor parts which would complicate this diagram.

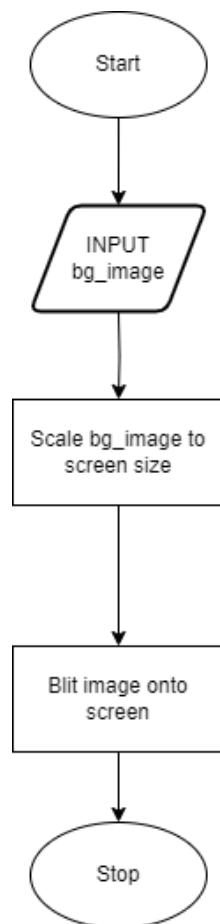


Success Criteria 1: Background Image

Justification

When the user initially opens the game, an image needs to be set for the user which will act as the background that the user will see when selecting their character and fighting. This image will need to be drawn on the screen throughout the time the user is playing my game. The image I will be using will not be the same size as the screen which is why it will need to be scaled as part of this process of drawing the background on the screen. The flowchart below is the design of the subroutine that I will be using.

Flowchart



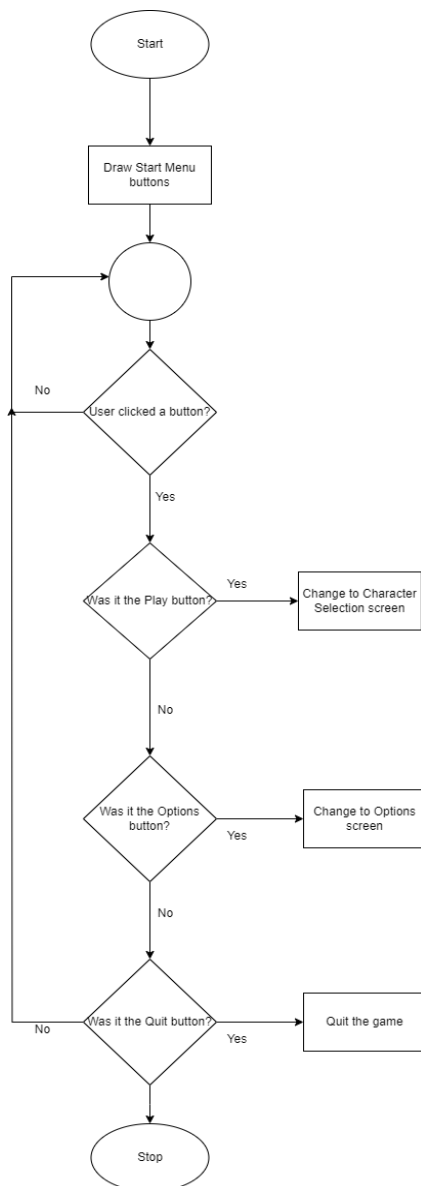
In practice, this subroutine will be called 60 times per second in the main game loop, so there would be no "stop". But to clearly show the functionality of this subroutine, it is included.

Success Criteria 2: Start Menu

Justification

When the user first starts the game, they must first be presented with a start menu that allows them to navigate through the game. They should be able to choose to either play the game, go to the options screen, or quit the game. To do this, the start menu buttons must be drawn onto the screen and the algorithm should wait for the user to click a button to refer them to the correct screen when they have clicked the corresponding button. The design of the main menu will be included in the next section; however, the logic of the screen is displayed below.

Flowchart

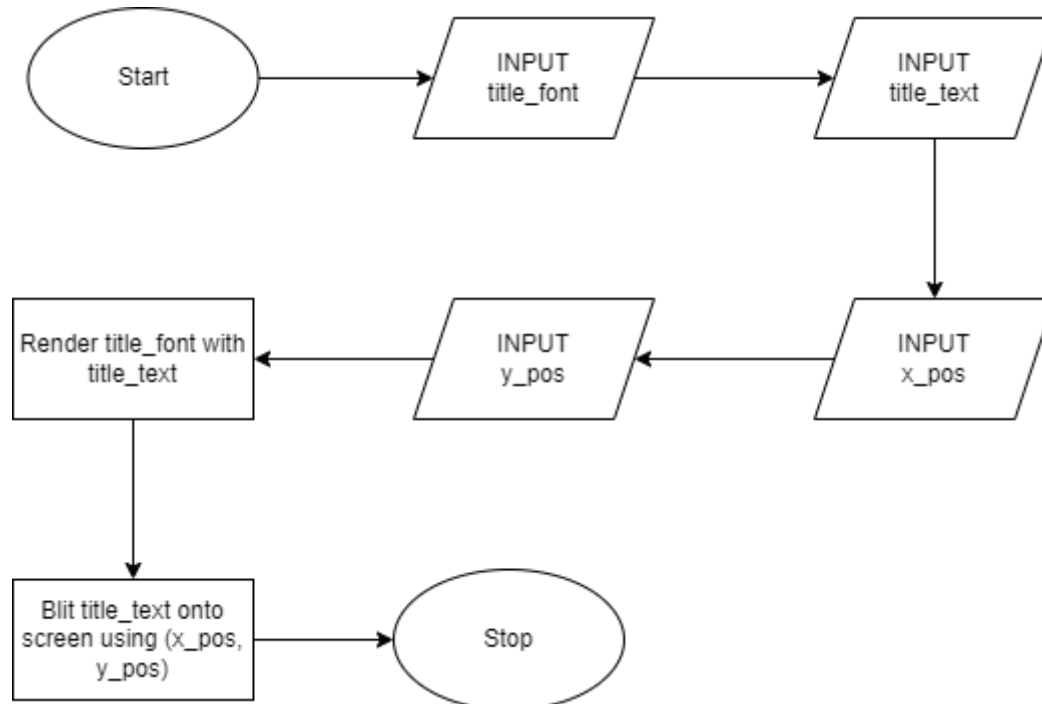


Success Criteria 3: Game title

Justification

The user should be able to see the title of my game as an introduction so that they know what the game is broadly about and to allow them to reference the game correctly. The game title will need to be drawn on to the top-centre of the screen in a large size to capture the attention of the user. To do this, I will need to take in inputs of x and y positions to then blit the text onto the screen. To add text to my project using pygame, I will first need to add the font to the text then it will need to be rendered.

Flowchart

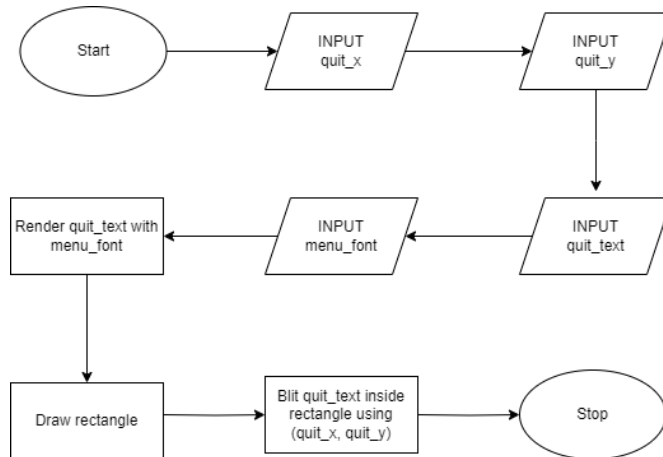


Success Criteria 4: Quit button on Main Menu

Justification

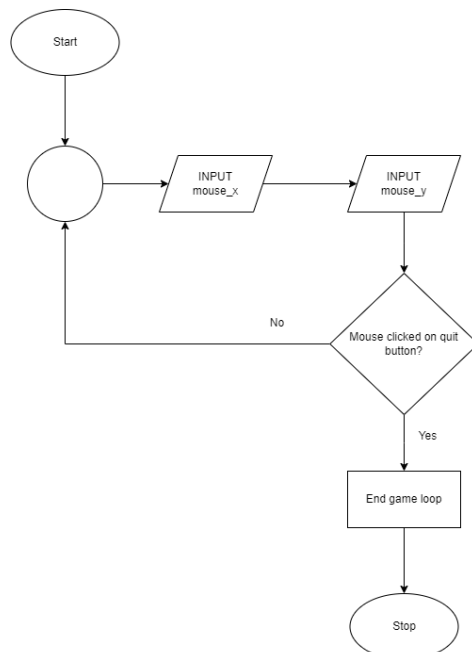
If the user finishes their session on my game, they should be able to quit the game at any point whilst on the main menu. The algorithm to draw the button is similar to the previous success criteria, but here there will need to be a rectangle around the text to show that it is a button that needs to be pressed.

Flowchart 1



In terms of functionality, the button will need to be able to quit the game. To do this, the button will need to terminate the game loop when clicked.

Flowchart 2



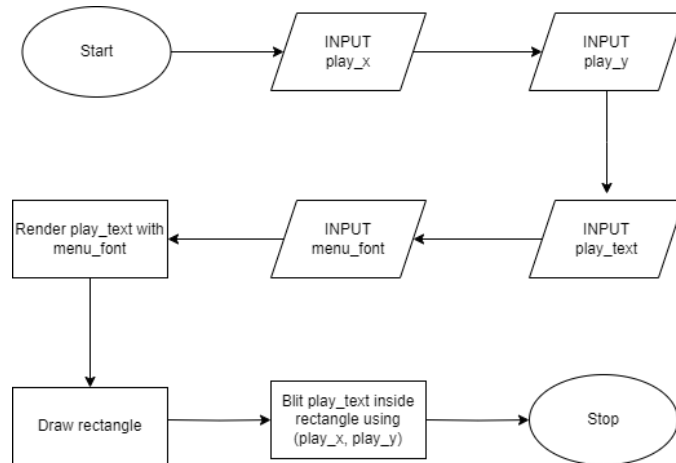
Success Criteria 5: Play button on Main Menu

Justification

When the user wants to advance to the character screen, this button should be pressed to switch the screen from the main menu to the character selection screen.

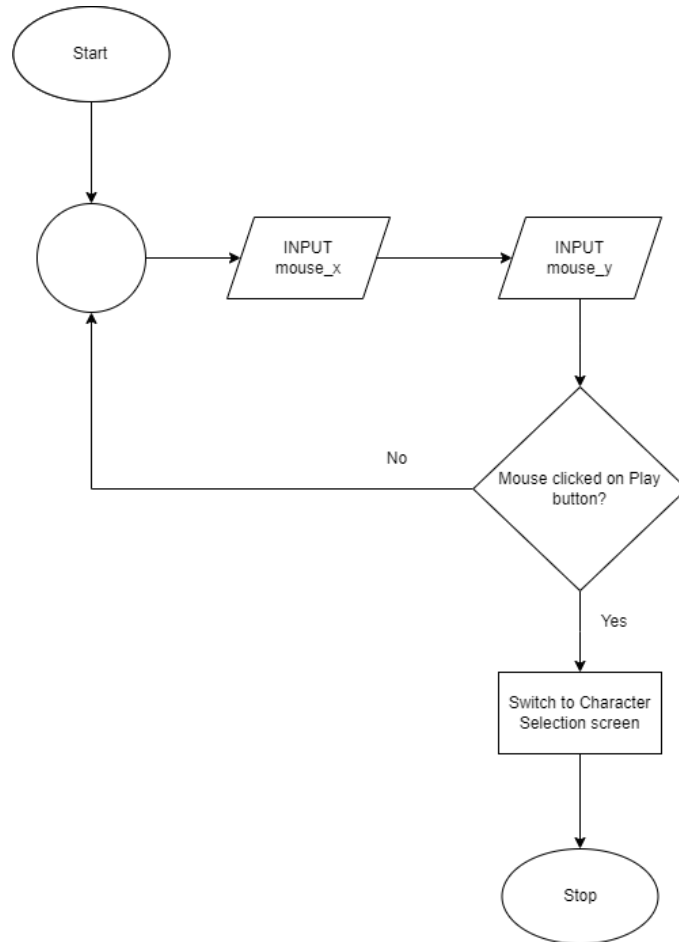
The drawing of this button is similar to the success criteria above, but it has different text.

Flowchart 1



The logic of the play button is similar to the previous success criteria, but the function is to switch to the character selection screen instead of quitting the game.

Flowchart 2



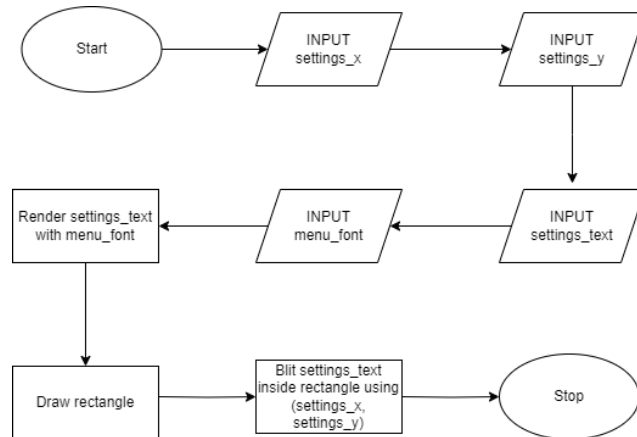
Success Criteria 6: Settings button on Main Menu

Justification

The user may want to change some settings, such as muting the music or changing their preferred button for attacking. A new screen will be used to display these options and the settings button on the main menu will need to signify this change of screen.

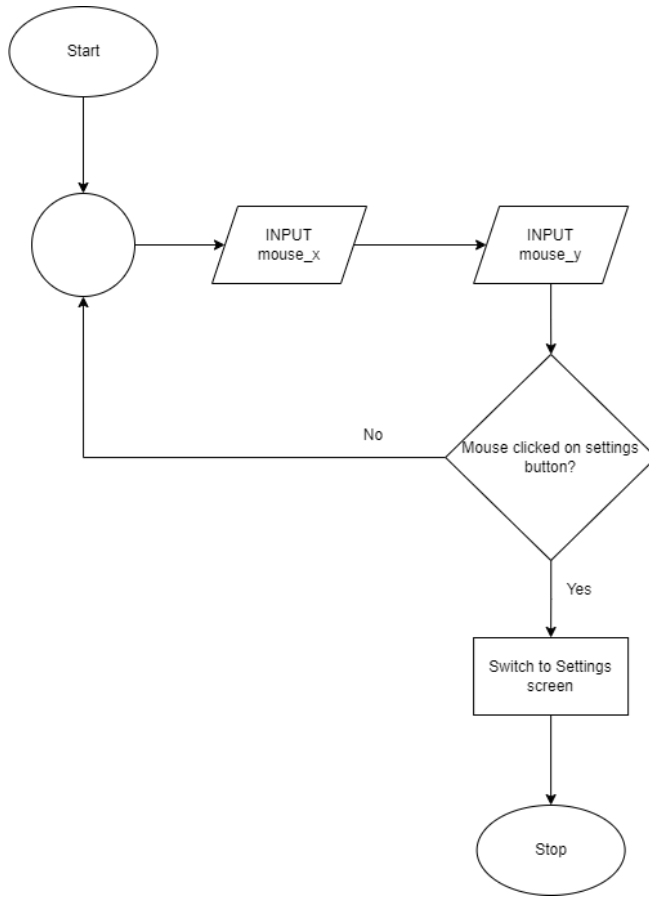
The drawing, again, is similar to the success criteria above.

Flowchart 1



The logic and functionality of this button is also like the success criteria above, but the button signifies a switch to the settings screen.

Flowchart 2

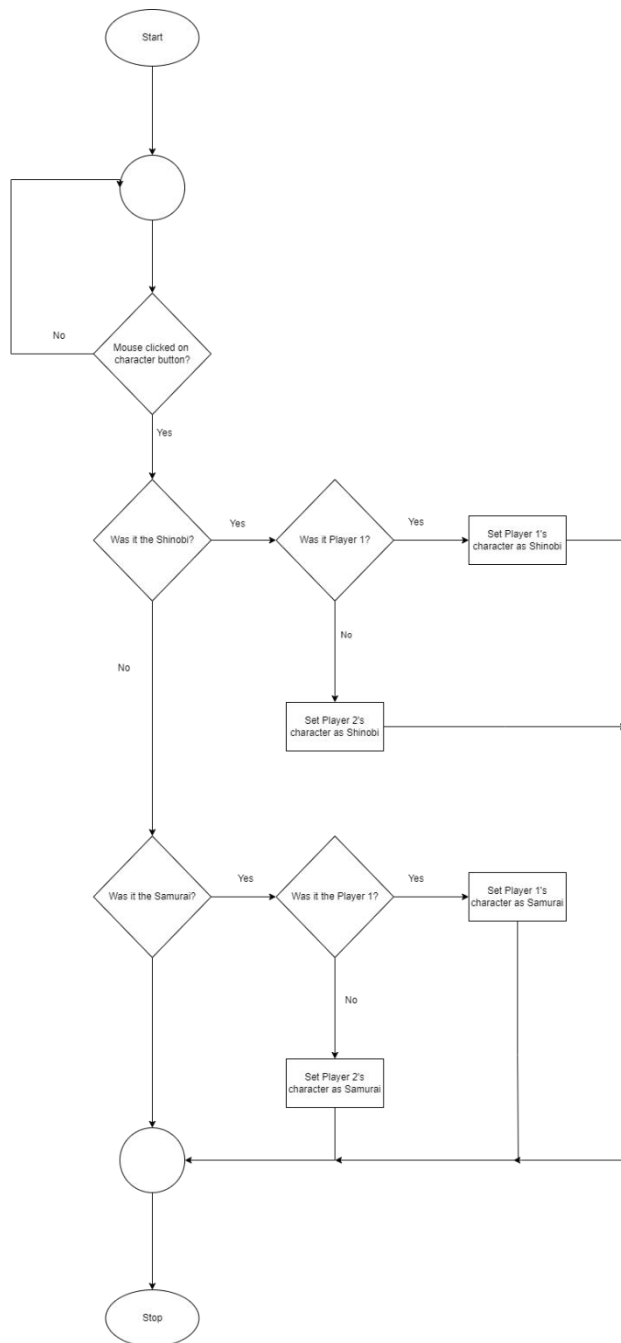


Success Criteria 7: Character Selection screen

Justification

The user will have a selection of two characters to choose from to battle with. These characters will be displayed on the Character Selection screen and the user will be able to click them to signify their character choice. Once both characters have been selected for each player, the user will be able to move onto the battle screen via the Start Game button. The flowchart below will show the logic of how character selection will work.

Flowchart

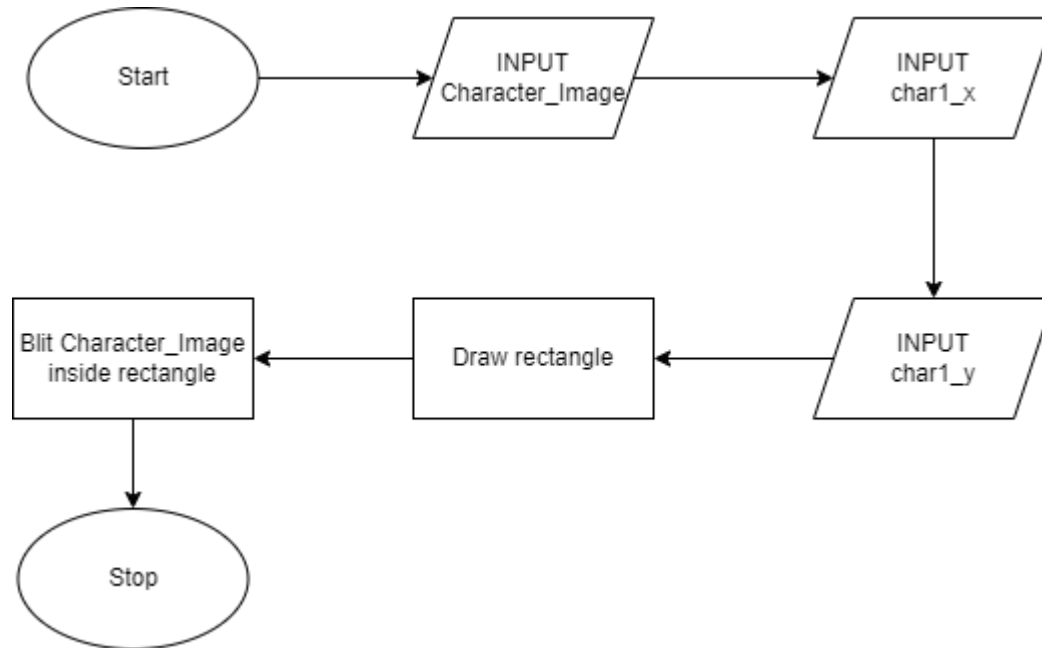


Success Criteria 8: Character 1 Selection Button

Justification

If the user wants to select the Shinobi (Character 1), a button will be available to press to set the Shinobi to one of the players. As there are two players, there will need to be two of the same buttons on either side of the screen to differentiate selecting the Shinobi for player 1 and for player 2. This can be done by reusing this algorithm but with a different x value (char1_x).

Flowchart

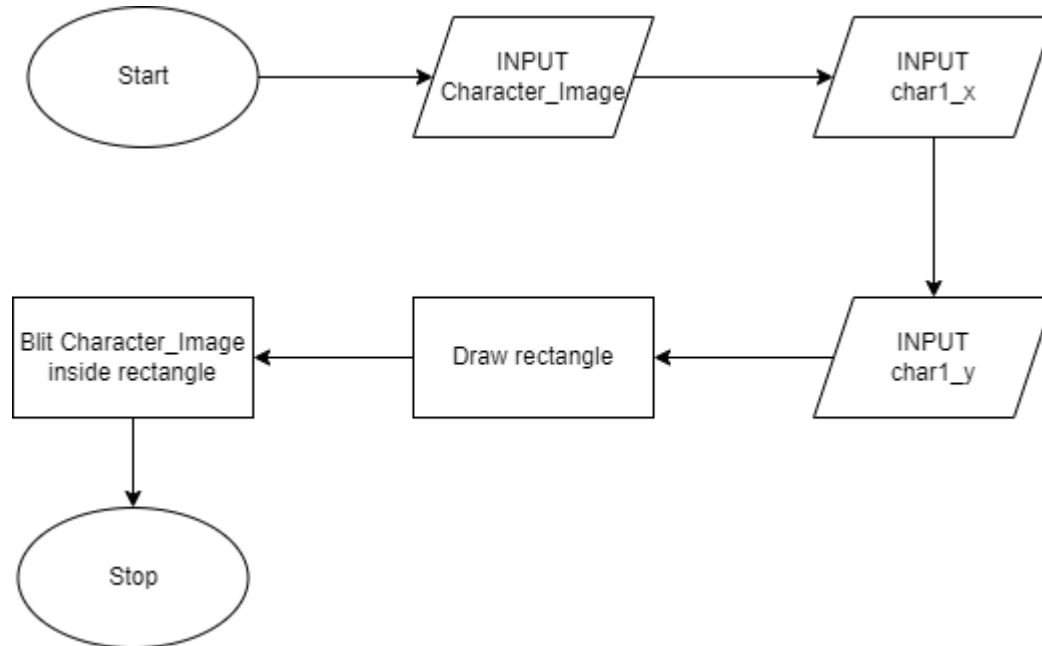


Success Criteria 9: Character 2 Selection Button

Justification

If the user wants to select the Samurai (Character 2), a button will be available to press to set it to one of the players. As there are two players, there will need to be two of the same buttons on either side of the screen to differentiate selecting the Samurai for player 1 and for player 2.

Flowchart

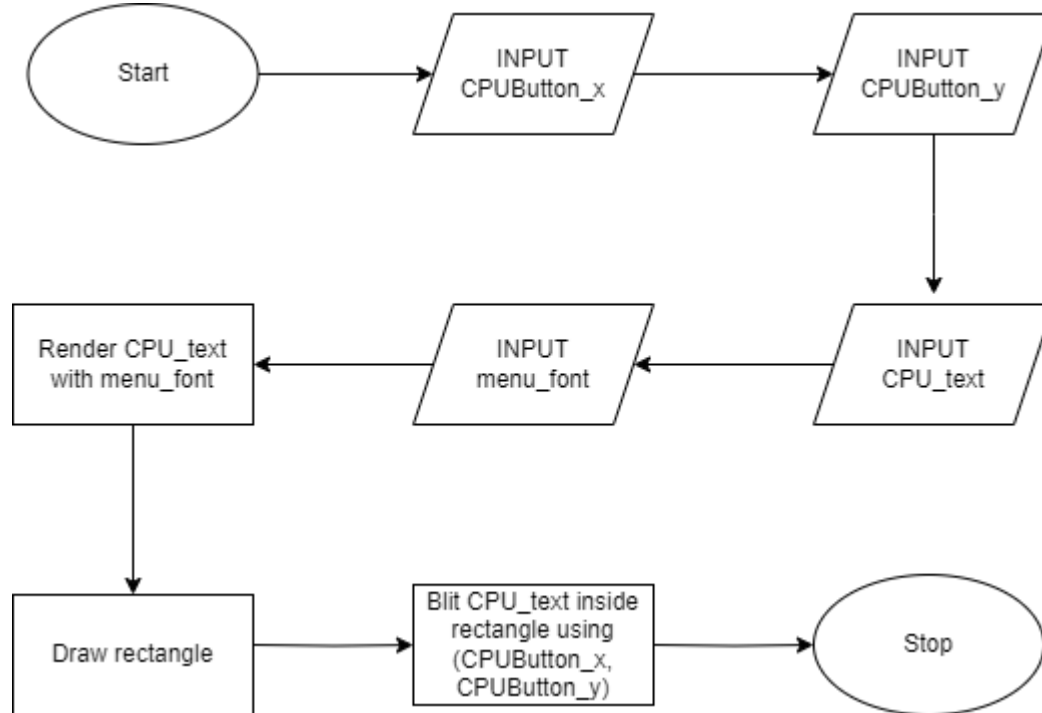


Success Criteria 10: CPU Selection Button

Justification

If the user chooses to battle against a CPU enemy, there will be a button to allow this selection in the Character Selection screen. This option will be able to be selected for one or both players, depending on user preference. and

Flowchart

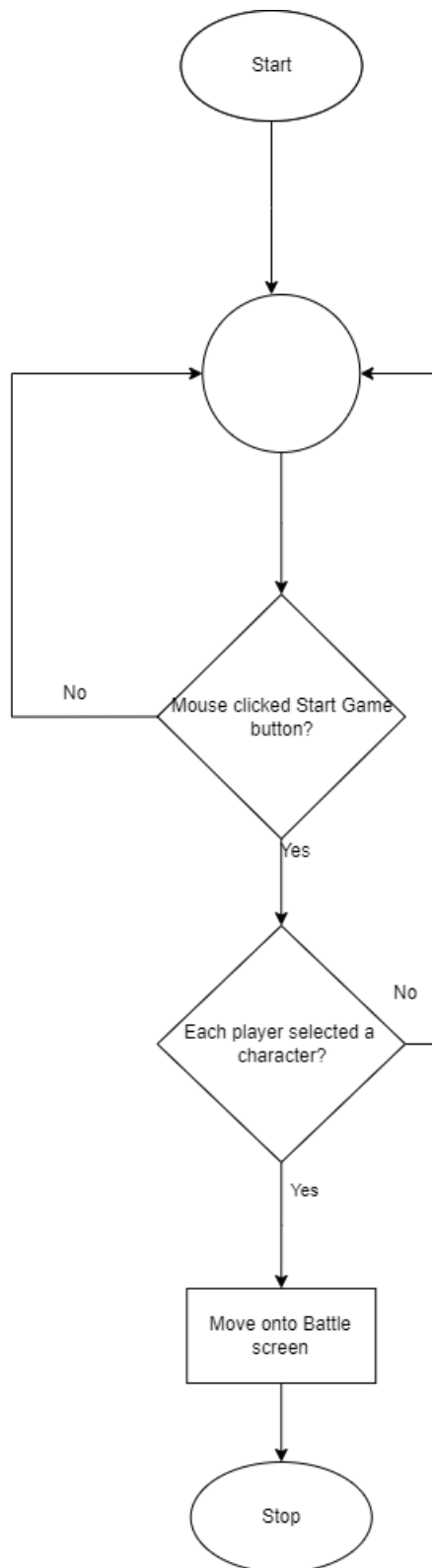


Success Criteria 11: Start Battle Button

Justification

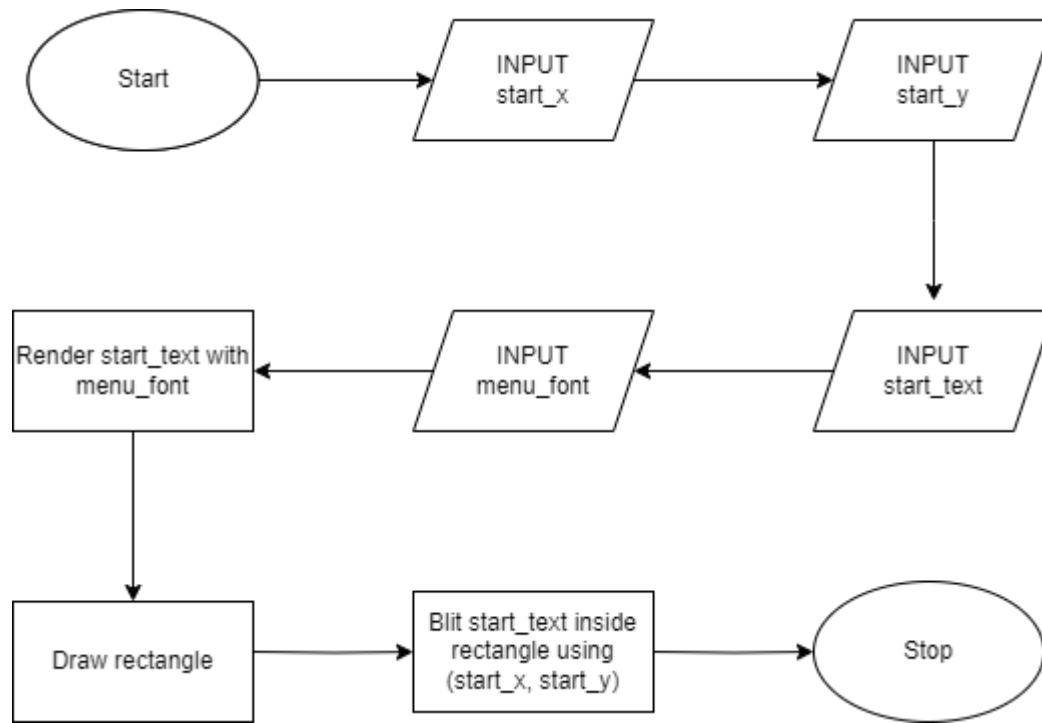
When the user wants to start the battle, there will be a button at the bottom of the Character Selection screen that will enable the user to move onto the Battle screen once both players have a character selected.

Flowchart 1



The Start Game button will be drawn like the buttons drawn on the main menu.

Flowchart 2

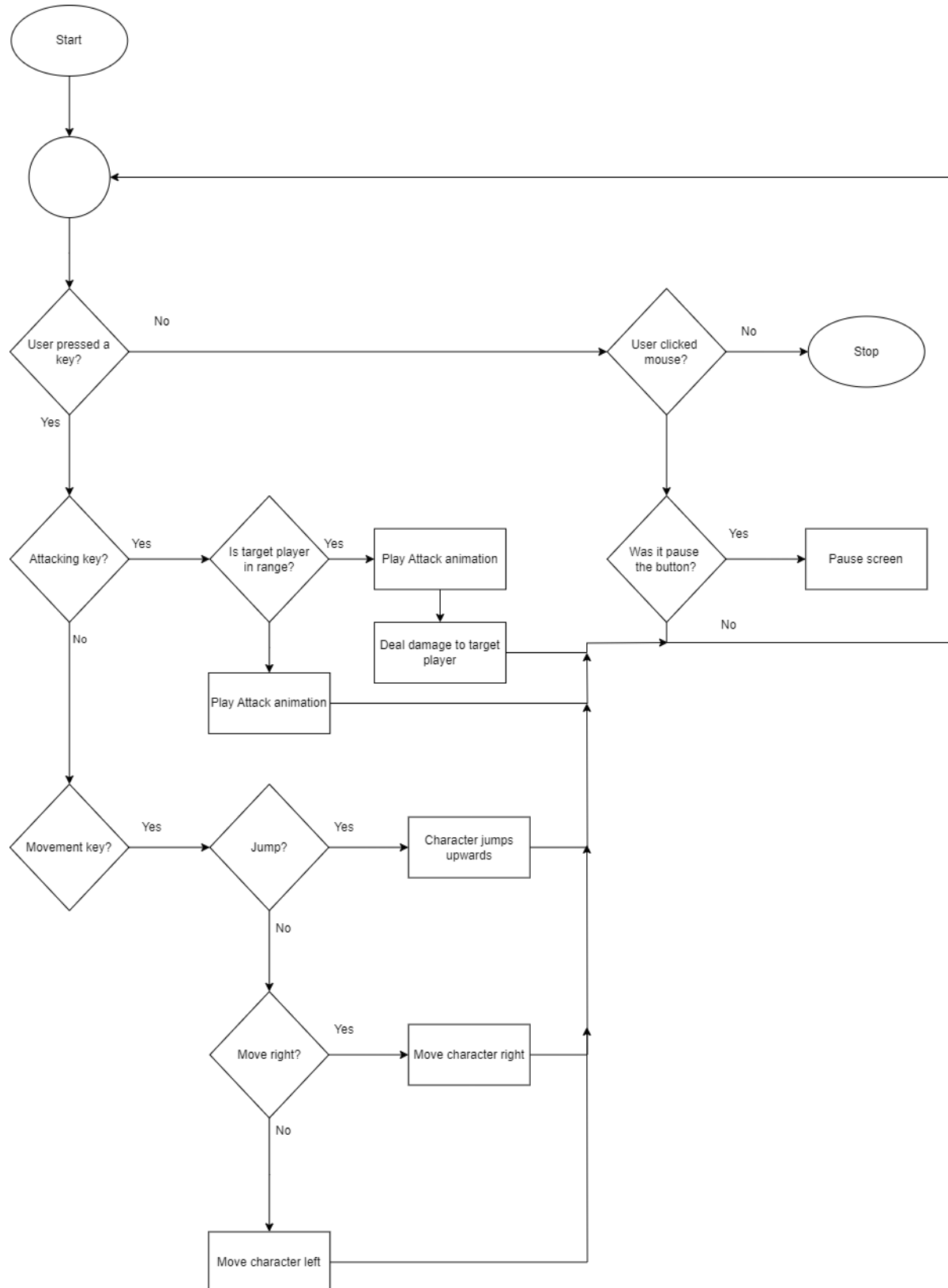


Success Criteria 12: Battle Screen

Justification

When the characters have been selected by the user and the Start Battle button has been pressed, the user will move onto the Battle Screen. In this screen, the user will use their character to battle the selected opponent, either another user or CPU. The logic of this screen is shown below.

Flowchart 1

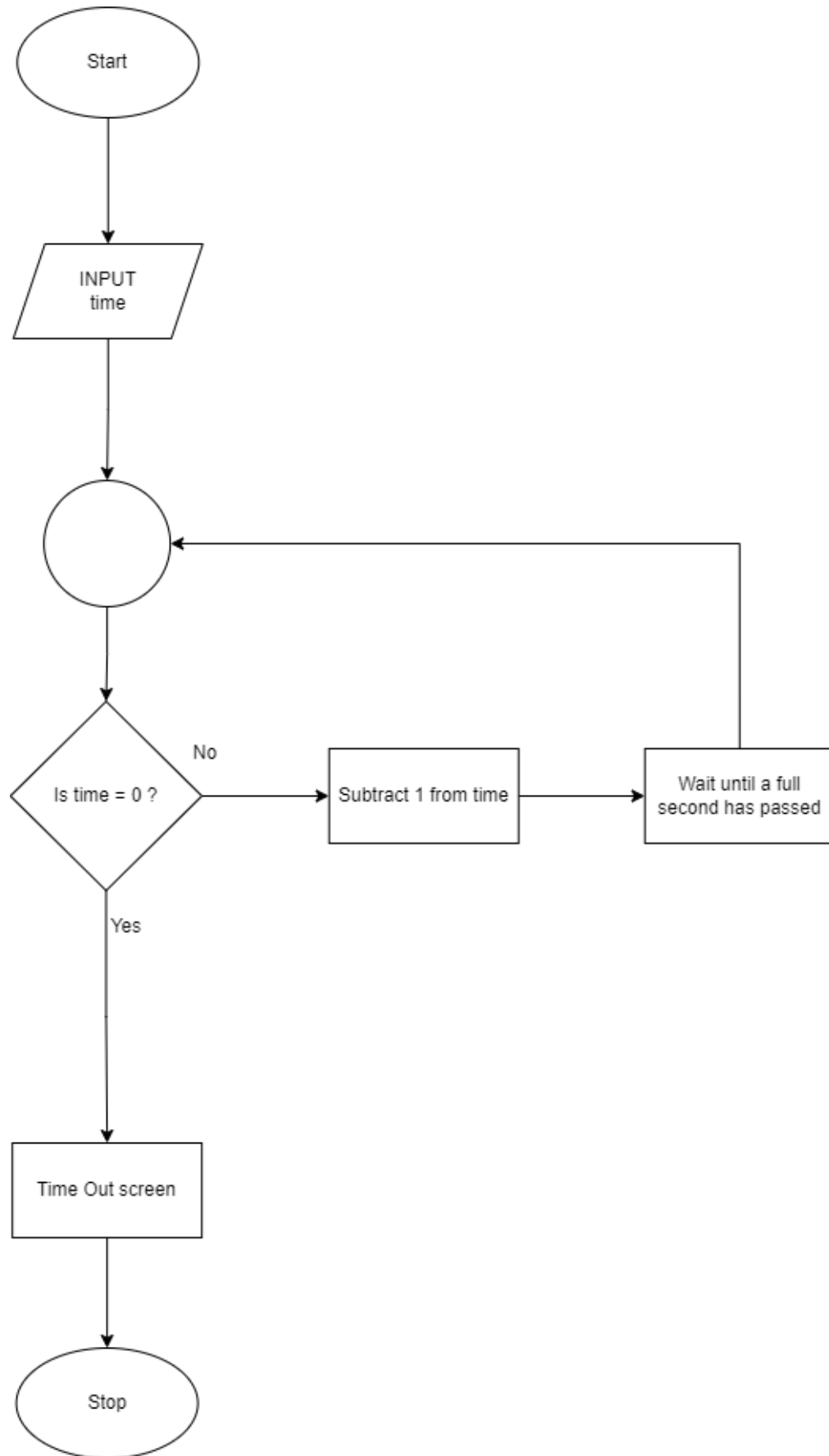


Success Criteria 13: Timer/Timeout Screen

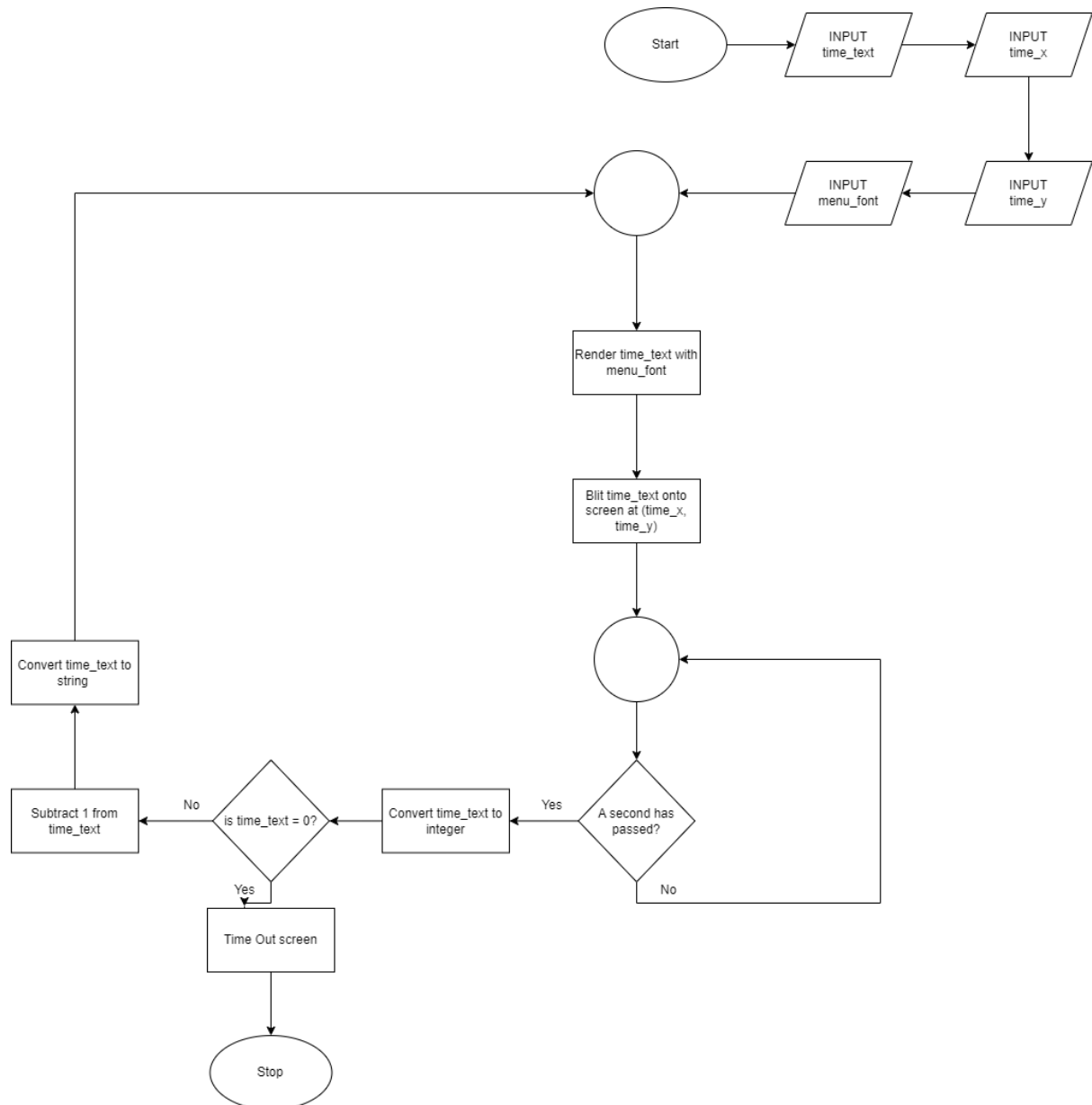
Justification

Whilst the user is battling, a timer will be counting to the default time constraint of 45 seconds. However, this time can be adjusted in the options screen. The timer encourages the user to attack their opponent and defeat them as fast as possible before they run out of time. If the user runs out of time, they will move onto the Timeout Screen which will show that the battle result was a draw because a character wasn't defeated in time. Both the logic and drawing of the timer and the Timeout Screen are shown below.

Flowchart 1



Flowchart 2

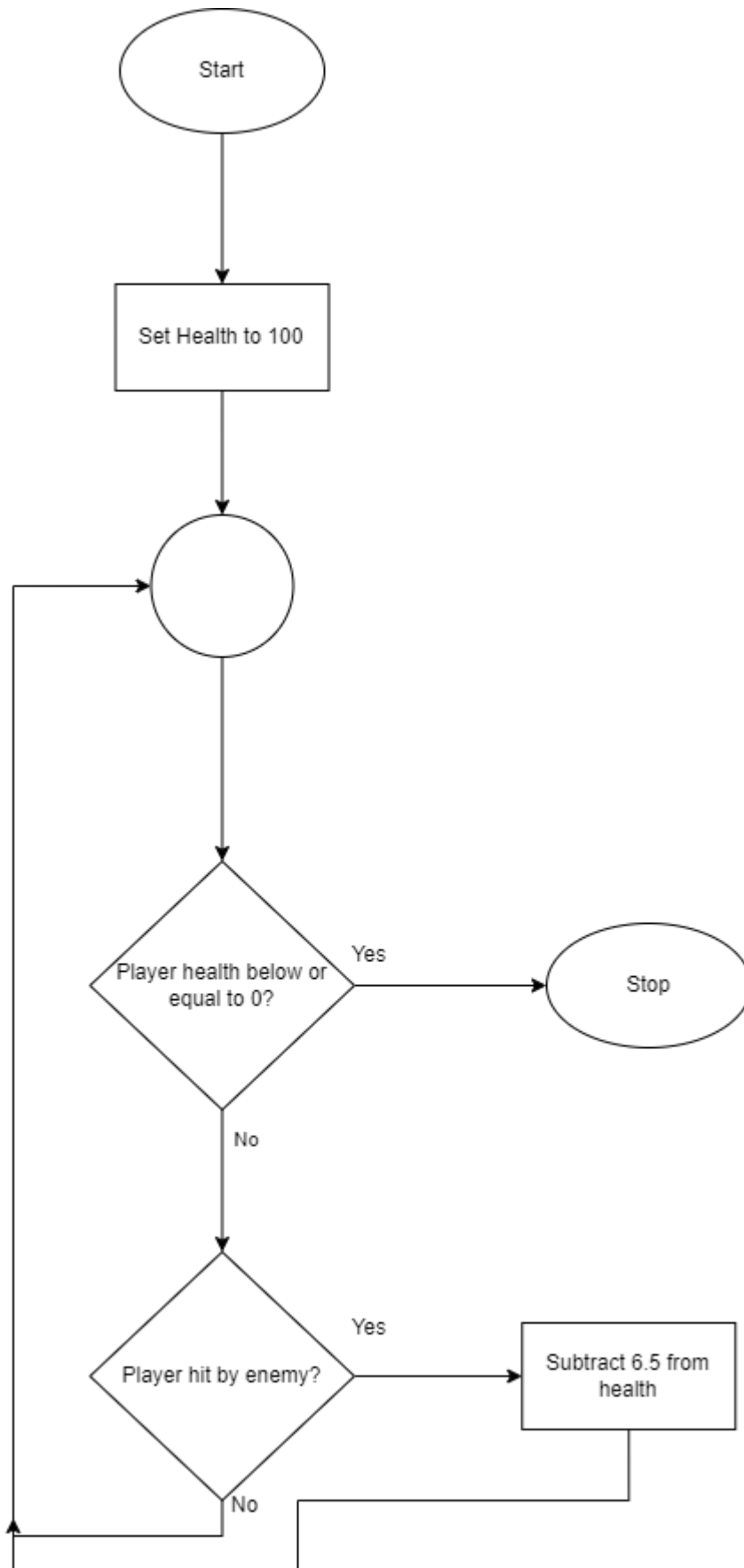


Success Criteria 14 & 15: Health Bars for Characters

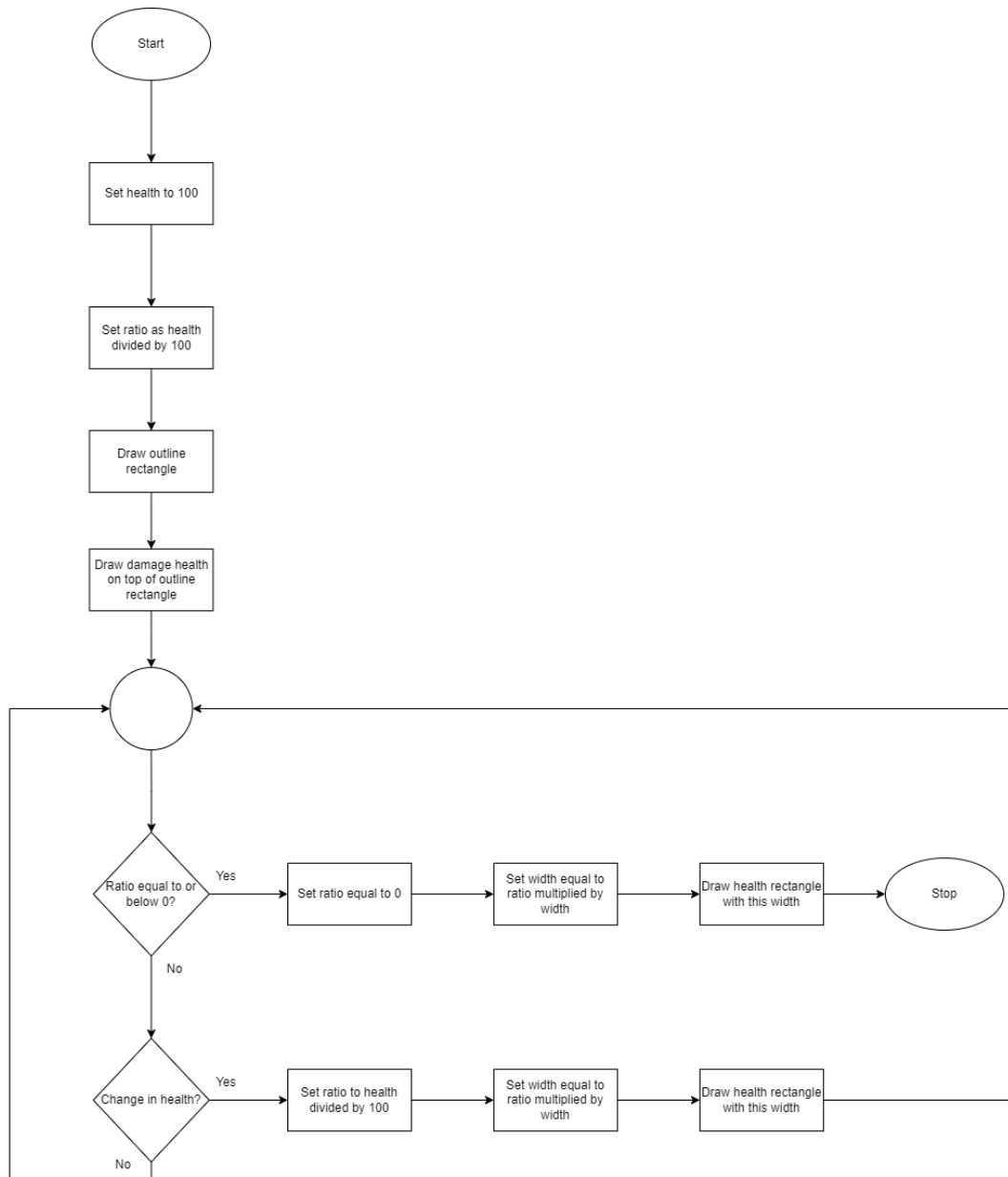
Justification

When users are battling each other, they will need to keep track of their health and the opponent's health. I will make this possible by having two health bars drawn on the screen and the user will be able to see their health (in green) and the damage they have taken (in red). Both characters start with 100 health and the damage for each hit will subtract 6.5 from the health of the target. If the health bar is all red, then that means the character has been defeated. The logic and drawing of the Health Bar are displayed below.

Flowchart 1



Flowchart 2

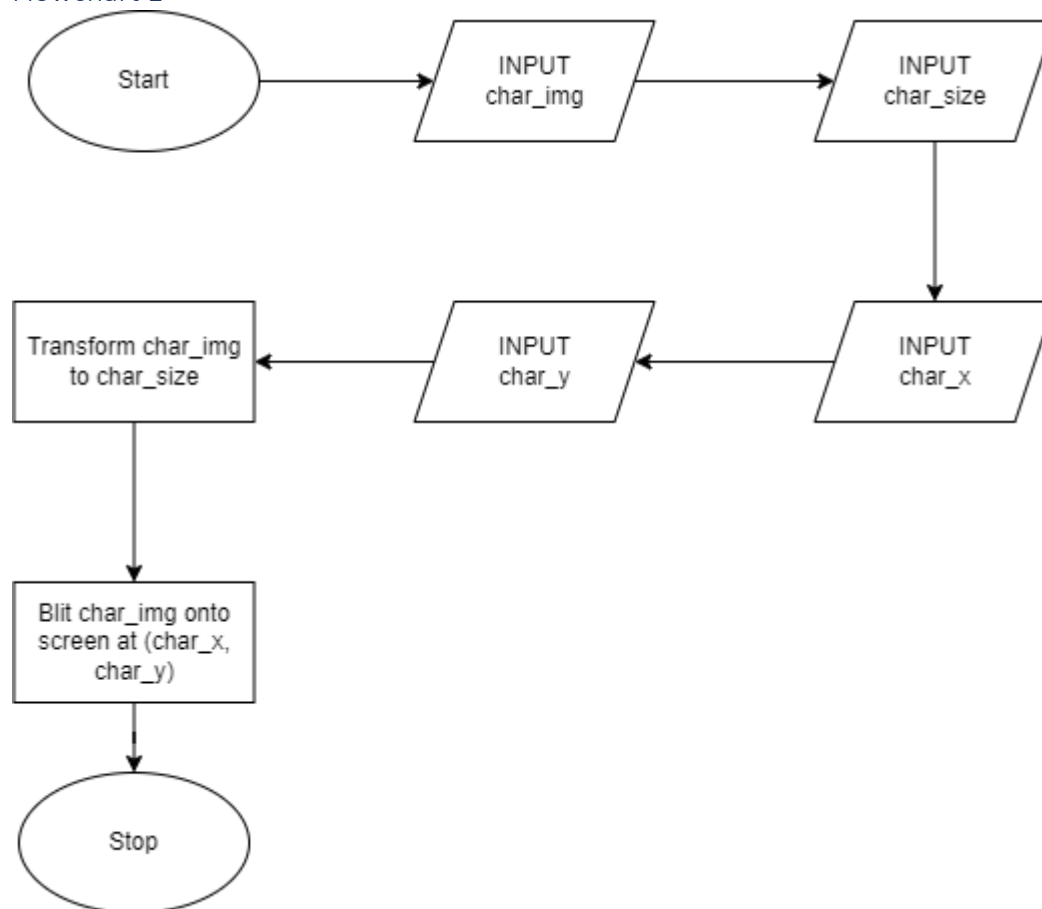


Success Criteria 16 & 17: Character Image on Battle Screen

Justification

When the user enters the Battle Screen, they must be able to see the character that they are fighting with and against. Thus, the image of the characters must be displayed on the Battle Screen at their respective x and y positions, and in a size that can be easily seen by the user.

Flowchart 1

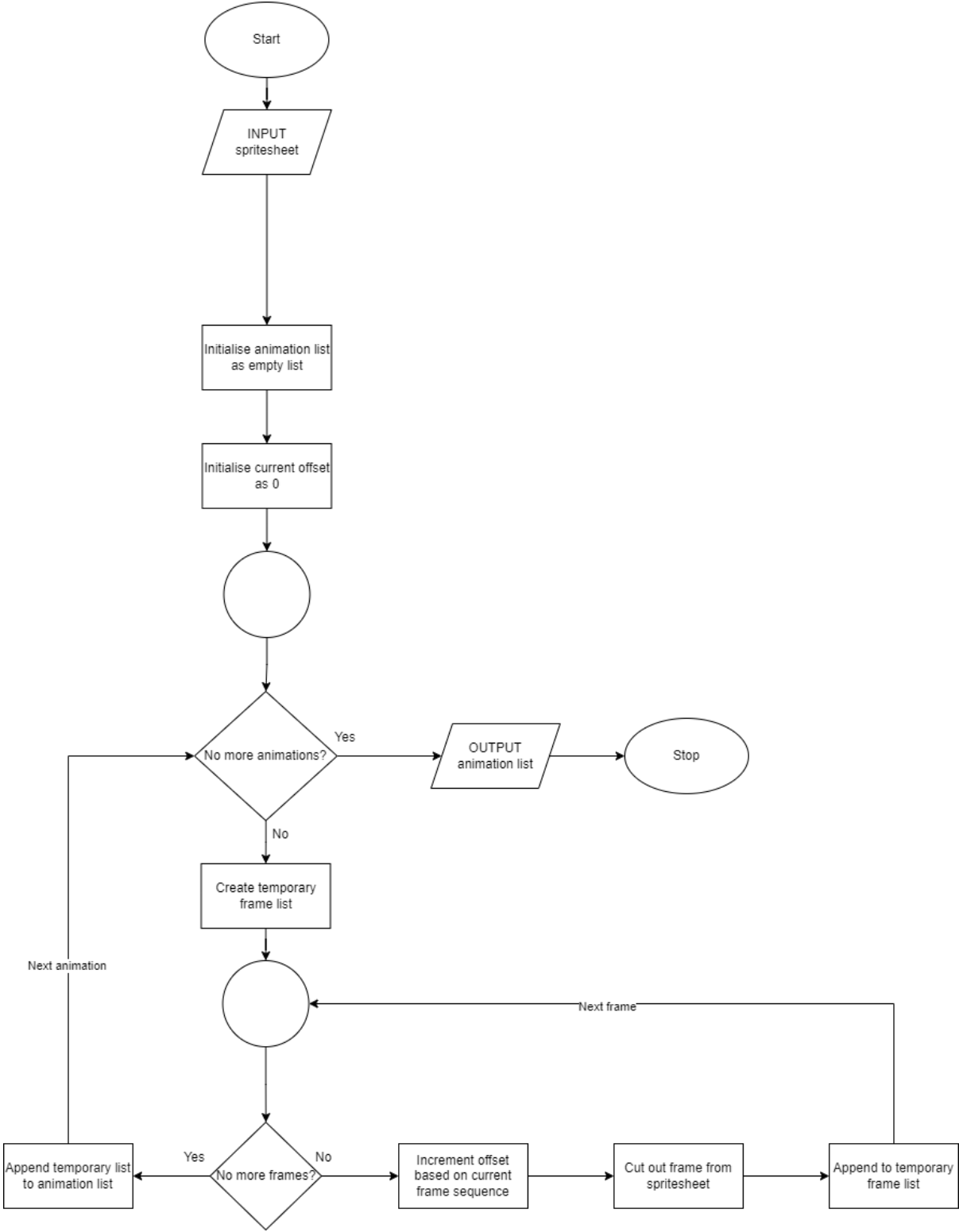


Success Criteria 18 & 23: Character Idle Animation

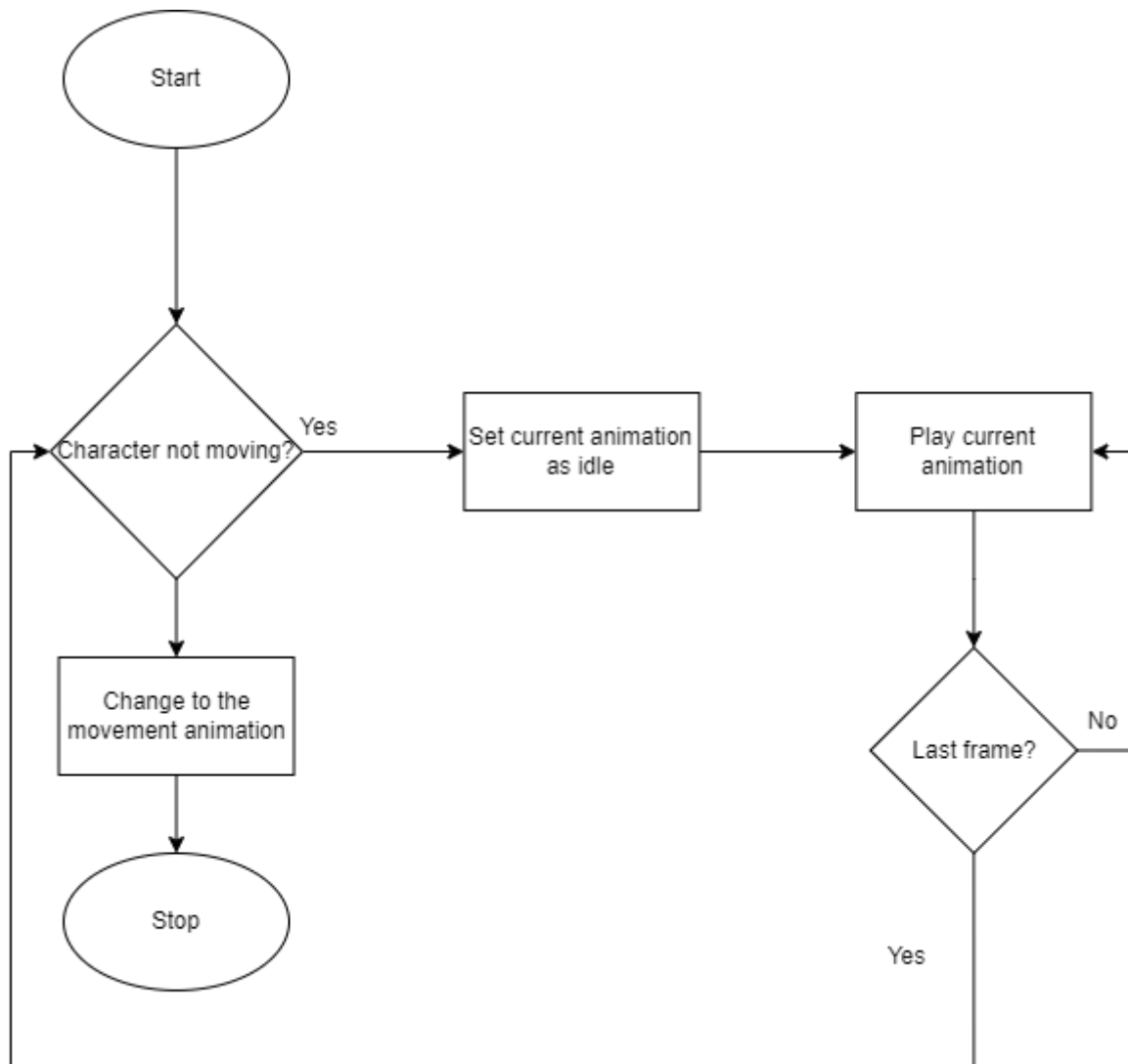
Justification

When the character is not moving whilst on the Battle Screen, this animation will be played to signify this state. Animations will play a major role on the Battle Screen for each character, as such I have included the logic of creating animation lists from sprite sheets. The next flowchart will describe the process of playing the idle animation.

Flowchart 1



Flowchart 2

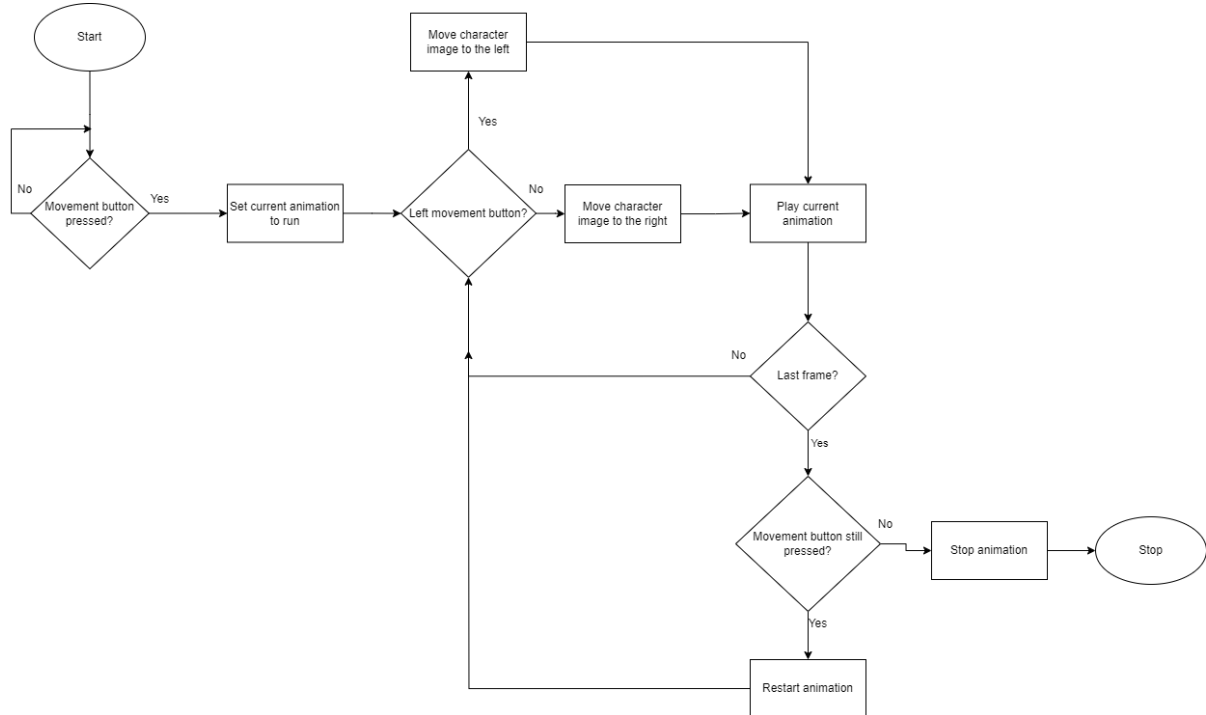


Success Criteria 19 & 24: Character Run Animation

Justification

When the user presses a movement button in either the left or right direction, the character's animation will change into the run animation and the image will move in the movement direction pressed.

Flowchart 1

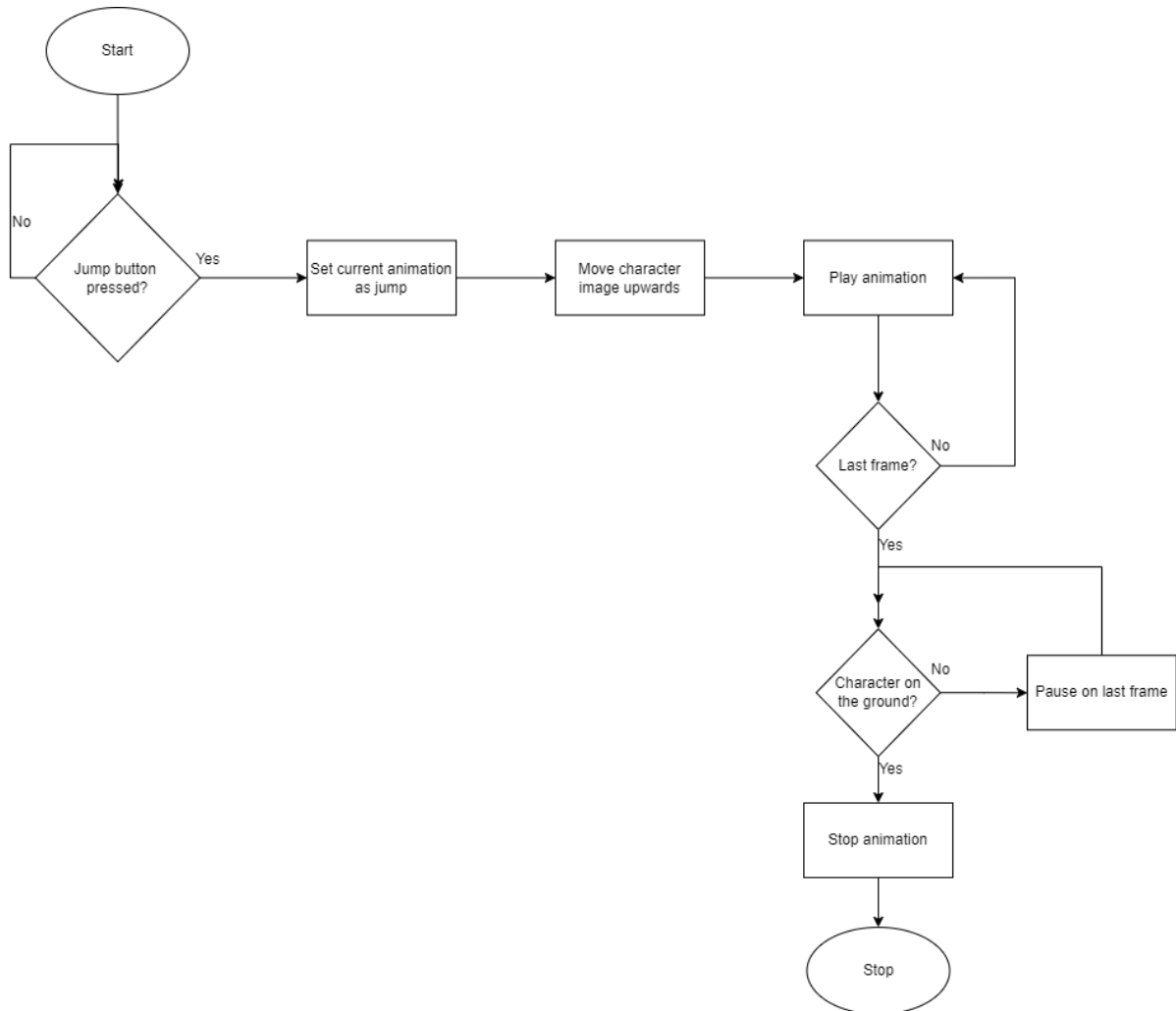


Success Criteria 20 & 25: Character Jump Animation

Justification

When the user wants to jump (either to dodge an attack or to engage in midair combat), a jumping animation will be displayed by the character as soon as the jump button is pressed.

Flowchart 1

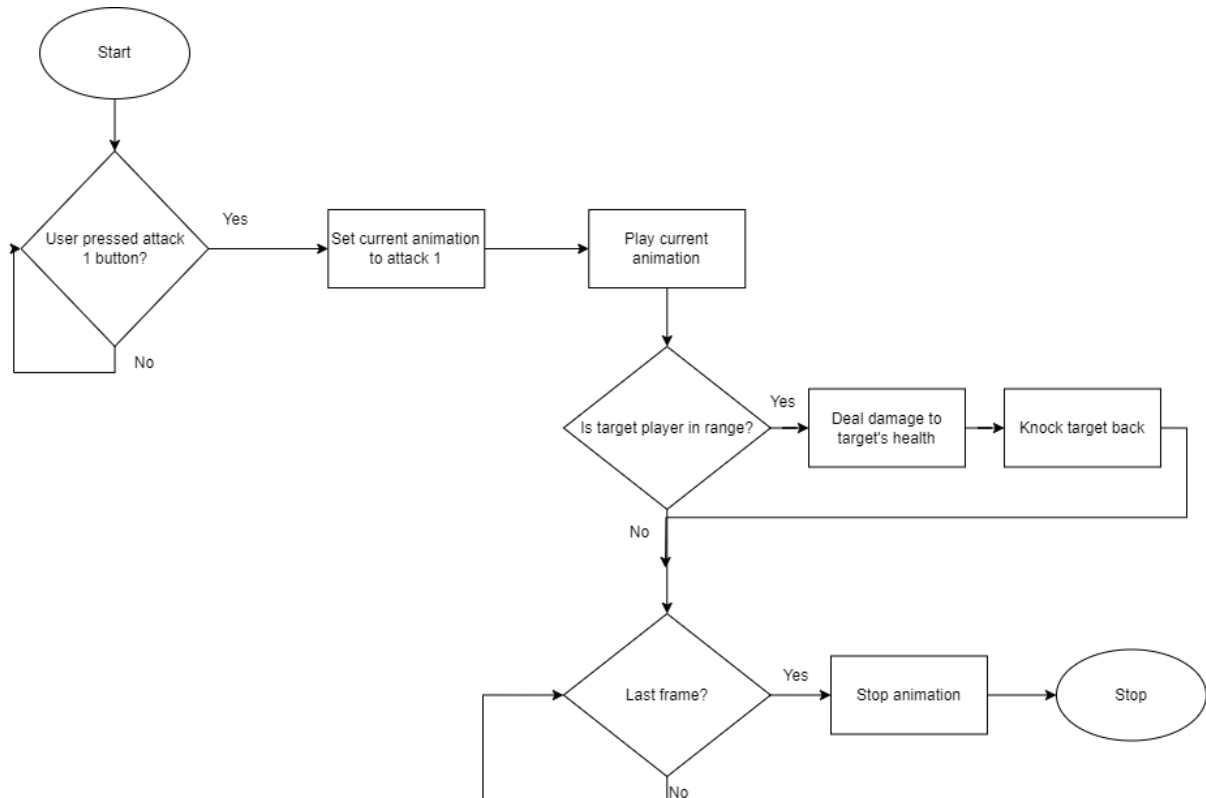


Success Criteria 21 & 26: Character Attack 1 Animation

Justification

To attack the opponent, the user will need to press their player's attack button. Both players are given two attack buttons. When the first attack button is pressed, an animation will play for the corresponding character that the user is playing as. If the opponent is in range, they will be knocked backwards by the attack and their health will be depleted by 6.5.

Flowchart 1

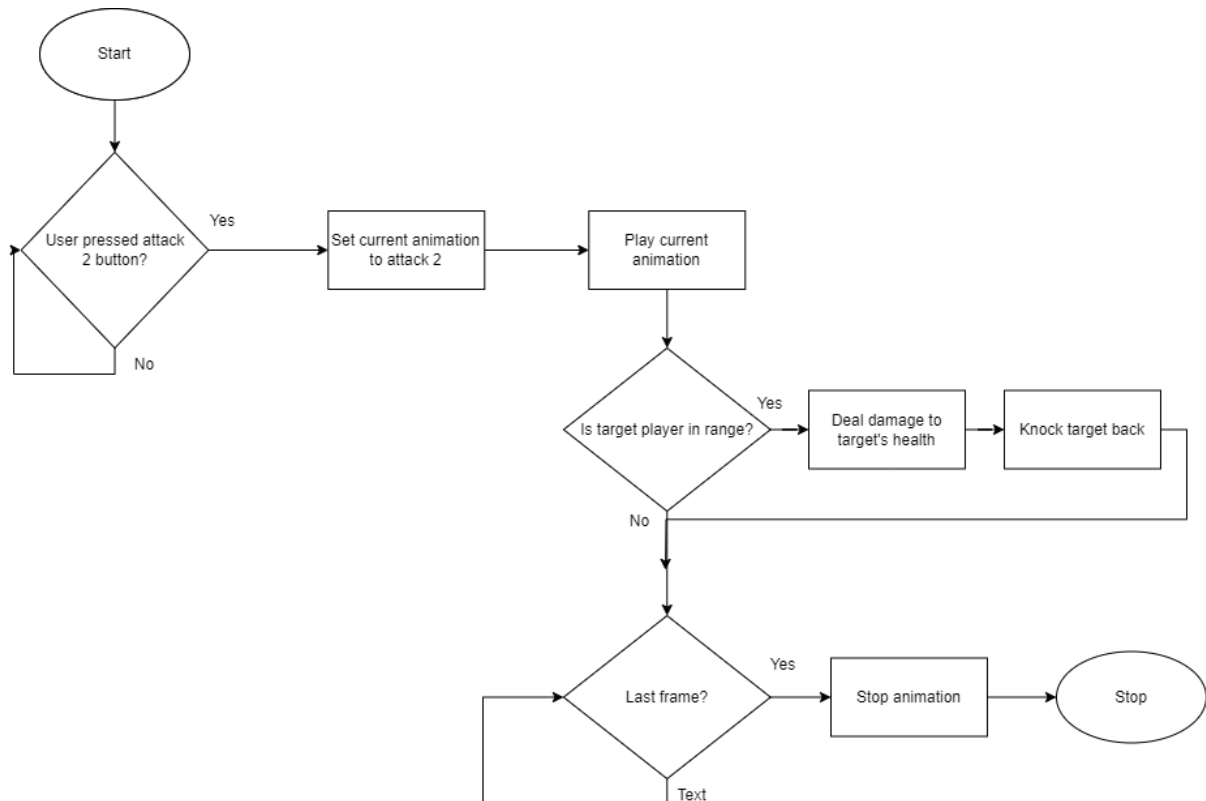


Success Criteria 22 & 27: Character Attack 2 Animation

Justification

Similarly, when the user presses the second attack option, an animation will play for the user's character and if the opponent is in range, they will be knocked back and have their health reduced by the same rate.

Flowchart 1

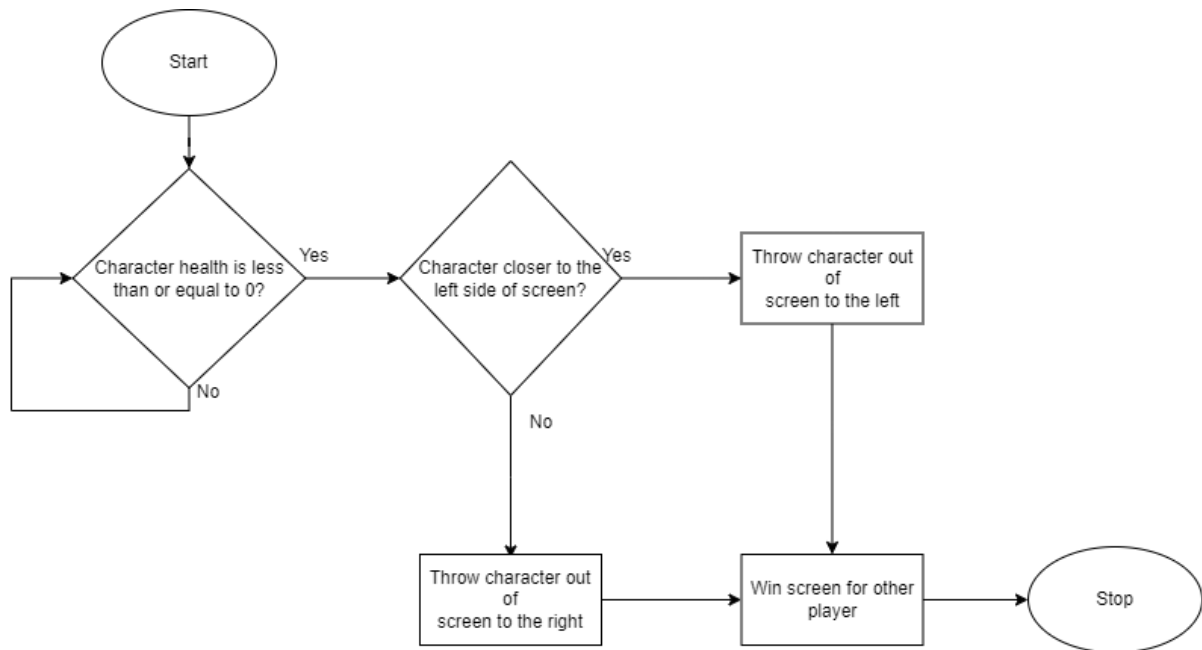


Success Criteria 28: Knock Out Death

Justification

When a player gets defeated, the last attack they received will have an extra powerful knock back effect which will knock the player's character image out of the screen.

Flowchart 1

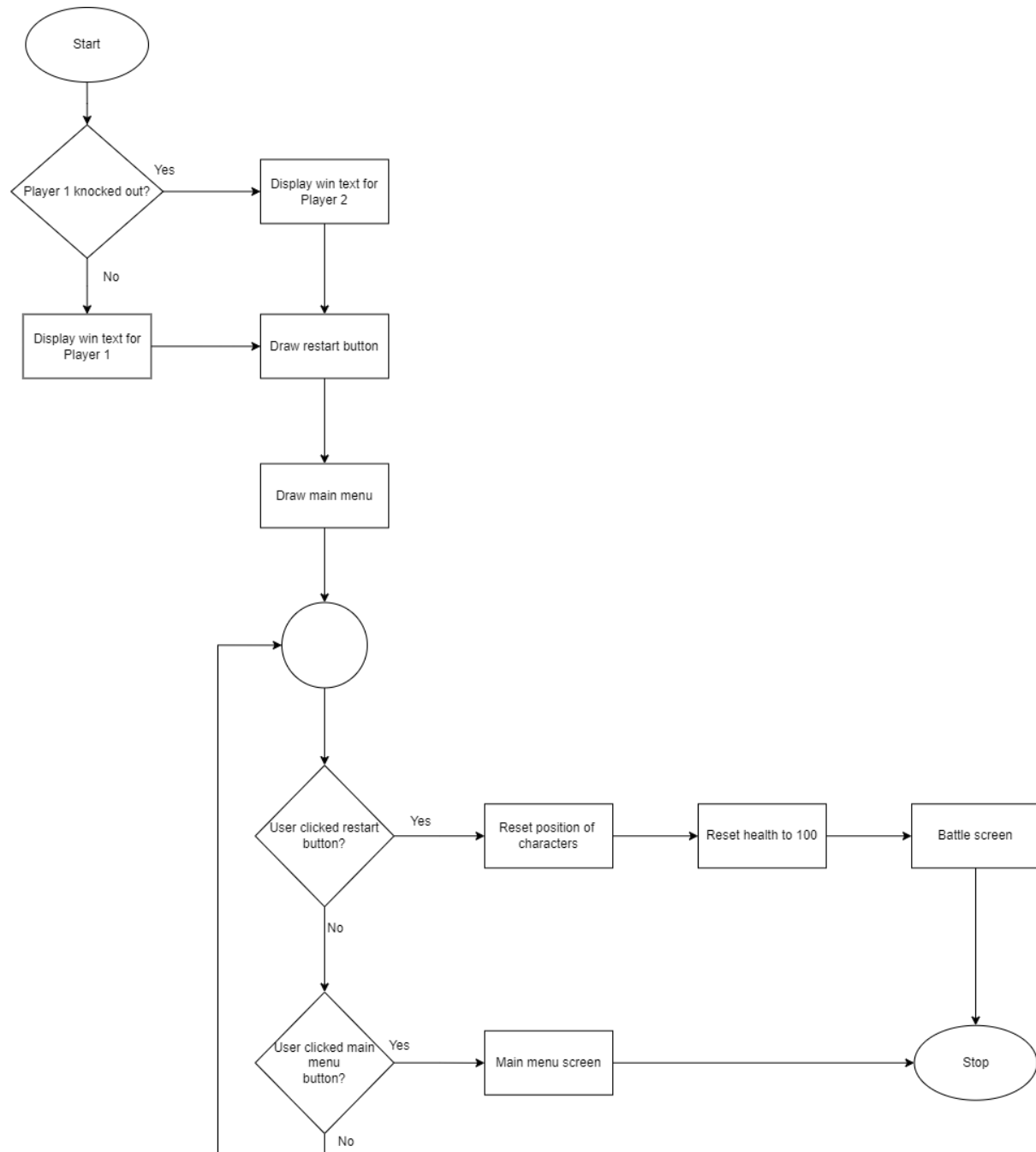


Success Criteria 29: Win Screen

Justification

When a player gets defeated, the screen will display a win message for the other player as well as menu buttons to restart the battle and a main menu button. The logic of this screen and the menu buttons are displayed below.

Flowchart 1

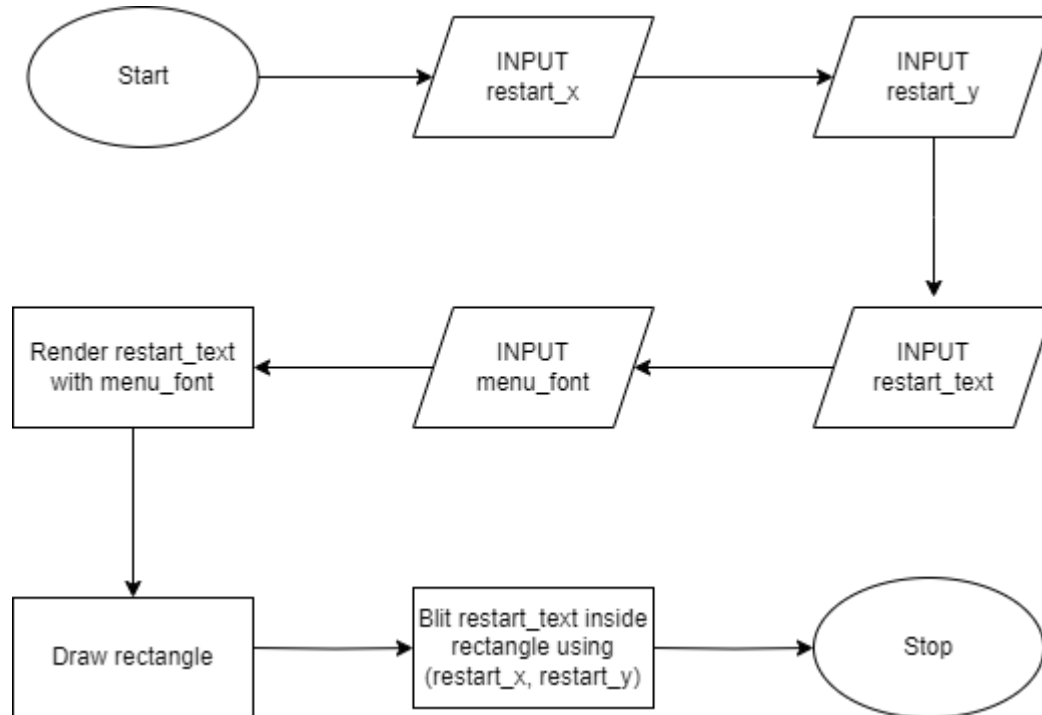


Success Criteria 30: Restart Button on Win Screen

Justification

On the Win Screen, there will be a button to restart the battle. When pressed, it will take the user back to the Battle screen. As the logic was already mentioned in the flowchart above, the drawing method will be shown below.

Flowchart 1

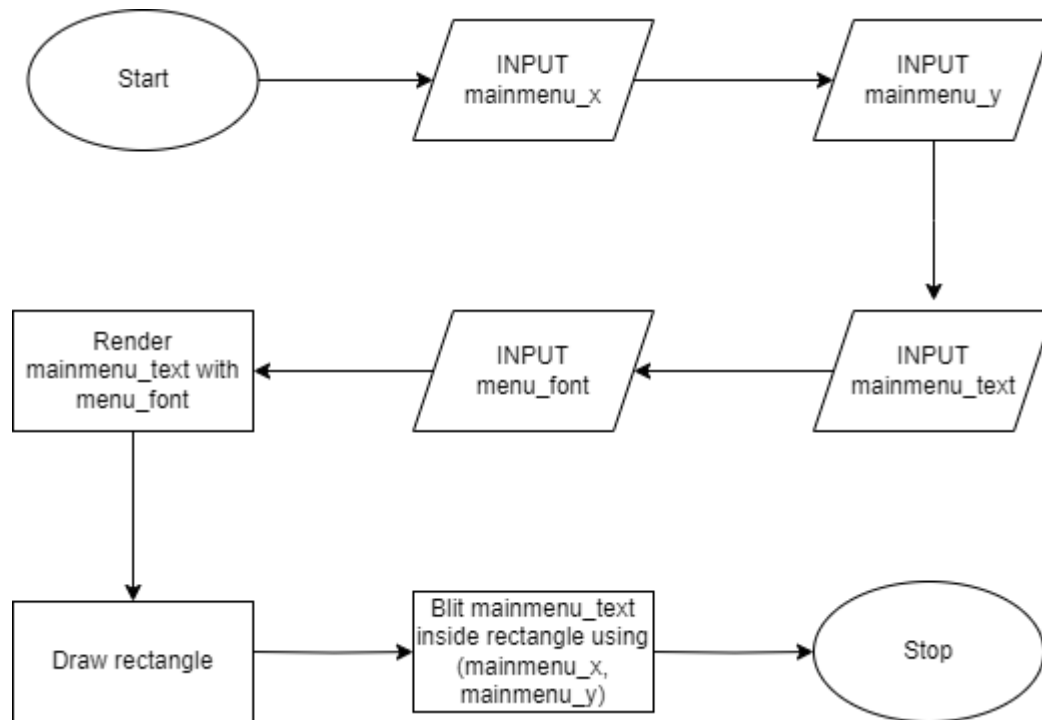


Success Criteria 31: Main Menu Button on Win Screen

Justification

On the Win Screen, there will also be a Main Menu Button which will return the user to the Main Menu Screen when pressed.

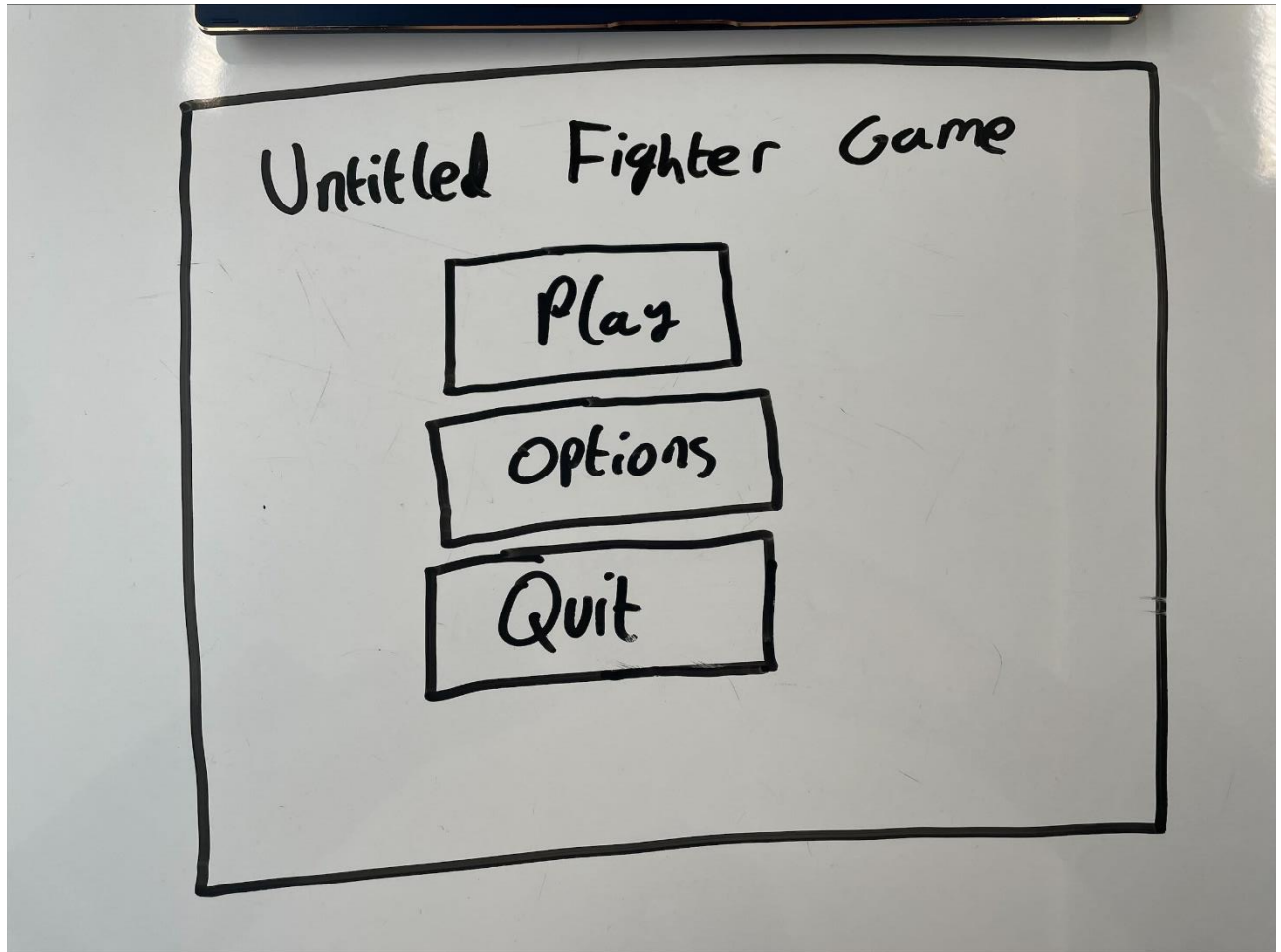
Flowchart 1



Screen Designs

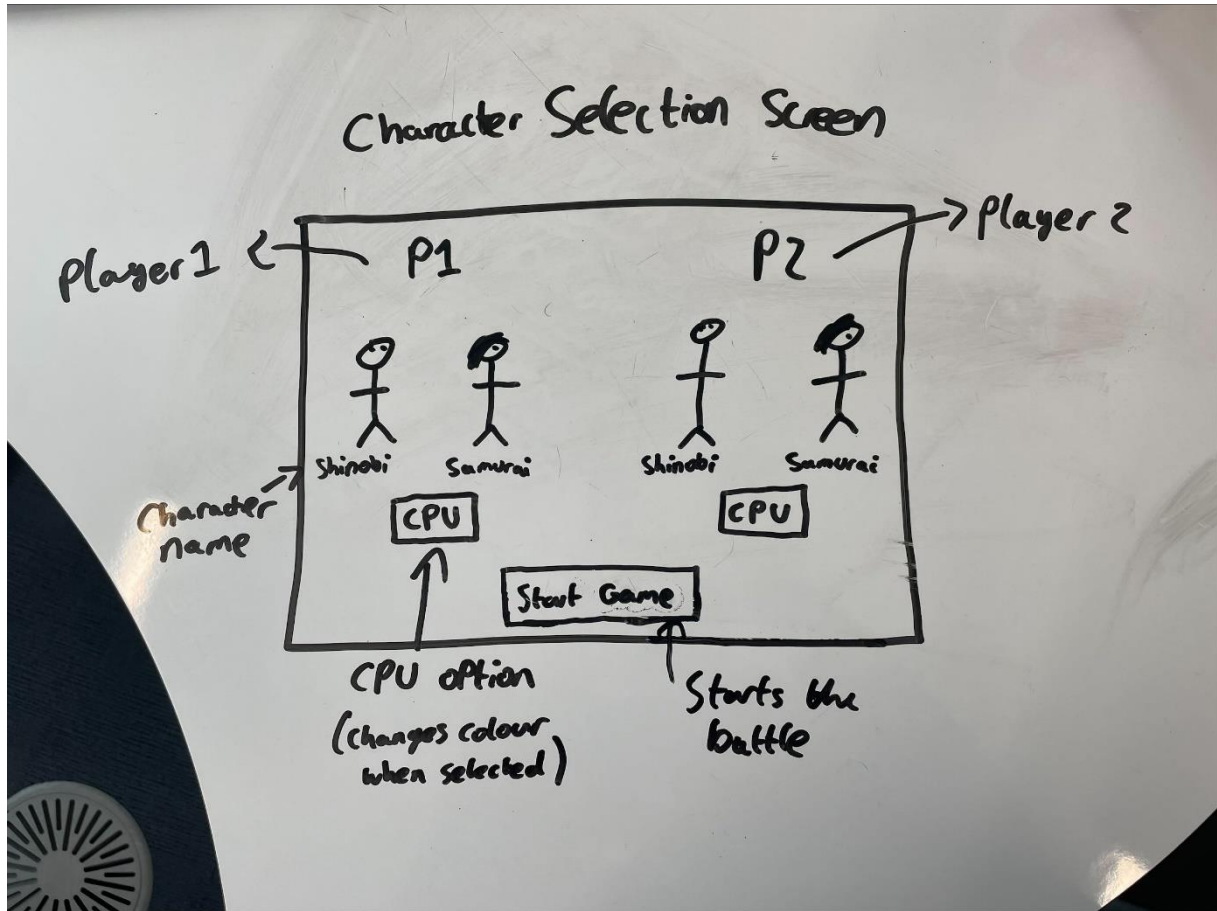
For this section of the design, I have drawn rough designs of each screen that the user will go through in my game. I took inspiration from the existing games in my research made in the analysis section, as well as the advice given to me by my interviewees. The designs below are the rough foundations from which I will try to build upon in during the development of my fighting game.

Main Menu



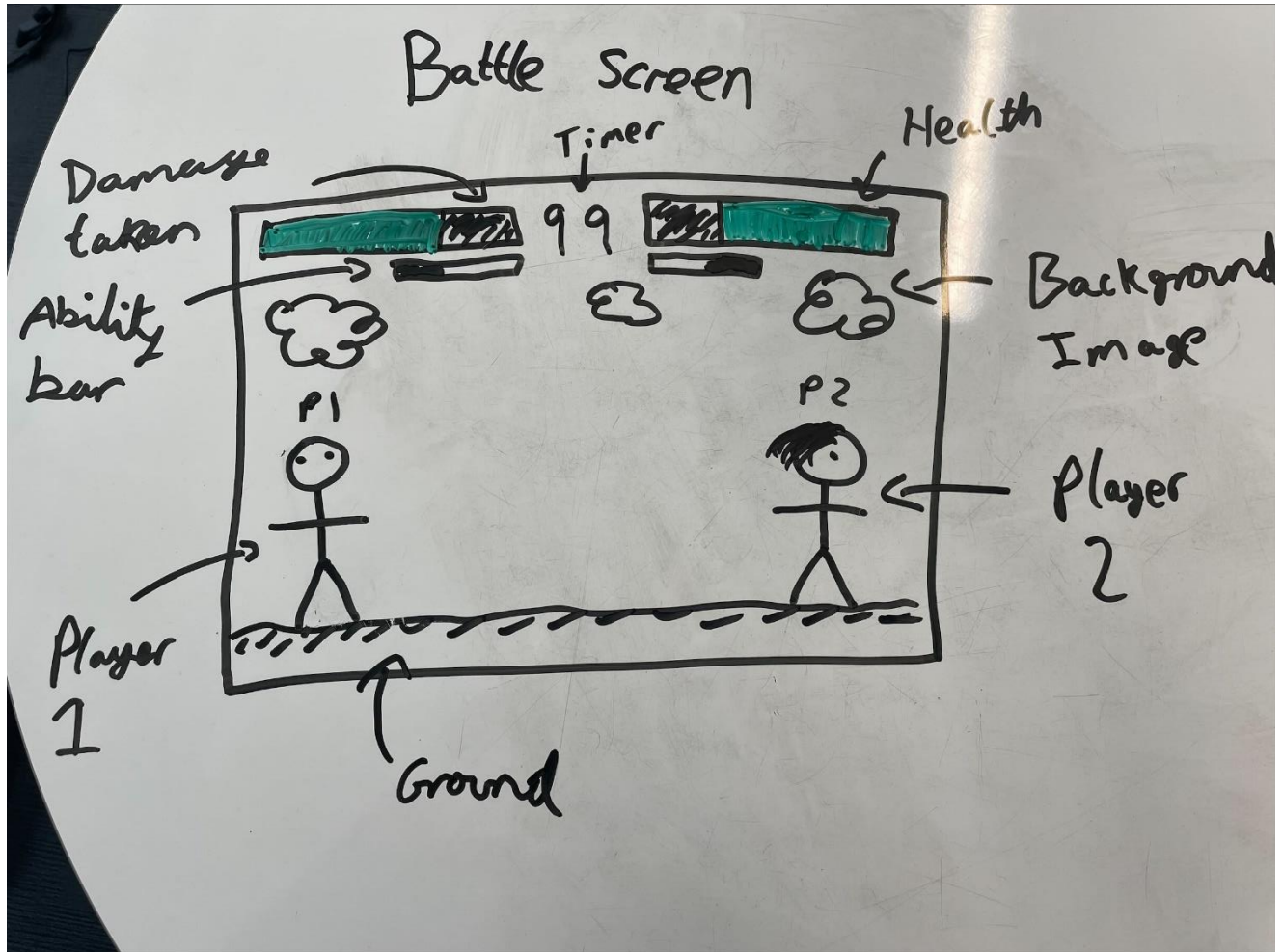
This is the rough main menu design. The title is located at the top of the screen and the three buttons (Play, Options and Quit) are available for the user to press below the title.

Character Selection



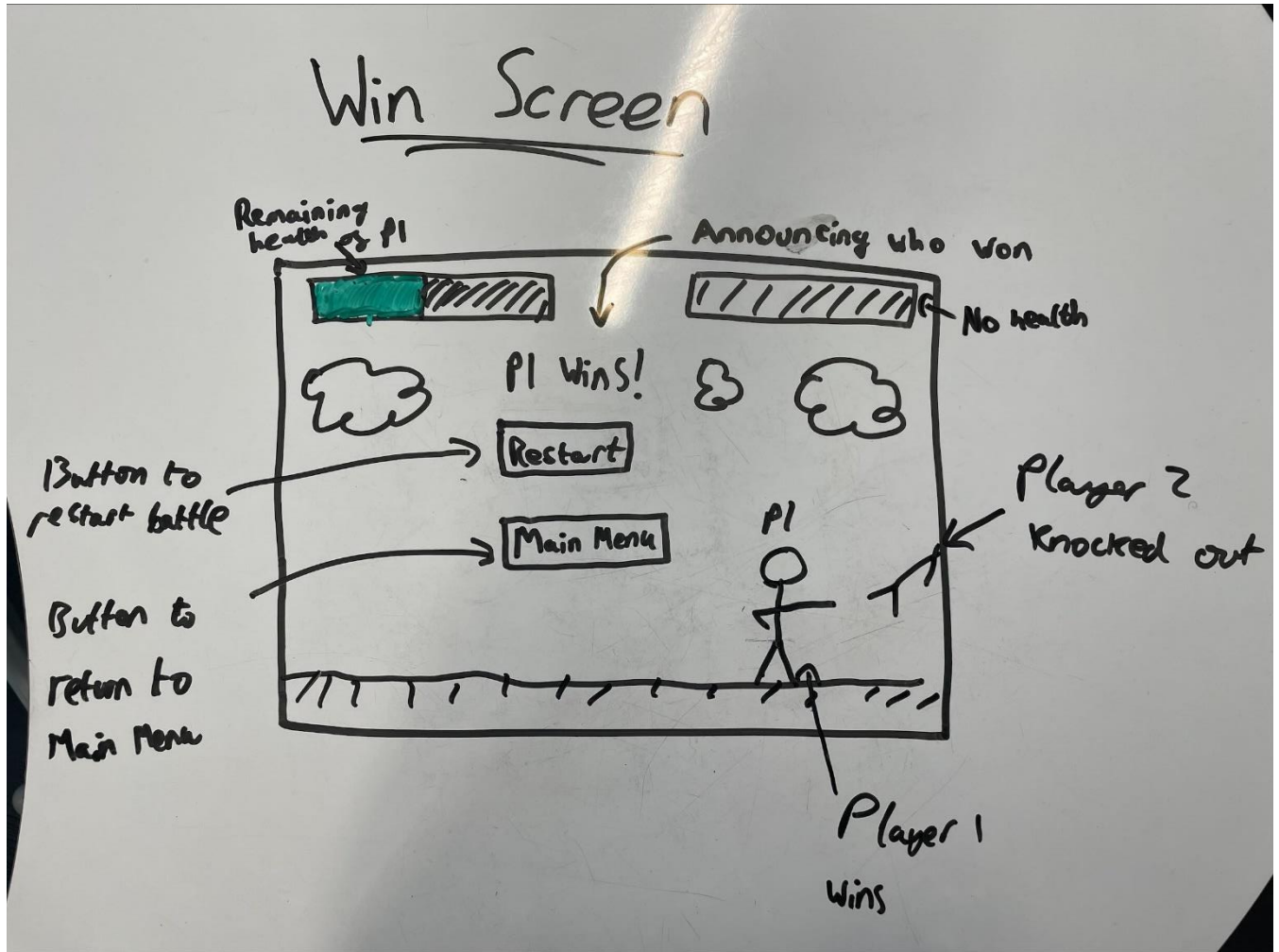
Here is the Character Selection screen. This is the screen that the user will see after they press the 'Play' button on the Main Menu. In this screen, the user can choose the character that they want to use as well as the character they want to battle against. The user can also choose to enable the CPU option which will make the character's fully computer controlled. Once both characters have been set and the user is ready to battle, the 'Start Game' button will be readily available to press at the bottom of the screen. Each button on this screen will change colour to green when pressed to indicate that it has been selected by the user.

Battle Screen



Once the user presses Start Game, they are shown this screen, the Battle screen. Here, the user's objective is to defeat the other player before the time runs out. The timer is at the top (will count up to 45 by default, not 99) of the screen and is located in between each user's health bar. When a character takes damage, the green health bar gets smaller, showing that their health is going down. The ability bar is underneath the health bar. The character's will initially be at opposite ends to each other on the screen and will have an animation playing at all times.

Win Screen



When a player wins a match, the opponent's character will be knocked out of the screen and the user will be presented with this screen. In this screen, there will be a message announcing who won the match and two buttons (Restart and Main Menu) that can will redirect the user to different screens when pressed. If the Restart button is pressed, the user will be redirected back to the battle screen with the fighter's health and positions being reset. If the user presses Main Menu, they will be directed to the Main Menu screen.

Data Structures

Menu Variables

These are the initial menu variables I will use to decide which screen appears at any one moment. The variable Main Menu is true because it is the first menu screen that will appear for the user. The rest will have to be toggled to True later on in the program.

Name	Data Type	Initial Setting
MainMenu	Boolean	True
options	Boolean	False
quit	Boolean	False
CharacterSelect	Boolean	False
restart	Boolean	False
MenuReturn	Boolean	False
Timeout	Boolean	False

Fighter Class

Instantiation

This will be the base class that the Player objects will use. It will take inputs for x, y positions, Player1, flip, sprite sheet, CPU, size and scale. The table below describes the attributes that will be included in the class.

Name	Data Type	Initial Setting
size	Integer	INPUT
image_scale	Integer	INPUT
offset	Integer	INPUT
animationlist	List	[]
running	Boolean	False
action	Integer	0
frame_index	Integer	0
image	Pygame Surface	animationlist[action][frame_index]
currentFrame	Integer	0
cooldown_timer	Integer	500
jump	Boolean	False
attacking	Boolean	False
attacktype	Integer	0
hit	Boolean	False
dead	Boolean	False
health	Integer	100
player	Boolean	INPUT
CPU	Boolean	INPUT

Draw Method

This method takes in the image attribute as an input to blit the image onto the screen.

Name	Data Type	Initial Setting
img	Surface	

Load Images Method

This method takes in the sprite sheet and animationsteps attributes from the Instantiation. This function returns the animation list value for its attribute.

Name	Data Type	Initial Setting
animationlist	List	[]
Temp_img_list	List	[]
Temp_img	Surface	

Move Method

This method will update the player position on the screen.

Name	Data Type	Initial Setting
speed	Integer	30
dx	Integer	0
dy	Integer	0
running	Boolean	False
key	String	''
CPU	Boolean	False
Dead	Boolean	False
attacking	Boolean	False
player	Boolean	True

Attack Method

This method will update the target player's health and whether the current player is attacking or not.

Name	Data Type	Initial Setting
attacking	Boolean	True
health	Integer	100
hit	Boolean	False

Post-development Data

I will use an evaluation user interview after implementing my game to determine how users will play my game, collecting information on what is their favourite part of my game and any errors that they face while using it.

It would also make sense for me to try the game using another device to test compatibility and ensure users of all devices that meet the requirements can play my game.

Test Plan

Total tests: 130

Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome
1	01	To see if the background image appears in the game window	Run the program and inspect the output screen for the image	Background image is seen inside game window
2	02 & 03	Title On Screen	Run the program and inspect the screen to see if the title is visible	Title can be seen on the screen
3	03	Centred Title	Check if the Title is in the middle of the output screen	Title is centred and is big
4	03	Title text is the game title	Check code text and output text match	Title says "Untitled Fighter Game"
5	03	Title is White	Inspect output screen for a white title font	The colour of the title is white
6	05	Play button that the user can see	Inspect output screen to see a rectangular button	Rectangular button visible on screen
7	05	"Play" Text inside rectangle button	Inspect screen for "Play" text inside button	Play text visible inside rectangular button
8	05	Centred play button	Check button is in the middle of the screen below title	Button located below game title at the centre of screen

9	05 & 07	Play button goes to Character Selection Screen	Press play button in multiple areas within the button with left mouse button	Change from main menu screen to Character Selection
10	05 & 07	Any clicks outside play button does not take user to Character Selection	Click left mouse button all around the play button	No change in screen
11	06	Options button that the user can see	Check output screen for rectangular button	Visible options button on screen
12	06	"Options" text inside rectangle	Check output screen for "Options" text	Options text visible inside rectangular button
13	06	Centred options button	Check output screen for options button below	Button located below play button and centred
14	06	Options button goes to settings screen	Click multiple areas within the options button with left mouse button	Change from main menu screen to options screen
15	06	Any clicks outside options button does not go to settings screen	Click around the options button with left mouse button	No change in screen
16	04	Quit button that user can see	Check output screen for rectangular button	Visible quit button on screen
17	04	"Quit" text inside rectangle	Check output screen for "Quit" text	Quit text visible inside rectangular
18	04	Centred quit button	Check output screen to see quit button in middle of the screen below options button	Button located below options button and centred
19	04	Quit button exits program when clicked	Click multiple areas within quit button with left mouse button	Program is closed after Quit button is clicked
20	04	Any clicks outside quit button does not exit program	Click around quit button with left mouse button	No change in screen
21	08	Player 1, Shinobi Button on screen	Check output screen	Visible rectangular button on the left side of screen
22	08	Player 1, Shinobi Button selected	Click multiple areas of the Shinobi button when not	Button changes colour from black to green

			selected (the button is black)	
23	08	Player 1, Shinobi image inside button	Check output screen – all of image should be inside the button	Visible Shinobi character sprite image inside rectangular button
24	08	Player 1, Shinobi Button deselected	Click multiple areas of the Shinobi button after it has been selected (the button is already green)	Button changes colour from green to black
25	08	Player 1, Shinobi not selected when user clicks outside its button	Click multiple areas outside and around Shinobi button	No change to green for this specific button
26	09	Player 1, Samurai Button on screen	Check output screen	Visible rectangular button adjacent to Player 1 Shinobi button
27	09	Player 1, Samurai Button Selected	Click multiple areas of the Samurai button (button is black already)	Button changes colour from black to green
28	09	Player 1, Samurai image inside button	Check output screen – all of image should be inside button	Visible Samurai character sprite image inside rectangular button
29	09	Player 1, Samurai Button deselected	Click multiple areas Of the Samurai button (button is green already)	Button changes colour from green to black
30	09	Player 1, Samurai not selected when user clicks outside its button	Click multiple areas outside and around Samurai button	No change to green for this specific button
31	08	Player 2, Shinobi Button on screen	Check output screen	Rectangular button on the right-hand side of the screen
32	08	Player 2, Shinobi Button selected	Click multiple areas of the Shinobi button when not selected (the button is black)	Button changes colour from black to green
33	08	Player 2, Shinobi image inside button	Check output screen – all of image should be inside button	Visible Shinobi character sprite image inside rectangular button

34	08	Player 2, Shinobi Button deselected	Click multiple areas of the Shinobi button after it has been selected (the button is already green)	Button changes colour from green to black
35	08	Player 2, Shinobi not selected when user clicks outside its button	Click multiple areas outside and around Shinobi button	No change to green for this specific button
36	09	Player 2, Samurai Button on screen	Check output screen	Rectangular button on the right-hand side of the screen adjacent to Shinobi Player 2 button
37	09	Player 2, Samurai Button selected	Click multiple areas of the Samurai button (button is black already)	Button changes colour from black to green
38	09	Player 2, Samurai image inside button	Check output screen – all of image should be inside button	Visible Samurai character sprite image inside rectangular button
39	09	Player 2, Samurai Button deselected	Click multiple areas Of the Samurai button (button is green already)	The Samurai button goes from green to black
40	09	Player 2, Samurai not selected when user clicks outside its button	Click multiple areas outside and around Samurai button	No colour change of the button
41	08	Player 1, Shinobi name under button	Check output screen for Shinobi name under button	Visible character name “Shinobi” under the button
42	09	Player 1, Samurai name under button	Check output screen for Samurai name under button	Visible “Samurai” name under button
43	08	Player 2, Shinobi name under button	Check output screen Shinobi name	Visible “Shinobi” under button
44	09	Player 2, Samurai name under button	Check output screen for Samurai name under button	Visible “Samurai” under button
45	10	CPU Button under Player 1-character buttons	Check output screen for CPU button under player 1-character buttons	Visible black button under player 1 character buttons
46	10	CPU Button under Player 2 character buttons	Check output screen for CPU button under player 1 character buttons	Visible black button under player 2 character buttons

47	10	CPU Button selection	Click multiple areas of the CPU button (button is black already)	CPU button colour changes from black to green
48	10	CPU deselection	Click multiple areas Of the CPU button (button is green already)	CPU button colour changes from green to black (while green)
49	10	CPU button only selected when pressed	Click multiple areas outside and around CPU button. Then press CPU button.	CPU button changes to green only when pressed. No change in colour otherwise.
50	10	CPU Difficulty Buttons after CPU selection	Select CPU option. Then check for CPU Difficulty buttons on output screen	All three buttons (Easy, Medium and Hard) appear after selecting the CPU button
51	10	CPU "Easy" Button	Check output screen for CPU Easy button	Visible black button under CPU button
52	10	"Easy" text on CPU Button	Check output screen for "Easy" text in button	Visible "Easy" text inside Easy button
53	10	CPU "Easy" Selection	Click multiple areas of the CPU Easy button (button is black already)	Button goes from black to green when selected
54	10	CPU "Easy" Deselection	Click multiple areas Of the CPU Easy button (button is green already)	Button goes from green to black when selected (while green)
55		CPU "Medium" Button	Check output screen for CPU Medium button	Visible black button under CPU button
56	10	"Medium" text on CPU Button	Check output screen for "Medium" text in button	Visible "Medium" text inside button
57	10	CPU "Medium" Selection	Click multiple areas of the CPU Medium button (button is black already)	Button goes from black to green when selected
58	10	CPU "Medium" Deselection	Click multiple areas Of the CPU Medium button (button is green already)	Button goes from green to black when selected (while green)
59	10	CPU "Hard" Button	Check output screen for CPU Hard button	Visible black button under CPU button

60	10	"Hard" text on CPU Button	Check output screen for "Hard" text in button	Visible "Hard" text inside button
61	10	CPU "Hard" Selection	Click multiple areas of the CPU Hard button (button is black already)	Button goes from black to green when selected
62	10	CPU "Hard" Deselection	Click multiple areas Of the CPU Hard button (button is green already)	Button goes from green to black when selected (while green)
63	11	Start Game button	Check output screen for Start Game button in Character Selection Screen	Visible black button
64	11	"Start Game" text on button	Check output screen for Start Game button in Character Selection Screen	Visible "Start Game" text on button
65	11	Start game button at bottom of screen	Check output screen for position of "Start Game" button	"Start Game" at the bottom of the screen and centred
66	11 & 12	Start game button goes to Battle Screen	Click "Start Game"	Interface of Character Selection screen replaced with Battle Screen interface.
67	11	Start game only works when pressed	Click multiple areas outside of the "Start Game" button then click the visible box area of the button	Screen only changes if start game pressed. No change otherwise.
68	13	Timer during battle screen	Check Battle Screen for timer	Visible timer
69	13	Timer centred on screen	Check Battle Screen for position of timer	Visibly at the top of screen and centred.
70	13	Timer text changes after a second	Stopwatch or Clock, measure against the timer and check for synchronisation	Timer matches with clock
71	13	Timer starts at 0	Quickly check timer when "Start Game" button pressed	When Battle screen starts, timer is visibly at 0
72	13	Timer counts to 45	Wait until timer runs out on the Battle Screen	45 is visible just before battle ends
73	13	Timer triggers Timeout screen after 45 seconds	Wait until timer reaches 45 seconds,	Battle screen replaced with timeout screen

			inspect for a screen change	after 45 is visible on timer
74	13	"Time ran out!" text on screen	Inspect screen	Visible "Time ran out!" text
75	13	"Time ran out!" around the centre of the screen	Inspect screen	Centred text
76	13	Restart button	Inspect screen for button	Visible black button
77	13	"Restart" text in restart button (Timeout screen)	Inspect screen for "Restart" text	Visible "Restart" text inside the restart button
78	13	"Main menu" button (Timeout screen)	Inspect screen for "Main menu" button	Visible black button
79	13	"Main menu" text inside Main Menu button (Timeout screen)	Inspect screen for "Main menu" text	Visible "Main Menu" text inside Main menu button
80	13	Main Menu button goes to Main Menu Screen (from Timeout screen)	Click Main Menu button	Should return to Main Menu screen
81	13	Restart button goes back to Battle Screen (from Timeout screen)	Click restart button	Should return to battle screen
82	14 & 15	Health bar on screen	Inspect screen	Visible green bar on screen
83	14 & 15	Health bar at opposite sides at the top of the screen	Inspect screen	Visible green bar at top sides of the screen at both ends
84	14 & 15	Health bar starts at full capacity when Battle screen starts	Go through Main Menu -> Char Selection -> Battle Screen and inspect health at the start of the battle	Green health bar should be full (no red visible)
85	14 & 15	Green health bar goes down by 6.5 each hit	Use player 2 character to hit player 1 character and inspect health	Should go down by 6.5
86	14 & 15	Battle ends when one health bar is below or equal to zero	Use player 2 character to defeat player 1. Inspect for a change of screen.	Battle screen should transition to Win screen
87	14 & 15	Health bar resets to full capacity at restart	Use player 2 character to defeat player 1. Click restart button on Win screen. Inspect health bar.	Should be full green bars for both players (no red visible)

88	16 & 17	Shinobi Character visible	Inspect Battle screen	Shinobi sprite should be seen
89	16 & 17	Samurai Character visible	Inspect Battle screen	Samurai sprite should be seen
90	16 & 17	Shinobi Character on the ground	Inspect character in the context of the background image in Battle screen	Feet should be on the green grass
91	16 & 17	Samurai Character on the ground	Inspect character in the context of the 92background image in Battle screen	Feet should be on the green grass
92	18 & 23	Shinobi Idle Animation plays and loops animation when not moving	Inspect animation of Shinobi sprite while not moving in Battle screen	Idle animation should be slight breathing movements of the sprite.
93	18 & 23	Samurai Idle Animation plays and loops animation when not moving	Inspect animation of Samurai sprite while not moving in Battle screen	Idle animation should be slight breathing movements of the sprite.
94	18 & 23	Shinobi Idle Animation stops when moving/jumping	Use forwards movement key (D for player 1, < arrow key for player 2) and inspect the animation of the moving character. Then use backwards movement key (A for player 1, > arrow key for player 2) and inspect animation of moving character. Then use jump movement key (W for player 1, ^ for player 2) and inspect animation of moving character.	Idle animation should stop and run animation should play
95	18 & 23	Samurai Idle Animation stops when moving/jumping	Use forwards movement key (D for player 1, < arrow key for player 2) and inspect the animation of the moving character. Then use backwards	Idle animation should stop, and the relevant animation should play (run animation for forwards and backwards keys, jump animation for

			movement key (A for player 1, > arrow key for player 2) and inspect animation of moving character. Then use jump movement key (W for player 1, ^ for player 2) and inspect animation of moving character.	jump movement key)
96	19 & 24	Shinobi Run Animation plays when moving forwards	Use forwards movement key (D for player 1, < arrow key for player 2) and inspect player animation	Player's character should move forwards in their relative direction and run animation should play
97	19 & 24	Shinobi Run Animation plays when moving backwards	Use backwards movement key (A for player 1, > arrow key for player 2)	Player's character should move backwards in their relative direction and run animation should play
98	19 & 24	Shinobi Run Animation stops when not moving	Use forwards movement key (D for player 1, < arrow key for player 2), then stop and inspect player animation. Then use backwards movement key (A for player 1, > arrow key for player 2), stop and inspect player animation.	Animation should transition from run to idle
99	19 & 24	Shinobi Run Animation stops when jumping	Use jump movement key (W for player 1, ^ for player 2), wait for landing, then inspect animation.	Animation should transition from jump to idle when landing.
100	19 & 24	Samurai Run Animation plays when moving forwards	Use forwards movement key (D for player 1, < arrow key for player 2) and inspect player animation	Player's character should move forwards in their relative direction and run animation should play

101	19 & 24	Samurai Run Animation plays when moving backwards	Use backwards movement key (A for player 1, > arrow key for player 2)	Player's character should move backwards in their relative direction and run animation should play
102	19 & 24	Samurai Run Animation stops when not moving	Use forwards movement key (D for player 1, < arrow key for player 2), then stop and inspect player animation. Then use backwards movement key (A for player 1, > arrow key for player 2), stop and inspect player animation.	Animation should transition from run to idle
103	19 & 24	Samurai Run Animation stops when jumping	Use jump movement key (W for player 1, ^ for player 2), wait for landing, then inspect animation.	Animation should transition from jump to idle when landing.
104	20 & 25	Shinobi Jump Animation plays when jumping off the ground	Use jump movement key (W for player 1, ^ for player 2), then inspect animation.	Jump animation should play while in the air
105	20 & 25	Shinobi Jump Animation stops when character lands	Use jump movement key (W for player 1, ^ for player 2), wait for landing, then inspect animation.	Animation should transition from jump to idle when landing.
106	20 & 25	Shinobi can Jump forwards	Use jump movement key (W for player 1, ^ for player 2) and the forwards movement key (D for player 1, < arrow key for player 2), then inspect the movement of the character.	Character should be moving forwards whilst jumping, only having jump animation playing
107	20 & 25	Shinobi can Jump backwards	Use jump movement key (W for player 1, ^ for player 2) and the	Character should be moving backwards whilst jumping, only

			backwards movement key (A for player 1, > arrow key for player 2), then inspect the movement of the character.	having jump animation playing
108	20 & 25	Samurai Jump Animation plays when jumping off the ground	Use jump movement key (W for player 1, ^ arrow key for player 2) and inspect animation of the jumping character	Jump animation should play while in air
109	20 & 25	Samurai Jump Animation stops when character lands	Use jump movement key (W for player 1, ^ arrow key for player 2), wait until character lands then inspect animation of the jumping character	Jump animation should stop and switch to idle animation
110	20 & 25	Samurai can Jump forwards	Use jump movement key (W for player 1, ^ for player 2) and the forwards movement key (D for player 1, < arrow key for player 2), then inspect the movement of the character.	Character should be moving forwards whilst jumping, only having jump animation playing
111	20 & 25	Samurai can Jump backwards	Use jump movement key (W for player 1, ^ for player 2) and the backwards movement key (A for player 1, > arrow key for player 2), then inspect the movement of the character.	Character should be moving backwards whilst jumping, only having jump animation playing
112	21 & 26	Shinobi Attack 1 Animation plays when 'e' key pressed (Player 1) or 'n' key (Player 2)	Use Attack 1 for Player 1 and inspect animation. Repeat for Player 2.	Attack 1 animation should play once then switch back to idle.
113	21 & 26	Shinobi Attack 1 Animation damages	Use Attack 1 for Player 1 against Player 2.	Player 2 health should decrease by 6.5

		opponent in range of hitbox (-6.5)		
114	21 & 26	Samurai Attack 1 Animation plays when 'e' key pressed (Player 1) or 'n' key (Player 2)	Use Attack 1 for Player 1 and inspect animation. Repeat for Player 2.	Attack 1 animation should play once then switch back to idle.
115	21 & 26	Samurai Attack 1 Animation damages opponent in range of hitbox (-6.5)	Use Attack 1 for Player 1 against Player 2.	Player 2 health should decrease by 6.5
116	22 & 27	Shinobi Attack 2 Animation plays when 'r' key pressed (Player 1) or 'm' key (Player 2)	Use Attack 2 for Player 1 and inspect animation. Repeat for Player 2.	Attack 2 animation should play once then switch back to idle.
117	22 & 27	Shinobi Attack 2 Animation damages opponent in range of hitbox (-6.5)	Use Attack 2 for Player 1 against Player 2.	Player 2 health should decrease by 6.5
118	22 & 27	Samurai Attack 2 Animation plays when 'r' key pressed (Player 1) or 'm' key (Player 2)	Use Attack 2 for Player 1 and inspect animation. Repeat for Player 2.	Attack 2 animation should play once then switch back to idle.
119	22 & 27	Samurai Attack 2 Animation damages opponent in range of hitbox (-6.5)	Use Attack 2 for Player 1 against Player 2.	Player 2 health should decrease by 6.5
120	28	Character image gets knocked out from screen when health is 0 (dead)	Use any attack for Player 1 to defeat Player 2 and inspect the movement of the character image	The character image should move out of the screen and should not be visible
121	28	Dead character triggers Win screen	Use Player 1 character to defeat player 2	Battle screen should change to Win screen
122	29	Player 1 declared the winner when player 2 dies	Use Player 1 character to defeat player 2	"Player 1 Wins!" Text should appear on Win screen
123	29	Player 2 declared the winner when player 1 dies	Use Player 2 character to defeat player 1	"Player 2 Wins!" Text should appear on Win screen
124	29	Both 'restart' and 'main menu' buttons appear for use	Use Player 1 character to defeat player 2, then inspect Win screen.	Visible black 'restart' and 'main menu' buttons should appear
125	30	Restart button	Inspect Win screen	Visible black button
126	30	"Restart" text in restart button (Win screen)	Inspect Win screen	Visible "Restart" text inside restart button

127	30	Restart button goes back to Battle Screen (from Win screen)	Click restart button on Win screen	Screen change from Win screen to Battle screen
128	31	Main Menu button	Inspect Win screen	Visible black button
129	31	"Main Menu" text inside main menu button	Inspect Win screen	Visible "Main Menu" text inside restart button
130	31	Main Menu button goes to Main Menu when clicked (from Win screen)	Click Main Menu button on Win screen	Screen change from Win screen to Main Menu

Implementation

Overview

In this stage of my project, I will be addressing and implementing each algorithm I created in the design stage. After creating each screen, I will review what I have done and what I plan to do after. I will test each algorithm I have implemented against to check it functions as required.

My project will go through five main stages:

Stage One – Main Menu Screen

Stage Two – Battle Screen ([here](#))

Stage Three – Character Selection Screen ([here](#))

Stage Four – Win/Timeout Screen ([here](#))

Stage One: Main Menu Screen

I will start off implementing the main menu screen which the user will interact with to navigate the areas of my fighting game.

Base template

I started off by writing the base template for the game where the pygame library is imported and initialised, and there is a loop for the game to run in.

```
1  import pygame
2
3  pygame.init()
4  |
5  #game loop
6  run = True
7  while run:
8
9      #events
10     for event in pygame.event.get():
11         if event.type == pygame.quit:
12             run = False
13
14     #update display
15     pygame.display.update()
16
17 #exit pygame
18 pygame.quit()
```

Success Criteria 1: Background Image

The first success criteria address how I will display a background image on the screen on which the players will stand in. To do this, I needed to assign the loaded background image to a variable then output it using the blit function in the pygame library.

Attempt 1

My first attempt is below:

```

20 #load background - from: https://wallpapercave.com/landscape-pixel-art-wallpapers
21 bg_image = pygame.image.load("C:\Users\MatinaJ\OneDrive - Dulwich College\Desktop\Untitled Fighter Game\Images\ImageBG.jpg").convert_alpha()
22
23 #define background drawing function
24 def drawBG():
25     screen.blit(bg_image,(0,0)) #draws the image
26
27 # Used to manage how fast the screen updates
28 clock = pygame.time.Clock()
29
30

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

File "c:\Users\MatinaJ\.vscode\extensions\ms-python.debugpy-2024.0.0-win32-x64\bundled\libs\debugpy\_vendored\pydevd\_pydevd_bundle\pydevd_runpy.py", line 320, in run_path
  code, fname = _get_code_from_file(run_name, path_name)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "c:\Users\MatinaJ\.vscode\extensions\ms-python.debugpy-2024.0.0-win32-x64\bundled\libs\debugpy\_vendored\pydevd\_pydevd_bundle\pydevd_runpy.py", line 294, in _get_code_from_file
  code = compile(f.read(), fname, 'exec')
                ^^^^^^^^^^^^^^^^^^^^^^^^^
File "c:\Users\MatinaJ\OneDrive - Dulwich College\Desktop\Untitled Fighter Game\UFG1.py", line 21
  bg_image = pygame.image.load("C:\Users\MatinaJ\OneDrive - Dulwich College\Desktop\Untitled Fighter Game\Images\ImageBG.jpg").convert_alpha()
                                   ^
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 2-3: truncated \UXXXXXXX escape
PS C:\Users\MatinaJ\OneDrive - Dulwich College\Desktop\Untitled Fighter Game>

```

Error 1

I had gotten an error on line 21 because the location of the background image could not be found. To fix this, I had to use double slashes (//) instead of one slash. I had also forgotten to scale the background image to fit the screen size which I had added in my second attempt.

Attempt 2 (success)

My second attempt is below:

```

1  import pygame
2
3  pygame.init()
4
5  # Define some colors
6  BLACK = (0, 0, 0)
7  WHITE = (255, 255, 255)
8  GREEN = (0, 255, 0)
9  RED = (255, 0, 0)
10 YELLOW = (255,255,0)
11
12 #game window
13 screen_width = 1000
14 screen_height = 600
15 screen = pygame.display.set_mode((screen_width,screen_height))
16
17 #set the game window caption
18 pygame.display.set_caption("Untitled Fighter Game")
19
20 #load background - from: https://wallpapercave.com/landscape-pixel-art-wallpapers
21 bg_image = pygame.image.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Images\\ImageBG.jpg").convert_alpha()

```

```

20  https://wallpapercave.com/landscape-pixel-art-wallpapers
21  :\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Images\\ImageBG.jpg").convert_alpha()

```

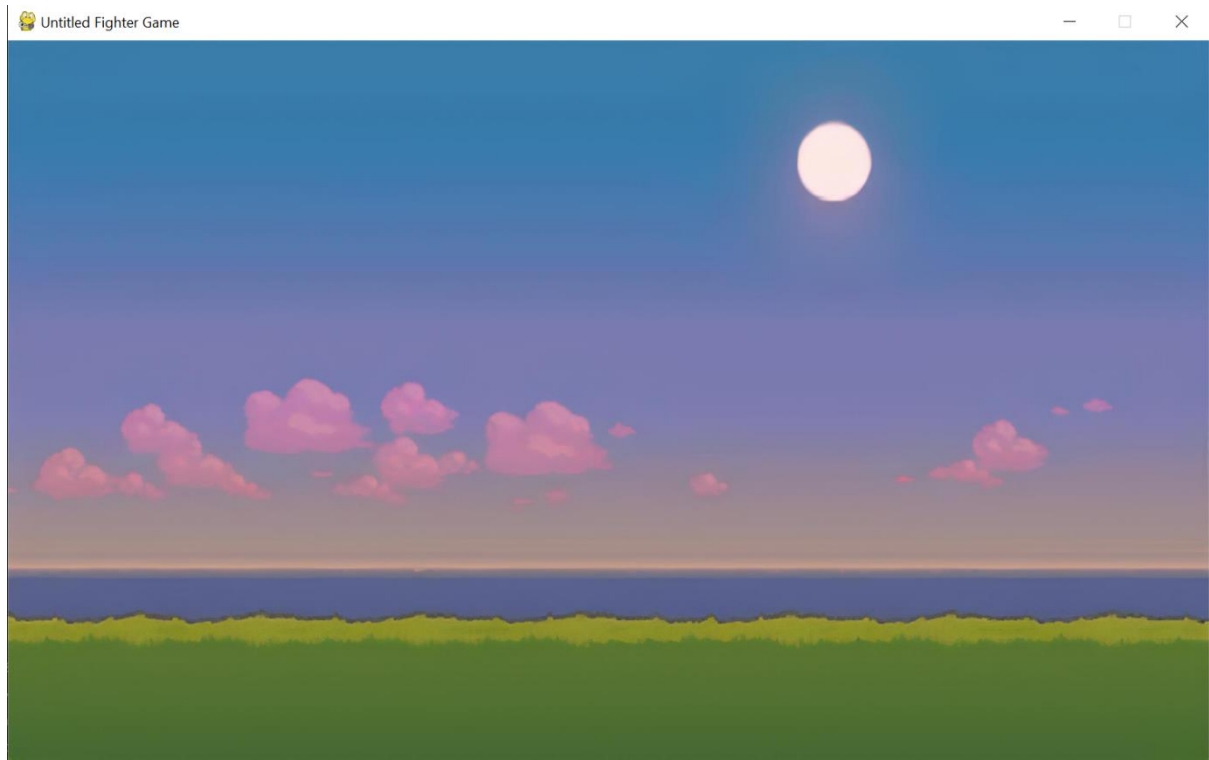
```

23 #define background drawing function
24 def drawBG():
25     scaled_bg = pygame.transform.scale(bg_image,(screen_width,screen_height)) #scales the image up to fit the window
26     screen.blit(scaled_bg,(0,0)) #draws the image

```

Test

And here is the output when the code was run:



The background image was outputted as expected and it fits perfectly in the window screen size, so it is clear for the user where the boundaries of battle are.

Success Criteria 2: Main Menu

The next success criteria concerns the start menu which is the first menu that the user sees. I focused on initialising the menu text font, rendering the text, initialising the y position and the colour of the text. My aim is to have the menu located at the centre of the screen so that it is visible to the user.

Attempt 1

I initialised the menu font and rendered the start menu text here. The y positions are also initialised here too.

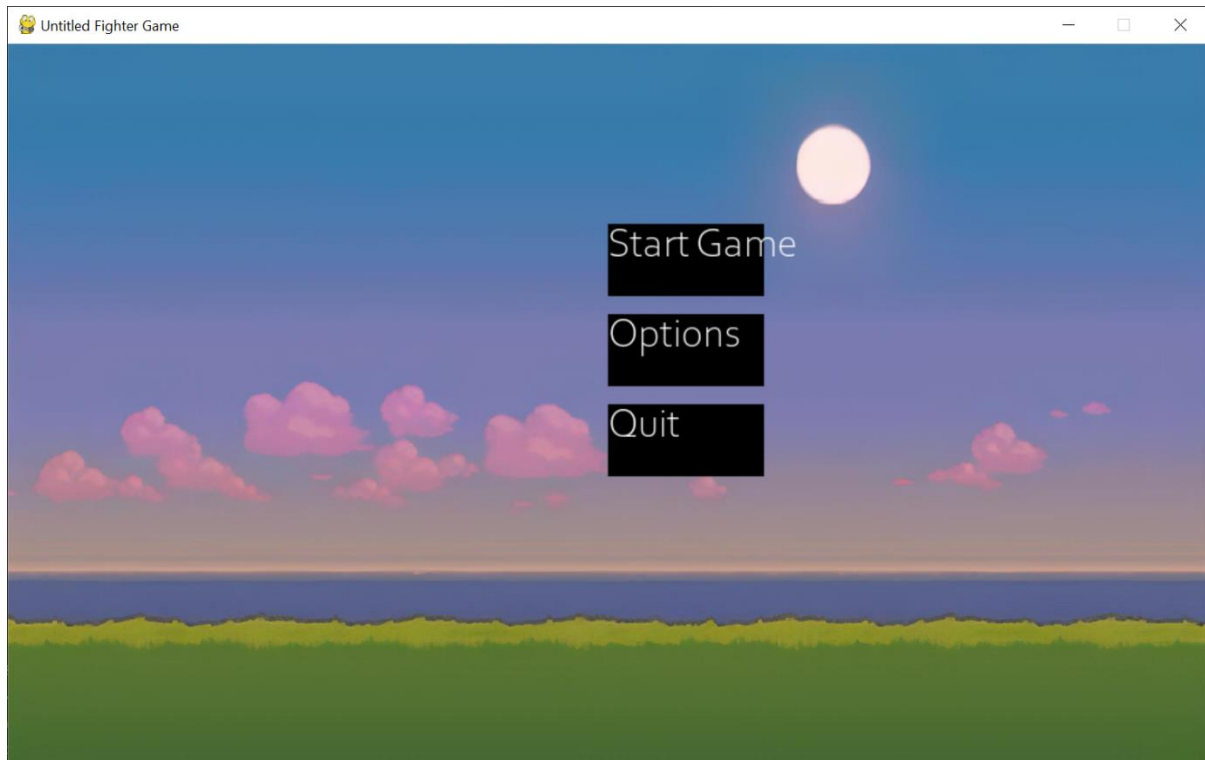
```
13 #game window
14 screen_width = 1000
15 screen_height = 600
16 screen = pygame.display.set_mode((screen_width,screen_height))
17
18 # defining menu font
19 MenuFont = pygame.font.SysFont('Corbel',35 )
20 #render menu font
21 text1 = MenuFont.render('Start Game' , True , WHITE)
22 text2 = MenuFont.render('Options' , True , WHITE)
23 text3 = MenuFont.render('Quit' , True , WHITE)
24
25 #menu buttons y pos
26 starttextY = (screen_height/2)-150
27 optionsY = starttextY+75
28 quitY = optionsY+75
29
30 menuColor1 = BLACK
31 menuColor2 = BLACK
32 menuColor3 = BLACK
```

Then I created the drawStart() function which draws and outputs the start menu buttons and text.

```
50 def drawBG():
51     scaled_bg = pygame.transform.scale(bg_image,(screen_width,screen_height))
52     screen.blit(scaled_bg,(0,0)) #draws the image from the top left of the window
53
54 #draw menu buttons + box around it
55 def drawStart():
56     pygame.draw.rect(screen,menuColor1,(screen_width/2,starttextY,130,60))
57     screen.blit(text1 , (screen_width/2,starttextY))
58     pygame.draw.rect(screen,menuColor2,(screen_width/2,optionsY,130,60))
59     screen.blit(text2 , (screen_width/2,optionsY))
60     pygame.draw.rect(screen,menuColor3,(screen_width/2,quitY,130,60))
61     screen.blit(text3 , (screen_width/2,quitY))
62
```

Error 1

Here is me testing the output.



The text does not fit inside the buttons and the buttons are not central, so the test has failed. I will need to increase the width of the black box and reduce the height of the box so that the text fits better inside the button.

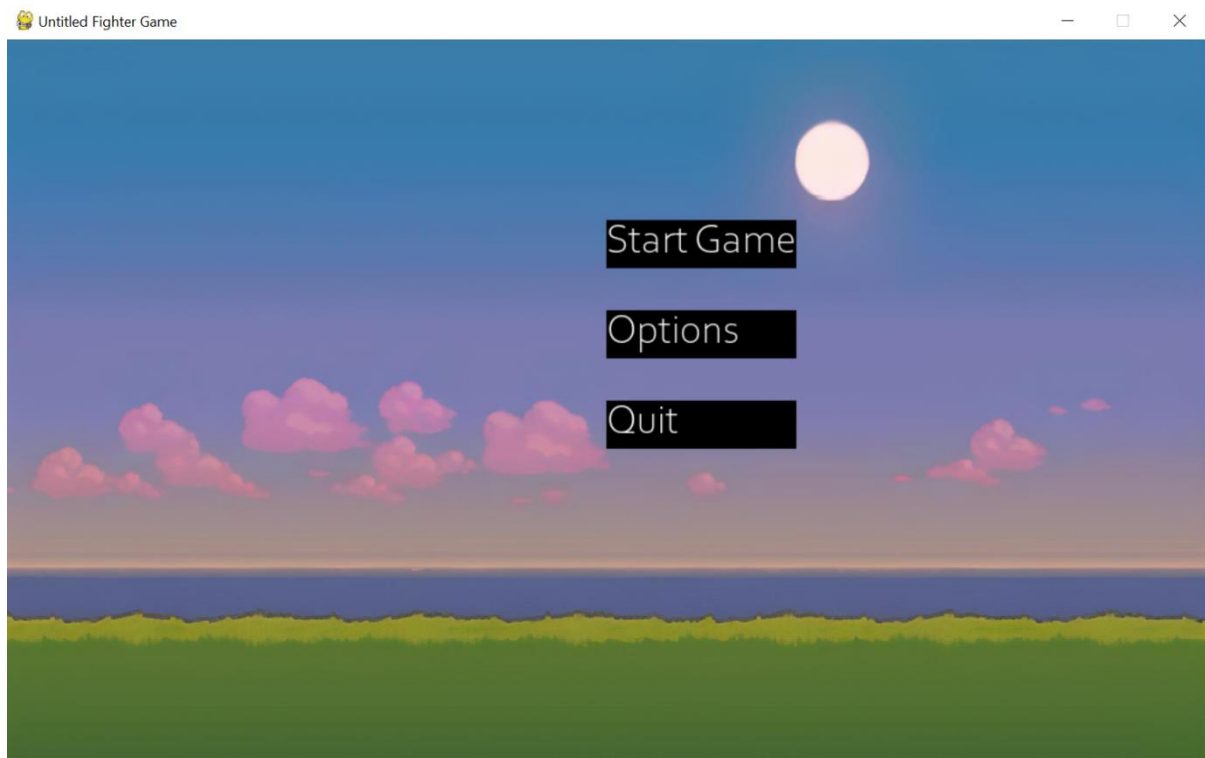
Attempt 2

I increased the width and reduced the height in the code below.

```
54 #draw menu buttons + box around it
55 def drawStart():
56     pygame.draw.rect(screen,menuColor1,(screen_width/2,starttextY,158,40))
57     screen.blit(text1 , (screen_width/2,starttextY))
58     pygame.draw.rect(screen,menuColor2,(screen_width/2,optionsY,158,40))
59     screen.blit(text2 , (screen_width/2,optionsY))
60     pygame.draw.rect(screen,menuColor3,(screen_width/2,quitY,158,40))
61     screen.blit(text3 , (screen_width/2,quitY))
62
```

Error 2

And here is the output.



Each piece of text is fully encompassed by their respective rectangular button. The buttons are on top of each other and equally distanced, but they are not centred in the middle of the screen so I will need to adjust their x positions further.

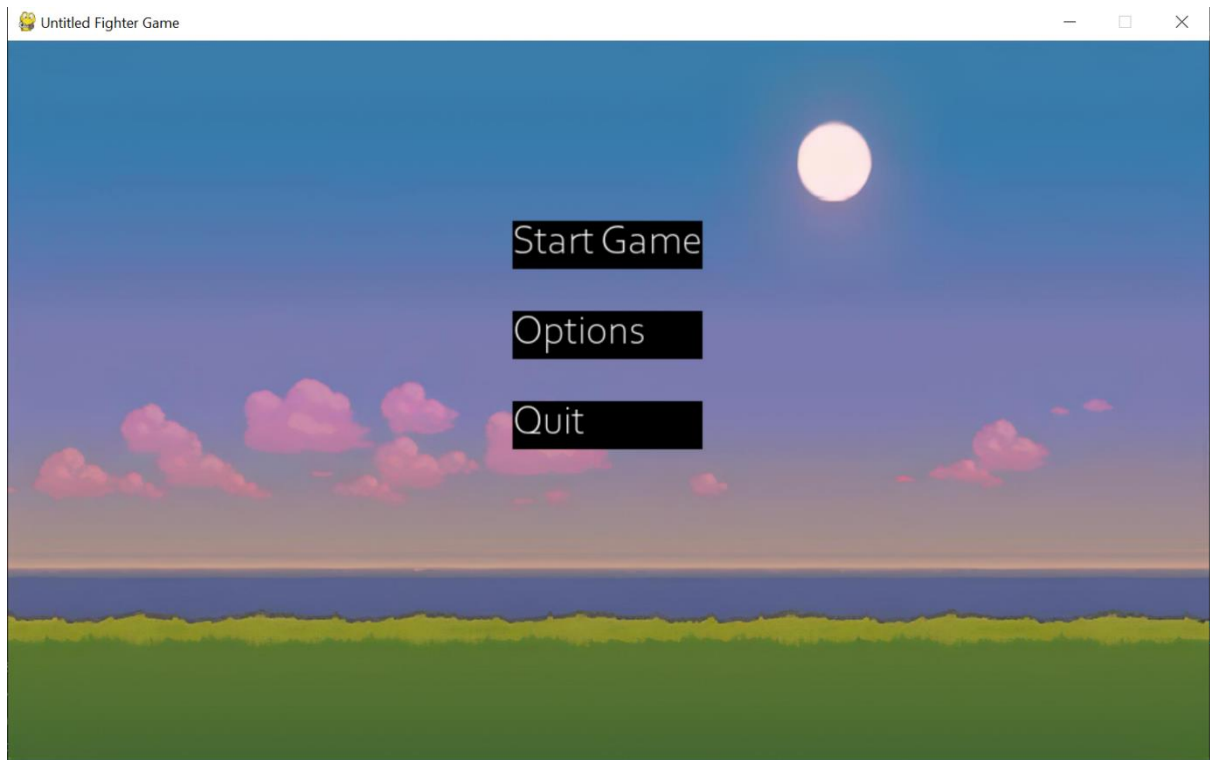
Attempt 3

Here is my attempted readjustment.

```
55 def drawStart():
56     pygame.draw.rect(screen,menuColor1,(screen_width/2-80,starttextY,158,40))
57     screen.blit(text1 , (screen_width/2-80,starttextY))
58     pygame.draw.rect(screen,menuColor2,(screen_width/2-80,optionsY,158,40))
59     screen.blit(text2 , (screen_width/2-80,optionsY))
60     pygame.draw.rect(screen,menuColor3,(screen_width/2-80,quitY,158,40))
61     screen.blit(text3 , (screen_width/2-80,quitY))
62
```

Test

The output is below.



Success Criteria 4,5,6

The test above was successful and I managed to meet success criteria 4,5 and 6 by having all the main menu buttons visible on the screen and centred so that the user can navigate my game with ease.

Success Criteria 3: Game Title

The last addition I have planned for the main menu is the game title which is needed for the user to see what the name of the game they will be playing is.

Attempt 1

The code for the font of the title is below:

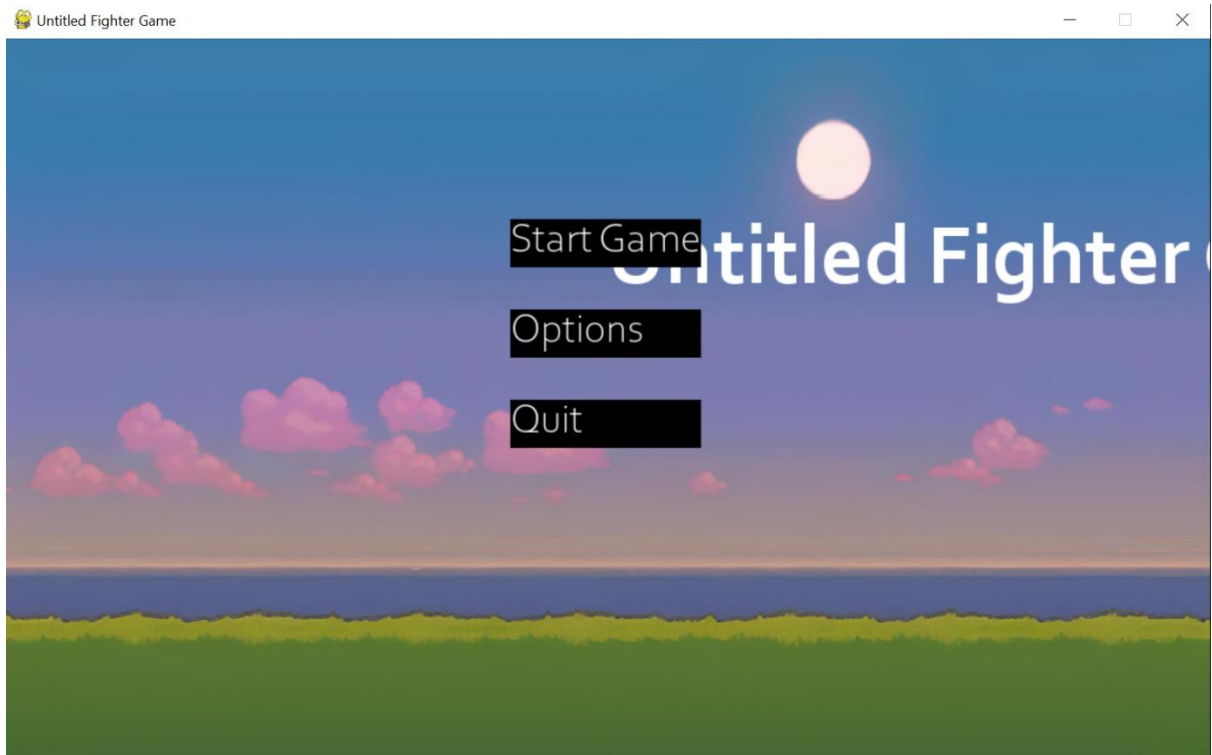
```
18 # defining menu font
19 TitleFont = pygame.font.SysFont('Corbel',70,True)
20 MenuFont = pygame.font.SysFont('Corbel',35 )
21 #render menu font
22 text1 = MenuFont.render('Start Game' , True , WHITE)
23 text2 = MenuFont.render('Options' , True , WHITE)
24 text3 = MenuFont.render('Quit' , True , WHITE)
25 text4 = TitleFont.render('Untitled Fighter Game' , True , WHITE)
26
27
```

I had to amend the drawStart() function to also output the game title with the other main menu buttons:

```
57 #draw menu buttons + box around it
58 def drawStart():
59     screen.blit(text4, (screen_width/2,screen_height/2-150))
60     pygame.draw.rect(screen,menuColor1,(screen_width/2-80,starttextY,158,40))
61     screen.blit(text1 , (screen_width/2-80,starttextY))
62     pygame.draw.rect(screen,menuColor2,(screen_width/2-80,optionsY,158,40))
63     screen.blit(text2 , (screen_width/2-80,optionsY))
64     pygame.draw.rect(screen,menuColor3,(screen_width/2-80,quitY,158,40))
65     screen.blit(text3 , (screen_width/2-80,quitY))
66
```

Error 1

Here is the output.



The title is on the screen and is big in size, but it is not fully visible or centred at all, so this test failed. I will need to change the x and y positions further in my next attempt.

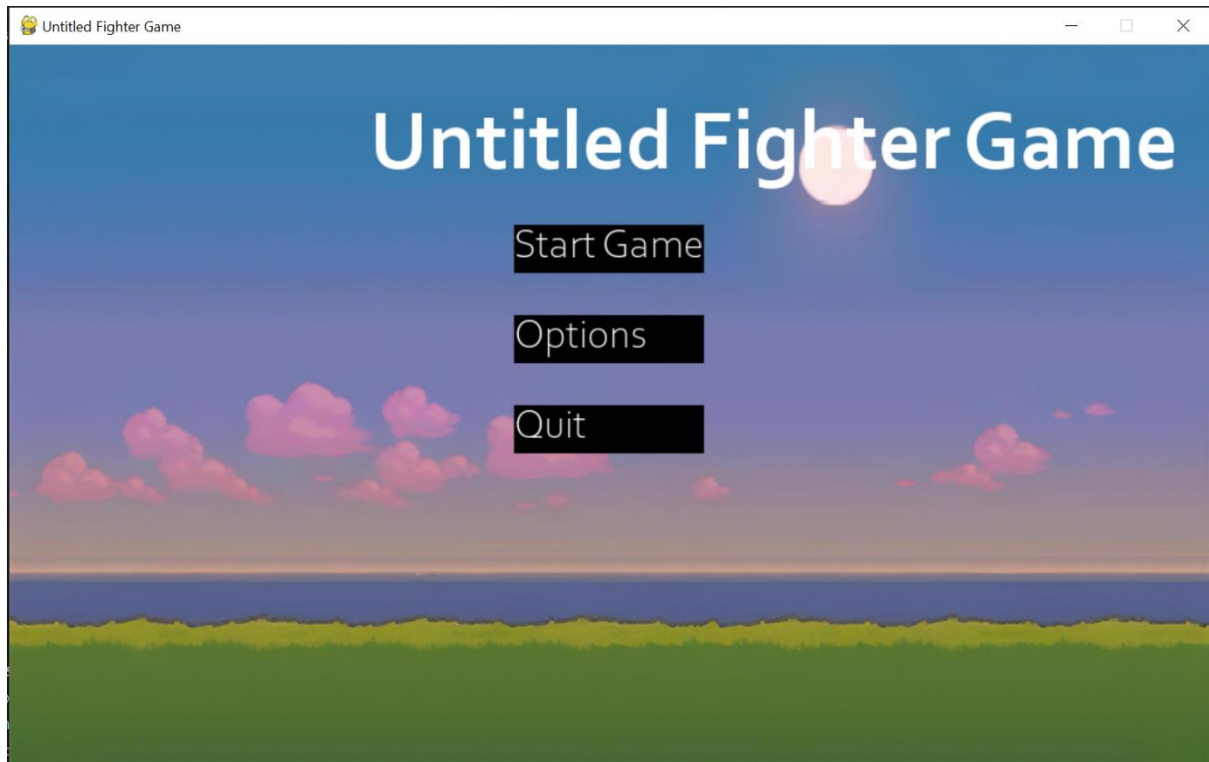
Attempt 2

I lowered the x and y values of the game title text position further:

```
57 #draw menu buttons + box around it
58 def drawStart():
59     screen.blit(text4, ([screen_width/2-200,screen_height/2-250]))
60     pygame.draw.rect(screen,menuColor1,(screen_width/2-80,starttextY,158,40))
61     screen.blit(text1 , (screen_width/2-80,starttextY))
62     pygame.draw.rect(screen,menuColor2,(screen_width/2-80,optionsY,158,40))
63     screen.blit(text2 , (screen_width/2-80,optionsY))
64     pygame.draw.rect(screen,menuColor3,(screen_width/2-80,quitY,158,40))
65     screen.blit(text3 , (screen_width/2-80,quitY))
66
```

Error 2

The output:



The title is still not central so I will need to adjust the x position further in my code.

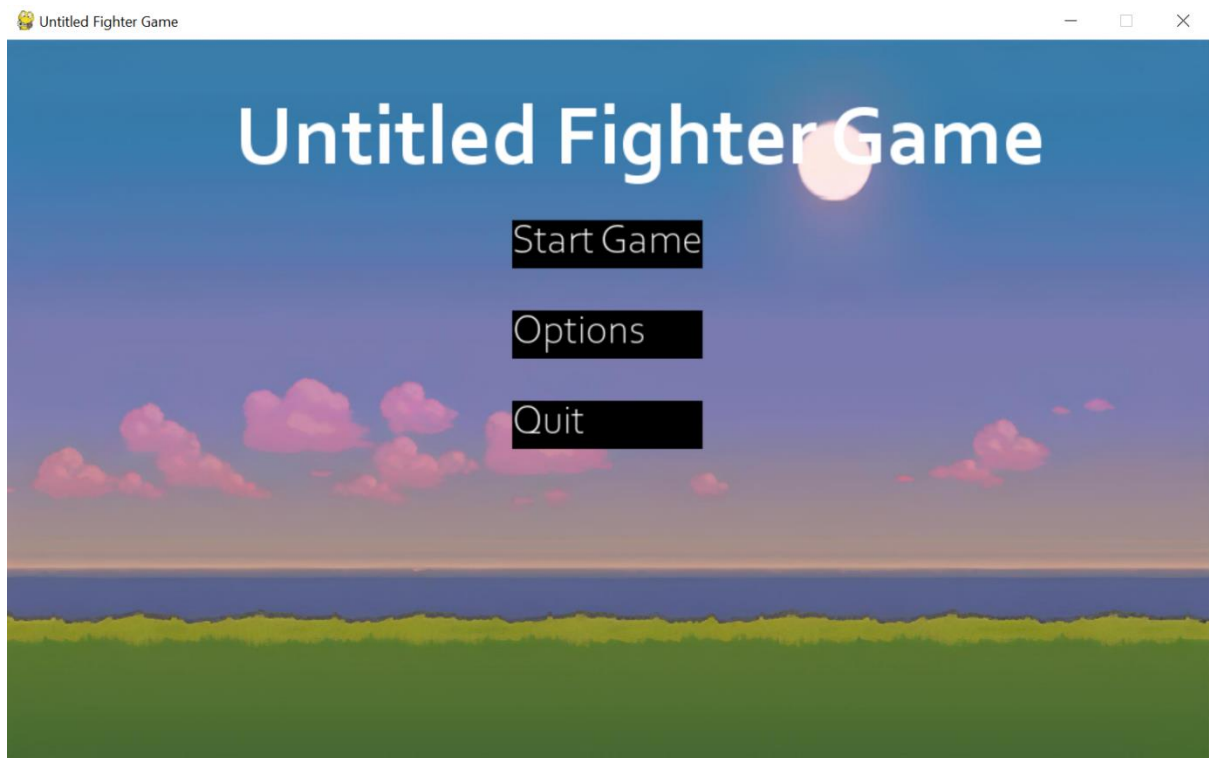
Attempt 3

Here are the adjustments made.

```
57 #draw menu buttons + box around it
58 def drawStart():
59     screen.blit(text4, (screen_width/2-310,screen_height/2-250))
60     pygame.draw.rect(screen,menuColor1,(screen_width/2-80,starttextY,158,40))
61     screen.blit(text1 , (screen_width/2-80,starttextY))
62     pygame.draw.rect(screen,menuColor2,(screen_width/2-80,optionsY,158,40))
63     screen.blit(text2 , (screen_width/2-80,optionsY))
64     pygame.draw.rect(screen,menuColor3,(screen_width/2-80,quitY,158,40))
65     screen.blit(text3 , (screen_width/2-80,quitY))
66
```

Test

Here is the output.



The title is centred on the screen and is on top of the main menu buttons, so this test is a success as it meets the success criteria.

Success Criteria 4: Quit Button

For this success criterion to be fully met, the quit button needs to end the program for the user.

Here is my attempt at making sure the quit button ends the game loop.

```
68 #game loop
69 done = False
70 while not done:
71
72     #get mouse position
73     mouse = pygame.mouse.get_pos()
74
75
76     #events + game logic
77     for event in pygame.event.get():
78         if event.type == pygame.QUIT:
79             done = True
80
81         #logic
82         if event.type == pygame.MOUSEBUTTONDOWN:
83
84             if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and quitY <= mouse[1] <= quitY+50:#PRESSED QUIT
85                 if MainMenu: #CHECKS IF ON MAIN MENU SCREEN
86                     done = True
87
88     #drawing code -----
89     if firstLaunch:
90         MainMenu = True
91         firstLaunch = False
92
93
94     drawBG()
95
```

Test

Here is a video of this code working.



IMG_9032.MOV

The quit button instantly exits the program, so this test was a success.

Success Criteria 5: Play Button

The play button will be used to enter the character selection screen so it will need to be able to remove all the main menu buttons and show the character selection screen (which will be a blank screen at this stage).

My first attempt at implementing this is below.

```
39  #define game variables
40  MainMenu = False
41  firstLaunch = True
42  charSelect = False
43  startBattle = False
44
45

68  P2 = Fighter(700,450)
69  # Used to manage how fast the screen updates
70  clock = pygame.time.Clock()
71  #game loop
72  done = False
73  while not done:
74
75      #get mouse position
76      mouse = pygame.mouse.get_pos()
77
78
79      #events + game logic
80      for event in pygame.event.get():
81          if event.type == pygame.QUIT:
82              done = True
83
84          #logic
85          if event.type == pygame.MOUSEBUTTONDOWN:
86
87              if (screen_width/2)-100 <= mouse[0] <= (screen_width/2)+100 and starttextY <= mouse[1] <= starttextY+75:#pressed start game
88                  if startBattle:#check if game already started
89                      pass
90                  elif MainMenu:
91                      charSelect = True
92                      MainMenu = False
93
94                  if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and quitY <= mouse[1] <= quitY+50:#PRESSED QUIT
95                      if MainMenu: #CHECKS IF ON MAIN MENU SCREEN
96                          done = True
97
98
99
100
101
102  #drawing code -----
103  if firstLaunch:
104      MainMenu = True
105      firstLaunch = False
106
107
108  drawBG()
109
110
111
112  if MainMenu:
113      #draw start menu
114      drawStart()
115
116  if startBattle:
117      P1.draw(screen,YELLOW)
118      P2.draw(screen,YELLOW)
119
120  #update display
121  pygame.display.flip()
122
```

Test

The output is recorded here:



IMG_9395.mp4

As seen in the video, the main menu screen successfully transitions into the character selection screen (which is empty for now) so the test is a success here.

(Should have) Success Criteria 13: Highlighted buttons

Whilst I was testing the output above, it would be appropriate to implement success criteria 13 of the 'Should have' section to make it easier for the ser to know what button they are about to press on each menu.

Attempt 1

In this attempt, I had tried to program the buttons to change from black to blue once the mouse hovers over it.

```
34 menuColor1 = BLACK
35 menuColor2 = BLACK
36 menuColor3 = BLACK
37
38
39
40 #define game variables
41 MainMenu = False
42 firstLaunch = True
43 charSelect = False
44 startBattle = False
45
46
47 #set the game window caption
48 pygame.display.set_caption("Untitled Fighter Game")
49
50 #load background - from: https://wallpapercave.com/landscape-pixel-art-wallpapers
51 bg_image = pygame.image.load("C:\\Users\\Matina\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Images\\ImageBG.jpg").convert_alpha()
52
53 #define background drawing function
54 def drawBG():
55     scaled_bg = pygame.transform.scale(bg_image,(screen_width,screen_height))
56     screen.blit(scaled_bg,(0,0)) #draws the image from the top left of the window
57
58 #draw menu buttons + box around it
59 def drawStart():
60     screen.blit(text4, (screen_width/2-310,screen_height/2-250))
61     pygame.draw.rect(screen,menuColor1,(screen_width/2-80,starttextY,158,40))
62     screen.blit(text1, (screen_width/2-80,starttextY))
63     pygame.draw.rect(screen,menuColor2,(screen_width/2-80,optionsY,158,40))
64     screen.blit(text2, (screen_width/2-80,optionsY))
65     pygame.draw.rect(screen,menuColor3,(screen_width/2-80,quitY,158,40))
66     screen.blit(text3, (screen_width/2-80,quitY))
67
68
69     if (screen_width/2) <= mouse[0] <= (screen_width/2)+50 and starttextY <= mouse[1] <= starttextY+50:
70         menuColor1 = BLUE
71     else:
72         menuColor1 = BLACK
73     if (screen_width/2) <= mouse[0] <= (screen_width/2)+50 and optionsY <= mouse[1] <= optionsY+50:
74         menuColor2 = BLUE
75     else:
76         menuColor2 = BLACK
77     if (screen_width/2) <= mouse[0] <= (screen_width/2)+50 and quitY <= mouse[1] <= quitY+50:
78         menuColor3 = BLUE
79     else:
80         menuColor3 = BLACK
81
```

Error 1

Here is the error that I received once I ran my program.

```

58 #draw menu buttons + box around it
59 def drawStart():
60     screen.blit(text4, (screen_width/2-310,screen_height/2-250))
61     pygame.draw.rect(screen, menuColor1, (screen_width/2-80, starttextY, 158, 40))

Exception has occurred: UnboundLocalError ×
cannot access local variable 'menuColor1' where it is not associated with a value
File "C:\Users\MatinaJ\OneDrive - Dulwich College\Desktop\Untitled Fighter Game\UF61.py", line 61, in drawStart
    pygame.draw.rect(screen, menuColor1, (screen_width/2-80, starttextY, 158, 40))
    ^^^^^^^^^^^^^
File "C:\Users\MatinaJ\OneDrive - Dulwich College\Desktop\Untitled Fighter Game\UF61.py", line 129, in <module>
    drawStart()
UnboundLocalError: cannot access local variable 'menuColor1' where it is not associated with a value

62     screen.blit(text1, (screen_width/2-80, starttextY))
63     pygame.draw.rect(screen, menuColor2, (screen_width/2-80, optionsY, 158, 40))
64     screen.blit(text2, (screen_width/2-80, optionsY))
65     pygame.draw.rect(screen, menuColor3, (screen_width/2-80, quitY, 158, 40))
66     screen.blit(text3, (screen_width/2-80, quitY))
67
68
69     if (screen_width/2) <= mouse[0] <= (screen_width/2)+50 and starttextY <= mouse[1] <= starttextY+50:
70         menuColor1 = BLUE

```

It was an UnboundLocalError that I had experienced, and it was because I did not specify the scope of the variable 'menuColor1'. This led to Python implying that it was a local variable that was undefined in the function, causing an error.

Attempt 2

In this second attempt, I amended my mistake by declaring the scope of each 'menuColor' variable at the start of the 'drawStart' function.

```

57
58 #draw menu buttons + box around it
59 def drawStart():
60     global menuColor1
61     global menuColor2
62     global menuColor3
63     screen.blit(text4, (screen_width/2-310,screen_height/2-250))
64     pygame.draw.rect(screen, menuColor1, (screen_width/2-80, starttextY, 158, 40))
65     screen.blit(text1, (screen_width/2-80, starttextY))
66     pygame.draw.rect(screen, menuColor2, (screen_width/2-80, optionsY, 158, 40))
67     screen.blit(text2, (screen_width/2-80, optionsY))
68     pygame.draw.rect(screen, menuColor3, (screen_width/2-80, quitY, 158, 40))
69     screen.blit(text3, (screen_width/2-80, quitY))
70
71
72     if (screen_width/2) <= mouse[0] <= (screen_width/2)+50 and starttextY <= mouse[1] <= starttextY+50:
73         menuColor1 = BLUE
74     else:
75         menuColor1 = BLACK
76     if (screen_width/2) <= mouse[0] <= (screen_width/2)+50 and optionsY <= mouse[1] <= optionsY+50:
77         menuColor2 = BLUE
78     else:
79         menuColor2 = BLACK
80     if (screen_width/2) <= mouse[0] <= (screen_width/2)+50 and quitY <= mouse[1] <= quitY+50:
81         menuColor3 = BLUE
82     else:
83         menuColor3 = BLACK
84

```

Error 2

I managed to get an output screen and tested my code as seen below.



IMG_9396.mp4

The buttons do go blue but there are some areas where the mouse hovers over the button, but the button does not turn blue. So, in my next attempt I will need to increase the size of the area in which the button turns blue.

Attempt 3

```
58 #draw menu buttons + box around it
59 def drawStart():
60     global menuColor1
61     global menuColor2
62     global menuColor3
63     screen.blit(text4, (screen_width/2-310,screen_height/2-250))
64     pygame.draw.rect(screen,menuColor1,(screen_width/2-80,starttextY,158,40))
65     screen.blit(text1 , (screen_width/2-80,starttextY))
66     pygame.draw.rect(screen,menuColor2,(screen_width/2-80,optionsY,158,40))
67     screen.blit(text2 , (screen_width/2-80,optionsY))
68     pygame.draw.rect(screen,menuColor3,(screen_width/2-80,quitY,158,40))
69     screen.blit(text3 , (screen_width/2-80,quitY))
70
71
72 if (screen_width/2)-125 <= mouse[0] <= (screen_width/2)+100 and starttextY <= mouse[1] <= starttextY+48:
73     menuColor1 = BLUE
74 else:
75     menuColor1 = BLACK
76 if (screen_width/2)-125 <= mouse[0] <= (screen_width/2)+100 and optionsY <= mouse[1] <= optionsY+48:
77     menuColor2 = BLUE
78 else:
79     menuColor2 = BLACK
80 if (screen_width/2)-125 <= mouse[0] <= (screen_width/2)+100 and quitY <= mouse[1] <= quitY+48:
81     menuColor3 = BLUE
82 else:
83     menuColor3 = BLACK
84
```

Error 3

Here is the output that I tested.



IMG_9400.mp4

My mouse is outside of the button, but the buttons still get highlighted, so I will need to reduce the area of which the buttons get highlighted slightly.

Attempt 4

```
58 #draw menu buttons + box around it
59 def drawStart():
60     global menuColor1
61     global menuColor2
62     global menuColor3
63     screen.blit(text4, (screen_width/2-310,screen_height/2-250))
64     pygame.draw.rect(screen,menuColor1,(screen_width/2-80,starttextY,158,40))
65     screen.blit(text1 , (screen_width/2-80,starttextY))
66     pygame.draw.rect(screen,menuColor2,(screen_width/2-80,optionsY,158,40))
67     screen.blit(text2 , (screen_width/2-80,optionsY))
68     pygame.draw.rect(screen,menuColor3,(screen_width/2-80,quitY,158,40))
69     screen.blit(text3 , (screen_width/2-80,quitY))
70
71
72     if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and starttextY <= mouse[1] <= starttextY+48:
73         menuColor1 = BLUE
74     else:
75         menuColor1 = BLACK
76     if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and optionsY <= mouse[1] <= optionsY+48:
77         menuColor2 = BLUE
78     else:
79         menuColor2 = BLACK
80     if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and quitY <= mouse[1] <= quitY+48:
81         menuColor3 = BLUE
82     else:
83         menuColor3 = BLACK
84
```

Test

The output is here in video format.

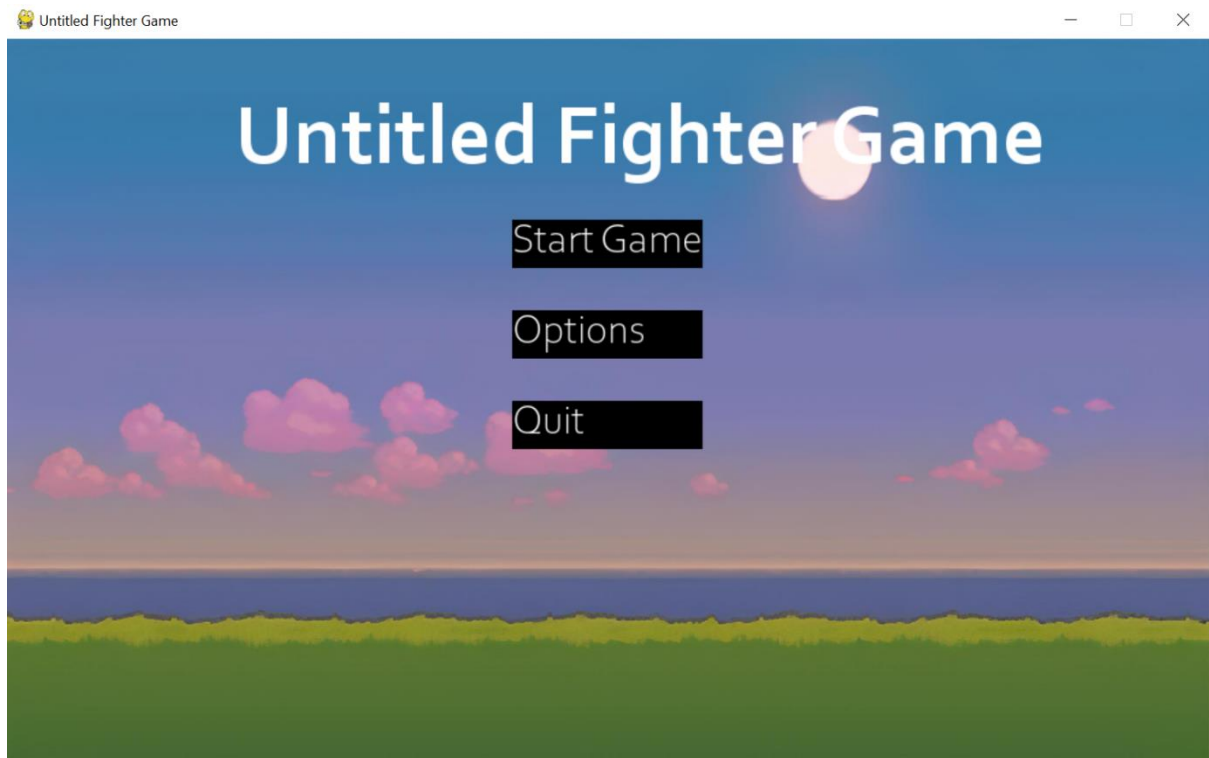


IMG_9660.mov

In the video, I tested the area at which the buttons change colour, and the area precisely matches the area of the actual button, so the test was a success

Main Menu Screen: Review

This is the current look of the main menu screen:



I added a background which is bright and attracts my target audience. This background will also be used in the battle screen when users are fighting each other or the CPU using their chosen character.

The title screen is big in size and centred at the top of the screen so that the user knows the name of the game. The “Start Game” button is the first menu button below the screen and is used to change the screen from the Main Menu to the Character Selection screen.

Next is the “Options” button. Here, the user will be able to configure some elements of the game such as volume and keys used to attack.

Lastly, the “Quit” button is available for users to exit the game when they want to stop playing.

The text inside the menu buttons is simple and it is easy to understand their function. Users will know to click the buttons when the mouse hovers over them and it highlights the button to a blue colour.

My next task will be to create the Battle screen which will be where the user spends the most amount of time in the game fighting either another player or a CPU enemy. I will be creating this screen next because it will be where most of the core functions of my game will be made, and it will be easier to work around this base prototype of my game after this stage is complete.

Success Criteria achieved

- Success Criteria 1: Background image
- Success Criteria 2: Main Menu screen
- Success Criteria 3: Game Title

- Success Criteria 4: Quit Button
- Success Criteria 5: Play Button
- Success Criteria 6: Options Button
- (Should have) Success Criteria 13: Highlighted buttons

Stage Two – Battle Screen

This stage will be the core part of this project since the user will spend most of their time battling with their chosen characters.

Success Criteria 17: Character Image

The user will need to see their character to control and coordinate their movements and attacks. To get the character image on screen, I will firstly need to draw a rectangle which the character will sit inside.

Attempt 1

I firstly created a class for the fighters in a new python file here:

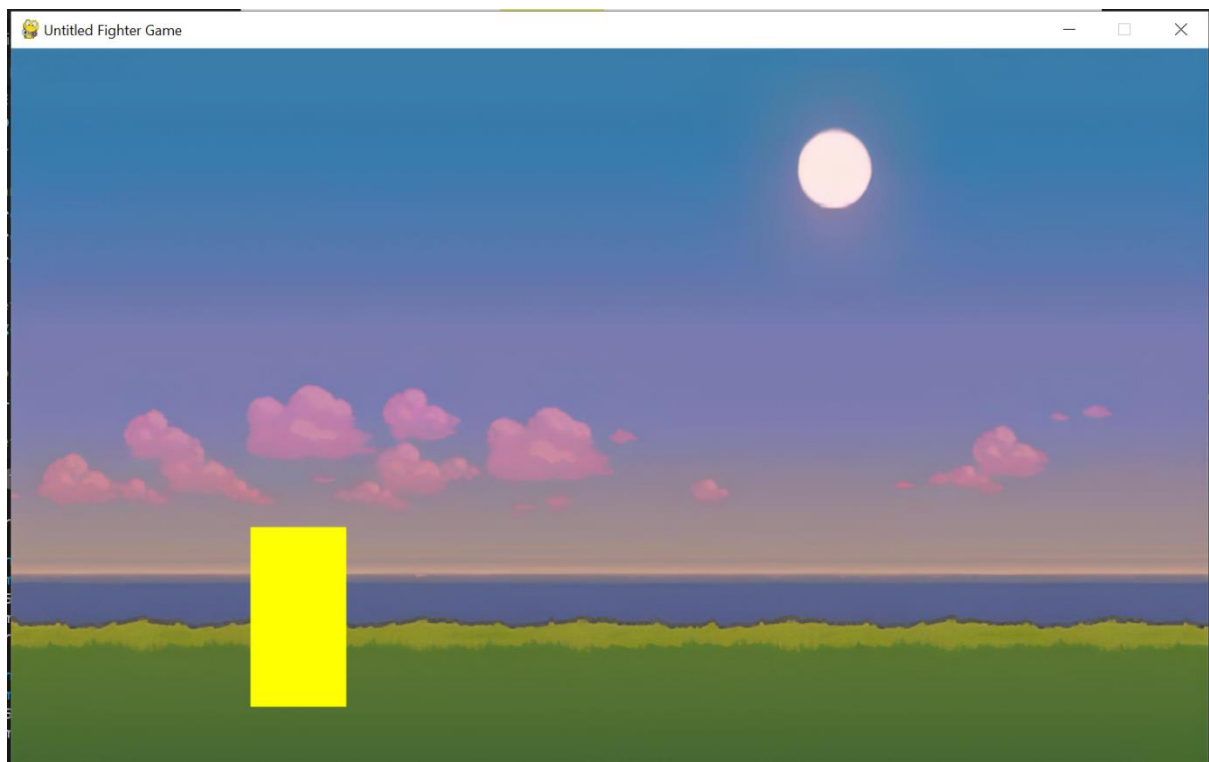
```
1 import pygame
2
3 class Fighter():
4     def __init__(self,x,y):
5         self.Rect = pygame.Rect(x,y,80,150)
6
7
8
9     def draw(self, surface, colour):
10         pygame.draw.rect(surface, colour, self.Rect)
11
12
```

Then I had to test some integers for the initial position of the characters. My first test is below:

```
29 P1 = Fighter(300,500)
30 # Used to manage how fast the screen updates
31 clock = pygame.time.Clock()
32 #game loop
33 done = False
34 while not done:
35     #events
36     for event in pygame.event.get():
37         if event.type == pygame.QUIT:
38             done = True
39     #drawing code -----
40     drawBG()
41     P1.draw(screen,YELLOW)
42     #update display
43     pygame.display.flip()
44
45     #60 frames per second
46     clock.tick(60)
47
48
49 #exit pygame
50 pygame.quit()
```

Test

The output is here:



The rectangle is too far above the ground level and a little too close to the left side of the screen. I lowered the height and adjusted the rectangle's x position in my next attempt.

Attempt 2

My second attempt is below:

```
26     scaled_bg = pygame.transform.scale(bg_image,(screen_width,screen_height))
27     screen.blit(scaled_bg,(0,0)) #draws the image from the top left of the window
28
29     P1 = Fighter([200,400])
30     # Used to manage how fast the screen updates
31     clock = pygame.time.Clock()
32     #game loop
33     done = False
34     while not done:
35         #events
36         for event in pygame.event.get():
37             if event.type == pygame.QUIT:
38                 done = True
39         #drawing code -----
40         drawBG()
41         P1.draw(screen,YELLOW)
42         #update display
43         pygame.display.flip()
44
45         #60 frames per second
46         clock.tick(60)
47
48
49 #exit pygame
50 pygame.quit()
```

Test

The output is below:



Here, the rectangle is below the bottom of the screen but it is in an acceptable x position, so my next attempt only requires further adjusting to the y position.

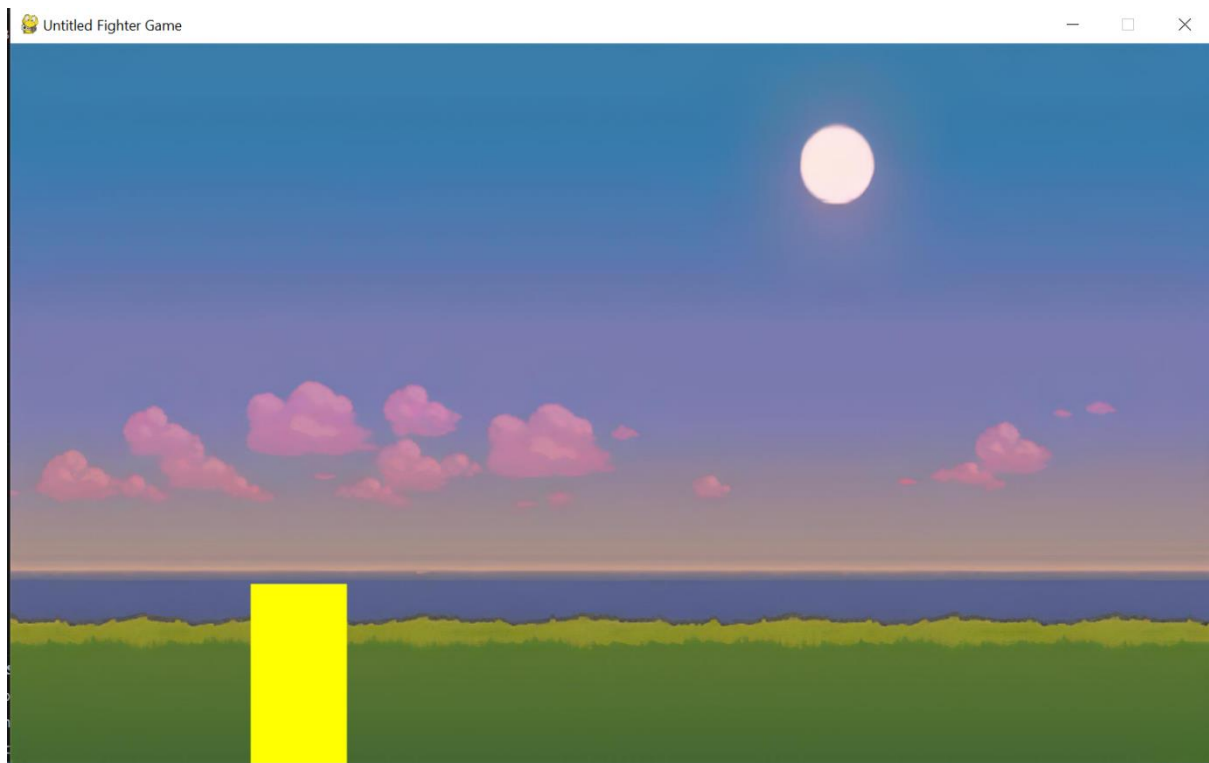
Attempt 3

My third attempt is here:

```
29 P1 = Fighter(200,450)
30 # Used to manage how fast the screen updates
31 clock = pygame.time.Clock()
32 #game loop
33 done = False
34 while not done:
35     #events
36     for event in pygame.event.get():
37         if event.type == pygame.QUIT:
38             done = True
39     #drawing code -----
40     drawBG()
41     P1.draw(screen,YELLOW)
42     #update display
43     pygame.display.flip()
44
45     #60 frames per second
46     clock.tick(60)
47
48
49 #exit pygame
50 pygame.quit()
```

Test

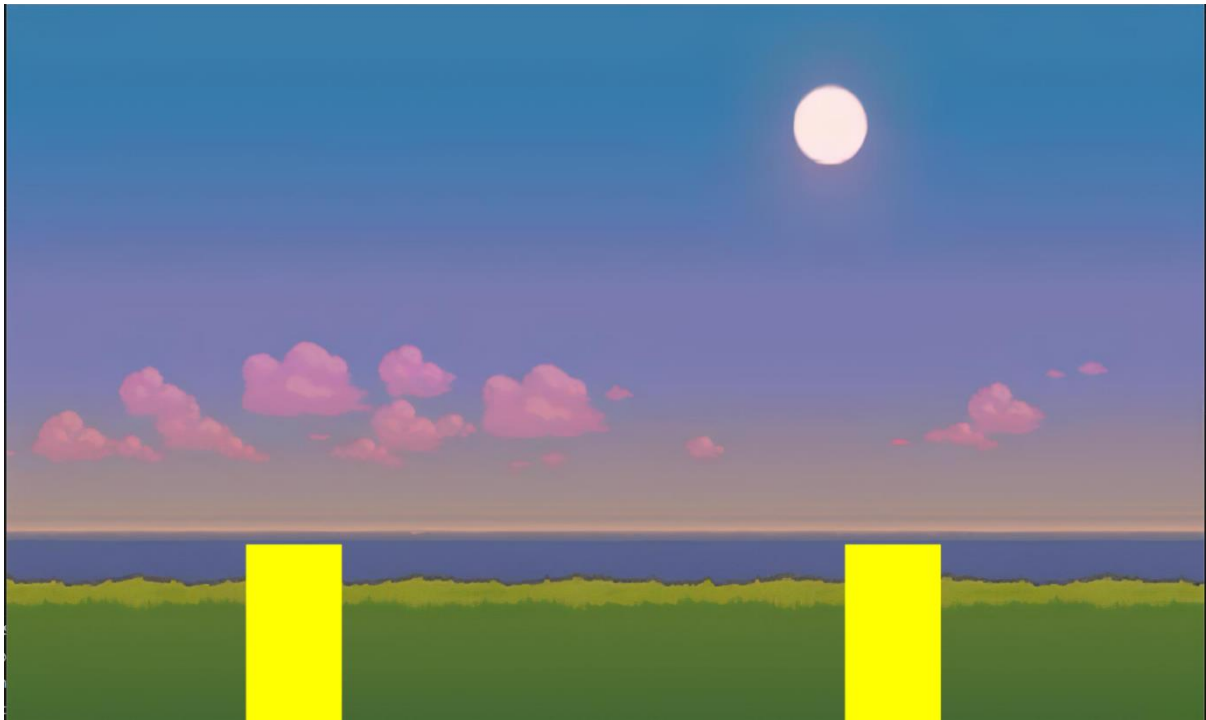
The output is here:



The rectangle was successfully outputted on the ground.

```
29 P1 = Fighter(200,450)
30 P2 = Fighter(700,450)
31 # Used to manage how fast the screen updates
32 clock = pygame.time.Clock()
33 #game loop
34 done = False
35 while not done:
36     #events
37     for event in pygame.event.get():
38         if event.type == pygame.QUIT:
39             done = True
40     #drawing code -----
41     drawBG()
42     P1.draw(screen,YELLOW)
43     P2.draw(screen,YELLOW)
44     #update display
45     pygame.display.flip()
46
47     #60 frames per second
48     clock.tick(60)
49
50
51 #exit pygame
52 pygame.quit()
```

I used the same successful y position that I got from the previous test and added 500 to the x position so the second character is expected to be on the other side of the screen.



The character boxes are now clearly at either of the battle screen and this sets me up to now add the character images on the screen.

To output the character image, I will need to first create an animation list and an algorithm to iterate through different frames of an animation. I will need to visually count the number of steps in an animation then use this as an input to the algorithm.



Here are the sprite sheet variables for both characters:

```
54 #load sprite sheets - from: https://craftpix.net/freebies/free-shinobi-sprites-pixel-art/
55 shinobi_spritesheet = pygame.image.load("C:\\Users\\Matina\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Images\\spritesheet (1).png").convert_alpha()
56 shinobi_attacksheet = pygame.image.load("C:\\Users\\Matina\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Images\\ShinobiAttackSheet.png").convert_alpha()
57 samurai_spritesheet = pygame.image.load("C:\\Users\\Matina\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Images\\samuraisheet.png").convert_alpha()
```

I used the Photos app on Windows to determine the width and height of each animation by dividing the width by the number of animations to determine the width and height of one animation step. I then selected 3 as the scale up and selected an offset as well:

```
46 # fighter variables
47 Shinobi_Size = 128 #size of animation image in pixels
48 ShinobiScale = 3 #scaling up image size to be visible
49 ShinobiOffset = [48, 78] #offsetting original blitted image to be on the ground
50 ShinobiData = [Shinobi_Size, ShinobiScale, ShinobiOffset]
51 Samurai_size = 128
52 SamuraiScale = 3
53 SamuraiOffset = [52, 78]
54 SamuraiData = [Samurai_size, SamuraiScale, SamuraiOffset]
```

The animation steps for each animation are below:

```
68
69
70 #num of animation steps (Idle,Run,Jump, Attack1, Attack2, Attack3, Hurt, Dead)
71 shinobi_Animationsteps = (6,8,10,6,3,4,6,3)
72
73 samurai_Animationsteps = (6,8,12,6,4,4,5,3)
```

Added the extra parameters needed in the Fighter class for Player 1 and Player 2:

```
106
107 P1 = Fighter(200,450, ShinobiData, shinobi_spritesheet,shinobi_Animationsteps)
108 P2 = Fighter(700,450,SamuraiData, samurai_Animationsteps, samurai_Animationsteps)
109 # Used to manage how fast the screen updates
```

Error 1

I then ran the code and received this error:

```
18 def load_images(self, sprite_sheet, animationsteps):
19     animationlist = []
20     for animation in animationsteps:
21         temp_img_list = []
22         for x in range(animation):
23             temp_img = sprite_sheet.subsurface(x*self.size, x*self.size, self.size, self.size)

Exception has occurred: ValueError
subsurface rectangle outside surface area

File "C:\Users\Matina\OneDrive - Dulwich College\Desktop\Untitled Fighter Game\UFGfighter.py", line 23, in load_images
    temp_img = sprite_sheet.subsurface(x*self.size, x*self.size, self.size, self.size)

File "C:\Users\Matina\OneDrive - Dulwich College\Desktop\Untitled Fighter Game\UFGfighter.py", line 8, in __init__
    self.animationlist = self.load_images(sprite_sheet, animationsteps)

File "C:\Users\Matina\OneDrive - Dulwich College\Desktop\Untitled Fighter Game\UFG1.py", line 107, in <module>
    P1 = Fighter(200,450, ShinobiData, shinobi_spritesheet,shinobi_Animationsteps)
ValueError: subsurface rectangle outside surface area

24         temp_img_list.append(pygame.transform.scale(temp_img, (self.size *self.image_scale, self.size*self.image_scale)))
25     animationlist.append(temp_img_list)
26     return animationlist
27
```

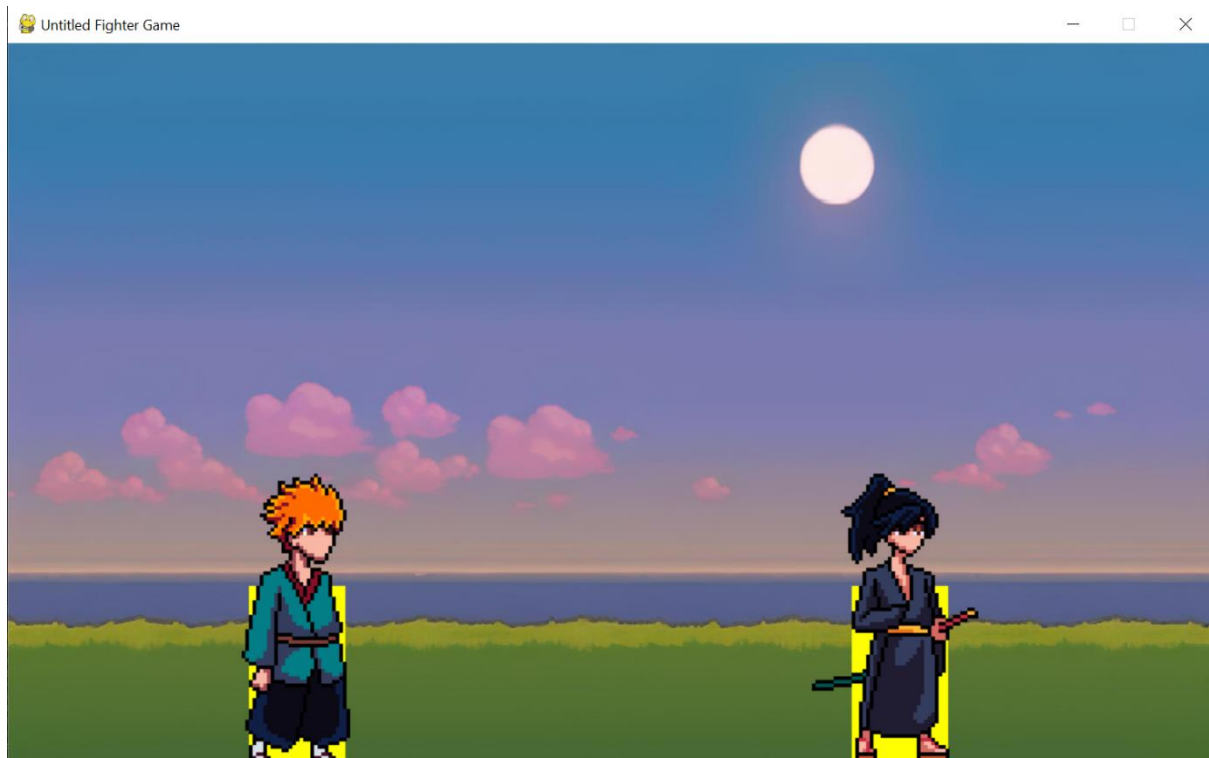
Looking back at my previous code, I realised that I had put 'samurai_Animationsteps' twice as a parameter for P2 instance of Fighter class.

Attempt 4

I corrected the parameters entered for the P2 Fighter here:

```
107 P1 = Fighter(200,450, ShinobiData, shinobi_spritesheet,shinobi_Animationsteps)
108 P2 = Fighter(700,450,SamuraiData, samurai_spritesheet, samurai_Animationsteps)
109 # Used to manage how fast the screen updates
```

Then I ran the code once more and received this output:



This is a successful test as the character images are visible and on the ground. A further improvement that I can make is to flip the samurai character so that both characters are facing each other, which will allow attacks to be visually correct later in development. I should also remove the yellow hitbox in the process.

Attempt 5

I added a flip parameter and a flip attribute to the Fighter class.

```
1  import pygame
2
3  class Fighter():
4      def __init__(self,x,y,flip, data, sprite_sheet, animationsteps):
5          self.size = data[0]
6          self.image_scale = data[1]
7          self.offset = data[2]
8          self.flip = flip
9          self.animationlist = self.load_images(sprite_sheet, animationsteps)
10         self.rect = pygame.Rect(x,y,80,150)
11         self.action = 0 #0
12         self.frame_index = 0
13         self.image = self.animationlist[self.action][self.frame_index]
14
15
16
```

I then updated the draw() procedure which now flips the image depending on the Boolean attribute self.flip.

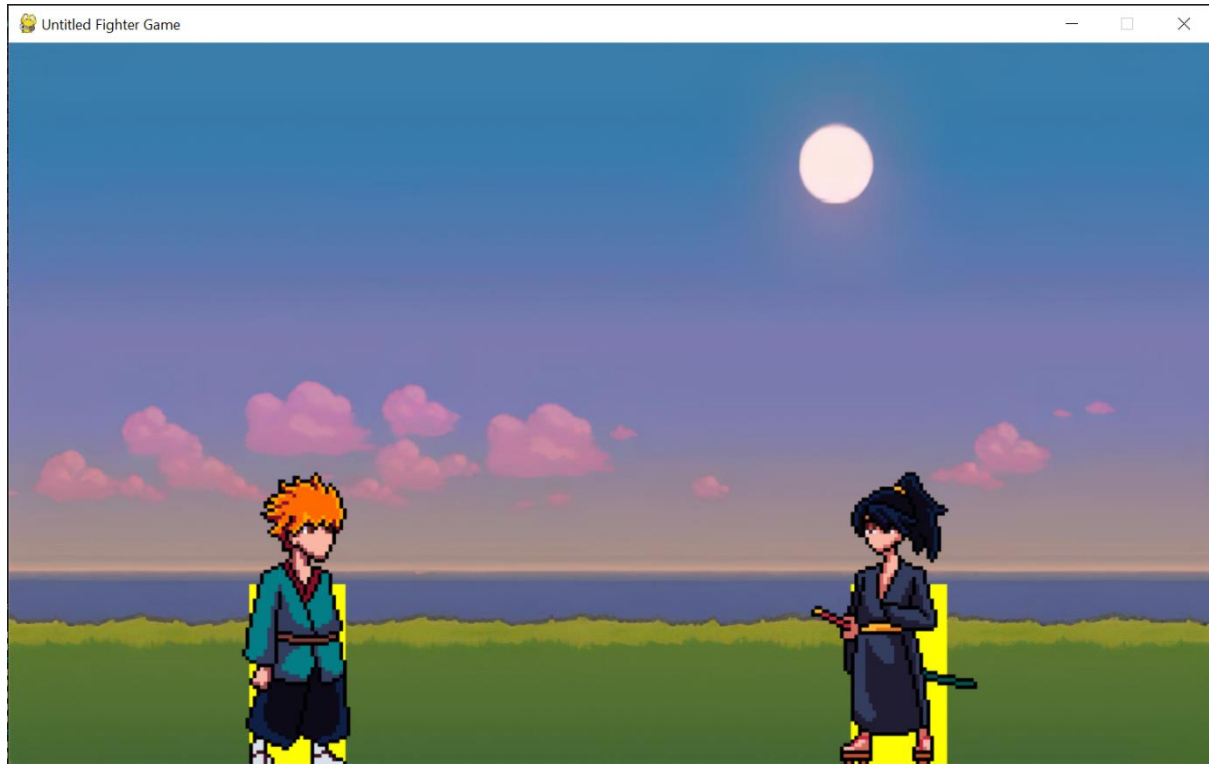
```
53
54
55
56     def draw(self, surface, colour):
57         pygame.draw.rect(surface, colour, self.rect)
58         img = pygame.transform.flip(self.image, self.flip, False)
59
60         surface.blit(img, (self.rect.x - (self.offset[0]*self.image_scale), self.rect.y - (self.offset[1] * self.image_scale)))
61
62
63
```

I updated the arguments given to P1 and P2 so that the flip parameter is given an input.

```
106
107     P1 = Fighter(200,450, False, ShinobiData, shinobi_spritesheet,shinobi_Animationsteps)
108     P2 = Fighter(700,450,True, SamuraiData, samurai_spritesheet, samurai_Animationsteps )
109
110
```

Test

After these changes, I ran my code and observed the changes.



The characters are now successfully facing each other, meaning that success criteria 16 & 17 have been fully achieved as both sprites are visible on the screen for the user to easily control. It is now time to incorporate those character animations and movements.

Success Criteria 18 & 23: Character Idle Animation

To first enable idle animations, I will need to update the character animation and ensure that no other animation is being currently prioritised.

As the idle animation is the only animation being focused on right now, there will be no conditions in my update() procedure in terms of prioritising animations.

Attempt 1

The current update method is below:

```
64     def update(self):
65         self.update_action(0)
66
67         animation_cooldown = 60 #milliseconds until next frame
68
69         if pygame.time.get_ticks() - self.updateTime > animation_cooldown: # if time passed has exceeded animation period, go to next frame
70             self.frame_index += 1
71             self.updateTime = pygame.time.get_ticks()
72         if self.frame_index >= len(self.animationlist[self.action])-1: #if last frame has been reached, go back to first frame
73             self.frame_index = 0
74
75         self.image = self.animationlist[self.action][self.frame_index] #current frame to be drawn
```

The update_action() method is below:

```
66
67
68     def update_action(self, newAction):
69
70         #check if new action is different to previous
71         if newAction != self.action:
72             self.action = newAction
73
74         #update animation settings
75         self.frame_index = 0
76         self.updateTime = pygame.time.get_ticks()
```

I then added the update method in the main game loop:

```
158         if startBattle:
159             P1.draw(screen,YELLOW)
160             P2.draw(screen,YELLOW)
161
162             P1.update()
163             P2.update()
164
```

Test

Now here is me testing the code:



IMG_9652.mov

The video shows both character idle animations playing and looping over the frames, meaning success criteria 18 & 23 have been fully achieved and I can now move on to character movement.

Success Criteria 19 & 24: Character Movement Function & Animation

Attempt 1

I firstly removed the yellow hitboxes behind each character so that I could see the animations clearer.

```
57     def draw(self, surface, colour):
58         #pygame.draw.rect(surface, colour, self.rect)
59         img = pygame.transform.flip(self.image, self.flip, False)
60
61         surface.blit(img, (self.rect.x - (self.offset[0]*self.image_scale), self.rect.y - (self.offset[1] * self.image_scale)))
62
```

To have movement for both players, I will need a variable to differentiate between Player 1 movement and Player 2 movement. This can eventually be used to determine the keyboard keys that will trigger attack animations for Player 1 and 2.

The parameter and attribute are added to the Fighter class here.

```
2
3 class Fighter():
4     def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1):
5         self.size = data[0]
6         self.image_scale = data[1]
7         self.offset = data[2]
8         self.flip = flip
9         self.animationlist = self.load_images(sprite_sheet, animationsteps)
10        self.rect = pygame.Rect(x,y,80,150)
11        self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
12        self.frame_index = 0
13        self.image = self.animationlist[self.action][self.frame_index]
14        self.updateTime = pygame.time.get_ticks()
15        self.player = Player1
16
```

I then used this attribute in the move method to differentiate the keys that will move Player 1 (A and D) and Player 2 (< and >). I set the speed to 15 and update the position of the characters at the end of the procedure.

```
36
37 def move(self):
38     speed = 15
39     dx = 0 #positive = going forward, negative = going backwards
40
41
42     #get key pressed on keyboard
43     key = pygame.key.get_pressed()
44
45     if self.player:
46
47         if key[pygame.K_a]:
48             dx = -speed # change in speed becomes negative, moves char backwards
49         if key[pygame.K_d]:
50             dx = speed
51     else:
52         if key[pygame.K_LEFT]:
53             dx = -speed # change in speed becomes negative, moves char backwards
54         if key[pygame.K_RIGHT]:
55             dx = speed
56
57
58
59     #update player position
60     self.rect.x += dx
61
62
```

In the main game program, I had updated the argument to include the Boolean variable for Player1.

```
106
107 P1 = Fighter(200,450, False, ShinobiData, shinobi_spritesheet,shinobi_Animationsteps, True)
108 P2 = Fighter(700,450,True, SamuraiData, samurai_spritesheet, samurai_Animationsteps, False)
109
```

I added the move() method in the main game loop

```
157
158     if startBattle:
159         P1.draw(screen,YELLOW)
160         P2.draw(screen,YELLOW)
161
162         P1.move()
163         P2.move()
164
165         P1.update()
166         P2.update()
167
168     #update display
169     pygame.display.flip()
170
171     #60 frames per second
172     clock.tick(60)
173
174
```

Test

The video test is here.



IMG_9665.mov

The characters clearly move along the ground forwards and backwards. All that is left to do to complete success criteria 19 & 24 is to add the animations for these movements.

Attempt 2

I added a running attribute to the Fighter class:

```
3  class Fighter():
4      def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1):
5          self.size = data[0]
6          self.image_scale = data[1]
7          self.offset = data[2]
8          self.flip = flip
9          self.animationlist = self.load_images(sprite_sheet, animationsteps)
10         self.rect = pygame.Rect(x,y,80,150)
11         self.running = False
12         self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
13         self.frame_index = 0
14         self.image = self.animationlist[self.action][self.frame_index]
15         self.updateTime = pygame.time.get_ticks()
16         self.player = Player1
17
```

I then used this attribute to determine whether the running animation should play if the player is running, else the idle animation should play:

```
73     def update(self):
74         self.update_action(0)
75         if self.running:
76             self.update_action(1)
77
78         animation_cooldown = 60 #milliseconds until next frame
79
80         if pygame.time.get_ticks() - self.updateTime > animation_cooldown: # if time passed has exceeded animation period, go to next frame
81             self.frame_index += 1
82             self.updateTime = pygame.time.get_ticks()
83             if self.frame_index >= len(self.animationlist[self.action])-1: #if last frame has been reached, go back to first frame
84                 self.frame_index = 0
85
86         self.image = self.animationlist[self.action][self.frame_index] #current frame to be drawn
87
88
89
```

I also added a trigger for the running attribute to become true when the movement buttons are being used by the user.

```
38     def move(self):
39         speed = 15
40         dx = 0 #positive = going forward, negative = going backwards
41
42
43         #get key pressed on keyboard
44         key = pygame.key.get_pressed()
45
46         if self.player:
47
48             if key[pygame.K_a]:
49                 dx = -speed # change in speed becomes negative, moves char backwards
50                 self.running = True
51             if key[pygame.K_d]:
52                 dx = speed
53                 self.running = True
54         else:
55             if key[pygame.K_LEFT]:
56                 dx = -speed # change in speed becomes negative, moves char backwards
57                 self.running = True
58             if key[pygame.K_RIGHT]:
59                 dx = speed
60                 self.running = True
61
62
```

Test

Here is a video of me testing the code above.



IMG_9666.mov

The test failed here because the characters do not show any running animation when the movement buttons are pressed. In fact, when the player stops moving, the idle animation doesn't continue playing either, suggesting there is a problem with animation triggers.

Attempt 3

I figured that there was no deactivation of the running attribute so after being set to True, it would never go back to being False when the character is idle. To fix this, I set the running attribute to False initially in the move method so that the idle animation could play as well.

```
37
38 def move(self):
39     self.running = False
40     speed = 15
41     dx = 0 #positive = going forward, negative = going backwards
42
43
44     #get key pressed on keyboard
45     key = pygame.key.get_pressed()
46
47     if self.player:
48
49         if key[pygame.K_a]:
50             dx = -speed # change in speed becomes negative, moves char backwards
51             self.running = True
52         if key[pygame.K_d]:
53             dx = speed
54             self.running = True
55     else:
56         if key[pygame.K_LEFT]:
57             dx = -speed # change in speed becomes negative, moves char backwards
58             self.running = True
59         if key[pygame.K_RIGHT]:
60             dx = speed
61             self.running = True
```

I also changed the update method so that the idle animation is encompassed in an if/else statement.

```
77 def update(self):
78     if self.running:
79         self.update_action(1)
80     else:
81         self.update_action(0)
82
83
84     animation_cooldown = 60 #milliseconds until next frame
85
86     if pygame.time.get_ticks() - self.updateTime > animation_cooldown: # if time passed has exceeded animation period, go to next frame
87         self.frame_index += 1
88         self.updateTime = pygame.time.get_ticks()
89     if self.frame_index >= len(self.animationlist[self.action])-1: #if last frame has been reached, go back to first frame
90         self.frame_index = 0
91
92     self.image = self.animationlist[self.action][self.frame_index] #current frame to be drawn
93
94
95
```

Test

Here is a video of me testing the updated code.



IMG_9667.mov

Both characters display their movement animations correctly when moving and this movement animation stops when the characters stop moving. This means that success criteria 19 & 24 have been fully achieved, allowing the user to see clearly when they are moving forwards or backwards.

Extra fix 1: Flipping the characters

However, a new problem has been created which will need to be solved now as it may cause issues down the line of development.

The characters face away from each other when they have gone beyond each other's x position. This is a problem because the user will not be able to attack the other player when behind them, thus limiting the usability of the game.



To fix this, I will need to use the flip attribute I had created in order to flip both characters once they have gone past each other's x positions.

Attempt 1

I first added a target parameter into the move method, and I ensured the Players flip if one player goes behind another player.

```

38     def move(self, target):
39         self.running = False
40         speed = 15
41         dx = 0 #positive = going forward, negative = going backwards
42
43
44         #get key pressed on keyboard
45         key = pygame.key.get_pressed()
46
47         if self.player:
48
49             if key[pygame.K_a]:
50                 dx = -speed # change in speed becomes negative, moves char backwards
51                 self.running = True
52             if key[pygame.K_d]:
53                 dx = speed
54                 self.running = True
55         else:
56             if key[pygame.K_LEFT]:
57                 dx = -speed # change in speed becomes negative, moves char backwards
58                 self.running = True
59             if key[pygame.K_RIGHT]:
60                 dx = speed
61                 self.running = True
62
63
64
65         #ensure player faces each other
66         if target.rect.centerx > self.rect.centerx:
67             self.flip = False
68         else:
69             self.flip = True
70
71
72         #update player position
73         self.rect.x += dx
74

```

I added the argument for this new target parameter in the main game loop and subsequently ran the code.

```

157
158     if startBattle:
159         P1.draw(screen, YELLOW)
160         P2.draw(screen, YELLOW)
161
162         P1.move(P2)
163         P2.move(P1)
164
165         P1.update()
166         P2.update()
167
168         #update display
169         pygame.display.flip()
170
171         #60 frames per second
172         clock.tick(60)
173
174
175 #exit pygame
176 pygame.quit()

```

Test

I ran the code tested if the characters correctly flipped.



IMG_9669.mov

This test was a great success as now the characters now face each other when one goes behind another.

Extra fix 2: Keeping Characters on Screen

Another problem that arose was that the characters could now go off screen if the user runs past the left and right borders of the screen window. This is a problem because it will not allow the user to see their chosen character that they need to use to defeat their opponent. Equally, the opponent will not be able to see the player's character which does not allow them to attempt to defeat the player.

Here is a video of this problem:



IMG_9670.mov

To fix this, I will need to set boundaries for both characters on the Battle screen so that they do not go outside of the screen and are visible for the user to see and use.

Attempt 1

I first used conditional statements on the left and right side of the sprite so that the sprite stays on screen if it attempts to move out of the screen borders.

```
62
63     #check player on screen
64     if self.rect.left + dx < 0: #if the speed and position of character beyond left screen border
65         dx = -self.rect.left #limit character speed to left border
66     if self.rect.right + dx > screen_width: #if the speed and position of character beyond right screen border
67         dx = screen_width - self.rect.right #limit character speed to right border
68
69
70
```

I then had to add the screen_width as a parameter in the move() function.

```
40
41     def move(self, screen_width, target):
42         self.running = False
43         speed = 15
44         dx = 0 #positive = going forward, negative = going backwards
```

Error

When running the code, I received an error in the main game loop:

```
151
152
153
154     if MainMenu: MainMenu = False
155         #draw start menu
156         drawStart()
157
158     if startBattle: startBattle = True
159         P1.draw(screen,YELLOW)
160         P2.draw(screen,YELLOW) screen = <Surface(1000x600x32 SW)>, YELLOW = (255, 255, 0)
161
162     P1.move(P2) P2 = <UFGfighter.Fighter object at 0x00000169FC987350>
Exception has occurred: TypeError X
Fighter.move() missing 1 required positional argument: 'target'
File "C:\Users\MatinaJ\OneDrive - Dulwich College\Desktop\Untitled Fighter Game\UFG1.py", line 162, in <module>
    P1.move(P2)
TypeError: Fighter.move() missing 1 required positional argument: 'target'
163
164     P2.move(P1)
165
166     P1.update()
167     P2.update()
168
169     #update display
170     pygame.display.flip()
171
172     #60 frames per second
173     clock.tick(60)
174
175     #exit pygame
176     pygame.quit()
```

I realised I had forgotten to add the argument for the screen_width in the main game loop for the move() function.

Attempt 2

I amended my mistake by correctly adding the screen_width argument in to the move() function called in the main game loop.

```
157
158     if startBattle:
159         P1.draw(screen,YELLOW)
160         P2.draw(screen,YELLOW)
161
162         P1.move(screen_width,P2)
163         P2.move(screen_width, P1)
164
165         P1.update()
166         P2.update()
167
168     #update display
169     pygame.display.flip()
170
171     #60 frames per second
172     clock.tick(60)
173
174
175 #exit pygame
176 pygame.quit()
```

Test

Here is a video of the code working.



IMG_9696.mov

As seen, the character now cannot leave the screen, so this fix is complete.

Success Criteria 20 & 25: Character Jump Function & Animation

Now that the characters can move forwards and backwards, I need to be able to allow them to jump which will help the users evade attacks from the enemy (as well as moving backwards)

Attempt 1

I first need to create a jump Boolean variable to signify when the character is jumping.

```
1 import pygame
2
3 class Fighter():
4     def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1):
5         self.size = data[0]
6         self.image_scale = data[1]
7         self.offset = data[2]
8         self.flip = flip
9         self.animationlist = self.load_images(sprite_sheet, animationsteps)
10        self.rect = pygame.Rect(x,y,80,150)
11        self.running = False
12        self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
13        self.frame_index = 0
14        self.image = self.animationlist[self.action][self.frame_index]
15        self.updateTime = pygame.time.get_ticks()
16        self.player = Player1
17        self.jump = False
18
```

I also needed a y velocity to show how far up the character will go when jumping.

```
3 class Fighter():
4     def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1):
5         self.size = data[0]
6         self.image_scale = data[1]
7         self.offset = data[2]
8         self.flip = flip
9         self.animationlist = self.load_images(sprite_sheet, animationsteps)
10        self.rect = pygame.Rect(x,y,80,150)
11        self.running = False
12        self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
13        self.frame_index = 0
14        self.image = self.animationlist[self.action][self.frame_index]
15        self.updateTime = pygame.time.get_ticks()
16        self.player = Player1
17        self.jump = False
18        self.vel_y = 0
19
20
```

I then added the keys that control jumping (W for player 1, ^ for player 2) into the move() function, ensuring that the jump Boolean variable is set to True when this key is pressed. I set the velocity to -30 initial and will see if this works when testing.

```
49
50     if self.player:
51
52         if key[pygame.K_a]:
53             dx = -speed # change in speed becomes negative, moves char backwards
54             self.running = True
55         if key[pygame.K_d]:
56             dx = speed
57             self.running = True
58         if key[pygame.K_w]:
59             self.vel_y = -30 #makes player jump up
60             self.jump = True
61     else:
62         if key[pygame.K_LEFT]:
63             dx = -speed # change in speed becomes negative, moves char backwards
64             self.running = True
65         if key[pygame.K_RIGHT]:
66             dx = speed
67             self.running = True
68         if key[pygame.K_UP]:
69             self.vel_y = -30 #makes player jump up
70             self.jump = True
71
```

I then added the velocity to the change in y coordinate (dy) then updated the character's y position at the end of the move function.

```
74     #update y coordinate
75     dy += self.vel_y
76
77     #check player on screen
78     if self.rect.left + dx < 0: #if the speed and position of character beyond left screen border
79         dx = -self.rect.left #limit character speed to left border
80     if self.rect.right + dx > screen_width: #if the speed and position of character beyond right screen border
81         dx = screen_width - self.rect.right #limit character speed to right border
82
83
84
85     #ensure player faces each other
86     if target.rect.centerx > self.rect.centerx:
87         self.flip = False
88     else:
89         self.flip = True
90
91
92     #update player position
93     self.rect.x += dx
94     self.rect.y += dy
95
96
97
```

Test

Here is the test I conducted on my code:



IMG_9695.mov

The video shows that when the jump key is pressed, the character does move upwards but does not come down. Also, the jump animation rightfully doesn't play yet as I have not moved on to showing that animation just yet.

Attempt 2

I need a way to add weight to the characters so that their y positions fall back down to ground level. A good way of doing this would be to add gravity into the game.

I added a gravity variable into the move() function.

```
40
41     def move(self, screen_width, target):
42         self.running = False
43         speed = 15
44         gravity = 9.8
45         dx = 0 #positive = going forward, negative = going backwards
46         dy = 0
47
48
```

This ensures that the characters are brought back down when jumping.

Test

I then ran the code and inspected the output.



IMG_9721.mov

The characters immediately start falling below the screen which means that I need to add a boundary to represent ground level so that the characters don't immediately fall. I also think that the gravity variable is possibly too high as the characters fell quite fast.

Attempt 3

I set the gravity down from 9.8 to 5 so that the characters do not fall too fast.

```
40
41     def move(self, screen_width, target):
42         self.running = False
43         speed = 15
44         gravity = 5
45         dx = 0 #positive = going forward, negative = going backwards
46         dy = 0
47
48
```

I also added the screen_height parameter in the move() method so that I can use it when checking if the character will fall past the ground level.

```

41     def move(self, screen_width, screen_height, target):
42         self.running = False
43         speed = 15
44         gravity = 5
45         dx = 0 #positive = going forward, negative = going backwards
46         dy = 0
47

```

I added a conditional statement to check if the character will fall past the screen, and if it does then its y position is limited to the screen height – its y position (so that the character can still be seen).

```

74
75     #update coordinate
76     self.vel_y += gravity
77     dy += self.vel_y
78
79     #check player on screen
80     if self.rect.left + dx < 0: #if the speed and position of character beyond left screen border
81         dx = -self.rect.left #limit character speed to left border
82     if self.rect.right + dx > screen_width: #if the speed and position of character beyond right screen border
83         dx = screen_width - self.rect.right #limit character speed to right border
84     if self.rect.bottom + dy > screen_height:
85         self.vel_y = 0
86         self.jump = False #character can jump again
87         dy = screen_height - self.rect.bottom #limit change in y
88
89
90

```

I then updated the argument for move() in the main game loop by adding screen height.

```

153
154     if MainMenu:
155         #draw start menu
156         drawStart()
157
158     if startBattle:
159         P1.draw(screen, YELLOW)
160         P2.draw(screen, YELLOW)
161
162         P1.move(screen_width, screen_height, P2)
163         P2.move(screen_width, screen_height, P1)
164
165         P1.update()
166         P2.update()
167
168     #update display
169     pygame.display.flip()
170
171     #60 frames per second
172     clock.tick(60)
173
174
175 #exit pygame
176 pygame.quit()

```

Test

Here is a video of me testing the code additions above.



IMG_9815.mov

The jump mechanism does work but not as intended. Pressing the respective jump button for the character does make them jump but holding this button also makes the character float endlessly which is not how the jump button should work. I also think the gravity is still quite strong so I will reduce this more.

Attempt 4

I added the condition 'not self.jump' into the jump if statement which only allows jumping to work once the character has landed.

```

52         if self.player:
53             if key[pygame.K_a]:
54                 dx = -speed # change in speed becomes negative, moves char backwards
55                 self.running = True
56             if key[pygame.K_d]:
57                 dx = speed
58                 self.running = True
59             if key[pygame.K_w] and not self.jump:
60                 self.vel_y = -30 #makes player jump up
61                 self.jump = True
62         else:
63             if key[pygame.K_LEFT]:
64                 dx = -speed # change in speed becomes negative, moves char backwards
65                 self.running = True
66             if key[pygame.K_RIGHT]:
67                 dx = speed
68                 self.running = True
69             if key[pygame.K_UP] and not self.jump:
70                 self.vel_y = -30 #makes player jump up
71                 self.jump = True
72
73

```

I reduced gravity further so that the character can jump higher for longer. This will help players to dodge attacks.

```

40
41     def move(self, screen_width, screen_height, target):
42         self.running = False
43         speed = 15
44         gravity = 1.4
45         dx = 0 #positive = going forward, negative = going backwards
46         dy = 0
47

```

I added a restriction for character movement beyond the top of the screen.

```

78
79     #check player on screen
80     if self.rect.left + dx < 0: #if the speed and position of character beyond left screen border
81         dx = -self.rect.left #limit character speed to left border
82     if self.rect.right + dx > screen_width: #if the speed and position of character beyond right screen border
83         dx = screen_width - self.rect.right #limit character speed to right border
84     if self.rect.bottom + dy > screen_height:
85         self.vel_y = 0
86         self.jump = False #character can jump again
87         dy = screen_height - self.rect.bottom #limit change in y
88     if self.rect.top + dy < 0:
89         dy = -self.rect.top
90

```

I also added the jump animation activation in the update() method

```
115
116 def update(self):
117     if self.running:
118         self.update_action(1)
119     elif self.jump:
120         self.update_action(2)
121     else:
122         self.update_action(0)
123
124
125
126
127
128     animation_cooldown = 60 #milliseconds until next frame
129
130     if pygame.time.get_ticks() - self.updateTime > animation_cooldown: # if time passed has exceeded animation period, go to next frame
131         self.frame_index += 1
132         self.updateTime = pygame.time.get_ticks()
133     if self.frame_index >= len(self.animationlist[self.action])-1: #if last frame has been reached, go back to first frame
134         self.frame_index = 0
135
136     self.image = self.animationlist[self.action][self.frame_index] #current frame to be drawn
137
```

Test

Here is me testing the code amendments made above.



IMG_9789.mov

In the test, I held the jump button, and the characters jumped correctly without floating. The jump animations also played correctly and ended when the character landed. The next animation I will need to implement is Attack 1. But for the function of this to work I will need to first implement the health bars for both players so that the attacks can deplete these bars.

Success Criteria 14 & 15: Character Health Bars

I will need to implement both character health bars in order to show how much damage each player has taken. Player health will start at full capacity and the battle ends when one health bar reaches 0 (if negative, will be set to 0).

Attempt 1

I created the drawHealth() procedure to be called in the main game loop to draw both players' health bars. The parameters health, x, and y are in the procedure and is used to position the health bar and determine how much of the green bar is left (health). The red bar is the damage taken and the white bar is just an outline for the health bar.

```
82
83 #define health bar function
84 def drawHealth(health,x,y):
85     ratio = health / 100
86
87     pygame.draw.rect(screen, WHITE, (x,y, 310,40))
88     pygame.draw.rect(screen, RED, (x,y,300,30))
89     pygame.draw.rect(screen, GREEN, ([x,y,300*ratio,30]))
90
```

I added the health attribute in the Fighter class and set it to 100 initially as the players will start at full health.

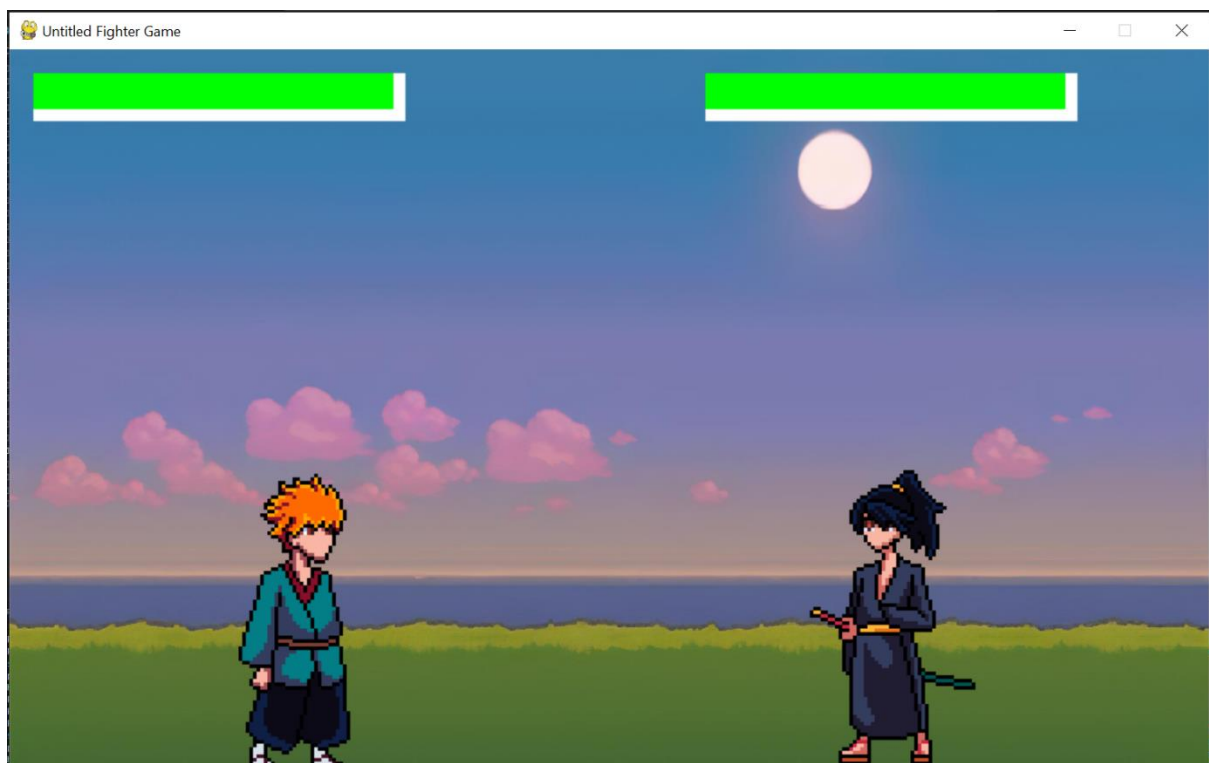
```
3 class Fighter():
4     def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1):
5         self.size = data[0]
6         self.image_scale = data[1]
7         self.offset = data[2]
8         self.flip = flip
9         self.animationlist = self.load_images(sprite_sheet, animationsteps)
10        self.rect = pygame.Rect(x,y,80,150)
11        self.running = False
12        self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
13        self.frame_index = 0
14        self.image = self.animationlist[self.action][self.frame_index]
15        self.updateTime = pygame.time.get_ticks()
16        self.player = Player1
17        self.jump = False
18        self.vel_y = 0
19        self.health = 100 |
20
21
```

I then called the drawHealth() procedure in the main game loop.

```
165     if MainMenu:
166         #draw start menu
167         drawStart()
168
169     if startBattle:
170         P1.draw(screen,YELLOW)
171         P2.draw(screen,YELLOW)
172
173         drawHealth(P1.health,20,20)
174         drawHealth(P2.health,580,20)
175
176         P1.move(screen_width,screen_height, P2)
177         P2.move(screen_width,screen_height, P1)
178
179         P1.update()
180         P2.update()
181
182     #update display
183     pygame.display.flip()
184
185     #60 frames per second
186     clock.tick(60)
187
188
189 #exit pygame
190 pygame.quit()
```

Test

Here is the output from running this code.



The health bars are visible on the screen, but they are not long enough, and the outline does not go around the whole green health bar.

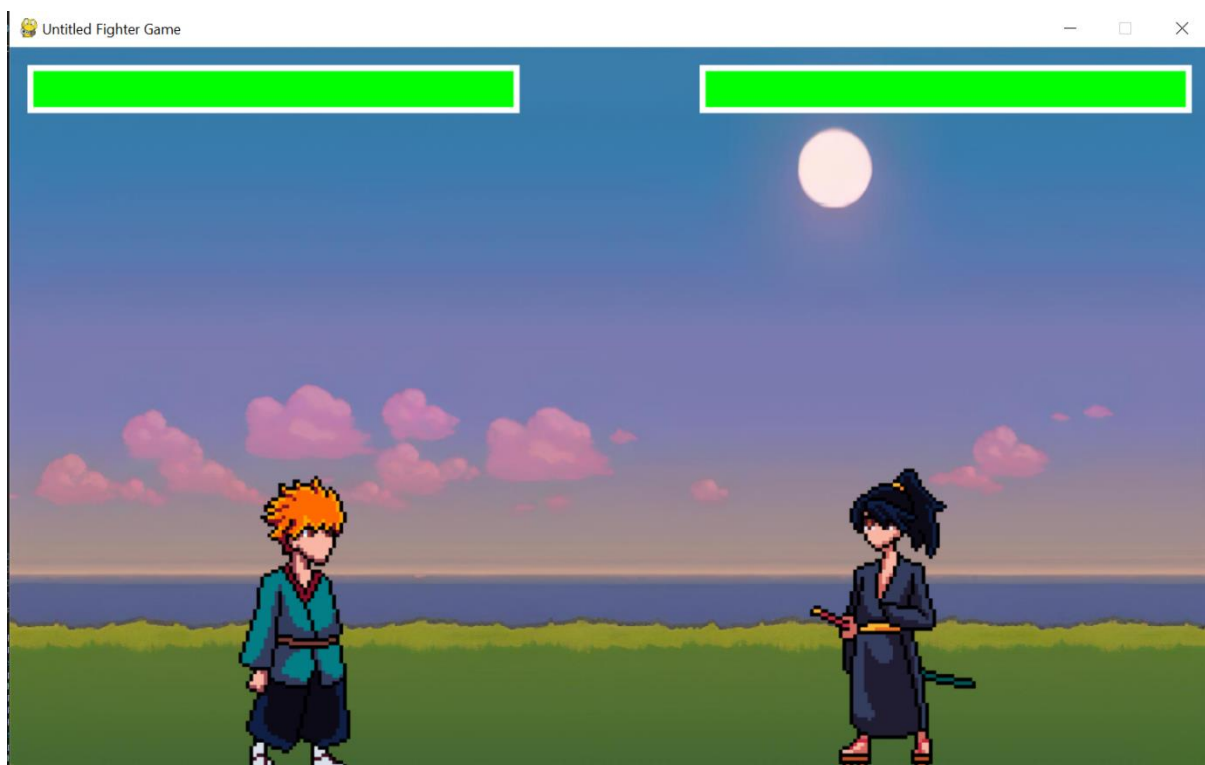
Attempt 2

I increased the width of all the bars and added an offset to the white outline bar so that it surrounds the green health bar

```
83 #define health bar function
84 def drawHealth(health,x,y):
85     ratio = health / 100
86
87     pygame.draw.rect(screen, WHITE, [x-5,y-5, 410,40])
88     pygame.draw.rect(screen, RED, (x,y,400,30))
89     pygame.draw.rect(screen, GREEN, (x,y,400*ratio,30))
90
```

Test

Here is the output of this code.



This test was a success at the health bar is correctly displayed and is long enough for the user to see. The white outline bar correctly surrounds the green health bar.

Success Criteria 21 & 26: Character Attack 1 Function & Animation

All the movement buttons have been implemented and the health bar has been correctly made, and now the focus is on character attacks. I will need to make sure the attack buttons work as intended and the right animation plays for both characters.

Attempt 1

I first added the attacking attribute in the constructor method and set it to False initial since the characters will not start the battle with an immediate attack.

```
3 class Fighter():
4     def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1):
5         self.size = data[0]
6         self.image_scale = data[1]
7         self.offset = data[2]
8         self.flip = flip
9         self.animationlist = self.load_images(sprite_sheet, animationsteps)
10        self.rect = pygame.Rect(x,y,80,150)
11        self.running = False
12        self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
13        self.frame_index = 0
14        self.image = self.animationlist[self.action][self.frame_index]
15        self.updateTime = pygame.time.get_ticks()
16        self.player = Player1
17        self.jump = False
18        self.attacking = False
19        self.vel_y = 0
20        self.health = 100
21
22
23
```

I created an attack method which has the parameters surface (the screen) and target (the target player receiving the attack). Each attack takes away 6.5 damage to the opponent player.

```
141 def attack(self, surface, target):
142     self.attacking = True
143     attackingRect = pygame.Rect(self.rect.centerx - (2*self.rect.width*self.flip), self.rect.y, 2*self.rect.width, self.rect.height)
144     if attackingRect.colliderect(target.rect):
145         target.health -= 6.5 #the damage players receive from attacks
146         pygame.draw.rect(surface,(0, 255, 0), attackingRect) #- shows the area which damage can occur to other player
147
148
```

I then added the attacktype attribute into the constructor method and this will be used to determine which attack will be used (attack 1 or attack 2).

```
3 class Fighter():
4     def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1):
5         self.size = data[0]
6         self.image_scale = data[1]
7         self.offset = data[2]
8         self.flip = flip
9         self.animationlist = self.load_images(sprite_sheet, animationsteps)
10        self.rect = pygame.Rect(x,y,80,150)
11        self.running = False
12        self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
13        self.frame_index = 0
14        self.image = self.animationlist[self.action][self.frame_index]
15        self.updateTime = pygame.time.get_ticks()
16        self.player = Player1
17        self.jump = False
18        self.attacking = False
19        self.attacktype = 0
20        self.vel_y = 0
21        self.health = 100
22
23
24
25
```

I added the surface parameter into the move method since I will also add the attack method inside the move method, and it needs the screen argument for the surface in the main game loop.

```
44     def move(self, screen_width, screen_height, target, surface):
45         self.running = False
46         speed = 15
47         gravity = 1.4
48         dx = 0 #positive = going forward, negative = going backwards
49         dy = 0
50
51
```

I added the attack conditions inside the move() method, including determining if an attack is being made and what attacktype is being made.

```
51
52     #get key pressed on keyboard
53     key = pygame.key.get_pressed()
54
55     if self.player:
56
57         if key[pygame.K_a]:
58             dx = -speed # change in speed becomes negative, moves char backwards
59             self.running = True
60         if key[pygame.K_d]:
61             dx = speed
62             self.running = True
63         if key[pygame.K_w] and not self.jump:
64             self.vel_y = -30 #makes player jump up
65             self.jump = True
66
67         #attack
68         if key[pygame.K_e] or key[pygame.K_r]:
69             self.attack(surface, target)
70             self.attacking = True
71
72         #determine attack type used
73         if key[pygame.K_e]:
74             self.attacktype = 4
75         if key[pygame.K_r]:
76             self.attacktype = 5
77     else:
78         if key[pygame.K_LEFT]:
79             dx = -speed # change in speed becomes negative, moves char backwards
80             self.running = True
81         if key[pygame.K_RIGHT]:
82             dx = speed
83             self.running = True
84         if key[pygame.K_UP] and not self.jump:
85             self.vel_y = -30 #makes player jump up
86             self.jump = True
87
88         #attack
89         if key[pygame.K_n] or key[pygame.K_m]:
90             self.attack(surface, target)
91             self.attacking = True
92
93         #determine attack type used
94         if key[pygame.K_n]:
95             self.attacktype = 4
96         if key[pygame.K_m]:
97             self.attacktype = 5
98
99
100     #update y coordinate
```

I also updated the update() method and incorporated the attack animations for both characters. If the attack type has been set to 4 then attack 1 will play. If the attacktype has been set to 5 then attack 2 will play.

```

141     def update(self):
142         if self.running:
143             self.update_action(1)
144         elif self.jump:
145             self.update_action(2)
146         elif self.attacking:
147             if self.attacktype == 4:
148                 self.update_action(4)
149             elif self.attacktype == 5:
150                 self.update_action(5)
151         else:
152             self.update_action(0)
153
154
155
156
157
158         animation_cooldown = 60 #milliseconds until next frame
159
160         if pygame.time.get_ticks() - self.updateTime > animation_cooldown: # if time passed has exceeded animation period, go to next frame
161             self.frame_index += 1
162             self.updateTime = pygame.time.get_ticks()
163         if self.frame_index >= len(self.animationlist[self.action])-1: #if last frame has been reached, go back to first frame
164             self.frame_index = 0
165
166         self.image = self.animationlist[self.action][self.frame_index] #current frame to be drawn
167
168

```

I then added the arguments for the surface parameter in the move() method.

```

151
152
153
154
155     #drawing code -----
156     if firstLaunch:
157         MainMenu = True
158         firstLaunch = False
159
160
161     drawBG()
162
163
164
165     if MainMenu:
166         #draw start menu
167         drawStart()
168
169     if startBattle:
170         P1.draw(screen, YELLOW)
171         P2.draw(screen, YELLOW)
172
173         drawHealth(P1.health, 20, 20)
174         drawHealth(P2.health, 580, 20)
175
176         P1.move(screen_width, screen_height, P2, screen)
177         P2.move(screen_width, screen_height, P1, screen)
178
179         P1.update()
180         P2.update()
181
182     #update display
183     pygame.display.flip()
184
185     #60 frames per second
186     clock.tick(60)
187
188
189     #exit pygame
190     pygame.quit()

```

Test

Here is a video of me testing the code above.



IMG_9808.mov

The attack animations do play but do not stop. The hitbox for the attacks also shows which is a success. The attacks also happen in too quick of a succession so I will first need to add a way to stop the attack animation once played then implement a cooldown for attacks.

Attempt 2

I added increased the animation cooldown for attacks to 125. I also reset the attacking attribute to false and the attacktype to 0 once the last attack frame has been displayed.

```
140
141     def update(self):
142         if self.running:
143             self.update_action(1)
144         elif self.jump:
145             self.update_action(2)
146         elif self.attacking:
147             if self.attacktype == 4:
148                 self.update_action(4)
149             elif self.attacktype == 5:
150                 self.update_action(5)
151         else:
152             self.update_action(0)
153
154
155
156
157
158         animation_cooldown = 60 #milliseconds until next
159         if self.attacking:
160             animation_cooldown = 125 #cooldown is longer for attacks
161
162         if pygame.time.get_ticks() - self.updateTime > animation_cooldown: # if time passed has exceeded animation period, go to next frame
163             self.frame_index += 1
164             self.updateTime = pygame.time.get_ticks()
165         if self.frame_index >= len(self.animationlist[self.action])-1: #if last frame has been reached, go back to first frame
166             self.frame_index = 0
167             if self.attacking:
168                 self.attacking = False #reset attacking status
169                 self.attack_type = 0 #reset attacktype
170
171         self.image = self.animationlist[self.action][self.frame_index] #current frame to be drawn
172
```

Test

Here is a video of the above amendments.



IMG_9809.mov

The attacks work as intended and the hitbox still appears as intended.

Here is a video of me testing attacks on the other player:



IMG_9812.mov

The attacks drain the other player's health too quickly. I will need to add a hit attribute to control whether the opponent character can take damage.

Attempt 3

I added a hit variable in the constructor method.

```
3 class Fighter():
4     def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1):
5         self.size = data[0]
6         self.image_scale = data[1]
7         self.offset = data[2]
8         self.flip = flip
9         self.animationlist = self.load_images(sprite_sheet, animationsteps)
10        self.rect = pygame.Rect(x,y,80,150)
11        self.running = False
12        self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
13        self.frame_index = 0
14        self.image = self.animationlist[self.action][self.frame_index]
15        self.updateTime = pygame.time.get_ticks()
16        self.player = Player1
17        self.jump = False
18        self.attacking = False
19        self.attacktype = 0
20        self.hit = False
21        self.vel_y = 0
22        self.health = 100
23
24
```

I added the condition 'if not self.attacking' in the move method so that the player cannot attack while currently attacking.

```

52
53     #get key pressed on keyboard
54     key = pygame.key.get_pressed()
55
56     if self.player:
57
58         if not self.attacking:
59
60             if key[pygame.K_a]:
61                 dx = -speed # change in speed becomes negative, moves char backwards
62                 self.running = True
63             if key[pygame.K_d]:
64                 dx = speed
65                 self.running = True
66             if key[pygame.K_w] and not self.jump:
67                 self.vel_y = -30 #makes player jump up
68                 self.jump = True
69
70             #attack
71             if key[pygame.K_e] or key[pygame.K_r]:
72                 self.attack(surface, target)
73                 self.attacking = True
74
75             #determine attack type used
76             if key[pygame.K_e]:
77                 self.attacktype = 4
78             if key[pygame.K_r]:
79                 self.attacktype = 5
80
81         else:
82             if not self.attacking:
83
84                 if key[pygame.K_LEFT]:
85                     dx = -speed # change in speed becomes negative, moves char backwards
86                     self.running = True
87                 if key[pygame.K_RIGHT]:
88                     dx = speed
89                     self.running = True
90                 if key[pygame.K_UP] and not self.jump:
91                     self.vel_y = -30 #makes player jump up
92                     self.jump = True
93
94                 #attack
95                 if key[pygame.K_n] or key[pygame.K_m]:
96                     self.attack(surface, target)
97                     self.attacking = True
98
99                 #determine attack type used
100                 if key[pygame.K_n]:
101                     self.attacktype = 4
102                 if key[pygame.K_m]:
103                     self.attacktype = 5

```

I added self.hit conditions in the update() method so that the hit status is reset to False once the player has been attacked.

```
157
158
159
160
161
162     animation_cooldown = 60 #milliseconds until next
163     if self.attacking:
164         animation_cooldown = 125 #cooldown is longer for attacks
165
166     if pygame.time.get_ticks() - self.updateTime > animation_cooldown: # if time passed has exceeded animation period, go to next frame
167         self.frame_index += 1
168         self.updateTime = pygame.time.get_ticks()
169     if self.frame_index >= len(self.animationlist[self.action])-1: #if last frame has been reached, go back to first frame
170         self.frame_index = 0
171     if self.attacking:
172         self.attacking = False #reset attacking status
173         self.attack_type = 0 #reset attacktype
174     if self.hit:
175         self.hit = False
176         self.attacking = False
177
178     self.image = self.animationlist[self.action][self.frame_index] #current frame to be drawn
179
180
181 def attack(self, surface, target):
182     self.attacking = True
183     attackingRect = pygame.Rect(self.rect.centerx - (2*self.rect.width*self.flip), self.rect.y, 2*self.rect.width, self.rect.height)
184     if attackingRect.colliderect(target.rect):
185         target.health -= 6.5 #the damage players receive from attacks
186         target.hit = True
187
188     pygame.draw.rect(surface,(0, 255, 0), attackingRect) #- shows the area which damage can occur to other player
189
190
```

I added the hit animation in the update() method as well once the player is hit by an attack.

```
144
145     def update(self):
146         if self.running:
147             self.update_action(1)
148         if self.hit:
149             self.update_action(7)
150         elif self.jump:
151             self.update_action(2)
152         elif self.attacking:
153             if self.attacktype == 4:
154                 self.update_action(4)
155             elif self.attacktype == 5:
156                 self.update_action(5)
157         else:
158             self.update_action(0)
159
160
161
162
```

I added a trigger for the hit attribute to be set to True once the player has been hit in the attack() method.

```
184 def attack(self, surface, target):
185     self.attacking = True
186     attackingRect = pygame.Rect(self.rect.centerx - (2*self.rect.width*self.flip), self.rect.y, 2*self.rect.width, self.rect.height)
187     if attackingRect.colliderect(target.rect) and not target.hit:
188         target.health -= 6.5 #the damage players receive from attacks
189         target.hit = True
190
191     pygame.draw.rect(surface,(0, 255, 0), attackingRect) #- shows the area which damage can occur to other player
192
193
194
```

Test

Here is a video of me testing the character attacks on each of the characters



IMG_9816.mov

The attacks clearly work as intended with an appropriate cooldown. Each attack deals the correct amount of damage to the opponent's health. Both attack 1 and 2 works so success criteria 21,22, 26 and 27 have all been achieved since both attacks for both characters work correctly according to the test plan.

Extra feature 1: Knockback on attacks

During the test I held down the attack button and the character would continuously attack the other player, depleting their health rather quicker than I had first imagined. I fear that this will not be fun for the user as the game will just become a game of who has the faster reaction time between the user that can hold the attack button first, without moving to avoid attacks. The user should be able to embrace all elements of my game, including movement, so I came up with an idea to add knockbacks on each attack. This will rid the problem of users solely holding down the attack button to defeat their opponents and will encourage users to jump, move forwards and backwards in order to avoid attacks from the opponent player.

Attempt 1

I added a knockFriction attribute to the fighter class.

```

1  import pygame
2
3  class Fighter():
4      def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1):
5          self.size = data[0]
6          self.image_scale = data[1]
7          self.offset = data[2]
8          self.flip = flip
9          self.animationlist = self.load_images(sprite_sheet, animationsteps)
10         self.rect = pygame.Rect(x,y,80,150)
11         self.running = False
12         self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
13         self.frame_index = 0
14         self.image = self.animationlist[self.action][self.frame_index]
15         self.updateTime = pygame.time.get_ticks()
16         self.player = Player1
17         self.jump = False
18         self.attacking = False
19         self.attacktype = 0
20         self.hit = False
21         self.vel_y = 0
22         self.health = 100
23         self.knockFriction = 0
24
25
26
27

```

The knockFriction attribute will be set to 10 after the player has been hit in the update() method.

```

167
168
169  def update(self):
170      if self.running:
171          self.update_action(1)
172      if self.hit:
173          self.knockFriction = 10
174          self.update_action(7)
175      elif self.jump:
176          self.update_action(2)
177      elif self.attacking:
178          if self.attacktype == 4:
179              self.update_action(4)
180          elif self.attacktype == 5:
181              self.update_action(5)
182      else:
183          self.update_action(0)
184
185
186
187

```

The knockback will then be added to the dx variable so that the character moves quickly when hit from an attack. Extra knockback will be added if the attack is attack 2. Then the knockFriction variable is decremented by 1.

```

105
106  #update y coordinate
107  self.vel_y += gravity
108  dy += self.vel_y
109
110  if self.hit:
111      self.vel_x = self.knockFriction
112      if target.attacktype == 5: #extra knockback for attack 2
113          self.vel_x += self.vel_x*20
114      dx += self.vel_x
115      self.knockFriction -= 1
116

```

Test

Here is a video of me testing the code.



IMG_9817.mov

The knockback is visible and there is a comfortable amount of it, but it does not go in the right way for both players. I will need to fix this using the flip variable to decide where the knockback happens.

Attempt 2

I used the flip attribute to determine the direction of knockback, even when flipped. I also added a reset knockback condition below this so that the knockFriction attribute is set to 0 when it has fallen below 0.25.

```
105
106     #update y coordinate
107     self.vel_y += gravity
108     dy += self.vel_y
109
110     if self.hit:
111         if not self.flip:
112             self.vel_x = -self.knockFriction
113
114             if target.attacktype == 5: #extra knockback for attack 2
115                 self.vel_x += self.vel_x*20
116                 dx += self.vel_x
117                 self.knockFriction -= 1
118
119         else:
120
121             self.vel_x = self.knockFriction
122             if target.attacktype == 5:
123                 self.vel_x += self.vel_x*20
124                 dx += self.vel_x
125                 self.knockFriction -= 1
126
127
128     if self.knockFriction <= 0.25: #reset knockback and velocity to 0 when knockback is negligible
129         self.knockFriction = 0
130         self.vel_x = 0
131
```

I also added height to the knockback in the attack() method so that the character gets knocked back at an angle to the horizontal ground.

```
208
209
210     def attack(self, surface, target):
211         self.attacking = True
212         attackingRect = pygame.Rect(self.rect.centerx - (2*self.rect.width*self.flip), self.rect.y, 2*self.rect.width, self.rect.height)
213         if attackingRect.colliderect(target.rect) and not target.hit:
214             target.health -= 6.5 #the damage players receive from attacks
215             target.hit = True
216             if target.flip:
217                 target.vel_y -= 20
218             else:
219                 target.vel_y += 20
220
221         #pygame.draw.rect(surface,(0, 255, 0), attackingRect) #- shows the area which damage can occur to other player
222
223
224
```

Test

Here is a video of me testing the knockback on both players.



IMG_9819.mov

The knockback works correctly and is more powerful when attack 2 is used. Therefore, I have fixed the issue of players being able to spam the attack button whilst standing still since there will be knockback on attacks which means that the attacking player will still have to move towards the opponent, and the opponent is given a chance to avoid attacks with the knockback.

Success Criteria 13: Timer

To make the battle more intense and engaging, I will need to implement a timer function so that players are encouraged to defeat their opponent before the 45 second limit.

Attempt 1

I added the game variables to control when the time starts and stops. These variables are initially set to False as they won't be needed in the Main Menu screen.

```
39
40 #define game variables
41 MainMenu = False
42 firstLaunch = True
43 charSelect = False
44 startBattle = False
45 timerStart = False
46 TimeOut = False
47 battleDone = False
48
```

I then created the timer function in the main game program. The seconds are defined by the difference from when the function gets called to the current time. This variable is reset when the battle finishes. If the timer runs over 45 seconds, the screen changes. Each second is displayed on the screen and is position at the top of the screen between both health bars.

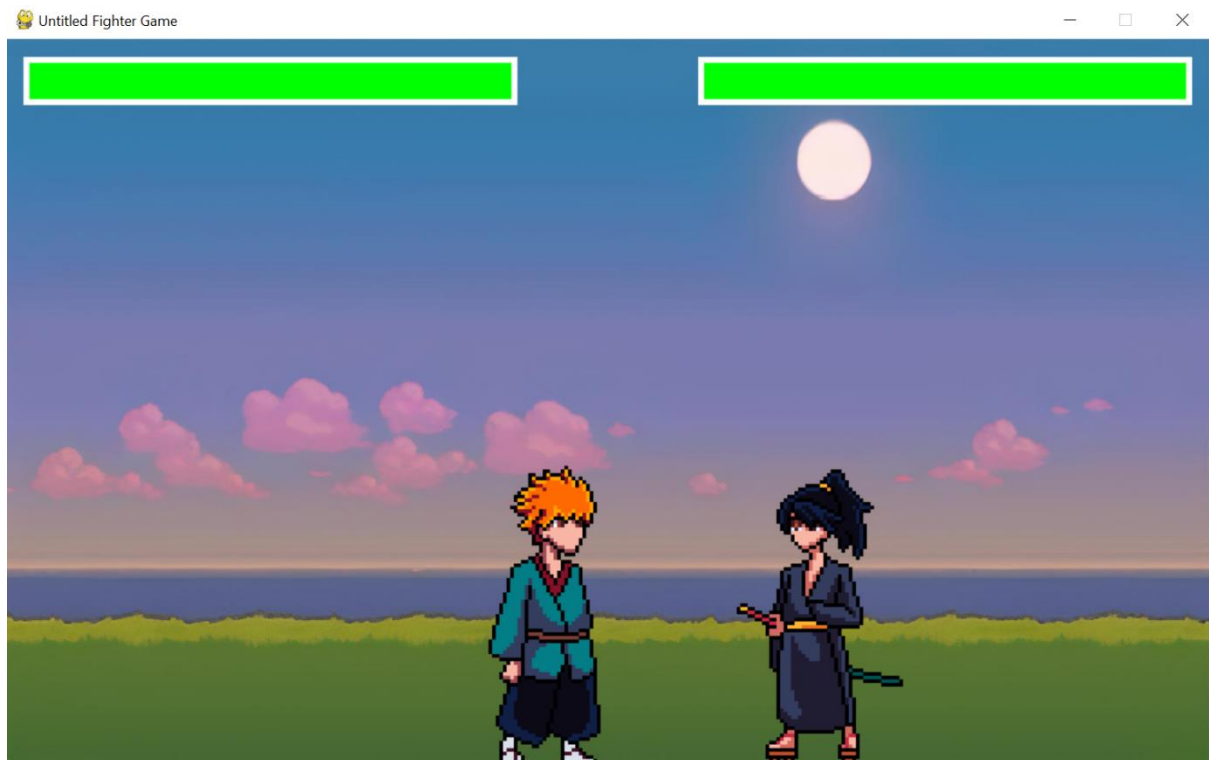
```
90 #draw timer function
91 def drawTimer():
92     if not timerStart and startBattle: #when the battle starts, start the timer
93         startTicks = pygame.time.get_ticks()
94         timerStart = True
95     if startBattle: #while the battle is ongoing, keep updating the value of seconds
96         seconds = (pygame.time.get_ticks()-startTicks)//1000
97
98     if battleDone: #when battle finishes, delete seconds
99         seconds = 0
100         del seconds
101     if seconds > 45 and startBattle: # if the timer goes over 45 seconds, end the battle
102         startTicks = pygame.time.get_ticks()
103         battleDone = True
104         TimeOut = True
105         startBattle = False
106         timerStart = False
107     if not battleDone: #While the battle is ongoing, keep outputing seconds on screen
108         timer = MenuFont.render(str(seconds), True, BLACK)
109         screen.blit(timer, [screen_width/2, screen_height/20])
110
```

I then called the function within the main game loop under the startBattle condition.

```
202     if startBattle:
203         drawTimer()
204     |
205     P1.draw(screen,YELLOW)
206     P2.draw(screen,YELLOW)
207
208     drawHealth(P1.health,20,20)
209     drawHealth(P2.health,580,20)
210
211     P1.move(screen_width,screen_height, P2, screen)
212     P2.move(screen_width,screen_height, P1, screen)
213
214     P1.update()
215     P2.update()
216
217     #update display
218     pygame.display.flip()
219
220     #60 frames per second
221     clock.tick(60)
222
223
224 #exit pygame
225 pygame.quit()
```

Test

Here is the output from my code.



The timer does not display on the screen. I think this is because the variables that I have used in the timer function were not declared as global variables so I will need to do this in the next attempt.

Attempt 2

I declared the global variables needed here at the start of the function.

```
89
90 #draw timer function
91 def drawTimer():
92     global startTicks
93     global battleDone
94     global TimeOut
95     global timerStart
96     global startBattle
97     if not timerStart and startBattle: #when the battle starts, start the timer
98         startTicks = pygame.time.get_ticks()
99         timerStart = True
100     if startBattle: #while the battle is ongoing, keep updating the value of seconds
101         seconds = (pygame.time.get_ticks()-startTicks)//1000
102
103     if battleDone: #when battle finishes, delete seconds
104         seconds = 0
105         del seconds
106     if seconds > 45 and startBattle: # if the timer goes over 45 seconds, end the battle
107         startTicks = pygame.time.get_ticks()
108         battleDone = True
109         TimeOut = True
110         startBattle = False
111         timerStart = False
112     if not battleDone: #While the battle is ongoing, keep outputting seconds on screen
113         timer = MenuFont.render(str(seconds), True, BLACK)
114         screen.blit(timer, (screen_width/2, screen_height/20))
115
```

Test

Here is a video of me testing the timer function.



IMG_9847.mov

The timer does work as intended but it is not aligned well between the health bars so I will need to decrease the y value and the x value more.

Attempt 3

I took away 12 from the x value and increase the value I divided the y value by from 20 to 50

```
91 def drawTimer():
92     global startTicks
93     global battleDone
94     global TimeOut
95     global timerStart
96     global startBattle
97     if not timerStart and startBattle: #when the battle starts, start the timer
98         startTicks = pygame.time.get_ticks()
99         timerStart = True
100     if startBattle: #while the battle is ongoing, keep updating the value of seconds
101         seconds = (pygame.time.get_ticks()-startTicks)//1000
102
103     if battleDone: #when battle finishes, delete seconds
104         seconds = 0
105         del seconds
106     if seconds > 45 and startBattle: # if the timer goes over 45 seconds, end the battle
107         startTicks = pygame.time.get_ticks()
108         battleDone = True
109         TimeOut = True
110         startBattle = False
111         timerStart = False
112     if not battleDone: #While the battle is ongoing, keep outputting seconds on screen
113         timer = MenuFont.render(str(seconds), True, BLACK)
114         screen.blit(timer, [screen_width/2 - 12, screen_height/50])
115
```

Test

Here is a video of the timer function working.



IMG_9848.mov

The timer works and is positioned correctly between both health bars. This will signify to users about how much time they have left to battle and will encourage them to defeat their opponent in time, adding an exciting element to the game.

(Should have) Success Criteria 1 & 3: Ability 1 Animation

Abilities act as another way to deal extra damage to opponents but must be used wisely as their cooldown will be much longer than attacks.

This will be the sprite sheet that I will use and animate as the ability.



Attempt 1

To animate the abilities, I will first need to implement ability bars for both players so that users can see when they can use abilities.

I added the relevant ability attributes in the constructor method of the Fighter class. This includes useAbility (signifies when user can use ability), ability_charge (will increment to control ability bar) and ability_cooldown (will control a period where ability cannot be used).

```
2
3 class Fighter():
4     def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1):
5         self.size = data[0]
6         self.image_scale = data[1]
7         self.offset = data[2]
8         self.flip = flip
9         self.animationlist = self.load_images(sprite_sheet, animationsteps)
10        self.rect = pygame.Rect(x,y,80,150)
11        self.running = False
12        self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
13        self.frame_index = 0
14        self.image = self.animationlist[self.action][self.frame_index]
15        self.updateTime = pygame.time.get_ticks()
16        self.player = Player1
17        self.jump = False
18        self.attacking = False
19        self.attacktype = 0
20        self.hit = False
21        self.vel_y = 0
22        self.health = 100
23        self.knockFriction = 0
24        self.useAbility = False
25        self.ability_charge = 0
26        self.ability_cooldown = False
27
28
29
```

I then created a function to draw the ability bar on to the screen in the main program. This includes a white outline and the ability bar will be blue.

```
126 #draw ability bar
127 def drawAbilityBar(x, y, charge): #add charge parameter
128     pygame.draw.rect(screen, WHITE, (x-5, y-5, 210, 40)) # Bar outline, increase visibility
129     pygame.draw.rect(screen, BLUE, (x, y, 200 * charge, 30)) # use charge to control how much bar is filled
130
```

I then called this function within the main game loop under the startBattle condition.

```
209     if startBattle:
210         drawTimer()
211
212         P1.draw(screen,YELLOW)
213         P2.draw(screen,YELLOW)
214
215         drawHealth(P1.health,20,20)
216         drawHealth(P2.health,580,20)
217
218         drawAbilityBar(20, 80, P1.ability_charge)
219         drawAbilityBar(580, 80, P2.ability_charge)
220
221         P1.move(screen_width,screen_height, P2, screen)
222         P2.move(screen_width,screen_height, P1, screen)
223
224         P1.update()
225         P2.update()
226
227     #update display
228     pygame.display.flip()
229
230     #60 frames per second
231     clock.tick(60)
232
233
```

Test

Here is the output of my code.



The ability bar outline does show up on the screen, but the blue bar does not grow yet. I will need to add the increment for the ability bar in the next attempt.

Attempt 2

I added the increment as 0.01 in the main game loop.

```
204
205     if MainMenu:
206         #draw start menu
207         drawStart()
208
209     if startBattle:
210         drawTimer()
211
212         P1.draw(screen,YELLOW)
213         P2.draw(screen,YELLOW)
214
215         drawHealth(P1.health,20,20)
216         drawHealth(P2.health,580,20)
217
218         drawAbilityBar(20, 80, P1.ability_charge)
219         drawAbilityBar(580, 80, P2.ability_charge)
220
221
222         P1.ability_charge = P1.ability_charge + 0.01
223
224         P2.ability_charge = P2.ability_charge + 0.01
225
226         P1.move(screen_width,screen_height, P2, screen)
227         P2.move(screen_width,screen_height, P1, screen)
228
229         P1.update()
```

Test

Here is a video of me testing this new addition.



IMG_9859 real.mp4

The ability bar grows way too quickly and it exceeds the limits of the outline bar. I will need to add a limit to the growth of the ability bar and I will need to lessen the increment for the ability charge.

Attempt 3

I first lessened the increment to 0.0025. I also utilised the min() function with the second argument being 1 so that the ability bar stops growing when it reaches full capacity.

```
206
207     if MainMenu:
208         #draw start menu
209         drawStart()
210
211     if startBattle:
212         drawTimer()
213
214         P1.draw(screen,YELLOW)
215         P2.draw(screen,YELLOW)
216
217         drawHealth(P1.health,20,20)
218         drawHealth(P2.health,580,20)
219
220         drawAbilityBar(20, 80, P1.ability_charge)
221         drawAbilityBar(580, 80, P2.ability_charge)
222
223
224         P1.ability_charge = min(P1.ability_charge + 0.0025, 1)
225
226
227         P2.ability_charge = min(P2.ability_charge + 0.0025, 1)
228
229         P1.move(screen_width,screen_height, P2, screen)
230         P2.move(screen_width,screen_height, P1, screen)
231
232         P1.update()
233         P2.update()
234
235     #update display
236     pygame.display.flip()
237
238     #60 frames per second
239     clock.tick(60)
240
241
242 #exit pygame
243 pygame.quit()
```

Test

Here is a video of my code working.



IMG_9860.mov

The ability bar grows slower, and it stops when at the outline bar. This is a success, and users will now know when they can use an ability. Now, I will need to display the ability animation for the user to see.

Attempt 4

I created a new Ability class file and added similar parameters in the constructor method of the Fighter class. The attributes are also similar but there are some new ones such as 'active' which indicates whether the ability should be displayed.

```

1  import pygame
2
3  class Ability():
4      def __init__(self, x, y, data, animationsteps, spritesheet, flip, debug=False):
5          self.size = data[0]
6          self.image_scale = data[1]
7          self.offset = data[2]
8          self.flip = flip
9          self.animationlist = self.load_images(spritesheet, animationsteps)
10         frame_width = spritesheet.get_width() // animationsteps
11         self.rect = pygame.Rect(
12             x - (self.offset[0] * self.image_scale), # apply X offset
13             y - (self.offset[1] * self.image_scale), # apply Y offset
14             frame_width * self.image_scale - 150, # use frame width
15             spritesheet.get_height() * self.image_scale + 200 # use frame height
16         )
17         self.frame_index = 0
18         self.image = self.animationlist[self.frame_index]
19         self.updateTime = pygame.time.get_ticks()
20         self.abilityBar = 0
21         self.active = False
22         self.cooldown = 7000
23         self.debug = debug # debug colour hitbox
24

```

The load_images method is similar to the load method in the Fighter class except that there is only one animation list needed as there is only one type of animation to display.

```
25     def load_images(self, sprite_sheet, animationsteps):
26         #Validate input
27         sheet_width, sheet_height = sprite_sheet.get_size()
28         frame_width = sheet_width // animationsteps
29         frame_height = sheet_height
30         animationlist = []
31
32         #Extract frames from the sprite sheet
33         for frame_index in range(animationsteps):
34             frame_x = frame_index * frame_width
35             temp_img = sprite_sheet.subsurface((frame_x, 0, frame_width, frame_height))
36             scaled_img = pygame.transform.scale(
37                 temp_img, (frame_width * self.image_scale, frame_height * self.image_scale)
38             )
39             animationlist.append(scaled_img)
40
41         return animationlist
42
```

The draw method is also similar to the Fighter class draw method but there is a condition to control where the ability moves. If the ability is facing left, the x coordinate decreases so that the ability moves left. If it is facing right, the x coordinate increases so that the ability moves right.

```
43     def draw(self, surface):
44         img = pygame.transform.flip(self.image, not self.flip, True)
45         if self.debug:
46             pygame.draw.rect(surface, (255, 255, 0), self.rect) #Debug hitbox
47         surface.blit(
48             img,
49             (self.rect.x - (self.offset[0] * self.image_scale),
50              self.rect.y - (self.offset[1] * self.image_scale))
51         )
52         if self.active:
53             if not self.flip:
54                 self.rect.x += 20
55             else:
56                 self.rect.x -= 20
57
```

The update method is somewhat similar to the Fighter class update method but the key difference is that the ability animation only loops 6 times then it is deactivated. This is to ensure that there aren't endless abilities using memory whilst being outside of the screen.

```
58
59     def update(self):
60         #Update animation only if active
61         if self.active:
62             cycles = 0
63             animation_cooldown = 80 #Time between frames (milliseconds)
64             if pygame.time.get_ticks() - self.updateTime > animation_cooldown:
65                 self.frame_index += 1
66                 self.updateTime = pygame.time.get_ticks()
67
68             #Reset to the first frame if the animation ends
69             if self.frame_index >= len(self.animationlist):
70                 cycles += 1
71                 self.frame_index = 0
72
73             if cycles > 5:
74                 cycles = 0
75                 self.active = False
76
77             self.image = self.animationlist[self.frame_index]
78
```

This method updates the ability and checks whether the ability has collided with the target – that being the player.

```
80     def AbilityOn(self, target):
81         if self.active:
82             self.update()
83             if self.rect.colliderect(target.rect):
84                 target.health -= 16
85                 self.active = False
86
```

I added the sprite sheet into the main game program similar to the shinobi and samurai sprite sheet.

```
76 #red punch sprite sheet
77 redpunch_spritesheet = pygame.image.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Images\\Red Fire\\red_fire_projectile.png")
78
79 #num of animation steps (Idle,Run,Jump, Attack1, Attack2, Attack3, Hurt, Dead)
80 shinobi_Animationsteps = (6,8,10,6,3,4,6,3)
81
82 samurai_Animationsteps = (6,8,12,6,4,4,5,3)
83
84
85 ability_data = [(32,5,[25,35]), 4, redpunch_spritesheet]
86
87
```

I then made an attribute in the Fighter class called ability and created an Ability object for this attribute. The x and y position of the ability is set to the character's x and y position since the ability will start from the character and move from there. The relevant data about the ability is assigned to the ability data as well as the character's current flip status.

```
4 class Fighter():
5     def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1, ability_data):
6         self.size = data[0]
7         self.image_scale = data[1]
8         self.offset = data[2]
9         self.flip = flip
10        self.animationlist = self.load_images(sprite_sheet, animationsteps)
11        self.rect = pygame.Rect(x,y,80,150)
12        self.running = False
13        self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
14        self.frame_index = 0
15        self.image = self.animationlist[self.action][self.frame_index]
16        self.updateTime = pygame.time.get_ticks()
17        self.player = Player1
18        self.jump = False
19        self.attacking = False
20        self.attacktype = 0
21        self.hit = False
22        self.vel_y = 0
23        self.health = 100
24        self.knockFriction = 0
25        self.useAbility = False
26        self.ability_charge = 0
27        self.ability_cooldown = False
28        self.ability = Ability(
29            x=self.rect.centerx,
30            y=self.rect.centery,
31            data=ability_data[0],
32            animationsteps=ability_data[1],
33            spritesheet=ability_data[2],
34            flip=flip,
35        )
36
37
38
```

I then added the ability_data argument in the instance of both Fighter objects for player 1 and 2.

```
168 P1 = Fighter(200,450, False, ShinobiData, shinobi_spritesheet,shinobi_Animationsteps, True, ability_data)
169 P2 = Fighter(700,450,True, SamuraiData, samurai_spritesheet, samurai_Animationsteps, False, ability_data)
```

I then created the `activate_ability()` method. It starts with a check to make sure the ability bar is full, that method being below. Then the ability frame starts at 0 and the charge is reset to 0 as well. The ability starts in front of the player so a check will need to be made for the flip Boolean to see if the 10 should be added or subtracted from the x position.

```
169 def activate_ability(self):
170     if self.checkAbilityBar(): # Use instance-based charge
171         self.ability.active = True
172         self.ability.frame_index = 0
173         self.ability_charge = 0 # Reset charge
174         # Position ability relative to fighter
175         if self.flip:
176             self.ability.rect.x = self.rect.x - 10
177         else:
178             self.ability.rect.x = self.rect.x + 10
179         self.ability.rect.y = self.rect.centery
180
181     def checkAbilityBar(self):
182         return self.ability_charge >= 1 # True when full
183
184
185
```

I called the activate_ability() method in the move method for the Fighter class under the attacks for each player. A check will be made if the ability button ('q' or 'b') has been press and if the ability bar is full before the activate_ability() method is called.

```

69         if self.player:
70             if not self.attacking:
71                 if key[pygame.K_a]:
72                     dx = -speed # change in speed becomes negative, moves char backwards
73                     self.running = True
74                 if key[pygame.K_d]:
75                     dx = speed
76                     self.running = True
77                 if key[pygame.K_w] and not self.jump:
78                     self.vel_y = -30 #makes player jump up
79                     self.jump = True
80
81             #attack
82             if key[pygame.K_e] or key[pygame.K_r]:
83                 self.attack(surface, target)
84                 self.attacking = True
85
86             #determine attack type used
87             if key[pygame.K_e]:
88                 self.attacktype = 4
89             if key[pygame.K_r]:
90                 self.attacktype = 5
91             if key[pygame.K_q] and self.ability_charge >= 1:
92                 self.activate_ability()
93
94         else:
95             if not self.attacking:
96                 if key[pygame.K_LEFT]:
97                     dx = -speed # change in speed becomes negative, moves char backwards
98                     self.running = True
99                 if key[pygame.K_RIGHT]:
100                     dx = speed
101                     self.running = True
102                 if key[pygame.K_UP] and not self.jump:
103                     self.vel_y = -30 #makes player jump up
104                     self.jump = True
105
106             #attack
107             if key[pygame.K_n] or key[pygame.K_m]:
108                 self.attack(surface, target)
109                 self.attacking = True
110
111             #determine attack type used
112             if key[pygame.K_n]:
113                 self.attacktype = 4
114             if key[pygame.K_m]:
115                 self.attacktype = 5
116             if key[pygame.K_b] and self.ability_charge >= 1:
117                 self.activate_ability()
118
119
120

```

I added the AbilityOn() function which updates the ability and checks whether it collides with the target player.

```
80     def AbilityOn(self, target):
81         if self.active:
82             self.update()
83             if self.rect.colliderect(target.rect):
84                 target.health -= 16
85                 self.active = False
86
```

I then called this function in the main game loop under the startBattle condition. Having P2 as the target for P1 and vice versa.

```
218
219     if startBattle:
220         drawTimer()
221
222         P1.draw(screen,YELLOW)
223         P2.draw(screen,YELLOW)
224
225         drawHealth(P1.health,20,20)
226         drawHealth(P2.health,580,20)
227
228         drawAbilityBar(20, 80, P1.ability_charge)
229         drawAbilityBar(580, 80, P2.ability_charge)
230
231
232         P1.ability_charge = min(P1.ability_charge + 0.0025, 1)
233
234
235         P2.ability_charge = min(P2.ability_charge + 0.0025, 1)
236
237         P1.ability.AbilityOn(P2)
238         P2.ability.AbilityOn(P1)
239
240         P1.move(screen_width,screen_height, P2, screen)
241         P2.move(screen_width,screen_height, P1, screen)
242
243         P1.update()
244         P2.update()
245
246     #update display
247     pygame.display.flip()
248
249     #60 frames per second
250     clock.tick(60)
251
252
253 #exit pygame
254 pygame.quit()
```

I then added the ability update() method in the Fighter update() method so that the ability animation starts updating when it has been used by the user.

```
198     def update(self):
199         if self.useAbility:
200             self.ability.update()
201             self.update_action(4)
202         if self.hit:
203             self.knockFriction = 10
204             self.update_action(7)
205
206         elif self.attacking:
207             if self.attacktype == 4:
208                 self.update_action(4)
209             elif self.attacktype == 5:
210                 self.update_action(5)
211         elif self.jump:
212             self.update_action(2)
213         elif self.running:
214             self.update_action(1)
215         else:
216             self.update_action(0)
217
218
```

Test

Here is a video of my code.



IMG_9863.mov

Here, I am pressing both abilities for both players and it is not showing up, even though the ability bar is resetting to 0 each time it is used. I realised I need to draw the ability for both players in the update() method so I will fix this in the next attempt.

Attempt 5

I added target and surface as parameters for the update() method in the Fighter class. I then added the ability draw() method in the Fighter update() method and used the surface parameter as its argument.

```

198
199     def update(self, target, surface):
200         if self.ability.active:
201             self.ability.draw(surface)
202             self.ability.update(target)
203             self.update_action(4)
204         if self.hit:
205             self.knockFriction = 10
206             self.update_action(7)
207
208         elif self.attacking:
209             if self.attacktype == 4:
210                 self.update_action(4)
211             elif self.attacktype == 5:
212                 self.update_action(5)
213         elif self.jump:
214             self.update_action(2)
215         elif self.running:
216             self.update_action(1)
217         else:
218             self.update_action(0)
219
220

```

I then added the relevant arguments in the update method called in the main game loop. The screen is the argument for the surface and will be used to draw the abilities and player.

```

59
60     def update(self, target):
61         #Update animation only if active
62         if self.active:
63             animation_cooldown = 80 #Time between frames (milliseconds)
64             if pygame.time.get_ticks() - self.updateTime > animation_cooldown:
65                 self.frame_index += 1
66                 self.updateTime = pygame.time.get_ticks()
67
68             #Reset to the first frame if the animation ends
69             if self.frame_index >= len(self.animationlist)-1:
70                 self.cycles += 1
71                 self.frame_index = 0
72
73             if self.cycles > 5:
74                 self.cycles = 0
75                 self.active = False
76
77             self.image = self.animationlist[self.frame_index]
78
79
80             if self.rect.colliderect(target.rect):
81                 target.health -= 16.5
82                 self.active = False
83
84
85
86

```

I merged the AbilityOn() function with the update method so that the collisions are handled in the update() method in the Fighter class.

```
59
60 def update(self, target):
61     #Update animation only if active
62     if self.active:
63         animation_cooldown = 80 #Time between frames (milliseconds)
64         if pygame.time.get_ticks() - self.updateTime > animation_cooldown:
65             self.frame_index += 1
66             self.updateTime = pygame.time.get_ticks()
67
68         #Reset to the first frame if the animation ends
69         if self.frame_index >= len(self.animationlist)-1:
70             self.cycles += 1
71             self.frame_index = 0
72
73         if self.cycles > 5:
74             self.cycles = 0
75             self.active = False
76
77         self.image = self.animationlist[self.frame_index]
78
79
80         if self.rect.colliderect(target.rect):
81             target.health -= 16.5
82             self.active = False
83
84
85
86
```

Test

Here is a video of my code working.



IMG_9864.mov

This video shows that the ability does work for both players and it flies at a good speed (given that it is in close range). However, the ability is too high on the screen so I will need to adjust the offset of the ability further to lower the ability.

This separate video shows another error.



IMG_9865.mov

This video shows that the ability instantly appears next to the opponent and damages them, giving the attacking player an advantage. I will have to add a function to reset and update the ability position to the player position every time the user is not using it.

Attempt 5

I lessened the ability y offset to 20.

```
84
85 ability_data = [(32,5,[25,20]), 4, redpunch_spritesheet]
86
87
```

I created a function to update the position of the player ability to the current player position so that the ability flies from the current player each time.

```
189
190 def updateAbilityPos(self):
191     if not self.ability.active:
192         self.ability.rect.x = self.rect.x
193         self.ability.rect.y = self.rect.y
194
195
196
```

I called this function in the update() method for the Fighter class.

```
204 def update(self, target, surface):
205     self.updateAbilityPos()
206     if self.ability.active:
207         self.ability.draw(surface)
208         self.ability.update(target)
209         self.update_action(4)
210     if self.hit:
211         self.knockFriction = 10
212         self.update_action(7)
213
214     elif self.attacking:
215         if self.attacktype == 4:
216             self.update_action(4)
217         elif self.attacktype == 5:
218             self.update_action(5)
219     elif self.jump:
220         self.update_action(2)
221     elif self.running:
222         self.update_action(1)
223     else:
224         self.update_action(0)
225
226
```

Test

Here is a video of my code working.



IMG_9867.mov

Here, the abilities work as intended for each character and they are properly positioned, activated and deactivated when they hit the opponent or when it flies out of the screen. I think one further feature I can easily implement is knockback from the ability, and this will help visually show the impact of the ability.

Attempt 6

I added a knocked attribute to the ability class as a Boolean variable to control when the knockback happens.

```
1  import pygame
2
3  class Ability():
4      def __init__(self, x, y, data, animationsteps, spritesheet, flip, debug=False):
5          self.size = data[0]
6          self.image_scale = data[1]
7          self.offset = data[2]
8          self.flip = flip
9          self.animationlist = self.load_images(spritesheet, animationsteps)
10         frame_width = spritesheet.get_width() // animationsteps
11         self.rect = pygame.Rect(
12             x - (self.offset[0] * self.image_scale), # apply X offset
13             y - (self.offset[1] * self.image_scale), # apply Y offset
14             frame_width * self.image_scale - 100, # use frame width
15             spritesheet.get_height() * self.image_scale # use frame height
16         )
17         self.frame_index = 0
18         self.image = self.animationlist[self.frame_index]
19         self.updateTime = pygame.time.get_ticks()
20         self.abilityBar = 0
21         self.active = False
22         self.cooldown = 7000
23         self.debug = debug # debug colour hitbox
24         self.cycles = 0
25         self.knocked = False
26
```

I added the same knockback code as previously used in the Fighter class on page [134](#). I used the knocked attribute to trigger this knockback when the opponent has been hit by the player's ability.

```

60
61 def update(self, target):
62     #Update animation only if active
63     if self.active:
64         animation_cooldown = 80 #Time between frames (milliseconds)
65         if pygame.time.get_ticks() - self.updateTime > animation_cooldown:
66             self.frame_index += 1
67             self.updateTime = pygame.time.get_ticks()
68
69         #Reset to the first frame if the animation ends
70         if self.frame_index >= len(self.animationlist)-1:
71             self.cycles += 1
72             self.frame_index = 0
73
74         if self.cycles > 5:
75             self.cycles = 0
76             self.active = False
77
78         self.image = self.animationlist[self.frame_index]
79
80
81         if self.rect.colliderect(target.rect):
82             target.health -= 16.5#damage of ability
83             target.hit = True
84             self.knocked = True
85             self.active = False
86
87         if self.knocked:#same knockback code as in fighter class
88             if not target.flip:
89                 target.vel_x = -target.knockFriction-20
90                 target.rect.x += target.vel_x
91                 target.knockFriction -= 1
92
93             else:
94
95                 target.vel_x = target.knockFriction+20
96                 target.rect.x += target.vel_x
97                 target.knockFriction -= 1
98
99
100         if target.knockFriction <= 0.25: #reset knockback and velocity to 0 when knockback is negligible
101             target.knockFriction = 0
102             target.vel_x = 0
103             self.knocked = False
104
105
106

```

Test

Here is a video of my code working.



20250330_1845110
00_iOS.MOV

The ability successfully knocks back the opponent player once it hits them and there are no visible signs of an error. This means that the ability success criterion has been completed as users have a power-up that they can use to give them an advantage in the game.

(Should have) Success Criteria 7: Battle Screen Music

Another extra feature I should add to the game is music and sound effects. This will help to bring out the atmosphere of each screen and adds to the fighting theme. Each music piece/ sound is from my research from the Analysis Stage.

Attempt 1

I initialised the mixer library function in pygame so that all the mixer functions can be used later on in the program.

```
1 import pygame
2 from UFGfighter import Fighter
3
4 pygame.init()
5 pygame.mixer.init()
6
```

I then added the musicPlaying variable into the game variables and it will be used to control what music plays at any one moment. I also loaded the battle music from my files and

```
42 #define game variables
43 MainMenu = False
44 firstLaunch = True
45 charSelect = False
46 startBattle = False
47 timerStart = False
48 TimeOut = False
49 battleDone = False
50 musicPlaying = False
51
52 #load music and sound effects
53 battleMusic = pygame.mixer.music.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Battle Music.mp3")
54
55
```

I used the musicPlaying variable to control whether the music plays in the battle screen. I set this variable to True inside the condition so that the music doesn't keep restarting.

```
222
223
224 if startBattle:
225
226
227     if not musicPlaying:
228         pygame.mixer.music.set_volume(0.8) #set volume to 80%
229         pygame.mixer.music.play(-1)#loop music
230         musicPlaying = True
231
232     drawTimer()
233
234     P1.draw(screen,YELLOW)
235     P2.draw(screen,YELLOW)
236
237     drawHealth(P1.health,20,20)
238     drawHealth(P2.health,580,20)
239
240     drawAbilityBar(20, 80, P1.ability_charge)
241     drawAbilityBar(580, 80, P2.ability_charge)
242
243
244     P1.ability_charge = min(P1.ability_charge + 0.0025, 1)
245
246     P2.ability_charge = min(P2.ability_charge + 0.0025, 1)
```

Test

Here is video of my code working



20250329_1718430
00_iOS.MOV

The music clearly plays after starting the battle and plays for the whole battle period. Although this success criterion is complete, I can also use this opportunity to complete all the other music/sound effect success criteria as well.

Attempt 2

I loaded all of the sound effects I needed and assigned them to their respective variables. I also set up sound channels so that sound effects can play simultaneously.

```
56 #load music and sound effects
57
58 mainMusic = pygame.mixer.music.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Main Menu Music.mp3")
59
60 punch1 = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Punch 1.mp3")
61 punch2 = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Punch 2.mp3")
62 punch_swing = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Punch Swing.mp3")
63 sword1 = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Sword Hit.mp3")
64 sword2 = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Sword 2.mp3")
65 sword_swing = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Sword swing.mp3")
66 ability_fire = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Ability Shoot.mp3")
67 ability_hit = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Ability Shoot.mp3")
68
69 P1channelAttack = pygame.mixer.Channel(1)
70 P1channelSwing = pygame.mixer.Channel(2)
71
72 P2channelAttack = pygame.mixer.Channel(3)
73 P2channelSwing = pygame.mixer.Channel(4)
74
```

I made a function to play all of the sounds through their respective channels depending on what triggers the user presses.

```

114 #check punch sound
115 def AttackSound(P1,P2):
116     key = pygame.key.get_pressed()
117     if P1.attacking:
118         if key[pygame.K_e] or key[pygame.K_r]:
119             if not P1channelSwing.get_busy():
120                 P1channelSwing.queue(punch_swing)
121                 P1channelSwing.play(punch_swing)
122             if key[pygame.K_e] and P2.hit:
123                 if not P1channelAttack.get_busy():
124                     P1channelAttack.queue(punch1)
125                     P1channelAttack.play(punch1)
126             if key[pygame.K_r] and P2.hit:
127                 if not P1channelAttack.get_busy():
128                     P1channelAttack.queue(punch2)
129                     P1channelAttack.play(punch2)
130         if P1.ability.active and not pygame.mixer.get_busy():
131             ability_fire.play()
132     if P1.ability.rect.colliderect(P2.rect):
133         ability_hit.play()
134
135     if P2.attacking:
136         if key[pygame.K_n] or key[pygame.K_m]:
137             if not P2channelSwing.get_busy():
138                 P2channelSwing.queue(sword_swing)
139                 P2channelSwing.play(sword_swing)
140             if key[pygame.K_n] and P1.hit:
141                 if not P2channelAttack.get_busy():
142                     P2channelAttack.queue(sword1)
143                     P2channelAttack.play(sword1)
144             if key[pygame.K_m] and P1.hit:
145                 if not P2channelAttack.get_busy():
146                     P2channelAttack.queue(sword2)
147                     P2channelAttack.play(sword2)
148         if P2.ability.active and not pygame.mixer.get_busy():
149             ability_fire.play()
150     if P2.ability.rect.colliderect(P1.rect):
151         ability_hit.play()
152
153

```

I then called this function in the main game loop under the startBattle condition.

```

296         musicPlaying = True
297
298         if startBattle:
299
300             AttackSound(P1,P2)

```

Test

Here is me testing the sound effects of my code.



20250329_1832200
00_iOS.MOV

The sound effects clearly work here, and I have managed to complete success criteria 5, 7, and 8 in the should have section. Now the user will be more immersed in their gaming experience of my game because of the sound effects.

Battle Screen: Review

This marks the end of focused development of the Battle screen in my game. On this screen, I have managed to add the character images, animations and functions of the characters, as well as the user interface of the Battle Screen and music/sound effects.

The images I used were from my analysis stage and I displayed them on the Battle screen with an appropriate and visible size. I managed to create a function for creating the animation lists for each animation for both characters (load_images) and created another function to update and display these animations (Update). I then had to use conditions such as running and jump to decide which animation should be played depending on what the user presses for their intended movement. These images will be helpful for the user to see what action their character is committing to and is an exciting visual element of the game.

The functions for the characters included running forwards and backwards, jumping, attacking and using player the player ability. The running function is controlled by the relevant user movement buttons (A and D for player 1, < and > for player 2) and controls whether the user's chosen character moves forwards or backwards. The jump movement function (W for player 1, ^ for player 2) moves the user's character upwards towards the top of the screen, as agreed in the design stage interview. Character attacks (E and R for Player 1, N and M for player 2) allow users to deal damage to their opponents and they now knock the opponent player backwards so that players aren't able to spam their attacks without using other movement buttons. Attack 1 is a normal attack whereas attack 2 deals a greater knockback so that players can escape attacks, giving opponents an opportunity to deal damage back to the player and ensuring a fair game. The player ability (Q for player 1, and B for player 2) is a power-up for the players to use wisely to deal extra damage to their opponent and knock them back, offering an opportunity to turn the tides of a battle. However, these abilities can be dodged so they do not always offer an advantage for the player using it – this ensures that fairness is kept in the game.

The user interface that I created during this stage included the health bars, ability bars and the timer function. The health bars show the character's health at any current moment. When attacks or abilities land on a player, their health will decrease by the respective damage of the attacks (6.5 for attacks, 16.5 for abilities). When I develop the Timeout Screen, I will make sure that the Battle screen ends when a character dies (health bar is all red) or the timer goes above 45 seconds. The ability bar shows the user when their ability will be ready. The timer acts as a boundary for players to aim to fight and defeat their opponent in time.

The music/sound effects added to the main menu and Battle screen aim to immerse the user more into the game with realistic sounds and thematic music in the background. The sound effects are responsive to character actions such as punching, swinging a sword and using abilities.

Here is the current prototype version of my game:



20250330_1645140
00_iOS.MOV

The next screen is the Character Selection screen, and this is where the user will be able choose their preferred character for both Player 1 and Player 2. In this screen, there will also be a CPU option for those single players who still want to play against an active opponent.

Success Criteria achieved

- Success Criteria 17: Character Image
- Success Criteria 14 & 15: Health Bar
- Success Criteria 18 & 23: Character Idle Animation
- Success Criteria 19 & 24: Character Movement
- Success Criteria 20 & 25: Character Jump
- Success Criteria 21 & 26: Character Attack 1
- Success Criteria 22 & 27: Character Attack 2
- Success Criteria 13: Timer
- Success Criteria 12: Battle Screen
- (Should have) Success Criteria 1 & 3: Ability 1
- (Should have) Success Criteria 5: Main Menu music
- (Should have) Success Criteria 7: Battle screen music
- (Should have) Success Criteria 8: Punch sound effect
- (Should have) Success Criteria 10: Damage sound effect
- (Should have) Success Criteria 11: Ability Bar

Stage Three – Character Selection Screen

This screen primarily focuses on user selection of their preferred character to use in the battle screen and also the use of CPU on characters when there is only one player.

Success Criteria 8: Character 1 Button

On the Character Selection screen, the user will need to be able to choose their character by clicking on a recognisable character. So, I will need to make these buttons for the user to click on.

Attempt 1

I created the necessary game variables for the character selection screen in the main game program.

```
44 #define game variables
45 MainMenu = False
46 firstLaunch = True
47 charSelect = False
48 startBattle = False
49 timerStart = False
50 TimeOut = False
51 battleDone = False
52 musicPlaying = False
53 P1sound = False
54 P2Sound = False
55 MainMenuMusic = False
56 battleMusic = False
57 P1Picked = False
58 P2Picked = False
59 shinobiPicked = False
60 samuraiPicked = False
61 CPU1 = False
62 CPU2 = False
63
```

I then created the charPortraits() function which outputs the character buttons on the Character Selection screen. I only did the left side so I could test if it works and is visible for the user.

```
225 #character selection
226 def charPortraits():
227     global menuColor1
228     global menuColor2
229     global menuColor3
230     portraitWidth, portraitHeight = 125, 175
231     global P2
232     global P1
233     P1.update_action(0) #image for character button
234     P2.update_action(0)
235     P1image = pygame.transform.flip(P1.image, False, False) #image face each other
236     P2image = pygame.transform.flip(P2.image, True, False)
237
238     #left side
239     portrait1 = pygame.transform.scale(P1image, (portraitWidth, portraitHeight))
240     portrait2 = pygame.transform.scale(P2image, (portraitWidth, portraitHeight))
241     pygame.draw.rect(screen, menuColor1, (50, 200, portraitWidth, portraitHeight))
242     pygame.draw.rect(screen, menuColor2, (250, 200, portraitWidth, portraitHeight))
243     screen.blit(portrait1, (50, 200))
244     screen.blit(portrait2, (250, 200))
245
246
247
```

I changed the function of the play button to open the character select screen instead of the battle screen.

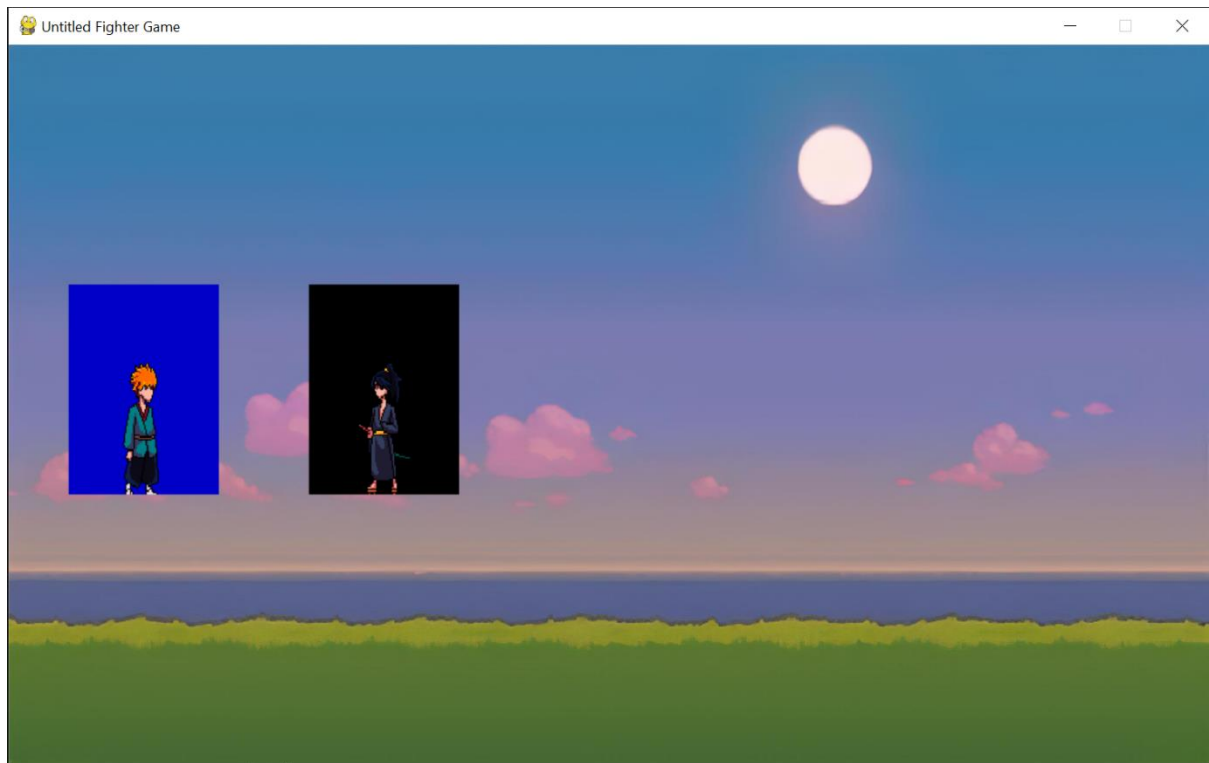
```
287 clock = pygame.time.Clock()
288 #game loop
289 done = False
290 while not done:
291
292     #get mouse position
293     mouse = pygame.mouse.get_pos()
294
295     #events + game logic
296     for event in pygame.event.get():
297         if event.type == pygame.QUIT:
298             done = True
299
300     #logic
301     if event.type == pygame.MOUSEBUTTONDOWN:
302         if (screen_width/2)-100 <= mouse[0] <= (screen_width/2)+100 and startttextY <= mouse[1] <= startttextY+75:#pr
303             if startBattle:#check if game already started
304                 pass
305             elif MainMenu:
306                 charSelect = True
307                 MainMenu = False
308
309
310
```

I then called the charPortraits() function under the charSelect condition.

```
344     if charSelect:
345         charPortraits()
346
347
```

Test

Here is the output of my code.



The character buttons do appear in acceptable positions for player 1 and the character images are visible. However, the button for the shinobi is highlighted blue without me adding any highlighted button code, so I will need to fix this by correcting the colour of the button beforehand and adding more menu colours for the other buttons I will create.

Attempt 2

I made the y position of the portrait a global variable and used it in the positioning of the buttons. I then made similar code for the right side, using 650 and 850 as the x coordinates.

```

229 def charPortraits():
230     global menuColor1
231     global menuColor2
232     global menuColor3
233     global menuColor4
234     portraitWidth, portraitHeight = 125, 175
235     global portraitY
236     global P2
237     global P1
238     P1.update_action(0) #image for character button
239     P2.update_action(0)
240     P1image = pygame.transform.flip(P1.image, False, False) #image face each other
241     P2image = pygame.transform.flip(P2.image, True, False)
242
243     #left side
244     portrait1 = pygame.transform.scale(P1image, (portraitWidth, portraitHeight))
245     portrait2 = pygame.transform.scale(P2image, (portraitWidth, portraitHeight))
246     pygame.draw.rect(screen, menuColor1, (50, 200, portraitWidth, portraitHeight))
247     pygame.draw.rect(screen, menuColor2, (250, 200, portraitWidth, portraitHeight))
248     screen.blit(portrait1, (50, portraitY))
249     screen.blit(portrait2, (250, portraitY))
250     #highlight buttons
251     if not P1Picked:
252         if 50 <= mouse[0] <= 180 and portraitY <= mouse[1] <= portraitY+180:
253             menuColor1 = BLUE
254         else:
255             menuColor1 = BLACK
256         if 250 <= mouse[0] <= 380 and portraitY <= mouse[1] <= portraitY+180:
257             menuColor2 = BLUE
258         else:
259             menuColor2 = BLACK
260
261
262     #right side
263     portrait3 = pygame.transform.scale(P1image, (portraitWidth, portraitHeight))
264     portrait4 = pygame.transform.scale(P2image, (portraitWidth, portraitHeight))
265     pygame.draw.rect(screen, menuColor3, (650, 200, portraitWidth, portraitHeight))
266     pygame.draw.rect(screen, menuColor4, (850, 200, portraitWidth, portraitHeight))
267     screen.blit(portrait3, (650, portraitY))
268     screen.blit(portrait4, (850, portraitY))
269     if not P2Picked:
270         if 650 <= mouse[0] <= 780 and portraitY <= mouse[1] <= portraitY+180:
271             menuColor3 = BLUE
272         else:
273             menuColor3 = BLACK
274         if 850 <= mouse[0] <= 980 and portraitY <= mouse[1] <= portraitY+180:
275             menuColor4 = BLUE
276         else:
277             menuColor4 = BLACK
278
279

```

I then made the functions for the buttons in the main game loop, ensuring that the buttons turn green when a player has confirmed their character selection.

```

339         elif MainMenu:
340             menuColor1 = BLACK
341             charSelect = True
342             MainMenu = False
343
344     if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and quitY <= mouse[1] <= quitY+50:#PRESSED QUIT
345         if MainMenu: #CHECKS IF ON MAIN MENU SCREEN
346             done = True
347
348
349     if 50 <= mouse[0] <= 180 and portraitY <= mouse[1] <= portraitY+180: #Character 1 Button for Player 1
350         if charSelect:
351             if not P1Picked:
352                 if not P2Picked:
353                     shinobiPicked = True
354                     P1Picked = True
355                     menuColor1 = GREEN
356                 elif P2Picked and not shinobiPicked:
357                     shinobiPicked = True
358                     P1Picked = True
359                     menuColor1 = GREEN
360             else:
361                 if not P2Picked:
362                     shinobiPicked = False
363                     P1Picked = False
364                     menuColor1 = BLACK
365
366
367     if 250 <= mouse[0] <= 380 and portraitY <= mouse[1] <= portraitY+180: #Character 2 Button for Player 1
368         if charSelect:
369             if not P1Picked:
370                 if not P2Picked:
371                     samuraiPicked = True
372                     P1Picked = True
373                     menuColor2 = GREEN
374                 elif P2Picked and not samuraiPicked:
375                     samuraiPicked = True
376                     P1Picked = True
377                     menuColor2 = GREEN
378             else:
379                 if not P2Picked:
380                     samuraiPicked = False
381                     P1Picked = False
382                     menuColor1 = BLACK
383
384
385

```

I used similar code from above to make the functions for the player 2 buttons.

```
383
384     if 650 <= mouse[0] <= 780 and portraitY <= mouse[1] <= portraitY+180: #Character 1 Button for Player 2
385         if charSelect:
386             if not P2Picked:
387                 if not P1Picked:
388                     shinobiPicked = True
389                     P2Picked = True
390                     menuColor3 = GREEN
391                 elif P1Picked and not shinobiPicked:
392                     shinobiPicked = True
393                     P2Picked = True
394                     menuColor3 = GREEN
395             else:
396                 if not P1Picked:
397                     shinobiPicked = False
398                     P2Picked = False
399                     menuColor1 = BLACK
400
401     if 850 <= mouse[0] <= 980 and portraitY <= mouse[1] <= portraitY+180: #Character 2 Button for Player 2
402         if charSelect:
403             if not P2Picked:
404                 if not P1Picked:
405                     samuraiPicked = True
406                     P2Picked = True
407                     menuColor4 = GREEN
408                 elif P1Picked and not samuraiPicked:
409                     samuraiPicked = True
410                     P2Picked = True
411                     menuColor4 = GREEN
412             else:
413                 if not P1Picked:
414                     samuraiPicked = False
415                     P2Picked = False
416                     menuColor1 = BLACK
417
418
```

Test

Here is a video of my code working.



20250330_1657040
00_iOS.MOV

The buttons appear correctly and are correctly highlighted blue when the mouse hovers over it and green when the user selects their character. This means that success criteria 8 and 9 are complete as users will be able to select their choice of character for the Battle screen.

Success Criteria 10: CPU Selection Button

This next success criterion deals with CPU selection of fighters. This will allow users to fight against a computer-controlled opponent.

Attempt 1

I created the CPU text as the text5 variable

```
21 # defining menu font
22 TitleFont = pygame.font.SysFont('Corbel',70,True)
23 MenuFont = pygame.font.SysFont('Corbel',35 )
24 #render menu font
25 text1 = MenuFont.render('Start Game' , True , WHITE)
26 text2 = MenuFont.render('Options' , True , WHITE)
27 text3 = MenuFont.render('Quit' , True , WHITE)
28 text4 = TitleFont.render('Untitled Fighter Game' , True , WHITE)
29 text5 = MenuFont.render('CPU' , True , WHITE)
30
31
```

I added the drawing code in the charPortraits() method. It uses similar code to that but I used the character portrait positions to accurately position the CPU buttons below them.

```
290 menuColor4 = BLACK
291 #CPU Buttons
292 pygame.draw.rect(screen,menuColor5,(125,450,175,50))
293 screen.blit(text5,(140,460))
294 pygame.draw.rect(screen,menuColor6,(725,447,175,50))
295 screen.blit(text5,(740,457))
296 if not CPU1:
297     if 125 <= mouse[0] <= 300 and 450 <= mouse[1] <= 500:
298         menuColor5 = BLUE
299     else:
300         menuColor5 = BLACK
301
302 if not CPU2:
303     if 725 <= mouse[0] <= 900 and 450 <= mouse[1] <= 500:
304         menuColor6 = BLUE
305     else:
306         menuColor6 = BLACK
307
308
309
310
```

I added the button function for the CPU buttons in the main game loop.

```
442         if 125 <= mouse[0] <= 300 and 450 <= mouse[1] <= 500:#player 1 CPU toggle
443             if charSelect:
444                 if not CPU1:
445                     CPU1 = True
446                     menuColor5 = GREEN
447                 else:
448                     CPU1 = False
449                     menuColor5 = BLACK
450
451         if 725 <= mouse[0] <= 900 and 450 <= mouse[1] <= 500:#player 2 CPU toggle
452             if charSelect:
453                 if not CPU2:
454                     CPU2 = True
455                     menuColor6 = GREEN
456                 else:
457                     CPU2 = False
458                     menuColor6 = BLACK
459
```

Test

Here is a video of the CPU buttons working.



20250330_1711130
00_iOS.MOV

The CPU buttons clearly appear on the Character Selection screen. They are black and turn blue when the mouse hovers over them, and also turn green when selected. They can be deselected as well. It would now make sense to make the actual CPU function in the Fighter class so that the button can work as intended.

Attempt 2

I added the CPU as a Boolean parameter and attribute to the Fighter class constructor method.

```

4  class Fighter():
5      def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1,CPU, ability_data):
6          self.size = data[0]
7          self.image_scale = data[1]
8          self.offset = data[2]
9          self.flip = flip
10         self.animationlist = self.load_images(sprite_sheet, animationsteps)
11         self.rect = pygame.Rect(x,y,80,150)
12         self.running = False
13         self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
14         self.frame_index = 0
15         self.image = self.animationlist[self.action][self.frame_index]
16         self.updateTime = pygame.time.get_ticks()
17         self.player = Player1
18         self.CPU = CPU
19         self.jump = False
20         self.attacking = False
21         self.attacktype = 0
22         self.hit = False
23         self.vel_y = 0
24         self.health = 100
25         self.knockFriction = 0
26         self.useAbility = False
27         self.ability_charge = 0
28         self.ability_cooldown = False
29         self.ability = Ability(
30             x=self.rect.centerx,
31             y=self.rect.centery,
32             data=ability_data[0],
33             animationsteps=ability_data[1],
34             spritesheet=ability_data[2],
35             flip=flip,
36         )
37
38

```

I then added these arguments in the main game program.

```

340
341 P1 = Fighter(200,450, False, ShinobiData, shinobi_spritesheet,shinobi_Animationsteps, True,CPU1, ability_data)
342 P2 = Fighter(700,450,True, SamuraiData, samurai_spritesheet, samurai_Animationsteps, False,CPU2, ability_data)
343
344
345 # Used to manage how fast the screen updates
346 clock = pygame.time.Clock()
347 #game loop
348 done = False
349 while not done:
350
351     #get mouse position
352     mouse = pygame.mouse.get_pos()
353
354

```

I added the random library because CPU movements will be random and unpredictable for users, adding a level of challenge to this option.

```
1 import pygame, random
2 from UFGability import Ability
3
```

I added the CPU function in the move() method of the Fighter class. CPU movement and attacks will be randomly made when in range of the player.

```
69
70     if self.CPU:
71
72         if not self.attacking:
73
74             #if CPU can attack player
75             if abs(target.rect.centerx - self.rect.centerx) < 45:
76                 self.attack(surface, target)
77                 self.attacking = True
78                 if random.randint(0,1) == 0:
79                     self.attack_type = 4
80                 else:
81                     self.attack_type = 5
82
83             if self.checkAbilityBar:
84                 self.activate_ability()
85
86
87         #if player is in the air, CPU jumps
88         if random.randint(0,45)--7:
89             if self.rect.bottom + dy > screen_height - 0 and not self.dead:
90                 self.vel_y = 0
91                 self.jump = False
92                 dy = (screen_height - 0) - self.rect.bottom
93             else:
94                 self.vel_y = -30
95                 self.jump = True
96         #CPU chases player
97         if target.rect.centerx+10 > self.rect.centerx:
98             dx = speed*5
99             self.running = True
100         elif target.rect.centerx-10 < self.rect.centerx:
101             dx = -speed*5
102             self.running = True
103     else:
104         if self.player:
105
106             if not self.attacking:
107
108                 if key[pygame.K_a]:
109                     dx = -speed # change in speed becomes negative, moves char backwards
110                     self.running = True
111                 if key[pygame.K_d]:
112                     dx = speed
113                     self.running = True
114                 if key[pygame.K_w] and not self.jump:
115                     self.vel_y = -30 #makes player jump up
116                     self.jump = True
117
118             #attack
119             if key[pygame.K_e] or key[pygame.K_r] or key[pygame.K_q]:
120                 self.attack(surface, target)
121                 self.attacking = True
122
```

I incorporated CPU attacking inside the sound checks for the characters.

```

134
135 #check_punch_sound
136 def AttackSound(P1,P2):
137     key = pygame.key.get_pressed()
138     if P1.attacking:
139         if key[pygame.K_e] or key[pygame.K_r] or P1.attacktype == 4 or P1.attacktype == 5:
140             if not P1channelSwing.get_busy():
141                 P1channelSwing.queue(punch_swing)
142                 P1channelSwing.play(punch_swing)
143             if (key[pygame.K_e] or P1.attacktype == 4) and P2.hit:
144                 if not P1channelAttack.get_busy():
145                     P1channelAttack.queue(punch1)
146                     P1channelAttack.play(punch1)
147             if (key[pygame.K_r] or P1.attacktype == 5) and P2.hit:
148                 if not P1channelAttack.get_busy():
149                     P1channelAttack.queue(punch2)
150                     P1channelAttack.play(punch2)
151             if P1.ability.used and not P1AbilityFly.get_busy():
152                 P1AbilityFly.queue(ability_fire)
153                 P1AbilityFly.play(ability_fire)
154             if P1.ability.rect.colliderect(P2.rect) and P1.ability.used and not P1AbilityHit.get_busy():
155                 P1AbilityHit.queue(ability_hit)
156                 P1AbilityHit.play(ability_hit)
157
158     if P2.attacking:
159         if key[pygame.K_n] or key[pygame.K_m] or P2.attacktype == 4 or P2.attacktype == 5:
160             if not P2channelSwing.get_busy():
161                 P2channelSwing.queue(sword_swing)
162                 P2channelSwing.play(sword_swing)
163             if (key[pygame.K_n] or P2.attacktype == 4) and P1.hit:
164                 if not P2channelAttack.get_busy():
165                     P2channelAttack.queue(sword1)
166                     P2channelAttack.play(sword1)
167             if (key[pygame.K_m] or P2.attacktype == 5) and P1.hit:
168                 if not P2channelAttack.get_busy():
169                     P2channelAttack.queue(sword2)
170                     P2channelAttack.play(sword2)
171             if P2.ability.used and not P2AbilityFly.get_busy():
172                 P2AbilityFly.queue(ability_fire)
173                 P2AbilityFly.play(ability_fire)
174             if P2.ability.rect.colliderect(P1.rect) and P2.ability.used and not P2AbilityHit.get_busy():
175                 P2AbilityHit.queue(ability_hit)
176                 P2AbilityHit.play(ability_hit)
177

```

Test

This test was completed after the development of the start game button. Page [175](#).

Success Criteria 11: Start Battle Button

This button will allow the user to transition into the Battle screen from the Character Selection screen.

Attempt 1

I added the start game text as a menu font render.

```
21 # defining menu font
22 TitleFont = pygame.font.SysFont('Corbel',70,True)
23 MenuFont = pygame.font.SysFont('Corbel',35 )
24 #render menu font
25 text1 = MenuFont.render('Start Game' , True , WHITE)
26 text2 = MenuFont.render('Options' , True , WHITE)
27 text3 = MenuFont.render('Quit' , True , WHITE)
28 text4 = TitleFont.render('Untitled Fighter Game' , True , WHITE)
29 text5 = MenuFont.render('CPU' , True , WHITE)
30 text6 = MenuFont.render('Start Battle!' , True , WHITE)
31
32
33
```

I then included the drawing function in the CharPortraits() method. I positioned the button near the bottom of the screen.

```
309 #start battle button
310 pygame.draw.rect(screen,menuColor7,(420,550,180,50))
311 screen.blit(text6,(425,555))
312 if 420 <= mouse[0] <= 600 and 550 <= mouse[1] <= 600:
313     menuColor7 = BLUE
314 else:
315     menuColor7 = BLACK
316
```

I added a switchedChar Boolean variable in the main program.

```
67 CPU1 = False
68 CPU2 = False
69 switchedChar = False
```

I added the function of the start battle button under the function of the cpu button in the main game loop.

```
473
474     if 420 <= mouse[0] <= 600 and 550 <= mouse[1] <= 600:#start game after characters picked
475         if P1Picked and P2Picked:
476             menuColor7 = GREEN
477             charSelect = False
478             if menuColor2 == GREEN or menuColor3 == GREEN:
479                 del P1
480                 del P2
481                 P1 = Fighter(200,450,False, SamuraiData, samurai_spritesheet, samurai_Animationsteps, True,CPU1, ability_data)
482                 P2 = Fighter(700,450, True, ShinobiData, shinobi_spritesheet, shinobi_Animationsteps, False,CPU2,ability_data)
483                 switchedChar = True
484                 startBattle = True
485
```

I had to use the switchedChar variable to switch the sound of the players if they were not in the original order assigned.

```
518     if startBattle:
519         if not switchedChar:
520             AttackSound(P1,P2)
521         else:
522             AttackSound(P2,P1)
523     if MainMenuMusic:
```

Test

Here is a video of my code working.



12-27-15.mp4

The start battle button does correctly switch the screen to the Battle screen and is positioned correctly. The CPU function also works as well so this completes success criteria 10 and 11.

Character Selection Screen: Review

The Character Selection screen is now complete, and I have successfully added the character buttons, the CPU buttons and the start game button for both players. The screen now allows users to select their preferred character to use in battle.

The character buttons allow the user to pick their preferred character for player 1 and player 2. They have the character images inside the buttons so that they are recognisable and easier to pick. There are two buttons for each player. The buttons are big and located along the middle of the screen so that the user knows to pick a character. When the mouse hovers over the buttons, they are highlighted blue, and when the user presses their mouse on the button, they are highlighted green.

The CPU buttons are directly below the character buttons for both players. They allow the user to decide which player will be a computer-controlled player with random movements and attacks being made. The buttons highlight blue when the mouse hovers over it, and it highlights green when selected by the user.

The start battle button is at the very bottom of the screen and allows the user to move on to the Battle screen after selecting their preferred character. The button highlights blue when the mouse hovers over it and is highlighted green when selected.

Here is the prototype version of this game up to this point:



12-24-23.mp4

Success Criteria achieved

- Success Criteria 8: Character 1 Button
- Success Criteria 9: Character 2 Button
- Success Criteria 10: CPU Button
- Success Criteria 11: Start Battle Button

The last problem that needs to be fixed is the replay ability of the game. Currently, the game just ends after one battle, but I want users to be able to experience many battles so I will need to add a win screen and a timeout screen for the end of the Battle screen

Stage Four – Win/Timeout Screen

This stage will focus on the ending of the Battle screen and what happens after, whether the user wants there to be a restarted battle or a return to the main menu.

Success Criteria 28: Knock Out Death

This success criterion introduces knock out deaths into the game which will serve as an indication of player death. This will also trigger the subsequent win screen for the player left alive.

Attempt 1

I added the dead attribute to the constructor class as a Boolean attribute.

```

4 class Fighter():
5     def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1,CPU, ability_data):
6         self.size = data[0]
7         self.image_scale = data[1]
8         self.offset = data[2]
9         self.flip = flip
10        self.animationlist = self.load_images(sprite_sheet, animationsteps)
11        self.rect = pygame.Rect(x,y,80,150)
12        self.running = False
13        self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead
14        self.frame_index = 0
15        self.image = self.animationlist[self.action][self.frame_index]
16        self.updateTime = pygame.time.get_ticks()
17        self.player = Player1
18        self.CPU = CPU
19        self.jump = False
20        self.attacking = False
21        self.attacktype = 0
22        self.hit = False
23        self.dead = False

```

I then used this variable to control CPU actions and player actions (can't move if they are dead)

```

67
68 #get key pressed on keyboard
69 key = pygame.key.get_pressed()
70 if not self.dead:
71
72     if self.CPU:
73
74         if not target.dead:
75
76             if not self.attacking:
77
78                 #if CPU can attack player
79                 if abs(target.rect.centerx - self.rect.centerx) < 45:
80                     self.attack(surface,target)
81                     self.attacking = True
82                     if random.randint(0,1) == 0:
83                         self.attack_type = 4
84                     else:
85                         self.attack_type = 5
86
87             if self.checkAbilityBar():
88                 self.activate_ability()
89
90
91         #if player is in the air, CPU jumps
92         if random.randint(0,45)==7:
93             if self.rect.bottom + dy > screen_height - 0 and not self.dead:
94                 self.vel_y = 0
95                 self.jump = False
96                 dy = (screen_height - 0) - self.rect.bottom
97             else:
98                 self.vel_y = -30
99                 self.jump = True
100
101         #CPU chases player
102         if target.rect.centerx+10 > self.rect.centerx:
103             dx = speed-5
104             self.running = True
105         elif target.rect.centerx+10 < self.rect.centerx:
106             dx = -speed + 5
107             self.running = True

```

I also used the dead variable to control if the character stays on the screen. As the character gets knocked out after dying, they will need to be allowed to pass the boundaries of the screen.

```
185
186     #check player on screen
187     if self.rect.left + dx < 0 and not self.dead: #if the speed and position of character beyond left screen border
188         dx = -self.rect.left #limit character speed to left border
189     if self.rect.right + dx > screen_width and not self.dead: #if the speed and position of character beyond right screen border
190         dx = screen_width - self.rect.right #limit character speed to right border
191     if self.rect.bottom + dy > screen_height and not self.dead:
192         self.vel_y = 0
193         self.jump = False #character can jump again
194         dy = screen_height - self.rect.bottom #limit change in y
195     if self.rect.top + dy < 0 and not self.dead:
196         dy = -self.rect.top
197
198
```

I added the local variable knock_x in the move method for the Fighter class. This will control the speed of the knockback of death.

```
60     def move(self, screen_width, screen_height, target, surface):
61         self.running = False
62         speed = 15
63         gravity = 1.4
64         dx = 0 #positive = going forward, negative = going backwards
65         dy = 0
66         knock_x = 0
67
68
```

If the player is alive, their player position will update as normal. If they are dead, they will be knocked out using a quadratic curve I made. The speed of the knock out will be 20.

```
207
208     #update player position
209     if not self.dead:
210         self.rect.x += dx
211         self.rect.y += dy
212     elif self.dead:
213         self.rect.y -= knock_x**2-5*knock_x - 20#fling player
214         if self.player:
215             if self.flip:
216                 knock_x = 10
217                 self.rect.x+=knock_x
218             else:
219                 knock_x = 10
220                 self.rect.x-=knock_x
221
222         if not self.player:
223             if self.flip:
224                 knock_x = 10
225                 self.rect.x+=knock_x
226             else:
227                 knock_x = 10
228                 self.rect.x-=knock_x
229
230
```

Finally, I needed to trigger the dead variable when the player's health reaches 0.

```
271         self.update_action()
272     if self.health <= 0:
273         self.dead = True
274         self.health = 0.1
275
```

Test

Here is a video of the knockout code working.



2025-04-01
22-36-54.mkv

Here, the character getting defeated gets knocked out of bounds when their health reaches 0 which is a success.

Success Criteria 29: Win Screen

I will now need to create a win screen for the player that is left alive.

Attempt 1

I first added the restart game variable since I hadn't done so already.

```
53
54 #define game variables
55 MainMenu = False
56 firstLaunch = True
57 charSelect = False
58 startBattle = False
59 timerStart = False timerStart = False
60 TimeOut = False TimeOut = False
61 battleDone = False
62 musicPlaying = False
63 P1sound = False P1sound = False
64 P2Sound = False P2Sound = False
65 MainMenuMusic = False
66 battleMusic = False
67 P1Picked = False
68 P2Picked = False
69 shinobiPicked = False
70 samuraiPicked = False
71 CPU1 = False
72 CPU2 = False
73 switchedChar = False
74 restart = False
```

I then rendered the different win texts and the texts for the buttons on the win screen.

```
20
21 # defining menu font
22 TitleFont = pygame.font.SysFont('Corbel',70,True)
23 MenuFont = pygame.font.SysFont('Corbel',35 )
24 #render menu font
25 text1 = MenuFont.render('Start Game' , True , WHITE)
26 text2 = MenuFont.render('Options' , True , WHITE)
27 text3 = MenuFont.render('Quit' , True , WHITE)
28 text4 = TitleFont.render('Untitled Fighter Game' , True , WHITE)
29 text5 = MenuFont.render('CPU' , True , WHITE)
30 text6 = MenuFont.render('Start Battle!' , True , WHITE)
31 text7 = MenuFont.render('P1 Wins!' , True , WHITE)
32 text8 = MenuFont.render('P2 Wins!' , True , WHITE)
33 text9 = MenuFont.render('Restart' , True , WHITE)
34 text10 = MenuFont.render('Main Menu' , True , WHITE)
35
36
```

I created the drawBattleDone() method in the main game program, taking a Boolean value as an argument to determine which player has won. The drawing code is otherwise similar to previous drawing methods in the Character Selection screen.

```

322 #win screen
323 def drawBattleDone(P1):
324     global menuColor8
325     global menuColor9
326
327     if P1: #if P1 wins, print 'P1 wins'
328         screen.blit(text7 , (screen_width/2-80,screen_height/2 - 80))
329     else: #Else print 'P2 wins'
330         screen.blit(text8 , (screen_width/2-80,screen_height/2 - 80))
331     pygame.draw.rect(screen,menuColor8,(screen_width/2-80,screen_height/2,158,40))
332     screen.blit(text9 , (screen_width/2-80,screen_height/2))
333     pygame.draw.rect(screen,menuColor9,(screen_width/2-80,screen_height/2 + 50,158,40))
334     screen.blit(text10 , (screen_width/2-80,screen_height/2 + 50))
335
336     if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 <= mouse[1] <= screen_height/2 +48:#restart button area for highlight
337         menuColor8 = BLUE
338     else:
339         menuColor8 = BLACK
340     if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 + 50 <= mouse[1] <= screen_height/2 + 98:#main menu button area for highlight
341         menuColor9 = BLUE
342     else:
343         menuColor9 = BLACK
344
345

```

I then made restorehealth() and resetposition() methods in the Fighter class as it will help to reset the character's attributes if the user wants to immediately restart the battle.

```

250     def restoreHealth(self):
251         self.dead = False
252         self.health = 100
253     def resetPosition(self,x,y):
254         self.rect = pygame.Rect([x,y,80,150])
255

```

The function for the restart button and main menu button is below. The restart button resets the position of the characters to their starting positions and restores their health from previous rounds. The main menu button resets all menucolors and resets the P1 and P2 object to the original state so that Character Selection doesn't cause an error.

```

513
514     if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 <= mouse[1] <= screen_height/2 +48:#restart game
515     if battleDone:
516         P1.restoreHealth()
517         P2.restoreHealth()
518         if P1.player:
519             P1.resetPosition(200,450)
520         else:
521             P1.resetPosition(700,450)
522         if P2.player:
523             P2.resetPosition(200,450)
524         else:
525             P2.resetPosition(700,450)
526         restart = True
527
528
529     if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 + 50 <= mouse[1] <= screen_height/2 + 98:#return to menu
530     if battleDone:
531         P1.restoreHealth()
532         P2.restoreHealth()
533         MainMenu = True
534         del P1
535         del P2
536         P1 = Fighter(200,450, False, ShinobiData, shinobi_spritesheet, shinobi_Animationsteps, True,CPU1,ability_data)
537         P2 = Fighter(700,450,True, SamuraiData, samurai_spritesheet, samurai_Animationsteps, False,CPU2, ability_data)
538         P1Picked = False
539         P2Picked = False
540         samuraiPicked = False
541         shinobiPicked = False
542         menuColor1,menuColor2,menuColor3,menuColor4,menuColor5,menuColor6,menuColor7,menuColor8,menuColor9 = BLACK,BLACK,BLACK,BLACK,BLACK,BLACK,BLACK,BLACK,BLACK
543         battleDone = False
544
545
546

```

I controlled the screens with the game variables in the game loop here. I also reset the ability charge when the battle restarts.

```

619
620
621     if restart:
622         drawTimer()
623         startBattle = True
624         battleDone = False
625         restart = False
626         timerStart = False
627         P1.ability_charge = 0
628         P2.ability_charge = 0
629
630     if P1.dead or P2.dead:
631         startBattle = False
632         battleDone = True
633
634     if battleDone:
635         if P1.dead:
636             drawBattleDone(False)
637         elif P2.dead:
638             drawBattleDone(True)
639         P1.update()
640         P2.update()
641
642         P1.draw(screen)
643         P2.draw(screen)
644
645         #move fighter
646         P1.move(screen_width, screen_height, screen, P2)
647         P2.move(screen_width, screen_height, screen, P1)
648
649         drawHealth(P1.health,20,20)
650         drawHealth(P2.health,580,20)
651
652     #update display
653     pygame.display.flip()
654
655     #60 frames per second
656     clock.tick(60)
657

```

I also reset the timer when the battle restarts here.

```
196 #draw timer function
197 def drawTimer():
198     global startTicks
199     global battleDone
200     global TimeOut
201     global timerStart
202     global startBattle
203     if not timerStart and startBattle: #when the battle starts, start
204         startTicks = pygame.time.get_ticks()
205         timerStart = True
206     if startBattle: #while the battle is ongoing, keep updating the v
207         seconds = (pygame.time.get_ticks()-startTicks)//1000
208
209     if battleDone:
210         seconds = 0
211         del seconds
212     if restart:
213         seconds = 0
214     if seconds > 45 and startBattle: # if the timer goes over 45 seco
215         startTicks = pygame.time.get_ticks()
216         battleDone = True
217         TimeOut = True
218         startBattle = False
219         timerStart = False
220     if not battleDone: #While the battle is ongoing, keep outputting s
221         timer = MenuFont.render(str(seconds), True, BLACK)
222         screen.blit(timer, (screen_width/2 - 12, screen_height/50))
223
```

Test

Here is a video of my code working.



12-18-29.mp4

This video shows that the game can be restarted after the battle screen and all characters will reset back to their starting positions with their initial attributes.

The video also shows that the user can go back to the main menu after the win screen and can choose what to do from there, whether that be quitting the game or restarting the game with different characters. This completes success criteria 29.

Success Criteria 13: TimeOut Screen

This last screen will occur when the timer runs over the allowed 45 second limit. It will have the same menu options as the win screen.

Attempt 1

I initialised the text variables I would need for the Timeout screen.

```
21 # defining menu font
22 TitleFont = pygame.font.SysFont('Corbel',70,True)
23 MenuFont = pygame.font.SysFont('Corbel',35 )
24 #render menu font
25 text1 = MenuFont.render('Start Game' , True , WHITE)
26 text2 = MenuFont.render('Options' , True , WHITE)
27 text3 = MenuFont.render('Quit' , True , WHITE)
28 text4 = TitleFont.render('Untitled Fighter Game' , True , WHITE)
29 text5 = MenuFont.render('CPU' , True , WHITE)
30 text6 = MenuFont.render('Start Battle!' , True , WHITE)
31 text7 = MenuFont.render('P1 Wins!' , True , WHITE)
32 text8 = MenuFont.render('P2 Wins!' , True , WHITE)
33 text9 = MenuFont.render('Restart' , True , WHITE)
34 text10 = MenuFont.render('Main Menu' , True , WHITE)
35 text11 = MenuFont.render('Time ran out!' , True , WHITE)
36 text12 = MenuFont.render('Its a Draw!' , True , WHITE)
37
38
39
```

I then created the drawTimeout() function in the main game program. This function uses similar drawing code to previous screen drawing functions except for the text being outputted.

```
351
352 #Time out screen
353 def drawTimeout():
354     global menuColor8
355     global menuColor9
356     if P1.health > P2.health:
357         screen.blit(text7 , (screen_width/2-80,screen_height/2 - 80))
358     elif P2.health > P1.health:
359         screen.blit(text8 , (screen_width/2-80,screen_height/2 - 80))
360     else:
361         screen.blit(text12 , (screen_width/2-80,screen_height/2 - 80))
362
363
364     screen.blit(text11 , (screen_width/2-80,screen_height/2 - 140))
365
366     pygame.draw.rect(screen,menuColor8,(screen_width/2-80,screen_height/2,158,40))
367     screen.blit(text9 , (screen_width/2-80,screen_height/2))
368     pygame.draw.rect(screen,menuColor9,(screen_width/2-80,screen_height/2 + 50,158,40))
369     screen.blit(text10 , (screen_width/2-80,screen_height/2 + 50))
370
371     if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 <= mouse[1] <= screen_height/2 +48:
372         menuColor8 = BLUE
373     else:
374         menuColor8 = BLACK
375     if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 + 50 <= mouse[1] <= screen_height/2 + 98:
376         menuColor9 = BLUE
377     else:
378         menuColor9 = BLACK
379
380
```

I used the TimeOut game variable to control when the Timeout screen shows in the main game loop.

```
669     if battleDone:
670         if P1.dead:
671             drawBattleDone(False)
672         elif P2.dead:
673             drawBattleDone(True)
674         elif TimeOut:
675             drawTimeOut()
676
677         P1.update(P2, screen)
678         P2.update(P1, screen)
679
680         P1.draw(screen, YELLOW)
681         P2.draw(screen, YELLOW)
682
683         # move fighter
684         # P1.move(screen_width,screen_height, P2, screen)
685         # P2.move(screen_width,screen_height, P1, screen)
686
687         drawHealth(P1.health,20,20)
688         drawHealth(P2.health,580,20)
689
690
691     #update display
692     pygame.display.flip()
693
694     #60 frames per second
695     clock.tick(60)
696
697
698 #exit pygame
699 pygame.quit()
```

Test

Here is a video testing the Timeout screen.



12-11-45.mp4

The timeout screen appears with the restart and main menu buttons visible. In this video I press restart, and it works as intended



12-15-37.mp4

In this video I test the main menu button, and it works as intended as the user is taken to the main menu. This completes success criteria 13.

Win/Timeout Screen: Review

This is the end of the last stage of implementation with the win and timeout screen being fully complete. These screens will allow the user to instantly replay the game or go back to the main menu.

The win screen shows the text “P1 Wins!” if player 2 is defeated and vice versa. This is shown in white above the menu buttons below it and signifies which player has managed to win the battle.

The restart button effectively resets both player positions, health and ability bars. The button is near the middle of the screen and is a black button that turns blue when the mouse hovers over it.

The main menu button returns back to the main menu and resets all the menu color variables so that Character Selection doesn’t cause errors. It is located below the restart button in both screens and highlights blue when the mouse hovers over it.

The Timeout screen has the same menu buttons as the win screen. The time out screen shows “Time ran out!” text and shows either “P1 Wins!” or “P2 Wins!” depending on who has more health left. If both players have the same amount of health left, then the text “It’s a Draw!” will appear.

Here is the final product of my game:



2025-04-02
01-33-38.mp4

Success Criteria achieved

- Success Criteria 28: Knock Out Death
- Success Criteria 29: Win Screen
- Success Criteria 30: Restart Button on Win Screen
- Success Criteria 31: Main Menu button on Win Screen
- Success Criteria 13: Timeout Screen

Evaluation

Now that the implementation of my project has been completed, I will need to assess how far each success criteria has been met from its creation in the analysis stage. I will assess this using the test plan created in my design stage. I will discuss improvements and maintenance inside this section as well.

Testing for Evaluation

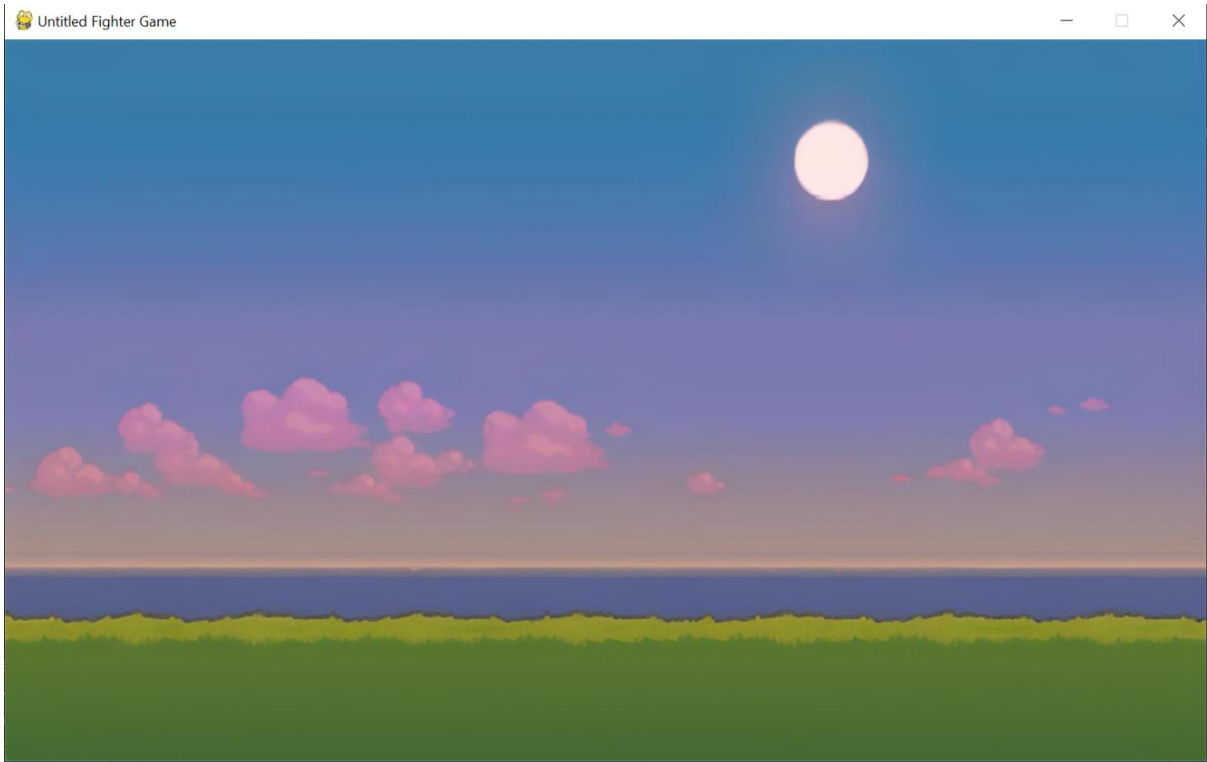
I will go through each success criteria and assess the extent of completion. I will highlight the test plan green if the test fulfils the expected outcome and success criteria, or red if it fails to meet the expected outcome or success criteria.

Success Criteria 1: Background Image

This success criteria concerns displaying the background image chosen in the analysis stage. This was needed because it would allow for the characters being made later on in the game to be visually standing on the ground presented by the background image. The image is also colourful and attractive which helps entice users into playing the game.

I implemented the image assigning it to a global variable using the `convert_alpha()` method after initially getting an error for not using double slashes (//) when locating the file. I then used the `blit()` function to output this onto the screen, making sure to scale the image beforehand to the preferred screen height and width. The

Success criteria 1 was completed on page [75](#) and correctly appears on every screen.



Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
1	01	To see if the background image appears in the game window	Run the program and inspect the output screen for the image	Background image is seen inside game window	Background image is seen inside game window	PASS

Success Criteria 2 & 3: Main Menu and Game Title

The main menu is the first screen that the user sees so it needs to be able to show all the points of navigation so that the user can choose where they want to go next in the game.

To complete this success criteria, I needed to add the game title at the centre of the screen near the top. This is in a big font size because it tells the user the name of the game. The colour of the title is white, so it contrasts the background image colour

The game title was completed on page [87](#) and the main menu on page [94](#).

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
2	02 & 03	Title On Screen	Run the program and inspect the screen to see if the title is visible	Title can be seen on the screen	Title seen on the screen	PASS
3	03	Centred Title	Check if the Title is in the middle of the output screen	Title is centred and is big	Title is big and centred	PASS
4	03	Title text is the game title	Check code text and output text match	Title says "Untitled Fighter Game"	Title says "Untitled Fighter Game"	PASS
5	03	Title is White	Inspect output screen for a white title font	The colour of the title is white	The title colour is white	PASS

Success Criteria 4: Quit

Next, the menu buttons were added with equal spacing and positioned in the middle of the screen. This is so that the buttons are instantly visible to the user, and they are compelled to navigate the game using these buttons.

The first button I added the functionality for was the quit button as this was relatively simple to achieve. All that was needed to do was to leave the game loop by setting the done variable to True. This ended the while loop that the game ran in and subsequently ended the program.

The quit button has a black rectangular box around it as it contrasts the background image colour and is therefore more visible to the user. It has the text "Quit" inside of it so that the user knows what clicking on it will do to the program.

The quit button was completed on page [88](#).

Here is the video for test 20:



2025-03-31
18-07-48.mkv

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
16	04	Quit button that user can see	Check output screen for rectangular button	Visible quit button on screen	Visible quit button	PASS
17	04	"Quit" text inside rectangle	Check output screen for "Quit" text	Quit text visible inside rectangular	Quit test inside button	PASS
18	04	Centred quit button	Check output screen to see quit button in middle of the screen below options button	Button located below options button and centred	Button is below both options and play buttons	PASS
19	04	Quit button exits program when clicked	Click multiple areas within quit button with left mouse button	Program is closed after Quit button is clicked	Program does close	PASS
20	04	Any clicks outside quit button does not exit program	Click around quit button with left mouse button	No change in screen	No change in screen made	PASS

Success Criteria 5: Play Button

The play button allows the user to go to the Character Selection screen to then eventually use those characters in battle. The play button triggers this transition of screen from the Main Menu to the Character Selection Screen.

I made this button alongside all the other main menu buttons, positioning them at the centre of the screen with the play button being on top of all the other buttons. I then added its functionality by making any clicks on the button turn the Main Menu game variable to false and eventually the CharSelect game variable to True. However, during initial development I made the play button set startBattle to True as I was focusing on the development of the battle screen.

The play button was initially completed on [91](#).

The test for test number 10 is below:

There is clearly no change in screen so this test passed. The rest of the tests are passed from



2025-03-31
18-17-51.mkv

inspection.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
6	05	Play button that the user can see	Inspect output screen to see a rectangular button	Rectangular button visible on screen	Play button is visible	PASS
7	05	"Play" Text inside rectangle button	Inspect screen for "Play" text inside button	Play text visible inside rectangular button	Play test visible inside button	PASS
8	05	Centred play button	Check button is in the middle of the screen below title	Button located below game title at the centre of screen	Button is below game title and above the rest of the	PASS

					menu buttons	
9	05 & 07	Play button goes to Character Selection Screen	Press play button in multiple areas within the button with left mouse button	Change from main menu to Character Selection	The button does take the user to the character selection screen	PASS
10	05 & 07	Any clicks outside play button does not take user to Character Selection	Click left mouse button all around the play button	No change in screen	No change at all	PASS

Success Criteria 6: Options Button

This success criteria concerns the options button on the main menu screen.

This button was made alongside the rest of the main menu buttons and is in between the play and quit button. It has the text “Options” inside of it and highlights blue when selected.

Although, this button does not have functionality, it is still visible for the user to see in the main menu and completes the button set for

This success criteria was partially met on page [86](#).

The video for test 14 is below:



2025-03-31
18-22-51.mkv

The options button doesn’t change the screen into the options screen so this test failed here. However, the button does appear so this success criterion is only partially met.

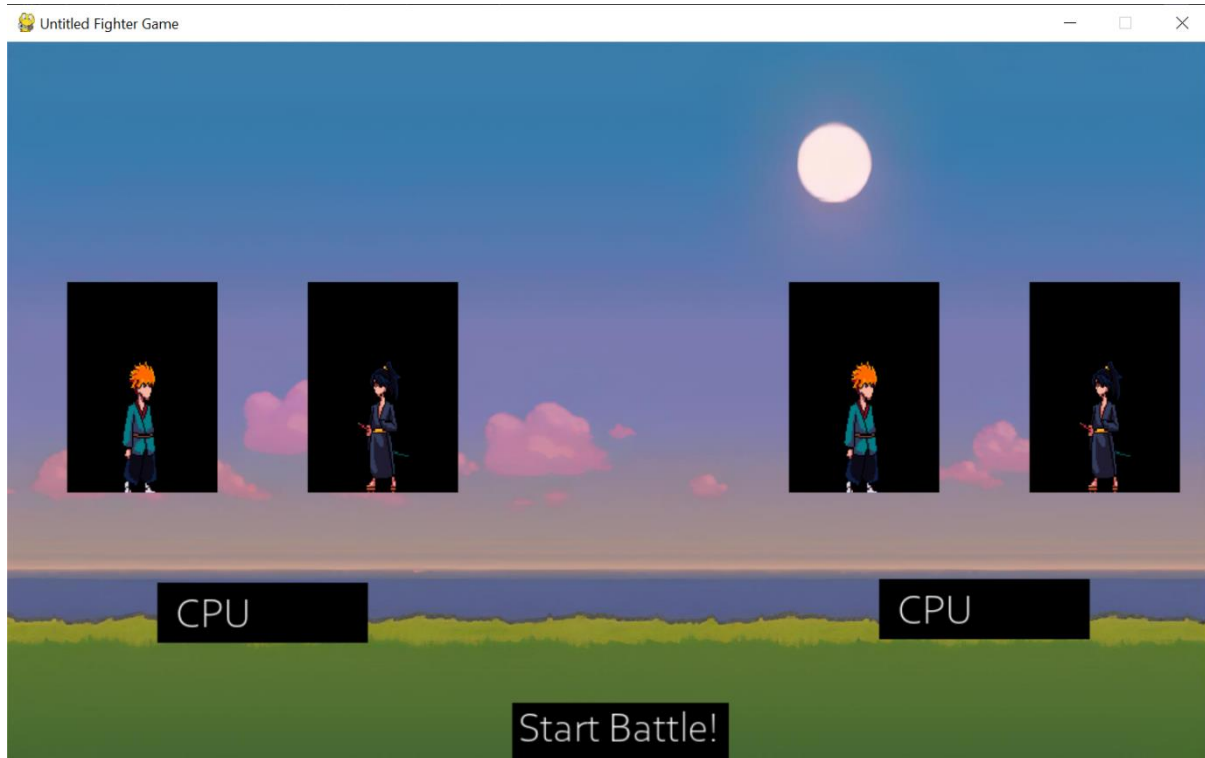
Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
-------------	------------------	-----------------	-------------	------------------	----------------	-----------

11	06	Options button that the user can see	Check output screen for rectangular button	Visible options button on screen	Visible options button	PASS
12	06	"Options" text inside rectangle	Check output screen for "Options" text	Options text visible inside rectangular button	Options text inside button	PASS
13	06	Centred options button	Check output screen for options button below	Button located below play button and centred	Button in between play and quit	PASS
14	06	Options button goes to settings screen	Click multiple areas within the options button with left mouse button	Change from main menu screen to options screen	No change in screen	FAIL
15	06	Any clicks outside options button does not go to settings screen	Click around the options button with left mouse button	No change in screen	No change in screen	PASS

Success Criteria 7: Character Selection screen

This success criterion is about the user's ability to choose their preferred character for battle in the battle screen.

Here is a view of the Character Selection screen:



This screen allows the user to select their character for player 1 and 2. The design is intuitive as the characters appear as a button as well as the CPU and start Battle button. The CPU buttons make the selected player a computer-controlled opponent. The start battle button transitions the Character Selection screen into the Battle screen.

The development of this screen was mainly about the positioning of the buttons and their function. There was a lot of Boolean variables that I had to keep track of such as P1Picked, P2Picked for the player choices and samuraiPicked and shinobiPicked to keep track of what character was picked. This screen is triggered by the play button in the main menu screen and this screen triggers the next screen, the battle screen, when start battle is pressed.

This success criteria was met on page [178](#).

The test for this on page [193](#).

Success Criteria 8: Character 1 Button

This success criteria is about the shinobi button on the Character Selection screen.

The Character 1 button confirms the user's selection of the shinobi character for either player 1 or 2. The button is black and highlights blue when the mouse hovers over it. It is highlighted green when selected and can be deselected if the user makes a mistake during selection.

The development of this button was not particularly difficult since the buttons used similar code to the main menu buttons except from different sizing and positioning and text.

This success criterion was completed on page [170](#).

Test for test number 22, 24, 25, 32,34,35 :



2025-03-31
18-48-14.mkv

The button does not change when not pressed. It turns green when pressed and turns back to black when deselected.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
21	08	Player 1, Shinobi Button on screen	Check output screen	Visible rectangular button on the left side of screen	Visible button for character 1	PASS
22	08	Player 1, Shinobi Button selected	Click multiple areas of the Shinobi button when not selected (the button is black)	Button changes colour from black to green	Buttons changes from black to green	PASS
23	08	Player 1, Shinobi image inside button	Check output screen – all of image should be inside the button	Visible Shinobi character sprite image inside rectangular button	Visible shinobi character inside button	PASS
24	08	Player 1, Shinobi Button deselected	Click multiple areas of the Shinobi button after it has been selected (the button is already green)	Button changes colour from green to black	Button changes from green to black	PASS
25	08	Player 1, Shinobi not selected when user clicks	Click multiple areas outside	No change to green for this specific button	No change	PASS

		outside its button	and around Shinobi button			
31	08	Player 2, Shinobi Button on screen	Check output screen	Rectangular button on the right-hand side of the screen	Visible button for character 1	PASS
32	08	Player 2, Shinobi Button selected	Click multiple areas of the Shinobi button when not selected (the button is black)	Button changes colour from black to green	Buttons changes from black to green	PASS
33	08	Player 2, Shinobi image inside button	Check output screen – all of image should be inside button	Visible Shinobi character sprite image inside rectangular button	Visible shinobi character inside button	PASS
34	08	Player 2, Shinobi Button deselected	Click multiple areas of the Shinobi button after it has been selected (the button is already green)	Button changes colour from green to black	Button changes from green to black	PASS
35	08	Player 2, Shinobi not selected when user clicks outside its button	Click multiple areas outside and around Shinobi button	No change to green for this specific button	No change	PASS

Success Criteria 9: Character 2 Button

This success criterion is about selecting the samurai character (Character 2).

This button performs a similar function to the Character 1 button except the character inside the button is a samurai character and the button selects this fighter for the battle screen.

The development was also similar to the Character 1 button with the main focus being on positioning and the function of the button in setting samuraiPicked to be true when selected and false when deselected. There was also some development of the menucolors being set to green or blue depending on being hovered on or selected.

This success criterion was completed on page [170](#).

The video [above](#) also shows Character 2 tests 27, 29, 30,37,39,40.

The clicks around the button show that it isn't selected when it is not pressed. The click on the button shows that Character 2 is selected when the Character 2 button is pressed. As the button performs its function correctly, this success criteria is successful.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
26	09	Player 1, Samurai Button on screen	Check output screen	Visible rectangular button adjacent to Player 1 Shinobi button	Visible button for character 2	PASS
27	09	Player 1, Samurai Button Selected	Click multiple areas of the Samurai button (button is black already)	Button changes colour from black to green	Buttons changes from black to green	PASS
28	09	Player 1, Samurai image inside button	Check output screen – all of image should be inside button	Visible Samurai character sprite image inside rectangular button	Visible samurai character inside button	PASS
29	09	Player 1, Samurai Button deselected	Click multiple areas Of the Samurai button (button is green already)	Button changes colour from green to black	Button changes from green to black	PASS
30	09	Player 1, Samurai not selected when user clicks	Click multiple areas outside and around Samurai button	No change to green for this specific button	No change	PASS

		outside its button				
36	09	Player 2, Samurai Button on screen	Check output screen	Rectangular button on the right-hand side of the screen adjacent to Shinobi Player 2 button	Visible button for character 2	PASS
37	09	Player 2, Samurai Button selected	Click multiple areas of the Samurai button (button is black already)	Button changes colour from black to green	Buttons changes from black to green	PASS
38	09	Player 2, Samurai image inside button	Check output screen – all of image should be inside button	Visible Samurai character sprite image inside rectangular button	Visible samurai character inside button	PASS
39	09	Player 2, Samurai Button deselected	Click multiple areas Of the Samurai button (button is green already)	The Samurai button goes from green to black	Button changes from green to black	PASS
40	09	Player 2, Samurai not selected when user clicks outside its button	Click multiple areas outside and around Samurai button	No colour change of the button	No change	PASS

Success Criteria 10: CPU Button

This success criteria aimed to make a button in the Character Selection screen which would trigger a character to be controlled by the computer in the battle screen depending on the user's choice of selection.

The development of the CPU button was similar to all the other buttons on the Character Selection screen as the main focus was positioning it under the Character buttons for both players. The function of the button was also quite simple as it only needed to set either CPU1 or CPU2 to True depending on which player wanted it to be selected. However, the actual CPU function on the battle screen required much more code in the Fighter class and using the random library to make the player have random movements and attack when in range

I did have ambitions to include different modes of difficulty for the CPU such as easy, medium and hard modes. However, due to the time constraint, having a focus on main game elements, and many errors during programming, I couldn't get around to developing this aspect of the CPU button. If I were to develop this , I would need to make three CPU variables (easy, medium, hard) and create six new CPU buttons (easy, medium and hard buttons for both players) under the CPU toggle button. If the user toggles on the CPU button, then they would have a selection of these three buttons to choose from. The difference between the modes would be an increase in attack damage for the CPU and an increase in their health so that they are more challenging to defeat in 45 seconds.

This success criteria was initially met on page [175](#).

The test for evaluation is here:



2025-04-01
14-02-44.mkv

The CPU buttons are visible under the Character buttons and highlight in a blue colour when the mouse hovers over it. They do go green when selected and back to black when unselected. This means that the user will know when they have chosen for a player to be a CPU character due to the green colour suggesting its selection. However, the difficulty buttons do not appear at all because I have not developed them. This means that this success criteria has only been partially met.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
45	10	CPU Button under Player 1-character buttons	Check output screen for CPU button under player 1-character buttons	Visible black button under player 1 character buttons	Visible black button under character 1 button	PASS
46	10	CPU Button under Player 2	Check output screen for CPU button under	Visible black button under player 2	Visible black button	PASS

		character buttons	player 1 character buttons	character buttons	under character 2 button	
47	10	CPU Button selection	Click multiple areas of the CPU button (button is black already)	CPU button colour changes from black to green	Button colour changes from black to green	PASS
48	10	CPU deselection	Click multiple areas Of the CPU button (button is green already)	CPU button colour changes from green to black (while green)	Button changes from green to black	PASS
49	10	CPU button only selected when pressed	Click multiple areas outside and around CPU button. Then press CPU button.	CPU button changes to green only when pressed. No change in colour otherwise.	No change when not pressed. Change in colour when pressed	PASS
50	10	CPU Difficulty Buttons after CPU selection	Select CPU option. Then check for CPU Difficulty buttons on output screen	All three buttons (Easy, Medium and Hard) appear after selecting the CPU button	N/A	FAIL
51	10	CPU "Easy" Button	Check output screen for CPU Easy button	Visible black button under CPU button	N/A	FAIL
52	10	"Easy" text on CPU Button	Check output screen for "Easy" text in button	Visible "Easy" text inside Easy button	N/A	FAIL
53	10	CPU "Easy" Selection	Click multiple areas of the CPU Easy button (button is black already)	Button goes from black to green when selected	N/A	FAIL
54	10	CPU "Easy" Deselection	Click multiple areas Of the CPU Easy button (button is green already)	Button goes from green to black when selected (while green)	N/A	FAIL

55		CPU "Medium" Button	Check output screen for CPU Medium button	Visible black button under CPU button	N/A	FAIL
56	10	"Medium" text on CPU Button	Check output screen for "Medium" text in button	Visible "Medium" text inside button	N/A	FAIL
57	10	CPU "Medium" Selection	Click multiple areas of the CPU Medium button (button is black already)	Button goes from black to green when selected	N/A	FAIL
58	10	CPU "Medium" Deselection	Click multiple areas Of the CPU Medium button (button is green already)	Button goes from green to black when selected (while green)	N/A	FAIL
59	10	CPU "Hard" Button	Check output screen for CPU Hard button	Visible black button under CPU button	N/A	FAIL
60	10	"Hard" text on CPU Button	Check output screen for "Hard" text in button	Visible "Hard" text inside button	N/A	FAIL
61	10	CPU "Hard" Selection	Click multiple areas of the CPU Hard button (button is black already)	Button goes from black to green when selected	N/A	FAIL
62	10	CPU "Hard" Deselection	Click multiple areas Of the CPU Hard button (button is green already)	Button goes from green to black when selected (while green)	N/A	FAIL

Success Criteria 11: Start Battle Button

This success criteria aimed to let the user start the battle after they have chosen their characters and CPU options in the Character Selection screen.

The start battle button completed this success criteria by allowing the transition from the Character Selection screen to the Battle screen. It was developed similarly to the other menu buttons with the focus being on its position at the bottom of the page and its function to switch the screen. The user had to select characters for both players in order for the game to start. The start battle button also switched the P1 and P2 objects if the user had chosen them the other way around than originally set.

This button was also completed on page [175](#).

Here is the test for evaluation:



2025-04-01
14-22-25.mkv

Here, the start button and text is clearly visible. It doesn't work when not pressed but does work when pressed by switching the screen. Therefore, this success criteria has been fully met as the user will be able to go to the Battle screen when they have selected their characters for both players.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
63	11	Start Game button	Check output screen for Start Game button in Character Selection Screen	Visible black button	Visible black button	PASS
64	11	"Start Game" text on button	Check output screen for Start Game button in Character Selection Screen	Visible "Start Game" text on button	Visible Start Battle text	PASS
65	11	Start game button at bottom of screen	Check output screen for position of "Start Game" button	"Start Game" at the bottom of the screen and centred	Visible at the bottom of the screen	PASS
66	11 & 12	Start game button goes to Battle Screen	Click "Start Game"	Interface of Character Selection screen replaced with Battle Screen interface.	Screen changes to battle screen	PASS

67	11	Start game only works when pressed	Click multiple areas outside of the "Start Game" button then click the visible box area of the button	Screen only changes if start game pressed. No change otherwise.	Screen doesn't change when not pressed.	PASS
----	----	------------------------------------	---	---	---	------

Success Criteria 12: Battle Screen

This success criteria aimed to have the user be able to go on to and use their character on the battle screen to defeat their opponent, whether that be another user or a CPU entity.

Development of this screen was a large part of my implementation stage with focus being on the user interface, character animations, attacks and movement functions.

The user interface consisted of the health bar for both players, ability bar for both players, and the timer. These aspects were reactive to character actions and the time of the battle, telling the user key information about their character's attributes during the battle such as health and ability bar usage. The background ground is used as the ground level for the characters.

The character animations show the user what their current action is and is visually accurate to what would happen in real life (punches making the opponent take damage). Their movements are quite fast, and they can jump as high as the screen border.

The test for evaluation is [above](#) for this success criteria. The battle screen clearly appears after the start battle button is used by the user and they will be able to use their chosen character, so this success criteria is a success.

One way I could improve this screen is by adding a countdown until the start of the battle. This will ensure that no time is wasted to battle the opponent. Currently, the game instantly starts when the start battle button is pressed which may leave the user with a few seconds unused to battle their opponent. I could make this function inside the timer function by using another variable for the countdown and displaying this in the middle of the screen. The countdown variable will be the difference from when the user pressed start battle until the current moment. If that difference is 3 seconds, then the battle can start.

Success Criteria 13: Timer/Timeout Screen

This success criteria aimed to display the time left of the current battle to the user and the screen that will appear once this runs out (timeout screen). This screen presents the user the opportunity to restart the battle or return to the main menu, allowing for better navigation of the game.

The development of the timer function was fairly easy to make as I had to update the display of the timer at the top of the screen with the seconds variable I was using to keep track of the seconds that have passed since the start of the battle. This being displayed was dependent on whether the screen was the battle screen or not.

The timeout screen was simple as well because I created the win screen before it and it used the same menu buttons with minor differences such as the text displayed. The text "Time ran out!" would display for this screen when the timer goes over 45 seconds, indicating that the user took too long to defeat their opponent.

The timer was completed on page [141](#) and the timeout screen was completed on page [187](#).

The test for evaluation is here:



2025-04-01
15-02-11.mkv

The battle screen starts, and the timer is at 0. I then wait until 45 seconds have passed and the timeout screen does appear. I press restart and the battle screen restarts with the timer at 0. After another 45 seconds, the timeout screen appears again. This time I press the main menu button, and the main menu screen appears. Since the screens change as intended and the buttons and text appear as intended, this success criterion has been achieved as the user can navigate from the timeout screen if they do not defeat their opponent in time and the timer works as a boundary for the players to defeat each other in.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
68	13	Timer during battle screen	Check Battle Screen for timer	Visible timer	Visible timer	PASS
69	13	Timer centred on screen	Check Battle Screen for position of timer	Visibly at the top of screen and centred.	Visible at top of screen	PASS
70	13	Timer text changes after a second	Stopwatch or Clock, measure against the timer and check for synchronisation	Timer matches with clock	Timer matches with clock	PASS
71	13	Timer starts at 0	Quickly check timer when "Start Game" button pressed	When Battle screen starts, timer is visibly at 0	Timer starts at 0	PASS

72	13	Timer counts to 45	Wait until timer runs out on the Battle Screen	45 is visible just before battle ends	Timer ends at 45	PASS
73	13	Timer triggers Timeout screen after 45 seconds	Wait until timer reaches 45 seconds, inspect for a screen change	Battle screen replaced with timeout screen after 45 is visible on timer	Timeout screen appears	PASS
74	13	"Time ran out!" text on screen	Inspect screen	Visible "Time ran out!" text	Visible text	PASS
75	13	"Time ran out!" around the centre of the screen	Inspect screen	Centred text	Centred	PASS
76	13	Restart button	Inspect screen for button	Visible black button	Visible button	PASS
77	13	"Restart" text in restart button (Timeout screen)	Inspect screen for "Restart" text	Visible "Restart" text inside the restart button	Visible text	PASS
78	13	"Main menu" button (Timeout screen)	Inspect screen for "Main menu" button	Visible black button	Visible black button	PASS
79	13	"Main menu" text inside Main Menu button (Timeout screen)	Inspect screen for "Main menu" text	Visible "Main Menu" text inside Main menu button	Visible text	PASS
80	13	Main Menu button goes to Main Menu Screen (from Timeout screen)	Click Main Menu button	Should return to Main Menu screen	Returns to Main Menu screen	PASS
81	13	Restart button goes back to Battle Screen (from Timeout screen)	Click restart button	Should return to battle screen	Returns to battle screen	PASS

Success Criteria 14 & 15: Health Bars

This success criterion aimed to display the health bars to the user for both players in the battle screen. This element of the user interface depicts character health and transforms it from an integer attribute into a visual image that the user will find easier to see. The colour of the player's health is in green, and the damage taken is in red. The outline of the health bars is in white just to make the health bars more visible.

I developed the drawing function of the health bars by taking in arguments of player health, x position and y position. The player health (100) was divided by 100 and used to determine the width of the green health bar while in the battle screen by multiplying this value by the width. As this ratio value decrease, the width of the green bar would decrease as well. I then called this function in the battle screen, expecting the health bar to update by shortening the green bar after any attacks have landed on the player. Testing showed that the health bars appeared correctly on the screen, but I will now need to test the health bars based on the test plan I have come up with in the Design stage.

This success criterion was completed on page [125](#).

The test for evaluation is in video form below:



2025-04-01
19-13-15.mkv

The health bars are visible at the top of the screen, displaying on either side to signify the health of player 1 and 2. The health bars deplete correctly by 6.5 (the damage set) and the battle screen goes to the win screen when one health bar is depleted. The health bars for both players are reset after restarting the game. This means that these success criteria have been complete for player 1 and 2 because users will now have an indication as to what their character's health is and they will know when they have defeated their opponent (their health bar is all red). Restarting the game will allow users to battle again with health bars being reset to full health for both players.

To improve this feature further, I could implement a way of showing the most recent damage taken from the opponent in the health bar. This bar would be the difference from the current health of the player and the health 6.5 or 16.5 above the current health, depending on if the character took damage from an attack or ability.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
82	14 & 15	Health bar on screen	Inspect screen	Visible green bar on screen	Visible green bar with white outline	PASS
83	14 & 15	Health bar at opposite sides at the top of the screen	Inspect screen	Visible green bar at top sides of the	Located at either side of the	PASS

				screen at both ends	screen at the top	
84	14 & 15	Health bar starts at full capacity when Battle screen starts	Go through Main Menu -> Char Selection -> Battle Screen and inspect health at the start of the battle	Green health bar should be full (no red visible)	Health bar starts at full capacity	PASS
85	14 & 15	Green health bar goes down by 6.5 each hit	Use player 2 character to hit player 1 character and inspect health	Should go down by 6.5	Health bar depletes by 6.5	PASS
86	14 & 15	Battle ends when one health bar is below or equal to zero	Use player 2 character to defeat player 1. Inspect for a change of screen.	Battle screen should transition to Win screen	Transitions to win screen menu	PASS
87	14 & 15	Health bar resets to full capacity at restart	Use player 2 character to defeat player 1. Click restart button on Win screen. Inspect health bar.	Should be full green bars for both players (no red visible)	Health bars are reset to full capacity.	PASS

Success Criteria 16 & 17: Character Image

This success criterion aimed to have the image of both characters display on the battle screen correctly at all times so that the user knows where their character is when using their movement and attacking keys. The image of the samurai and shinobi should be displayed in quite a large size and on the ground of the background image so that it is incorporated into the battle screen.

To develop this, I had to import the sprite sheets, count the animation steps of each animation, and add in all the sizing and positional data into the Fighter class instantiation method. I then created a load_images function which created all of the animation lists for all the separate animations of each character. I then made a draw() and update() function which would draw and eventually update the animations of each character according to the animation list. However, for this success criterion, success was just measured by visually seeing the characters on the screen.

This success criterion was achieved on page [106](#).

The test for evaluation is in video form below:



2025-04-01
19-45-40.mkv

Going to the battle screen, we see clearly that the characters are both visible and their feet are on the grass background on the screen. This success criterion is therefore a success since both characters are visible for the user to use and they are position correctly on the ground.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
88	16 & 17	Shinobi Character visible	Inspect Battle screen	Shinobi sprite should be seen	Shinobi sprite is seen	PASS
89	16 & 17	Samurai Character visible	Inspect Battle screen	Samurai sprite should be seen	Samurai sprite is seen	PASS
90	16 & 17	Shinobi Character on the ground	Inspect character in the context of the background image in Battle screen	Feet should be on the green grass	Feet is on the grass	PASS
91	16 & 17	Samurai Character on the ground	Inspect character in the context of the background image in Battle screen	Feet should be on the green grass	Feet is on the grass	PASS

Success Criteria 18 & 23: Character Idle Animation

This success criterion aims to show the user the idle animation of both characters in the battle screen. This animation should be the default animation that plays when the user is not moving or attacking. It will always play at the start of every battle.

I developed this using a function called `update_action()` and using the action attribute of the Fighter class which determines what action the character is on currently. The default idle action is 0 so in the `update()` method, I call the `update_action()` method with 0 as its argument and all this does is set the current action to 0 so that the character images that play are from the idle animation list. This animation has a cooldown of 80 milliseconds and plays each image with this cooldown so that it looks like the character is moving slightly up and down on the screen. Testing showed that this animation was visible and apparent, so this was initially a success there.

This success criterion was completed on page [107](#).

The test for evaluation is here:



2025-04-01
19-56-54.mkv

The characters clearly move up and down according to the idle animation list made by the `load_images()` function. This only plays when they are not moving so this success criterion has been fully achieved as the user will know when they are not using any action from their character.

Here is me testing 94 and 95:



2025-04-01
20-03-15.mkv

The character idle animation stops when moving and starts again when not moving. This means that the idle animation only plays when they are not moving, so this success criterion has been fully achieved as the user will know when they are not using any action from their character.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
92	18 & 23	Shinobi Idle Animation plays and loops animation when not moving	Inspect animation of Shinobi sprite while not moving in Battle screen	Idle animation should be slight breathing movements of the sprite.	Slight breathing movements noticed	PASS
93	18 & 23	Samurai Idle Animation plays	Inspect animation of	Idle animation	Slight breathing	PASS

		and loops animation when not moving	Samurai sprite while not moving in Battle screen	should be slight breathing movements of the sprite.	movements noticed	
94	18 & 23	Shinobi Idle Animation stops when moving/jumping	Use forwards movement key (D for player 1, < arrow key for player 2) and inspect the animation of the moving character. Then use backwards movement key (A for player 1, > arrow key for player 2) and inspect animation of moving character. Then use jump movement key (W for player 1, ^ for player 2) and inspect animation of moving character.	Idle animation should stop and run animation should play	Idle animation stops and run animation plays	PASS
95	18 & 23	Samurai Idle Animation stops when moving/jumping	Use forwards movement key (D for player 1, < arrow key for player 2) and inspect the animation of the moving character. Then use backwards movement key (A for player 1, > arrow key for player 2) and inspect animation of moving character. Then use jump	Idle animation should stop, and the relevant animation should play (run animation for forwards and backwards keys, jump animation for jump movement key)	Idle animation stops when not moving. Either runs, jumps or attacks	PASS

			movement key (W for player 1, ^ for player 2) and inspect animation of moving character.			
--	--	--	--	--	--	--

Success Criteria 19 & 24 : Character Run Animation

This success criterion aimed to show the user when their character is running when they have used a movement button. The character also moves along the screen when using this button and the user can use this to get closer to their opponent to then launch an attack.

I developed this function in the move() method in the Fighter class and set the speed to 15. This function checked if the user had pressed a key and if this key was a movement key, the player would move either forwards or backwards depending on the key and the player. I then called this function inside of the main game loop in the main program so that movement is checked all the time while the program is running, and the battle screen is set to True.

This success criterion was completed on page [112](#).

Here is the video for testing the test plans below:



2025-04-01
20-43-44.mkv

The video shows that the run animation plays correctly for both players, moving forwards and backwards. Jumping changes the run animation into the jump animation then the idle animation once landing. This means that this success criterion has been achieved as the user can visually see when their character is running forwards and backwards when they use their movement keys.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
96	19 & 24	Shinobi Run Animation plays when moving forwards	Use forwards movement key (D for player 1, < arrow key for player 2) and inspect player animation	Player's character should move forwards in their relative direction and run animation should play	Run animation plays and player moves forwards	PASS
97	19 & 24	Shinobi Run Animation plays when moving backwards	Use backwards movement key (A for player 1, > arrow key for player 2)	Player's character should move backwards in their relative direction and run animation should play	Run animation plays and player moves backwards	PASS
98	19 & 24	Shinobi Run Animation stops when not moving	Use forwards movement key (D for player 1, < arrow key for player 2), then stop and inspect player animation. Then use backwards movement key (A for player 1, > arrow key for player 2), stop	Animation should transition from run to idle	Animation changes from run to idle	PASS

			and inspect player animation.			
99	19 & 24	Shinobi Run Animation stops when jumping	Use jump movement key (W for player 1, ^ for player 2), wait for landing, then inspect animation.	Animation should transition from jump to idle when landing.	Animation changes from jump to idle	PASS
100	19 & 24	Samurai Run Animation plays when moving forwards	Use forwards movement key (D for player 1, < arrow key for player 2) and inspect player animation	Player's character should move forwards in their relative direction and run animation should play	Character moves forwards and run animation plays	PASS
101	19 & 24	Samurai Run Animation plays when moving backwards	Use backwards movement key (A for player 1, > arrow key for player 2)	Player's character should move backwards in their relative direction and run animation should play	Character moves backwards and run animation plays	PASS
102	19 & 24	Samurai Run Animation stops when not moving	Use forwards movement key (D for player 1, < arrow key for player 2), then stop and inspect player animation. Then use backwards movement key (A for player 1, > arrow key for player 2), stop and inspect player animation.	Animation should transition from run to idle	Animation changes from run to idle	PASS
103	19 & 24	Samurai Run Animation stops when jumping	Use jump movement key (W for player 1, ^ for player 2),	Animation should transition from jump to	Animation changes from jump to idle	PASS

			wait for landing, then inspect animation.	idle when landing.		
--	--	--	--	-----------------------	--	--

Success Criteria 20 & 25 : Character Jump Animation

This success criterion aimed to show the jumping animation of the user's chosen character and to show the upwards movement of the character when the jumping button has been pressed.

The development of this was made inside the move() function as well below all of the movement code. I added a vel_y attribute which would represent the velocity upwards of the player. This would update to 30 when the jump button for either player 1 or 2 would be used. When testing this code, I noticed that the player could go off screen when jumping so I updated the code to check for potential movement outside the screen and limited player movement from outside the screen to make sure that the user can see their character at all times, allowing them to know what action their character is making by its animation.

This success criterion was completed on page [122](#).

Here is the test for evaluation:



2025-04-01
21-48-05.mkv

This test shows that the jump animation plays when the jump buttons are pressed (W or ^). The players can jump both forwards and backwards with the jump animation playing for both actions. This test solidifies that this success criterion has been met as the user will be able to see when their character is jumping, and they will visually see their character jumping up. This action can be used to dodge attacks and get away from the opponent.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
104	20 & 25	Shinobi Jump Animation plays when jumping off the ground	Use jump movement key (W for player 1, ^ for player 2), then inspect animation.	Jump animation should play while in the air	Jump animations plays in air	PASS
105	20 & 25	Shinobi Jump Animation stops when character lands	Use jump movement key (W for player 1, ^ for player 2), wait for landing, then inspect animation.	Animation should transition from jump to idle when landing.	Animation transitions from jump to idle	PASS
106	20 & 25	Shinobi can Jump forwards	Use jump movement key (W for player 1, ^ for player 2) and the forwards movement key (D for player 1, < arrow key for player 2), then inspect the movement of the character.	Character should be moving forwards whilst jumping, only having jump animation playing	Character moves forwards whilst jumping, but only jumping animation plays	PASS
107	20 & 25	Shinobi can Jump backwards	Use jump movement key (W for player 1, ^ for player 2) and the backwards movement key (A for player 1, > arrow key for player 2), then inspect the	Character should be moving backwards whilst jumping, only having jump animation playing	Character moves backwards whilst jumping, but only jumping animation plays	PASS

			movement of the character.			
108	20 & 25	Samurai Jump Animation plays when jumping off the ground	Use jump movement key (W for player 1, ^ arrow key for player 2) and inspect animation of the jumping character	Jump animation should play while in air	Jump animations plays in air	PASS
109	20 & 25	Samurai Jump Animation stops when character lands	Use jump movement key (W for player 1, ^ arrow key for player 2), wait until character lands then inspect animation of the jumping character	Jump animation should stop and switch to idle animation	Animation transitions from jump to idle	PASS
110	20 & 25	Samurai can Jump forwards	Use jump movement key (W for player 1, ^ for player 2) and the forwards movement key (D for player 1, < arrow key for player 2), then inspect the movement of the character.	Character should be moving forwards whilst jumping, only having jump animation playing	Character moves forwards whilst jumping, but only jumping animation plays	PASS
111	20 & 25	Samurai can Jump backwards	Use jump movement key (W for player 1, ^ for player 2) and the backwards movement key (A for player 1, > arrow key for player 2), then inspect the movement of the character.	Character should be moving backwards whilst jumping, only having jump animation playing	Character moves backwards whilst jumping, but only jumping animation plays	PASS

Success Criteria 21, 22, 26 & 27 : Character 1 & 2, Attack 1 & 2 Animation

These success criteria aimed to show the user when their character is attacking. Doing this action at the right close distance from the opponent will deal damage to their health and cause the opponent bar to decrease. Attack 1 is a light attack that can be incorporated in an attack combo since it has little knockback whereas attack 2 has a high knockback to the other player. Both attacks have the same damage.

Development of these animations were similar to previous success criteria as I updated the code in the move() function to incorporate attacks with the use of the attacking Boolean attribute. I then used the attack() method which takes away the damage from the target when in range of the attack. The hitbox for the attack is an invisible rectangle in front of the player and is the same size as the player. Both attacks include a knockback to the target player but attack 2 has a larger knockback to reflect it being a heavier hit. All attacks also have distinct sound effects which I incorporated later on in development. The shinobi has a punch sound effect since its character image uses punches as attacks, whereas the samurai has sword sound effects as it uses a sword to deal damage to opponents.

These success criteria were completed on page [137](#)

The test for evaluation is below:



2025-04-01
22-12-57.mkv

In this test, it is clear that the attack 1 and 2 animation do play when their trigger button is pressed (E and R for player 1, N and M for player 2). Both attacks deplete the health by 6.5 correctly and attack 2 deals way more knockback than attack 1. So considering this test, I think it is fair to say

these success criteria have been completed as users can see their attacks happen on screen and can use them to deal damage to and defeat their opponent on the battle screen.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
112	21 & 26	Shinobi Attack 1 Animation plays when 'e' key pressed (Player 1) or 'n' key (Player 2)	Use Attack 1 for Player 1 and inspect animation. Repeat for Player 2.	Attack 1 animation should play once then switch back to idle.	Attack 1 animation plays and switches back to idle	PASS
113	21 & 26	Shinobi Attack 1 Animation damages opponent in range of hitbox (-6.5)	Use Attack 1 for Player 1 against Player 2.	Player 2 health should decrease by 6.5	Opponent health decreases by 6.5	PASS
114	21 & 26	Samurai Attack 1 Animation plays when 'e' key pressed (Player 1) or 'n' key (Player 2)	Use Attack 1 for Player 1 and inspect animation. Repeat for Player 2.	Attack 1 animation should play once then switch back to idle.	Attack 1 animation plays and switches back to idle	PASS
115	21 & 26	Samurai Attack 1 Animation damages opponent in range of hitbox (-6.5)	Use Attack 1 for Player 1 against Player 2.	Player 2 health should decrease by 6.5	Opponent health decreases by 6.5	PASS
116	22 & 27	Shinobi Attack 2 Animation plays when 'r' key pressed (Player 1) or 'm' key (Player 2)	Use Attack 2 for Player 1 and inspect animation. Repeat for Player 2.	Attack 2 animation should play once then switch back to idle.	Attack 2 animation plays and switches back to idle	PASS
117	22 & 27	Shinobi Attack 2 Animation damages opponent in range of hitbox (-6.5)	Use Attack 2 for Player 1 against Player 2.	Player 2 health should decrease by 6.5	Opponent health decreases by 6.5	PASS
118	22 & 27	Samurai Attack 2 Animation plays when 'r' key pressed (Player 1) or 'm' key (Player 2)	Use Attack 2 for Player 1 and inspect animation. Repeat for Player 2.	Attack 2 animation should play once then switch back to idle.	Attack 2 animation plays and switches back to idle	PASS

119	22 & 27	Samurai Attack 2 Animation damages opponent in range of hitbox (-6.5)	Use Attack 2 for Player 1 against Player 2.	Player 2 health should decrease by 6.5	Opponent health decreases by 6.5	PASS
-----	---------	---	---	--	----------------------------------	------

Success Criteria 28 : Knock Out Death

This success criterion aimed to display the defeated player being knocked out from the battle screen and also the transition to the win screen for the other player.

The development of this success criteria consisted of making the knock_x variable and using this in conjunction with the vel_y attribute as a quadratic curve to make the defeated player fling out of the screen. I also had to update the conditions of the screen borders to allow for this to happen by creating the dead attribute and using the negated version of this attribute to allow for the defeated player to be out of the screen.

This success criterion was completed on page [181](#).

The test for evaluation is here:



2025-04-01
23-22-25.mp4

This test shows player 1 using his attacks and ability to defeat the second player. Once player 2's health reaches 0, they are subsequently knocked out of the screen and the win screen shows on the screen. Thus, this success criterion has been achieved as players will be signified of their defeat by knockout, triggering the win screen and allowing for further navigation of the game.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
-------------	------------------	-----------------	-------------	------------------	----------------	-----------

120	28	Character image gets knocked out from screen when health is 0 (dead)	Use any attack for Player 1 to defeat Player 2 and inspect the movement of the character image	The character image should move out of the screen and should not be visible	Character image flung out of the screen and not visible	PASS
121	28	Dead character triggers Win screen	Use Player 1 character to defeat player 2	Battle screen should change to Win screen	Win screen shows up	PASS

Success Criteria 29 : Win Screen

This success criteria aimed to show user's which player has won the battle in the battle screen and allow for further navigation of the game to either restart the battle immediately or return to the main menu.

Development of this screen consisted of similar methods to the Character Selection screen with the development of menu buttons for restarting the battle and going to the main menu. The win screen also declared the user who survived the battle as the winner by saying either "P1 Wins!" or "P2 Wins!" at the centre of the screen. Below this text are the menu buttons "Restart" and "Main Menu" for further navigation.

This screen was made on page [185](#).

This is the test for evaluation:



2025-04-01
23-35-23.mp4

This test clearly shows that the win screen shows up when a player is defeated as I tested this for both players. If player 1 is defeated, the text shown is "P2 Wins!" with the menu buttons (restart and main menu) below it. But if player 2 is defeated, the text shown is "P1 Wins!" with the menu buttons below it. Therefore, this success criterion has been achieved because the screen declares which player has won a battle objectively and signifies this information to the user as text. As well as this, there are further menu navigation buttons for the user to use to go back to the main menu or immediately restart the battle.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
122	29	Player 1 declared the	Use Player 1 character to defeat player 2	"Player 1 Wins!" Text should	"P1 Wins!"	PASS

		winner when player 2 dies		appear on Win screen		
123	29	Player 2 declared the winner when player 1 dies	Use Player 2 character to defeat player 1	"Player 2 Wins!" Text should appear on Win screen	"P2 Wins!"	PASS
124	29	Both 'restart' and 'main menu' buttons appear for use	Use Player 1 character to defeat player 2, then inspect Win screen.	Visible black 'restart' and 'main menu' buttons should appear	Visible restart and main menu buttons	PASS

Success Criteria 30 : Restart Button

This success criterion aimed to show the restart button on the Win and Timeout screen so that the user can press this to immediately restart the battle.

Developing this button consisted of positioning it below the text shown above it (either player win declaration, or time ran out text) and making its function work properly by resetting fighter positions and attributes such as health. The button highlights blue when hovered over by the mouse to signify that the user would press this button if they clicked their mouse.

This success criterion was completed on page [187](#).

Here is a video for testing the test plan on this success criterion:



2025-04-01
23-46-48.mp4

The test shows me going from the Win screen to the Battle screen when using the restart button. Then I tested the Timeout screen, and it had worked the same way. Thus, this success criterion had been achieved as the user can now choose to immediately restart the battle if they choose to do so. This will reset their health and position, ensuring a fair game has restarted for them.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
125	30	Restart button	Inspect Win screen	Visible black button	Visible black button	PASS
126	30	"Restart" text in restart button (Win screen)	Inspect Win screen	Visible "Restart" text inside restart button	Visible "Restart" text	PASS
127	30	Restart button goes back to	Click restart button on Win	Screen change from	Screen change is	PASS

		Battle Screen (from Win /Timeout screen)	screen and Timeout screen	Win/Timeout screen to Battle screen	made from Win/Timeout screen	
--	--	---	---------------------------------	---	------------------------------------	--

Success Criteria 31 : Main Menu Button

This last success criterion aimed to have a visible main menu button on the Win Screen and the Timeout screen for the user to return to the main menu and navigate the game again from there. This can be done if the user wants to change their character or quit the game from inside the game without needing to close the window externally.

Developing this was similar to the success criterion above in terms of positioning and designing the button. However, this button triggered the main menu game variable instead and needed to reset all of the menuColor variables to ensure that all of the menu buttons that will be used subsequently will work as intended. The button highlights blue when hovered over by the mouse to signify that the user would press this button if they clicked their mouse.

This success criterion was completed on page [187](#).

This is the test for evaluation:



2025-04-01
23-57-22.mp4

This test shows me using the main menu button from the win screen and the timeout screen after. Both buttons are visible on both screens and switch the screen to the Main Menu screen when pressed by the user. The buttons work as intended in the main menu so this success criterion has been achieved as the user can navigate to the main menu after defeating their opponent or being defeated on the battle screen.

Test Number	Success Criteria	Purpose of Test	Test Method	Expected Outcome	Actual Outcome	PASS/FAIL
128	31	Main Menu button	Inspect Win screen	Visible black button	Visible black button	PASS
129	31	"Main Menu" text inside main menu button	Inspect Win screen	Visible "Main Menu" text inside restart button	Visible "Main Menu" text	PASS
130	31	Main Menu button goes to Main Menu when clicked (from Win screen)	Click Main Menu button on Win screen	Screen change from Win screen to Main Menu	Screen changes from win/timeout screen to	PASS

					main menu screen	
--	--	--	--	--	---------------------	--

User Interview: Usability & Maintenance

I brought back the 5 original students who I interviewed for the design stage and asked them to play my game without telling them how to do so at first, testing the ease of use of my game. I recorded the number of questions I received on each screen and what they asked.

Main Menu

No questions were asked at this stage and all the students had managed to click on the play button at the start of the game. This signifies to me that the main menu is easy to navigate and use for users playing my game. This means this screen was successful in its ease of use and efficiency.

Character Selection

One student had asked what the CPU button does. I explained to them that it makes the character selected on that side a computer-controlled character with automated movements and attacks. Apart from this, there were no other questions about how to use this screen. The students complemented the use of green to confirm selections and the blue highlight for the buttons they were going to press. All users managed to get to the battle screen when selecting their characters by pressing start battle. This tells me that the Character Selection screen is also easy to navigate and use for users to select their characters for battle. However, it could be that not all users know what CPU does or stands for, so I could use text to explain briefly what it does under the button since there is space there. I could also do this in the options screen if the user wanted help.

Battle Screen

Most users asked what buttons they needed to use to control the characters they had chosen. I showed them how to and they managed to pick up the skill fairly quickly and many were able to defeat the opponent they faced after learning the controls, both being another user or a CPU opponent. They utilised all elements of movement including running and jumping. They used light attacks and heavy attacks as well as abilities to defeat their opponents. This tells me that this screen is fairly easy to use when the users learn the controls, so there needs to be an initial explanation of the controls before the user battles. This could be in the form of a one-time screen or a screen in the options screen if I had developed it.

Win/Timeout Screen

Again, there were no questions asked on this screen and all 5 students knew that they had to press either restart to restart the battle immediately or main menu to navigate to the main menu. Users were also pleased to see either "P1 Wins!" or "P2 Wins!", signifying its key role in declaring their character winning the battle.

User Interview: General Feedback and Limitations

I then finally asked my interviewees what they liked about the game and what they wanted to see improved or added into the game to make it better. I have compiled the most common answers below.

CPU

The students that used the CPU toggle on the Character Selection screen and went into battle were surprised at the difficulty of the computer-controlled entity in battle as it was persistent and kept attacking them, even when they ran away. They did manage to defeat this opponent in the end but they had complimented this difficulty as it made them aim to defeat this entity after some suffered defeats initially.

Animation

The students also complimented the animations that I used and they worked well with the button I designated the certain animations to be triggered by. This tells me that the animations are visually appealing to the user and helps to show them what action they are completing on the screen

Sound FX & Music

The use of sound effects and music was also highly complimented. They said it set the mood for each screen (main menu and battle screen). The sound effects were also complimented as they matched the actions of the characters. The shinobi having the punch sound and the samurai having the sword sound effect was really noticed by the user's and added to their engagement with the game.

Ability

The use of abilities in the game was also complimented. The ability being a sparking fireball was visually appealing to the user and added a fun element into the game as users were excited to use their ability on their opponent and couldn't wait for their ability bar to be full!

Attack 1 & 2

The animation for both attack 1 and 2 was noticed and complimented by the students. They liked the knockback that the attacks had on the player and the extra knockback on attack 2 which had a large, comical effect on the opponent by sending them almost to the other side of the screen. They liked how it got the opponent off of their character and could use that opportunity to use the ability or launch a counterattack.

Background

Users did like the background screen but also mentioned that it could get boring to look at after a while of playing the game, no matter how bright and colourful it may be. This suggests to me that there should be a background selection screen developed so that there can be different backgrounds that the users can fight in, allowing for new environments and experiences in battle. New backgrounds could also bring about the opportunity to have effects on characters such as slowing their speed or increasing damages of abilities.

Pause button

Users also mentioned the lack of a pause button in the battle screen, which would freeze the battle screen for the user in the case that there is a disruption which doesn't allow them to battle or if they want to immediately restart or go to the main menu. I see how this feature would prove to be useful and will note it for post-development of this game after evaluation.

Character Selection

The students suggested that there should be more than two characters available to choose from in the Character Selection screen since they feel it will get boring after a while of using only two characters in the game. I actually did have another sprite for this game, but I did not have enough time to implement this sprite into the game and found it much easier given my time constraint to implement two characters. However, for post-development, it would prove useful and beneficial to the game to include more characters for battle so that the user can experience new attacks and abilities.

Options Screen

Users also noted the lack of an options screen and what possibilities it could have, including a mute button, key binding and a how to play section. I noted down these possibilities and will keep these as objectives for post-development.

User Interview: Conclusion

With the user interview concluded, I gathered all the data about what was liked about the game and what needed to be added or improved.

What Went Well

The usability of the main menu and the win/timeout screen was really successful and easy to use for the students I interviewed. They had no questions about how the screen worked and they intuitively managed to press the menu buttons that they intended to press, navigating the menu as they wanted.

The CPU element of my game was really popular and used in my game as players found it challenging to defeat it. Some took several tries but it managed to introduce a fun element to my game for single-players, achieving the initial problem I wanted to solve, so this was a success here.

The animation was also complimented as visually appealing, as well as the abilities and attacks of the characters. Users found it fun to see their player fight the opponent visually. With their character performing their intended action using the buttons I taught them to use.

The sound effects and music was also greatly appreciated by the users. They found the music to be a match with the atmosphere of the game, it being a fighting game. The sound effects also matched the character being used, whether that be the shinobi or samurai.

Even Better If...

The background could change in some way, but preferably by background selection screen that the user can control for themselves. Seeing one background all the time would get boring for most users after a while, and they need the screen to switch colours to make it more engaging.

There was a pause button on the battle screen so that users can control when they want to continue a battle or quit and restart or return to the main menu. Currently, the user is forced to play or wait the entire 45 seconds until the game is over and the timeout screen appears for navigation.

There was more than two characters to choose from as this would make players more open to selecting different characters in the Character Selection screen. This would also introduce different attacking animations and the possibility for new abilities as well.

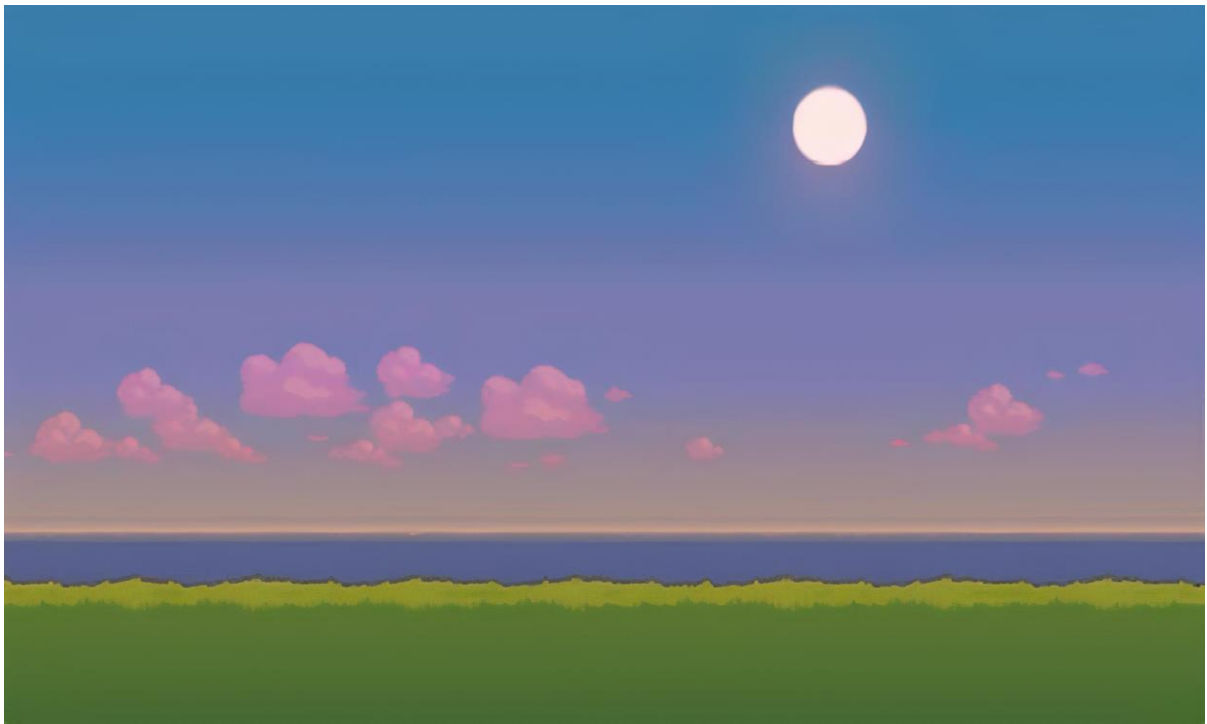
A working options screen was there as it would help users reassign the original keybinds I had set if they preferred another method of controlling their characters. This screen would also allow for volume control of music and sound effects, as well as tips on how to play the game.

Bibliography

- [Program Arcade Games With Python And Pygame](#)
- [Pygame Front Page — pygame v2.6.0 documentation](#)
- [2D Game Assets Store & Free - CraftPix.net](#)
- [Top free game assets tagged 2D and Animation - itch.io](#)
- [Free Effect Bullet Impact Explosion 32x32 by BDragon1727 \(itch.io\)](#)
- [Free Sword Sound Effects Download - Pixabay](#)

Appendix

Background image

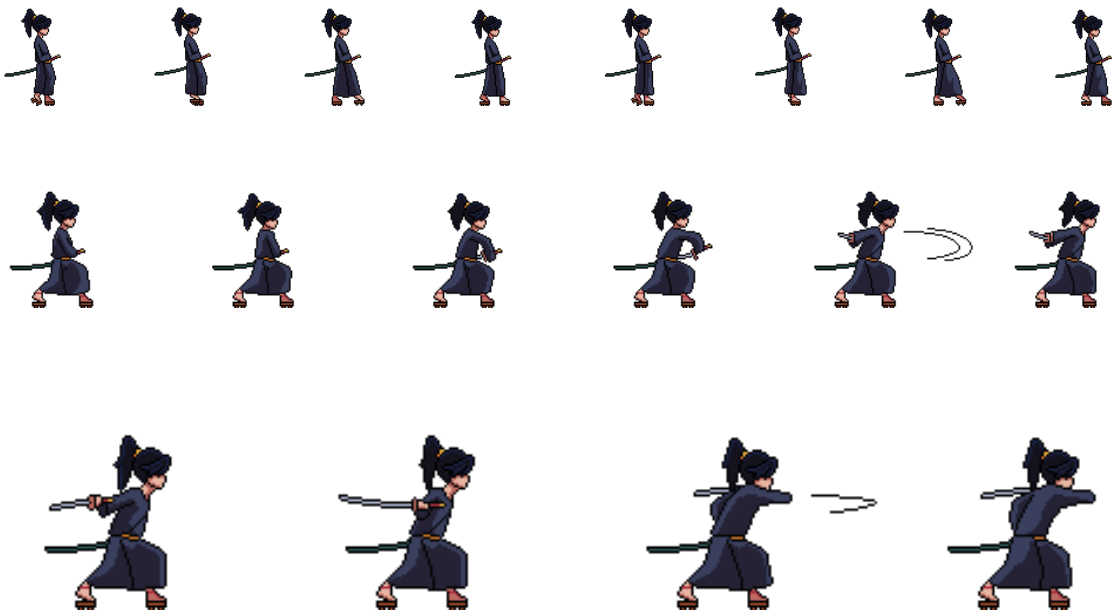


Shinobi



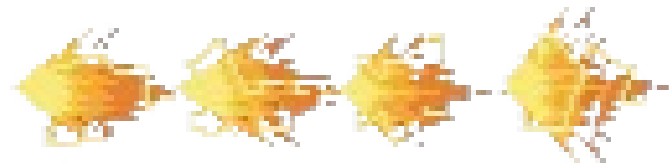


Samurai





Red Fire Ball



UFG1.py

```
import pygame
```

```
from UFGfighter import Fighter
```

```
pygame.init()
```

```
pygame.mixer.init()
```

```
# Define some colors
```

```
BLACK = (0, 0, 0)
```

```
WHITE = (255, 255, 255)
```

```
GREEN = (0, 255, 0)
```

```
RED = (255, 0, 0)
```

```
YELLOW = (255,255,0)
```

```
BLUE = (0,0,200)
```

```
#game window
```

```
screen_width = 1000
```

```
screen_height = 600
```

```
screen = pygame.display.set_mode((screen_width,screen_height))
```

```
# defining menu font
```

```
TitleFont = pygame.font.SysFont('Corbel',70,True)
```

```
MenuFont = pygame.font.SysFont('Corbel',35 )
```

```
#render menu font
```

```
text1 = MenuFont.render('Start Game' , True , WHITE)
```

```
text2 = MenuFont.render('Options' , True , WHITE)
```

```
text3 = MenuFont.render('Quit' , True , WHITE)
```

```
text4 = TitleFont.render('Untitled Fighter Game' , True , WHITE)
```

```
text5 = MenuFont.render('CPU' , True , WHITE)
```

```
text6 = MenuFont.render('Start Battle!' , True , WHITE)
```

```
text7 = MenuFont.render('P1 Wins!' , True , WHITE)
```

```
text8 = MenuFont.render('P2 Wins!' , True , WHITE)
```

```
text9 = MenuFont.render('Restart' , True , WHITE)
```

```
text10 = MenuFont.render('Main Menu' , True , WHITE)
```

```
text11 = MenuFont.render('Time ran out!', True, WHITE)
```

```
text12 = MenuFont.render('Its a Draw!' , True , WHITE)
```

```
#menu buttons y pos
```

```
starttextY = (screen_height/2)-150
```

```
optionsY = starttextY+75
```

```
quitY = optionsY+75
```

```
menuColor1 = BLACK
```

```
menuColor2 = BLACK
```

```
menuColor3 = BLACK
```

```
menuColor4 = BLACK
```

```
menuColor5 = BLACK
```

```
menuColor6 = BLACK
```

```
menuColor7 = BLACK
```

```
menuColor8 = BLACK
```

```
menuColor9 = BLACK
```

```
#define game variables
```

```
MainMenu = False
```

```
firstLaunch = True
```

```
charSelect = False
```

```
startBattle = False
```

```
timerStart = False
```

```
TimeOut = False
```

```
battleDone = False
```

```
musicPlaying = False
```

```
P1sound = False
```

```
P2Sound = False
```

```
MainMenuMusic = False
```

```
battleMusic = False
```

```
P1Picked = False
```

```
P2Picked = False
```

```
shinobiPicked = False
```

```
samuraiPicked = False
```

```
CPU1 = False
```

```
CPU2 = False
```

```
switchedChar = False
```

```
restart = False
```

```
#load music and sound effects
```

```
pygame.mixer.set_num_channels(10)
```

```
mainMusic = pygame.mixer.music.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Audio\\Main Menu Music.mp3")
```

```
punch1 = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Audio\\Punch 1.mp3")
```

```
punch2 = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Audio\\Punch 2.mp3")
```

```
punch_swing = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Audio\\Punch Swing.mp3")
```

```
sword1 = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Audio\\Sword Hit.mp3")
```

```
sword2 = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Audio\\Sword 2.mp3")
```

```
sword_swing = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Audio\\Sword swing.mp3")
```

```
ability_fire = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Audio\\Ability Shoot.mp3")
```

```
ability_hit = pygame.mixer.Sound("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Audio\\Ability Hit.mp3")
```

```
P1channelAttack = pygame.mixer.Channel(1)
```

```
P1channelSwing = pygame.mixer.Channel(2)
```

```
P2channelAttack = pygame.mixer.Channel(3)
```

```
P2channelSwing = pygame.mixer.Channel(4)
```

```
P1AbilityFly = pygame.mixer.Channel(6)
```

```
P2AbilityFly = pygame.mixer.Channel(5)
```

```
P1AbilityHit = pygame.mixer.Channel(8)
```

```
P2AbilityHit = pygame.mixer.Channel(7)
```

```
# fighter variables
```

```
Shinobi_Size = 128 #size of animation image in pixels
```

```
ShinobiScale = 3 #scaling up image size to be visible
```

```
ShinobiOffset = [48, 78] #offsetting original blitted image to be on the ground
```

```
ShinobiData = [Shinobi_Size, ShinobiScale, ShinobiOffset]
```

```
Samurai_size = 128
```

```
SamuraiScale = 3
```

```
SamuraiOffset = [52, 78]
```

```
SamuraiData = [Samurai_size, SamuraiScale, SamuraiOffset]
```

```
#set the game window caption
```

```
pygame.display.set_caption("Untitled Fighter Game")
```

```
#load background - from: https://wallpapercave.com/landscape-pixel-art-wallpapers
```

```
bg_image = pygame.image.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Images\\ImageBG.jpg").convert_alpha()
```

```
#load sprite sheets - from: https://craftpix.net/freebies/free-shinobi-sprites-pixel-art/
```

```
shinobi_spritesheet = pygame.image.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Images\\spritesheet (1).png").convert_alpha()
```

```
#shinobi_attacksheet = pygame.image.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Images\\ShinobiAttackSheet.png").convert_alpha()
```

```
samurai_spritesheet = pygame.image.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Images\\samuraisheet.png").convert_alpha()
```

```
#red punch sprite sheet
```

```
redpunch_spritesheet = pygame.image.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich  
College\\Desktop\\Untitled Fighter Game\\Images\\Red Fire\\red_fire_projectile.png")
```

```
#num of animation steps (Idle,Run,Jump, Attack1, Attack2, Attack3, Hurt, Dead)
```

```
shinobi_Animationsteps = (6,8,10,6,3,4,6,3)
```

```
samurai_Animationsteps = (6,8,12,6,4,4,5,3)
```

```
ability_data = [(32,5,[25,20]), 4, redpunch_spritesheet]
```

```
#check punch sound
```

```
def AttackSound(P1,P2):
```

```
    key = pygame.key.get_pressed()
```

```
    if P1.attacking:
```

```
        if key[pygame.K_e] or key[pygame.K_r] or P1.attacktype == 4 or P1.attacktype == 5:
```

```
if not P1channelSwing.get_busy():
    P1channelSwing.queue(punch_swing)
    P1channelSwing.play(punch_swing)
if (key[pygame.K_e] or P1.attacktype == 4) and P2.hit:
    if not P1channelAttack.get_busy():
        P1channelAttack.queue(punch1)
        P1channelAttack.play(punch1)
if (key[pygame.K_r] or P1.attacktype == 5) and P2.hit:
    if not P1channelAttack.get_busy():
        P1channelAttack.queue(punch2)
        P1channelAttack.play(punch2)
if P1.ability.used and not P1AbilityFly.get_busy():
    P1AbilityFly.queue(ability_fire)
    P1AbilityFly.play(ability_fire)
if P1.ability.rect.colliderect(P2.rect) and P1.ability.used and not P1AbilityHit.get_busy():
    P1AbilityHit.queue(ability_hit)
    P1AbilityHit.play(ability_hit)

if P2.attacking:
    if key[pygame.K_n] or key[pygame.K_m] or P2.attacktype == 4 or P2.attacktype == 5:
        if not P2channelSwing.get_busy():
            P2channelSwing.queue(sword_swing)
            P2channelSwing.play(sword_swing)
        if (key[pygame.K_n] or P2.attacktype == 4) and P1.hit:
            if not P2channelAttack.get_busy():
                P2channelAttack.queue(sword1)
                P2channelAttack.play(sword1)
        if (key[pygame.K_m] or P2.attacktype == 5) and P1.hit:
            if not P2channelAttack.get_busy():
                P2channelAttack.queue(sword2)
                P2channelAttack.play(sword2)
if P2.ability.used and not P2AbilityFly.get_busy():
```

```
P2AbilityFly.queue(ability_fire)

P2AbilityFly.play(ability_fire)

if P2.ability.rect.colliderect(P1.rect) and P2.ability.used and not P2AbilityHit.get_busy():
    P2AbilityHit.queue(ability_hit)
    P2AbilityHit.play(ability_hit)


#define background drawing function
scaled_bg = pygame.transform.scale(bg_image,(screen_width,screen_height))
def drawBG():
    global scaled_bg
    screen.blit(scaled_bg,(0,0)) #draws the image from the top left of the window


startTicks = pygame.time.get_ticks()


#draw timer function
def drawTimer():
    global startTicks
    global battleDone
    global TimeOut
    global timerStart
    global startBattle

    if not timerStart and startBattle: #when the battle starts, start the timer
        startTicks = pygame.time.get_ticks()
        timerStart = True

    if startBattle: #while the battle is ongoing, keep updating the value of seconds
        seconds = (pygame.time.get_ticks()-startTicks)//1000


    if battleDone:
        seconds = 0
        del seconds
```

if restart:

seconds = 0

if seconds > 45 and startBattle: # if the timer goes over 45 seconds, end the battle

startTicks = pygame.time.get_ticks()

battleDone = True

TimeOut = True

startBattle = False

timerStart = False

if not battleDone: #While the battle is ongoing, keep outputting seconds on screen

timer = MenuFont.render(str(seconds), True, BLACK)

screen.blit(timer, (screen_width/2 - 12, screen_height/50))

#define health bar function

def drawHealth(health,x,y):

ratio = health / 100

pygame.draw.rect(screen, WHITE, (x-5,y-5, 410,40))

pygame.draw.rect(screen, RED, (x,y,400,30))

pygame.draw.rect(screen, GREEN, (x,y,400*ratio,30))

#draw ability bar

def drawAbilityBar(x, y, charge): #add charge parameter

pygame.draw.rect(screen, WHITE, (x-5, y-5, 210, 40)) # Bar outline, increase visibility

pygame.draw.rect(screen, BLUE, (x, y, 200 * charge, 30)) # use charge to control how much bar is filled

portraitY = 200

#character selection

```
def charPortraits():
    global menuColor1
    global menuColor2
    global menuColor3
    global menuColor4
    global menuColor5
    global menuColor6
    global menuColor7
    portraitWidth, portraitHeight = 125, 175
    global portraitY
    global P2
    global P1
    P1.update_action(0) #image for character button
    P2.update_action(0)
    P1image = pygame.transform.flip(P1.image, False, False) #image face each other
    P2image = pygame.transform.flip(P2.image, True, False)

    #left side
    portrait1 = pygame.transform.scale(P1image,(portraitWidth,portraitHeight))
    portrait2 = pygame.transform.scale(P2image,(portraitWidth,portraitHeight))
    pygame.draw.rect(screen,menuColor1,(50,200,portraitWidth,portraitHeight))
    pygame.draw.rect(screen,menuColor2,(250,200,portraitWidth,portraitHeight))
    screen.blit(portrait1,(50,portraitY))
    screen.blit(portrait2,(250,portraitY))
    #highlight buttons
    if not P1Picked:
        if 50 <= mouse[0] <= 180 and portraitY <= mouse[1] <= portraitY+180:
            menuColor1 = BLUE
        else:
            menuColor1 = BLACK
        if 250 <= mouse[0] <= 380 and portraitY <= mouse[1] <= portraitY+180:
            menuColor2 = BLUE
```

```
else:
    menuColor2 = BLACK

#right side
portrait3 = pygame.transform.scale(P1image,(portraitWidth,portraitHeight))
portrait4 = pygame.transform.scale(P2image,(portraitWidth,portraitHeight))
pygame.draw.rect(screen,menuColor3,(650,200,portraitWidth,portraitHeight))
pygame.draw.rect(screen,menuColor4,(850,200,portraitWidth,portraitHeight))
screen.blit(portrait3,(650,portraitY))
screen.blit(portrait4,(850,portraitY))
if not P2Picked:
    if 650 <= mouse[0] <= 780 and portraitY <= mouse[1] <= portraitY+180:
        menuColor3 = BLUE
    else:
        menuColor3 = BLACK
    if 850 <= mouse[0] <= 980 and portraitY <= mouse[1] <= portraitY+180:
        menuColor4 = BLUE
    else:
        menuColor4 = BLACK

#CPU Buttons
pygame.draw.rect(screen,menuColor5,(125,450,175,50))
screen.blit(text5,(140,460))
pygame.draw.rect(screen,menuColor6,(725,447,175,50))
screen.blit(text5,(740,457))
if not CPU1:
    if 125 <= mouse[0] <= 300 and 450 <= mouse[1] <= 500:
        menuColor5 = BLUE
    else:
        menuColor5 = BLACK

if not CPU2:
    if 725 <= mouse[0] <= 900 and 450 <= mouse[1] <= 500:
```

```
    menuColor6 = BLUE

else:

    menuColor6 = BLACK


#start battle button

pygame.draw.rect(screen,menuColor7,(420,550,180,50))

screen.blit(text6,(425,555))

if 420 <= mouse[0] <= 600 and 550 <= mouse[1] <= 600:

    menuColor7 = BLUE

else:

    menuColor7 = BLACK


#Win screen

def drawBattleDone(P1):

    global menuColor8

    global menuColor9


    if P1: #If P1 wins, print 'P1 wins'

        screen.blit(text7 , (screen_width/2-80,screen_height/2 - 80))

    else: #Else print 'P2 wins'

        screen.blit(text8 , (screen_width/2-80,screen_height/2 - 80))

    pygame.draw.rect(screen,menuColor8,(screen_width/2-80,screen_height/2,158,40))

    screen.blit(text9 , (screen_width/2-80,screen_height/2))

    pygame.draw.rect(screen,menuColor9,(screen_width/2-80,screen_height/2 + 50,158,40))

    screen.blit(text10 , (screen_width/2-80,screen_height/2 + 50))


    if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 <= mouse[1] <= screen_height/2 +48:#restart button area for highlight

        menuColor8 = BLUE

    else:

        menuColor8 = BLACK


    if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 + 50 <= mouse[1] <= screen_height/2 + 98:#main menu button area for highlight
```

```
    menuColor9 = BLUE
else:
    menuColor9 = BLACK

#Time out screen
def drawTimeOut():
    global menuColor8
    global menuColor9
    if P1.health > P2.health:
        screen.blit(text7 , (screen_width/2-80,screen_height/2 - 80))
    elif P2.health > P1.health:
        screen.blit(text8 , (screen_width/2-80,screen_height/2 - 80))
    else:
        screen.blit(text12 , (screen_width/2-80,screen_height/2 - 80))

    screen.blit(text11 , (screen_width/2-80,screen_height/2 - 140))

    pygame.draw.rect(screen,menuColor8,(screen_width/2-80,screen_height/2,158,40))
    screen.blit(text9 , (screen_width/2-80,screen_height/2))
    pygame.draw.rect(screen,menuColor9,(screen_width/2-80,screen_height/2 + 50,158,40))
    screen.blit(text10 , (screen_width/2-80,screen_height/2 + 50))

    if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 <= mouse[1] <=
screen_height/2 +48:
        menuColor8 = BLUE
    else:
        menuColor8 = BLACK

    if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 + 50 <=
mouse[1] <= screen_height/2 + 98:
        menuColor9 = BLUE
    else:
        menuColor9 = BLACK
```

```
#draw menu buttons + box around it

def drawStart():
    global menuColor1
    global menuColor2
    global menuColor3
    screen.blit(text4, (screen_width/2-310,screen_height/2-250))
    pygame.draw.rect(screen,menuColor1,(screen_width/2-80,starttextY,158,40))
    screen.blit(text1 , (screen_width/2-80,starttextY))
    pygame.draw.rect(screen,menuColor2,(screen_width/2-80,optionsY,158,40))
    screen.blit(text2 , (screen_width/2-80,optionsY))
    pygame.draw.rect(screen,menuColor3,(screen_width/2-80,quitY,158,40))
    screen.blit(text3 , (screen_width/2-80,quitY))

    if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and starttextY <= mouse[1] <=
starttextY+40:
        menuColor1 = BLUE
    else:
        menuColor1 = BLACK

    if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and optionsY <= mouse[1] <=
optionsY+40:
        menuColor2 = BLUE
    else:
        menuColor2 = BLACK

    if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and quitY <= mouse[1] <= quitY+40:
        menuColor3 = BLUE
    else:
        menuColor3 = BLACK

P1 = Fighter(200,450, False, ShinobiData, shinobi_spritesheet,shinobi_Animationsteps, True,CPU1,
ability_data)

P2 = Fighter(700,450,True, SamuraiData, samurai_spritesheet, samurai_Animationsteps, False,CPU2,
ability_data)
```

```
# Used to manage how fast the screen updates
clock = pygame.time.Clock()

#game loop
done = False
while not done:

    #get mouse position
    mouse = pygame.mouse.get_pos()

    #events + game logic
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            done = True

    #logic
    if event.type == pygame.MOUSEBUTTONDOWN:

        if (screen_width/2)-100 <= mouse[0] <= (screen_width/2)+100 and starttextY <= mouse[1] <=
starttextY+75:#pressed start game

            if startBattle:#check if game already started

                pass

            elif MainMenu:

                menuColor1 = BLACK

                charSelect = True

                MainMenu = False

        if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and quitY <= mouse[1] <=
quitY+50:#PRESSED QUIT

            if MainMenu: #CHECKS IF ON MAIN MENU SCREEN

                done = True
```

if 50 <= mouse[0] <= 180 and portraitY <= mouse[1] <= portraitY+180: #Character 1 Button for Player 1(Shinobi)

if charSelect:

if not P1Picked:

if not P2Picked:

shinobiPicked = True

P1Picked = True

menuColor1 = GREEN

elif P2Picked and not shinobiPicked:

shinobiPicked = True

P1Picked = True

menuColor1 = GREEN

else:

if not P2Picked:

shinobiPicked = False

P1Picked = False

menuColor1 = BLACK

if 250 <= mouse[0] <= 380 and portraitY <= mouse[1] <= portraitY+180: #Character 2 Button for Player 1(Samurai)

if charSelect:

if not P1Picked:

if not P2Picked:

samuraiPicked = True

P1Picked = True

menuColor2 = GREEN

elif P2Picked and not samuraiPicked:

samuraiPicked = True

P1Picked = True

menuColor2 = GREEN

else:

if not P2Picked:

samuraiPicked = False

P1Picked = False

menuColor1 = BLACK

if 650 <= mouse[0] <= 780 and portraitY <= mouse[1] <= portraitY+180: #Character 1 Button
for Player 2

if charSelect:

if not P2Picked:

if not P1Picked:

shinobiPicked = True

P2Picked = True

menuColor3 = GREEN

elif P1Picked and not shinobiPicked:

shinobiPicked = True

P2Picked = True

menuColor3 = GREEN

else:

if not P1Picked:

shinobiPicked = False

P2Picked = False

menuColor1 = BLACK

if 850 <= mouse[0] <= 980 and portraitY <= mouse[1] <= portraitY+180: #Character 2 Button
for Player 2

if charSelect:

if not P2Picked:

if not P1Picked:

samuraiPicked = True

P2Picked = True

menuColor4 = GREEN

elif P1Picked and not samuraiPicked:

samuraiPicked = True

P2Picked = True

```
    menuColor4 = GREEN
else:
    if not P1Picked:
        samuraiPicked = False
        P2Picked = False
        menuColor1 = BLACK

if 125 <= mouse[0] <= 300 and 450 <= mouse[1] <= 500:#player 1 CPU toggle
    if charSelect:
        if not CPU1:
            CPU1 = True
            menuColor5 = GREEN
        else:
            CPU1 = False
            menuColor5 = BLACK

if 725 <= mouse[0] <= 900 and 450 <= mouse[1] <= 500:#player 2 CPU toggle
    if charSelect:
        if not CPU2:
            CPU2 = True
            menuColor6 = GREEN
        else:
            CPU2 = False
            menuColor6 = BLACK

if 420 <= mouse[0] <= 600 and 550 <= mouse[1] <= 600:#start game after characters picked
    if P1Picked and P2Picked:
        menuColor7 = GREEN
        charSelect = False
        if menuColor2 == GREEN or menuColor3 == GREEN:
            del P1
```

del P2

P1 = Fighter(200,450,False, SamuraiData, samurai_spritesheet,
samurai_Animationsteps, True,CPU1, ability_data)

P2 = Fighter(700,450, True, ShinobiData, shinobi_spritesheet, shinobi_Animationsteps,
False,CPU2,ability_data)

switchedChar = True

else:

switchedChar = False

startBattle = True

if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 <= mouse[1] <= screen_height/2 +48:#restart game

if battleDone:

P1.restoreHealth()

P2.restoreHealth()

if P1.player:

P1.resetPosition(200,450)

else:

P1.resetPosition(700,450)

if P2.player:

P2.resetPosition(200,450)

else:

P2.resetPosition(700,450)

restart = True

if (screen_width/2)-95 <= mouse[0] <= (screen_width/2)+85 and screen_height/2 + 50 <= mouse[1] <= screen_height/2 + 98:#return to menu

if battleDone:

P1.restoreHealth()

P2.restoreHealth()

MainMenu = True

del P1

```
del P2

P1 = Fighter(200,450, False, ShinobiData, shinobi_spritesheet, shinobi_Animationsteps,
True,CPU1,ability_data)

P2 = Fighter(700,450,True, SamuraiData, samurai_spritesheet, samurai_Animationsteps,
False,CPU2, ability_data)

P1Picked = False

P2Picked = False

samuraiPicked = False

shinobiPicked = False

menuColor1,menuColor2,menuColor3,menuColor4,menuColor5,menuColor6,menuColor
7,menuColor8,menuColor9 = BLACK,BLACK,BLACK,BLACK,BLACK,BLACK,BLACK,BLACK,BLACK

battleDone = False
```

```
#drawing code -----
```

```
if firstLaunch:
```

```
    MainMenu = True
```

```
    MainMenuMusic = True
```

```
    firstLaunch = False
```

```
drawBG()
```

```
if MainMenu:
```

```
    #draw start menu
```

```
    drawStart()
```

```
if battleMusic:
```

```
    pygame.mixer.music.unload()
```

```
    MainMenuMusic = True
```

```
    musicPlaying = False
```

```
    battleMusic = False
```

if not musicPlaying and MainMenuMusic:

```
mainMusic = pygame.mixer.music.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Main Menu Music.mp3")
```

```
pygame.mixer.music.set_volume(0.8) #set volume to 80%
```

```
pygame.mixer.music.play(-1)#loop music
```

```
musicPlaying = True
```

if charSelect:

```
charPortraits()
```

if startBattle:

if not switchedChar:

```
AttackSound(P1,P2)
```

else:

```
AttackSound(P2,P1)
```

if MainMenuMusic:

```
pygame.mixer.music.unload()
```

```
MainMenuMusic = False
```

```
musicPlaying = False
```

```
battleMusic = True
```

if not musicPlaying and battleMusic:

```
Musicbattle = pygame.mixer.music.load("C:\\Users\\MatinaJ\\OneDrive - Dulwich College\\Desktop\\Untitled Fighter Game\\Audio\\Battle Music.mp3")
```

```
pygame.mixer.music.set_volume(0.8) #set volume to 80%
```

```
pygame.mixer.music.play(-1)#loop
```

```
startTicks = pygame.time.get_ticks()
```

```
musicPlaying = True
```

```
drawTimer()
```

```
P1.draw(screen,YELLOW)
```

```
P2.draw(screen,YELLOW)
```

```
drawHealth(P1.health,20,20)
```

```
drawHealth(P2.health,580,20)
```

```
drawAbilityBar(20, 80, P1.ability_charge)
```

```
drawAbilityBar(580, 80, P2.ability_charge)
```

```
P1.ability_charge = min(P1.ability_charge + 0.0025, 1)
```

```
P2.ability_charge = min(P2.ability_charge + 0.0025, 1)
```

```
P1.move(screen_width,screen_height, P2, screen)
```

```
P2.move(screen_width,screen_height, P1, screen)
```

```
P1.update(P2, screen)
```

```
P2.update(P1, screen)
```

```
if restart:
```

```
    drawTimer()
```

```
    startBattle = True
```

```
    battleDone = False
```

```
    restart = False
```

```
    timerStart = False
```

```
    P1.ability_charge = 0
```

```
    P2.ability_charge = 0
```

```
if P1.dead or P2.dead:
```

```
    startBattle = False
```

```
    battleDone = True
```

```
if battleDone:
    if P1.dead:
        drawBattleDone(False)
    elif P2.dead:
        drawBattleDone(True)
    elif TimeOut:
        drawTimeOut()

P1.update(P2, screen)
P2.update(P1, screen)

P1.draw(screen, YELLOW)
P2.draw(screen, YELLOW)

# #move fighter

P1.move(screen_width,screen_height, P2, screen)
P2.move(screen_width,screen_height, P1, screen)

drawHealth(P1.health,20,20)
drawHealth(P2.health,580,20)

#update display
pygame.display.flip()

#60 frames per second
clock.tick(60)

#exit pygame
pygame.quit()
```

UFGfighter.py

import pygame, random

from UFGability import Ability

class Fighter():

def __init__(self,x,y,flip, data, sprite_sheet, animationsteps, Player1,CPU, ability_data):

self.size = data[0]

self.image_scale = data[1]

self.offset = data[2]

self.flip = flip

self.animationlist = self.load_images(sprite_sheet, animationsteps)

self.rect = pygame.Rect(x,y,80,150)

```
self.running = False

self.action = 0 #0=idle, 1=run, 2=jump, 3=attack1, 4=attack2, 5=hit, 6=dead

self.frame_index = 0

self.image = self.animationlist[self.action][self.frame_index]

self.updateTime = pygame.time.get_ticks()

self.player = Player1

self.CPU = CPU

self.jump = False

self.attacking = False

self.attacktype = 0

self.hit = False

self.dead = False

self.vel_y = 0

self.vel_x = 0

self.health = 100

self.knockFriction = 0

self.useAbility = False

self.ability_charge = 0

self.ability_cooldown = False

self.ability = Ability(
    x=self.rect.centerx,
    y=self.rect.centery,
    data=ability_data[0],
    animationsteps=ability_data[1],
    spritesheet=ability_data[2],
    flip=flip,
)

def load_images(self, sprite_sheet, animationsteps):
    runningtotal = 0

    animationlist = [] #List of lists of all character animations in the spritesheet
```

```
for y, animation in enumerate(animationsteps):
    temp_img_list = [] #List for one animation
    for x in range(animation):
        if runningtotal > 0: #If the first frame has been already cut, then go to next frame
            temp_img = sprite_sheet.subsurface(runningtotal+(x*self.size),0,self.size,self.size)
        else:
            temp_img = sprite_sheet.subsurface(x*self.size,0,self.size,self.size)
        temp_img_list.append(pygame.transform.scale(temp_img, (self.size *self.image_scale,
self.size*self.image_scale)))#Scale up frame and add to temp_list
        if x == animation - 1:
            runningtotal += x*self.size
        animationlist.append(temp_img_list)
    return animationlist
```

```
def move(self, screen_width, screen_height, target, surface):
    self.running = False
    speed = 15
    gravity = 1.4
    dx = 0 #positive = going forward, negative = going backwards
    dy = 0
    knock_x = 0

    #get key pressed on keyboard
    key = pygame.key.get_pressed()

    if self.CPU:

        if not self.attacking:

            #if CPU can attack player
            if abs(target.rect.centerx - self.rect.centerx) < 45 and not self.dead and not target.dead:
                self.attack(surface,target)
```

```
self.attacking = True

if random.randint(0,1) == 0:
    self.attack_type = 4
else:
    self.attack_type = 5

if self.checkAbilityBar:
    self.activate_ability()

#if player is in the air, CPU jumps
if random.randint(0,45)==7:
    if self.rect.bottom + dy > screen_height - 0 and not self.dead and not target.dead:
        self.vel_y = 0
        self.jump = False
        dy = (screen_height - 0) - self.rect.bottom
    else:
        self.vel_y = -30
        self.jump = True
#CPU chases player
if target.rect.centerx+10 > self.rect.centerx and not self.dead and not target.dead:
    dx = speed-5
    self.running = True
elif target.rect.centerx+10 < self.rect.centerx and not self.dead and not target.dead:
    dx = -speed + 5
    self.running = True
else:
    if self.player:

        if not self.attacking:

            if key[pygame.K_a]:
                dx = -speed # change in speed becomes negative, moves char backwards
```

```
        self.running = True
    if key[pygame.K_d]:
        dx = speed
        self.running = True
    if key[pygame.K_w] and not self.jump:
        self.vel_y = -30 #makes player jump up
        self.jump = True

    #attack
    if key[pygame.K_e] or key[pygame.K_r] or key[pygame.K_q]:
        self.attack(surface, target)
        self.attacking = True

    #determine attack type used
    if key[pygame.K_e]:
        self.attacktype = 4
    if key[pygame.K_r]:
        self.attacktype = 5
    if key[pygame.K_q] and self.ability_charge >= 1:
        self.activate_ability()
else:
    if not self.attacking:
        if key[pygame.K_LEFT]:
            dx = -speed # change in speed becomes negative, moves char backwards
            self.running = True
        if key[pygame.K_RIGHT]:
            dx = speed
            self.running = True
        if key[pygame.K_UP] and not self.jump:
            self.vel_y = -30 #makes player jump up
            self.jump = True
```

```
#attack
if key[pygame.K_n] or key[pygame.K_m] or key[pygame.K_b]:
    self.attack(surface, target)
    self.attacking = True

#determine attack type used
if key[pygame.K_n]:
    self.attacktype = 4
if key[pygame.K_m]:
    self.attacktype = 5
if key[pygame.K_b] and self.ability_charge >= 1:
    self.activate_ability()

#update y coordinate
self.vel_y += gravity
dy += self.vel_y

if self.hit:
    if not self.flip:
        self.vel_x = -self.knockFriction

    if target.attacktype == 5: #extra knockback for attack 2
        self.vel_x += self.vel_x-20
    dx += self.vel_x
    self.knockFriction -= 1

else:

    self.vel_x = self.knockFriction
    if target.attacktype == 5:
        self.vel_x += self.vel_x+20
```

```
dx += self.vel_x

self.knockFriction -= 1

if self.knockFriction <= 0.25: #reset knockback and velocity to 0 when knockback is negligible
    self.knockFriction = 0
    self.vel_x = 0

#check player on screen

if self.rect.left + dx < 0 and not self.dead: #if the speed and position of character beyond left
screen border
    dx = -self.rect.left #limit character speed to left border

if self.rect.right + dx > screen_width and not self.dead: #if the speed and position of character
beyond right screen border
    dx = screen_width - self.rect.right #limit character speed to right border

if self.rect.bottom + dy > screen_height and not self.dead:
    self.vel_y = 0
    self.jump = False #character can jump again
    dy = screen_height - self.rect.bottom #limit change in y

if self.rect.top + dy < 0:
    dy = -self.rect.top


#ensure player faces each other

if target.rect.centerx > self.rect.centerx:
    self.flip = False
else:
    self.flip = True


#update player position

if not self.dead:
    self.rect.x += dx
```

```
self.rect.y += dy
else:
    self.rect.y -= knock_x**2-5*knock_x - 20
    if self.player:
        if self.flip:
            knock_x = 10
            self.rect.x+=knock_x
        else:
            knock_x = 10
            self.rect.x-=knock_x

    if not self.player:
        if self.flip:
            knock_x = 10
            self.rect.x+=knock_x
        else:
            knock_x = 10
            self.rect.x-=knock_x

def activate_ability(self):
    if self.checkAbilityBar(): # Use instance-based charge
        self.ability.used = True
        self.ability.active = True
        self.ability.frame_index = 0
        self.ability_charge = 0 # Reset charge
        # Position ability relative to fighter

        # if self.flip:
        #     self.ability.rect.x = self.rect.x - 10
        # else:
        #     self.ability.rect.x = self.rect.x + 10
        # self.ability.rect.y = self.rect.centery
```

```
def checkAbilityBar(self):
    return self.ability_charge >= 1 # True when full

def updateAbilityPos(self):
    if not self.ability.active:
        self.ability.rect.x = self.rect.x
        self.ability.rect.y = self.rect.y
def restoreHealth(self):
    self.dead = False
    self.health = 100
def resetPosition(self,x,y):
    self.rect = pygame.Rect((x,y,80,150))

def draw(self, surface, colour):
    #pygame.draw.rect(surface, colour, self.rect)
    img = pygame.transform.flip(self.image, self.flip, False)

    surface.blit(img, (self.rect.x - (self.offset[0]*self.image_scale),self.rect.y - (self.offset[1] *
self.image_scale)))

def update(self, target, surface):
    self.updateAbilityPos()
    if self.ability.active:
        self.ability.draw(surface)
        self.ability.update(target)
        self.update_action(4)
    if self.health<= 0:
        self.dead = True
        self.health = 0.1
    if self.hit:
```

```
self.knockFriction = 10
self.update_action(7)

elif self.attacking:
    if self.attacktype == 4:
        self.update_action(4)
    elif self.attacktype == 5:
        self.update_action(5)
elif self.jump:
    self.update_action(2)
elif self.running:
    self.update_action(1)
else:
    self.update_action(0)

animation_cooldown = 60 #milliseconds until next
if self.attacking:
    animation_cooldown = 125 #cooldown is longer for attacks

if pygame.time.get_ticks() - self.updateTime > animation_cooldown: # if time passed has
exceeded animation period, go to next frame
    self.frame_index += 1
    self.updateTime = pygame.time.get_ticks()
    if self.frame_index >= len(self.animationlist[self.action])-1: #if last frame has been reached, go
back to first frame
        self.frame_index = 0
    if self.attacking:
        self.attacking = False #reset attacking status
        self.attack_type = 0 #reset attacktype
    if self.hit:
        self.hit = False
```

```
self.attacking = False
```

```
self.image = self.animationlist[self.action][self.frame_index] #current frame to be drawn
```

```
def attack(self, surface, target):
```

```
    self.attacking = True
```

```
    attackingRect = pygame.Rect(self.rect.centerx - (2*self.rect.width*self.flip), self.rect.y,
2*self.rect.width, self.rect.height)
```

```
    if attackingRect.colliderect(target.rect) and not target.hit:
```

```
        target.health -= 6.5 #the damage players receive from attacks
```

```
        target.hit = True
```

```
        if target.flip:
```

```
            target.vel_y -= 20
```

```
        else:
```

```
            target.vel_y -= 20
```

```
    #pygame.draw.rect(surface,(0, 255, 0), attackingRect) #- shows the area which damage can
occur to other player
```

```
def update_action(self, newAction):
```

```
    #check if new action is different to previous
```

```
    if newAction != self.action:
```

```
        self.action = newAction
```

```
    #update animation settings
```

```
    self.frame_index = 0
```

```
    self.updateTime = pygame.time.get_ticks()
```

UFGability.py

import pygame

class Ability():

def __init__(self, x, y, data, animationsteps, spritesheet, flip, debug=False):

self.size = data[0]

self.image_scale = data[1]

self.offset = data[2]

self.flip = flip

self.animationlist = self.load_images(spritesheet, animationsteps)

frame_width = spritesheet.get_width() // animationsteps

self.rect = pygame.Rect(

x - (self.offset[0] * self.image_scale), # apply X offset

y - (self.offset[1] * self.image_scale), # apply Y offset

frame_width * self.image_scale - 100, # use frame width

spritesheet.get_height() * self.image_scale # use frame height

)

self.frame_index = 0

self.image = self.animationlist[self.frame_index]

self.updateTime = pygame.time.get_ticks()

self.abilityBar = 0

self.active = False

self.cooldown = 7000

self.debug = debug # debug colour hitbox

self.cycles = 0

self.knocked = False

self.used = False

def load_images(self, sprite_sheet, animationsteps):

#Validate input

sheet_width, sheet_height = sprite_sheet.get_size()

```
frame_width = sheet_width // animationsteps
frame_height = sheet_height
animationlist = []

#Extract frames from the sprite sheet
for frame_index in range(animationsteps):
    frame_x = frame_index * frame_width
    temp_img = sprite_sheet.subsurface((frame_x, 0, frame_width, frame_height))
    scaled_img = pygame.transform.scale(
        temp_img, (frame_width * self.image_scale, frame_height * self.image_scale)
    )
    animationlist.append(scaled_img)

return animationlist

def draw(self, surface):
    img = pygame.transform.flip(self.image, not self.flip, True)
    if self.debug:
        pygame.draw.rect(surface, (255, 255, 0), self.rect) #Debug hitbox
    surface.blit(
        img,
        (self.rect.x - (self.offset[0] * self.image_scale),
         self.rect.y - (self.offset[1] * self.image_scale))
    )
    # if self.active:
    #     if not self.flip:
    #         self.rect.x += 20
    #     else:
    #         self.rect.x -= 20

def update(self, target):
    #Update animation only if active
```

```
if self.active:

    animation_cooldown = 80 #Time between frames (milliseconds)

    if pygame.time.get_ticks() - self.updateTime > animation_cooldown:

        self.frame_index += 1

        self.updateTime = pygame.time.get_ticks()

#Reset to the first frame if the animation ends
if self.frame_index >= len(self.animationlist)-1:

    self.cycles += 1

    self.frame_index = 0

if self.cycles > 5:

    self.cycles = 0

    self.active = False

    self.used = False

self.image = self.animationlist[self.frame_index]

if target.rect.centerx > self.rect.centerx:

    self.rect.x += 20

else:

    self.rect.x -= 20

if self.rect.colliderect(target.rect):

    target.health -= 16.5#damage of ability

    target.hit = True

    self.active = False

    target.knocked = True

    self.used = False

if self.knocked:#same knockback code as in fighter class

    if not target.flip:
```

```
target.vel_x = -target.knockFriction-20
```

```
target.rect.x += target.vel_x
```

```
target.knockFriction -= 1
```

```
else:
```

```
target.vel_x = target.knockFriction+20
```

```
target.rect.x += target.vel_x
```

```
target.knockFriction -= 1
```

```
if target.knockFriction <= 0.25: #reset knockback and velocity to 0 when knockback is negligible
```

```
target.knockFriction = 0
```

```
target.vel_x = 0
```

```
target.knocked = False
```